# Engagement record

What was built, what was measured, and what is still unknown — a 1:1 port of AnyCompany's eight-specialist fraud underwriting platform onto Amazon Bedrock AgentCore.

*For two readers: the customer's engineer, who wrote the originals, and the executive and account audience who has to decide on the basis of them.*

AnyCompany runs agentic auto-loan fraud underwriting on Snowflake Intelligence — eight specialist fraud agents plus a master synthesizer, all written by their own engineer. This site records what happened when those nine agents were ported onto **Amazon Bedrock AgentCore** without rewriting a single prompt.

## Four headline results

| Result | Figure | Artifact |
|---|---|---|
| End-to-end adjudication latency | **31.45 s p50 / 46.55 s p95** (n=42) | `evals/results/pipeline_14x3_us-east-1.json` |
| Runs under the customer's 60 s target | **40 of 42** | same file |
| Cost per adjudication | **$0.1440 p50** (n=41 priced runs) | same file |
| Specialists held on Claude, on one deterministic metric | `_context` grounding **0.642 Claude vs 0.357 GPT** | `evals/results/set_comparison.json` |
| Master synthesis, on **GPT-5.6 Luna** | **4.20 s p50** (n=42), down from 55.2 s on Claude | `evals/results/pipeline_14x3_us-east-1.json` |

Derivation, per-specialist breakdown and every caveat: **Measured results**.

This is a **mixed-model system**, and both halves of that were measured rather than assumed: eight Claude specialists and one GPT-5.6 synthesizer. GPT-5.6 was also tested for all eight specialists across **72 blind-judged analyses** and declined — a projected 47.8% cost reduction turned down on one compliance measurement. **GPT-5.6 on this platform**.

## Start here

**Architecture overview**
Request lifecycle, the two-layer split, where each AWS service sits, and how the shape differs from the incumbent.

**Executive summary**
What was proven, what it costs, the one blocking question, and the Friday recommendation.

**Prompt fidelity**
Byte-identical prompts, sha256 verification, and the five mutation attempts that were caught.

**Open questions**
Five items that need a customer answer, and the exact sentence to ask each one with.

**GPT-5.6 on this platform**
Where OpenAI models run in production, where they were tested and declined, and every measurement on Luna, Terra and Sol.

## One rule this site follows

Every number traces to an artifact committed in the repository, and the page quoting it names the file. Where a figure was not measured, the page says **NOT MEASURED** instead of estimating. Where two artifacts disagree, both appear and the authoritative one is named.

That rule is stated up front because an earlier draft summary for this engagement broke it. It claimed *"130 analyses over 8 domains, judged by GPT-5.6 Sol, band 23/26 per model, 12.3 s end-to-end at $0.0508"* — and none of those figures appear in any artifact on disk. Our real cross-family study is **120 analyses over 4 domains with two judges**; the real

measured end-to-end is **31.45 s p50 at $0.1440 p50 over 42 runs**. Detecting that class of error is why every percentile here carries its `n`.

## One question that blocks the cost case

AnyCompany reports a projected **$700K–$1M** annual Snowflake AI spend. Measured cost here is **$0.1440 per adjudication** (p50, n=41). Comparing those two needs a figure the customer has not supplied: **how many applications per day take the full eight-agent forensic path.**

| Applications/day through full 8-agent underwriting | Annualised at measured p50 |
|---|---|
| ~3,000 | ~$158K |
| ~300,000 | ~$15.8M |

Arithmetic: volume x 365 x $0.14395847. The p50 is measured; the two volumes are illustrative brackets, not the customer's figures. Same architecture, opposite conclusion — which makes this the single most important open item: **Snowflake comparison**.

## Site map


**Fidelity**
Byte-identical prompts, the 21-key contract, the General Rules as code, and the defects the sweeps found in the prompts themselves.

**Performance**
The 42-run benchmark, why the 75 s tail exists, why it is irreducible, and which agent owns the critical path.

**Model selection**
Two blind studies, a tier assignment measurement overturned, and the judge methodology that makes either credible.

**Cost**
Real levers, three that look like levers and are not, and the volume question that decides the comparison.

**Platform**
AgentCore CLI commands that silently do nothing, Strands concurrency, the bedrock-

**Reference**
Fixture set, eval harness and its refusal rules, an index of every artifact, and a glossary.

mantle base URL, and the SDK upgrade.

Customer prompts and orchestration documents are confidential. They live in git-ignored `prompts/verbatim/` and are **not** reproduced here: pages describe them, name their files, and quote at most a short operative phrase where the exact wording is the point.

Architecture overview ›

One request end to end — signal layer, eight-way parallel fan-out, master synthesis, and the two ...

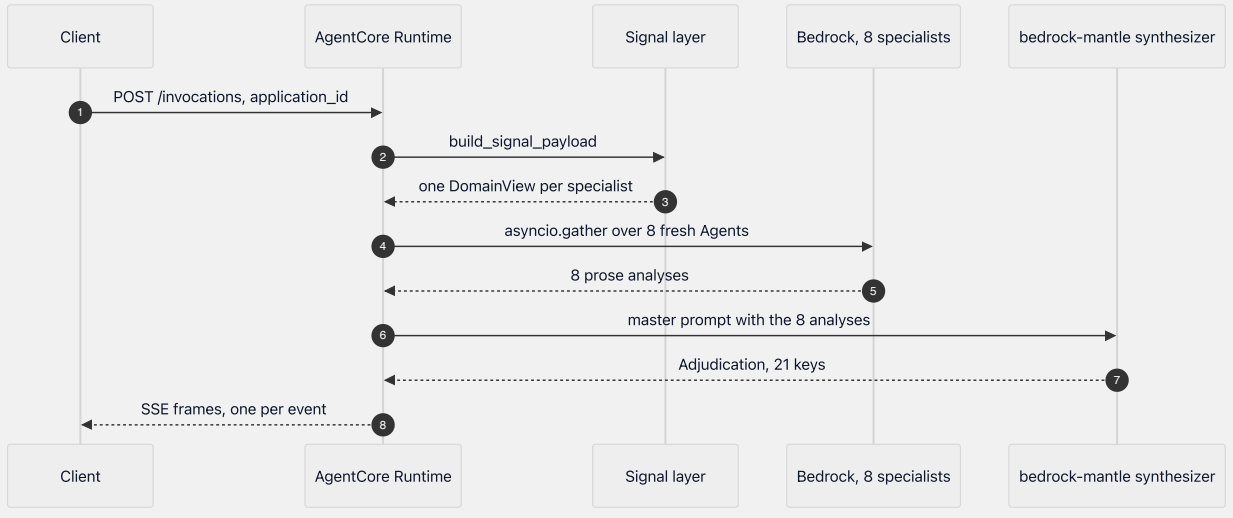
# Architecture overview

One request end to end — signal layer, eight-way parallel fan-out, master synthesis, and the two AgentCore Runtime entrypoints. Where each AWS service sits, and how the shape differs from the incumbent.

*For the AWS SA who has to explain this system in a room, and for the customer's engineer who has to recognise her own architecture in it.*

Nine agents, two layers, one boundary. Governed signals are computed first with no model involved; eight specialists then interpret their own slice of those signals in parallel; a synthesizer turns the eight narratives into one 21-key adjudication. End to end, that took **31.45 s at p50 over 42 measured runs** (`evals/results/pipeline_14x3_us-east-1.json`).

## Request lifecycle



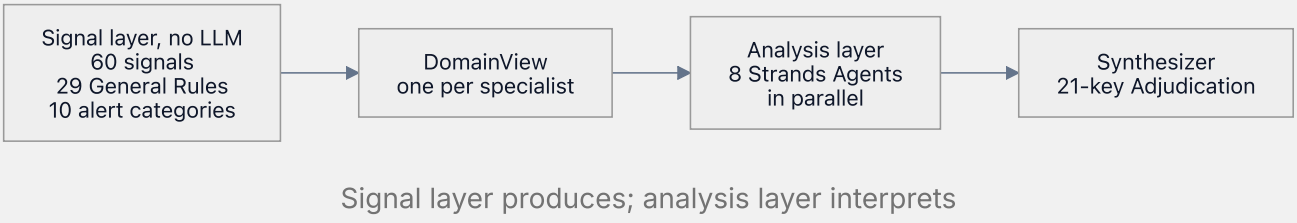

One application, start to finish

Steps 2 and 3 sit **outside** the timed window in every benchmark on this site, because the customer's own platform precomputes signals rather than querying per application. Every

latency figure here is agent latency, not pipeline-including-ETL latency.

Steps 4 and 5 are where the eight-way concurrency lives, and step 8 is why a client sees the first specialist land in single-digit seconds instead of waiting for all eight: an async-generator entrypoint is auto-converted to `text/event-stream`, one frame per yielded object.

## Two layers, one boundary

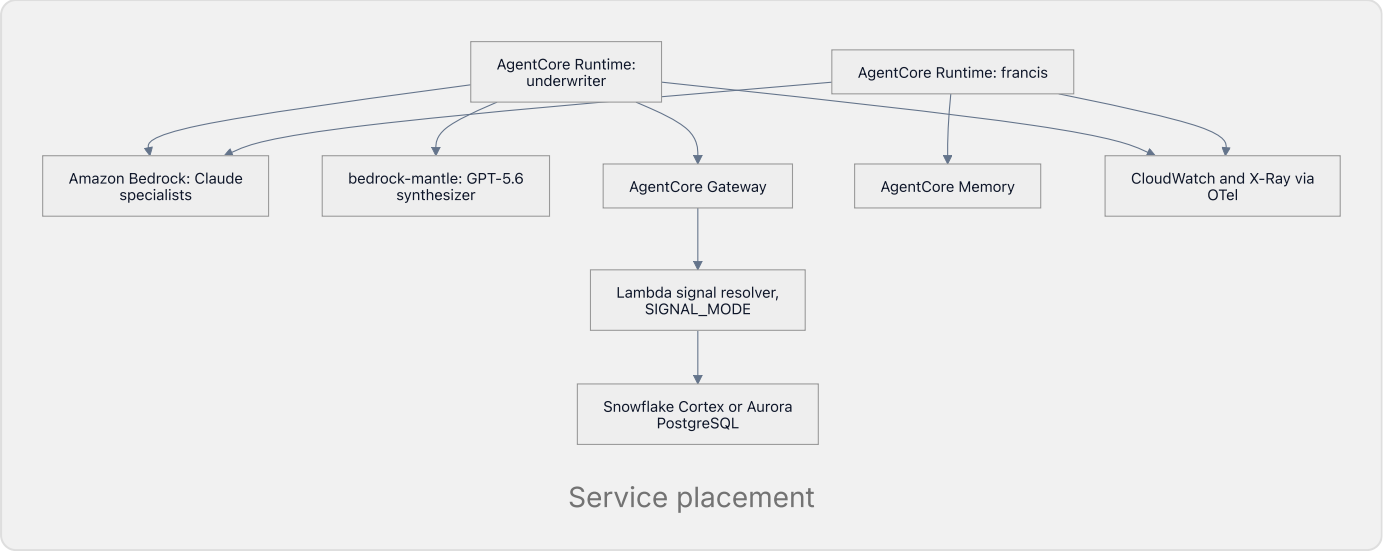



Signal layer produces; analysis layer interprets

**A specialist has no data tool at all.** `signal_layer.schema.DomainView` is the entire universe it reasons over: its own signals, their paired `_context` values, and its own fired alerts. "The agent only saw its own domain" is therefore a structural property of the code, not an instruction in a prompt that a model might ignore.

That split is the customer's own finding rather than an AWS preference, and it is also what keeps text-to-SQL out of the per-application hot path. Detail: Signal layer and Semantic layer.

# Where each AWS service sits




Service placement

| Component | Service | State today |
|---|---|---|
| Batch underwriting entrypoint | AgentCore Runtime, `CodeZip`, Python 3.12 | Authored, `agentcore validate` passes |
| Interactive router entrypoint | AgentCore Runtime, same build | Authored |
| Eight specialists | Amazon Bedrock, `global.*` inference profiles | Measured over 336 calls |
| Synthesizer | `bedrock-mantle` endpoint | Measured over 42 calls |
| Conversation memory for the router | AgentCore Memory, 7-day event expiry | Declared in `agentcore/agentcore.json`; unavailable under `agentcore dev` |
| Signal source | AgentCore Gateway → Lambda → Cortex or Aurora | **Recommended, not built.** `app/runtime_support.py::build_signal_payload` reads fixtures today and is the single swap point |
| Traces and spans | CloudWatch Transaction Search, X-Ray, OTel | Wiring documented; per-agent token and cost telemetry is our code's job because the SDK emits none |

Nothing above the signal layer changes when the data source does, and that is the whole point of putting `SIGNAL_MODE` on a Lambda rather than a data tool on an agent. See Deployment.

## Incumbent shape versus this shape

Same nine agents, same prompts, same output contract. What moves is where the queries happen, what runs concurrently, and what is measurable.

| Concern | Snowflake-shaped, as the customer describes it | AgentCore-shaped, this port | Recommendation |
|---|---|---|---|
| Signal retrieval | 8 Cortex Analyst messages plus 8 warehouse executions per application | **0** of each; signals precomputed by `signal_layer/` | **AgentCore.** The count is claimable without their credit rate; a dollar figure is not |
| Text-to-SQL per application | Regenerated per application. A published production decomposition of a Cortex Agent call attributes ~27 s of a 55 s total to Analyst SQL generation — **third-party, not our measurement** | Queries fixed at design time, outside the hot path | **AgentCore.** For a fixed signal set, regenerating SQL per application buys nothing |
| Specialist concurrency | Reported symptom: 4,500 applications took over 48 hours — **their figure** | `asyncio.gather` over 8 fresh `Agent` instances; measured p50 speed-up **3.64x** against the sequential counterfactual (n=42) | **AgentCore**, with the caveat that the counterfactual's arrangement is hypothetical |
| Partial failure | Not characterised by us | 7-of-8 graceful degradation, demonstrated live once in the benchmark | **AgentCore** |
| Prompt custody | Prompts live in the platform | Byte-identical files, sha256-verified at import, CI-diffed | **AgentCore**, and this is the fidelity argument |
| Per-call cost and token visibility | Not characterised by us | Captured per agent per call from real usage; `evals/bench.py` is the only code allowed to publish it | **AgentCore** |
| Total spend comparison | Reported projection **$700K–$1M** annual AI spend — **their figure** | Measured **$0.14396 per adjudication** (p50, n=41 priced runs) | **Undecided.** One missing input decides |

| Concern | Snowflake-shaped, as the customer describes it | AgentCore-shaped, this port | Recommendation |
|---|---|---|---|
| | | | it: Snowflake comparison |

> ⚠ **Two kinds of number appear in that table and they are labelled**
>
> Rows marked *their figure* are the customer's statements about their own platform. We never measured Snowflake, have no access to their account, and nothing on this site is a benchmark of their system. The ~27 s decomposition is a published third-party figure and carries the same label wherever it appears.

## Two entrypoints, two shapes

| | Batch underwriting | Interactive router |
|---|---|---|
| Path | `app/underwriter/main.py` | `app/francis/main.py` |
| Shape | Async-generator `@app.entrypoint` → SSE | Eight specialists as `@tool` wrappers |
| Concurrency | All eight, every request | **At most 2 tools per query**, from the customer's own routing rules |
| Output | 21-key `Adjudication` | Markdown, fixed section order |
| Timeout ceiling | 60 minutes streaming; 15 minutes synchronous and not adjustable | same |

Only one `@app.entrypoint` exists per app — a second one silently overwrites the first — and the second parameter must be literally named `context` or it is never injected. Detail: Runtime contract.

# Reading order from here

How the rest of the site is sequenced

Fidelity first is deliberate: the person reviewing this wrote the originals, so byte-identical prompts and a 21-of-21-key contract are the gate everything else sits behind.

**Component detail**
The eight specialists, the registry, the fan-out, the synthesizer, the repository map.

**Measured results**
42 live adjudications, 336 specialist calls, $6.4659 of Bedrock spend, every figure cited.

**Prompt fidelity**
What byte-identical means here and the five mutation attempts that were caught.

**Snowflake comparison**
Our measured cost, their reported spend, and the one input that decides between them.

## Still unknown at the architecture level

- **AgentCore Runtime overhead is not in any figure on this site.** Benchmarked calls went to Bedrock directly from the host recorded in `run.host`, so container cold start, session setup and SSE framing are excluded.
- **Throughput is unmeasured.** One application was in flight at a time, so nothing says what happens to p95 when adjudications overlap.
- **The Gateway path is a recommendation, not a deployment.** No Gateway or Lambda resource exists yet; `agentcore/aws-targets.json` still ships as an empty list.

‹ Engagement record
What was built, what was measured, and w...

Executive summary ›
For the account and executive audience — ...

# Executive summary

For the account and executive audience — what was proven on Amazon Bedrock, what it costs, the one question that blocks the cost comparison, and the Friday recommendation.

*For the executive and account audience. Every figure names the artifact it came from; the engineering detail behind each claim is one link away.*

Sub-minute forensic underwriting on Amazon Bedrock is **proven**. A lower cost than the incumbent is **not** — and that gap is one missing customer input, not a missing measurement.

The eight-specialist pipeline runs end to end in **31.45 seconds at p50 and 46.55 seconds at p95**, with **40 of 42 measured runs under the 60-second target**. Live measurement over 42 adjudications and 336 specialist model calls, not a projection.

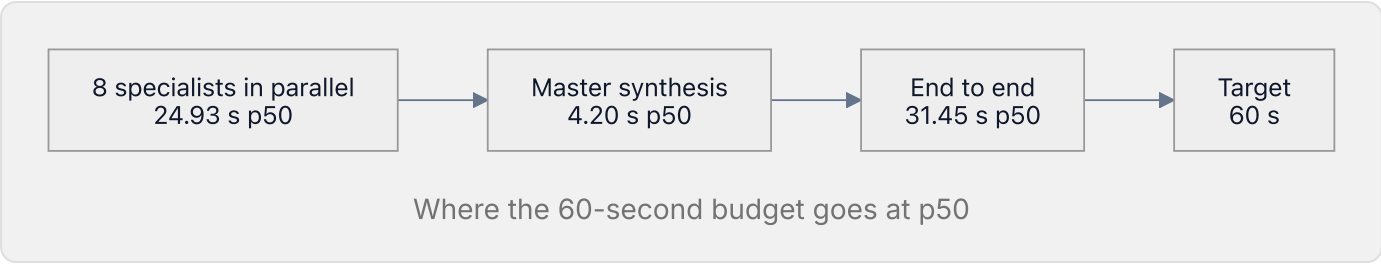
## Proven versus unproven

| Claim | Status | Evidence |
|---|---|---|
| Sub-minute forensic underwriting on Bedrock | **Proven** | 40 of 42 runs under 60 s; p50 31.45 s (n=42) |
| Fidelity to the customer's own prompts and contracts | **Proven** | Byte-identical prompts, sha256-verified; 21-key object validated on 42 of 42 runs |
| 7-of-8 graceful degradation under partial failure | **Proven** | One degraded run finished in 21.2 s and still validated all 21 keys |
| Lower cost than the incumbent | **Undecided** | Needs the daily volume figure. Detail |
| Behaviour when adjudications overlap | **Not measured** | One application in flight at a time throughout |
| Any p99 | **Refused by the harness** | Needs 100 samples; the run has 42 |

## What was measured

Every figure in this section: `evals/results/pipeline_14x3_us-east-1.json`, run `2026-07-28T07:42:42+00:00` in `us-east-1`, git `9bb569bac801`. 14 applications x 3 repetitions = **42 live adjudications, 336 specialist calls, $6.4659 of measured Bedrock spend.**

| | n | p50 | p95 | max |
|---|---|---|---|---|
| **End to end** | 42 | **31.45 s** | **46.55 s** | 74.97 s |
| Eight specialists in parallel (fan-out) | 42 | 24.93 s | 41.05 s | 70.62 s |
| Master synthesis | 42 | 4.20 s | 5.59 s | 15.21 s |
| Same eight run sequentially (counterfactual) | 42 | 85.62 s | 157.14 s | 173.00 s |




Where the 60-second budget goes at p50

Each box is an independent percentile over its own 42 samples, so the first two do not add to the third — the medians come from different runs.

**Cost per adjudication:** $0.1177 min, **$0.1440 p50**, $0.2536 max (n=41 fully priced runs; the one degraded run is excluded from cost and retained in latency).

**Parallel speed-up** over the sequential counterfactual: 1.85x min, **3.64x p50**, 5.11x max (n=42). It appears as a distribution rather than one number because a fan-out is bounded by its slowest member, so the ratio moves with which specialist ran long.

No p99 appears anywhere on this site. Percentiles are refused below 100 samples for a p99 and 20 for a p95, and at n=42 it recorded the refusal in the artifact verbatim: *"refused: 42 sample(s), need >= 100. A p99 over 42 sample(s) is not a p99."* Nothing here converts that refusal into a number.

## What the customer's engineer checks first

Fidelity is the product, not a detail. She wrote all nine production agents.

| Fidelity claim | Status |
|---|---|
| Her nine prompts | **Byte-identical**, sha256-verified at import, CI-diffed against source. Five mutation attempts, including regenerating the manifest, were all caught. |
| The adjudication object | **Exactly 21 keys in her order.** Validated on 42 of 42 runs, including the degraded one. |
| Her 29 numbered General Rules | **27 enforced as tested code**, 2 enforced by the verbatim prompt text. |
| Ten alert-ID categories | Data, not prose: 166 list entries, 148 distinct IDs, 18 multi-category. |
| Bust-out and rings output shape | Her prompts define **no title** for those two, and the port does not invent one. |

Detail: Prompt fidelity and Output contracts.

> **What the port replaced**
>
> Prior code in this repository **paraphrased all nine prompts** and invented a `RISK:` / `FINDINGS:` output format. Its parser defaulted to `LOW RISK` whenever it could not find that invented `RISK:` line — so feeding it the real prompts would have cleared every application silently. That is the first defect a fidelity review would have found, and it is gone.

## One question decides the cost case

AnyCompany reports a projected **$700K–$1M** annual Snowflake AI spend. Measured cost here is **$0.1440 per adjudication** (p50, n=41). Annualising needs one number nobody has given us: **applications per day through the full eight-agent forensic path.**

| Applications/day through full 8-agent underwriting | Annualised at measured p50 |
|---|---|
| ~3,000 | ~$158K |
| ~300,000 | ~$15.8M |

Arithmetic: volume x 365 x $0.14395847. The p50 is measured; the volumes are illustrative brackets, not the customer's figures.

Same architecture, opposite conclusion. **This is the most important open item in the engagement** and it cannot be closed from our side. Detail and the exact question to ask: Snowflake comparison.

One thing to keep off a cost slide: **per-agent model tiering saves $0.00 structurally.** Haiku-to-Sonnet is a $2/$10-per-million-token step down and Sonnet-to-Opus is the identical step up, so two light agents fund exactly two heavy agents at equal token counts. A test asserts that arithmetic so the claim cannot come back. Real levers are on cost levers; tiering is not one of them.

## Where the model choices came from

Two blind studies, both with measured spend recorded in the artifact.

| Study | Scope | Judges | Measured spend | Artifact |
|---|---|---|---|---|
| Claude-only tier study | 48 blind-judged analyses | Claude Opus 5 | **$4.9088** | `evals/results/tier_quality.json` |
| Cross-family study | 4 agents x 6 applications x 5 models = **120 analyses, 240 verdicts** | Claude Opus 5 and GPT-5.5 | **$16.8869** | `evals/results/set_comparison.json` |

Three results deserve executive attention.

1. **Specialists stay on Claude, and that was a decision rather than a default.** GPT-5.6 measured faster and cheaper. It was held on one deterministic metric — how often an analysis validates a signal against the paired `_context` column its own prompt requires: Claude **0.642**, GPT **0.357**, lower on 4 of 4 agents. No judge involved. Detail.
2. **Synthesizer selection also went to measurement, and Claude lost it.** On the same eight narratives, Claude Sonnet 5 took 55.2 s and GPT-5.6 Luna took 6.1 s for the same

verdict — the substitution that brings end-to-end under a minute. n=1 each on that comparison; the 42-run figure for Luna in the pipeline is 4.20 s p50. Detail.

3. **Measurement overturned a reasoning-based tier assignment on two agents.** Reading the prompts said fraud rings and synthetic identity needed the largest model. Measurement contradicted that on both. Detail.

## Risks, stated plainly

| Risk | Severity | Status |
|---|---|---|
| Volume figure that decides the cost comparison is unknown | **Blocking the cost case** | Customer input required |
| Two fixtures are mis-banded by **every model in both families** | High — a prompt defect, not a model choice | Fix is a sentence in her prompts; unmeasured |
| `master_agent_prompt.txt` and her process-guide PDF disagree on the mid-band enum | Medium — anything keying off the band string is wrong for one of them | Surfaced, not silently resolved |
| `full_underwriting_signal_dictionary.xlsx` never received | Medium | The 60-signal registry is reconstructed from the nine prompts |
| A ~0.6%-per-call server-side generation stall produced the two runs over 60 s | Medium | Irreducible; no client-side change removes it |

All of them, with the exact questions to ask: Open questions.

## Friday recommendation

**Demo on an application with a measured-stable latency profile.** Three of the fourteen fixtures ran three repetitions within a sub-two-second spread — shown sorted ascending: **APP-1010** (27.95 / 28.38 / 28.59 s), **APP-1002** (22.32 / 22.92 / 23.18 s) and **APP-1011** (21.21 / 22.40 / 22.79 s). Against those, APP-1007 ran 23.52 / 34.58 / **74.97** s and APP-1001 ran

18.93 / 24.25 / **70.24** s — spreads over 51 seconds on byte-identical input. Source: `evals/results/tail_analysis.json`. Stated with its limit: **n=3 per application makes this a low-variance observation, not a guarantee**, because the stall rate is per *call* and a run makes eight of them.

Four more things for the room:

1. **Lead with fidelity, not latency.** The customer's engineer is the gate, and byte-identical prompts plus a 21-of-21-key contract on 42 of 42 runs is the strongest thing here.
2. **Raise the two calibration failures before she finds them.** APP-1014 rings and APP-1004 bustout are mis-banded by all five models across both families, every one wrong the same way. Presenting it as a shared prompt defect we diagnosed lands very differently from her discovering it live. Detail.
3. **Ask the volume question in the room.** One number converts the cost slide from unresolvable to decided.
4. **Promise no p99 and no throughput figure.** Neither was measured. Applications ran one at a time, so nothing here says what happens when N adjudications overlap.

## Explicitly not measured

Carried from the artifact's own `caveats` array, so this is the benchmark's list rather than ours.

- **AgentCore Runtime overhead.** Benchmarked calls went to Bedrock directly from the host recorded in `run.host`, so container cold start, session setup and SSE framing are **not** in the end-to-end figures.
- **Signal generation.** Precomputed and outside the timed window, matching the customer's own architecture. End to end here is agent latency, not pipeline-including-ETL latency.
- **Throughput.** One application in flight at a time.
- **Any p99, anywhere.**

‹ Architecture overview
One request end to end — signal layer, eigh...

Component detail ›
Each piece of the port — the eight specialis...

# Component detail

Each piece of the port — the eight specialists and their anti-false-positive rules, the 60-signal registry, the fan-out, the synthesizer, the two entrypoints, and the repository map.

*For the customer's engineer, and for anyone who has read **Architecture overview** and now wants the parts list.*

Nine agents, one signal registry, two runtime entrypoints. This page is the component inventory; the system-level picture and the request lifecycle live on Architecture overview, and every number lives on Measured results.

## What "1:1" means here

**Prompts and output contracts are the customer's, byte for byte** — not "equivalent in intent". Prompt text loads from git-ignored `prompts/verbatim/` under sha256 verification and the port never rewrites it. Fidelity is the product, because the person reviewing this wrote the originals. See Prompt fidelity.

## Eight specialists

Each carries a persona name from the customer's own documents (`prompts.loader.AGENT_NAMES`) and an anti-false-positive rule its prompt requires it to honour. Reducing false positives is her stated quality goal: six of the eight prompts say outright that LOW should be the most common outcome — `bustout` and `rings` do not.

| Specialist | Persona | Detects | Anti-false-positive rule it must honour |
|---|---|---|---|
| identity | `ISAAC_IDENTITY` | SSN sharing, PII mismatch, identity volatility | Multiple alerts from one underlying data point count as one signal |
| dealer | `DIANA_DEALER` | Dealer-*orchestrated* fraud | Default / EPD / consortium scores are portfolio risk, not fraud |
| straw | `SAMMY_STRAW` | Proxy or nominee co-borrowers, role-switching | No co-borrower means straw risk is inherently minimal |
| employment | `EMILY_EMPLOYMENT` | Fabricated or shell employers | Missing employer fields are a data gap, not fraud |
| income | `IRIS_INCOME` | Overstatement, inconsistent reporting | Under 15% variance is normal; self-employment explains variance |
| synthetic | `SYLVIA_SYNTHETIC` | Fabricated identity plus manufactured credit | Thin files are legitimate for young adults and new immigrants |
| bustout | `BRUCE_BUSTOUT` | Rapid velocity, unsustainable exposure | Geography alone must not move the classification |
| rings | `RENEE_RINGS` | Shared identifiers, exact-unit reuse | Multi-unit housing and large employers explain overlap |

## Signal registry

`signal_layer/registry.py` is the contract every other layer resolves names through. Read from the module at import:

| | Count |
|---|---|
| Signals in `SIGNAL_OWNER` | 60 |
| Signals owned by exactly one domain | 55 |
| Signals legitimately shared by two domains (`SHARED_SIGNALS`) | 5 |

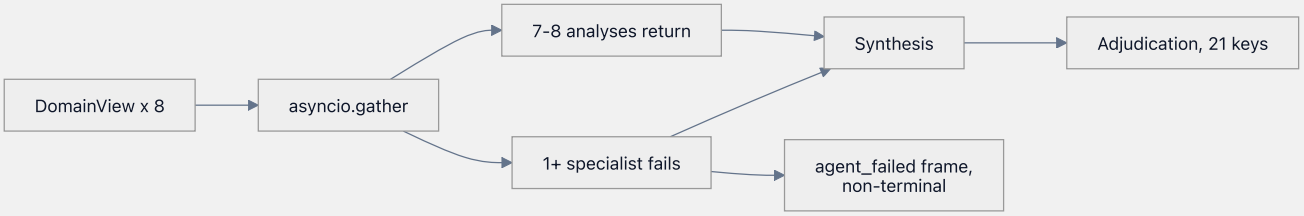
Per-domain ownership: dealer 14, synthetic 10, straw 8, employment 8, identity 7, income 7, bustout 5, rings 1. Rings owns one name of its own because most of what it reasons over is shared with identity — the three `*_reuse_distinct_ssn_count` signals.

**Near-duplicate names are both materialised, never aliased.** Her prompts spell the same *concept* differently in different domains, and collapsing those spellings silently degrades whichever agent uses the other name. So `employer_other_ssns_count` (rings) and `employer_distinct_other_ssns_count` (employment) are separate keys; so are `phone_ssn_count` (synthetic) and `phone_reuse_distinct_ssn_count` (identity, rings); so are `ssn_income_increase_20_percent_or_more` (income) and `ssn_income_increased_20_percent_last_30d_rapid_growth` (bustout). A specialist handed a signal its prompt never mentions is being invited to reason about something it has no interpretation for.

> **The registry is reconstructed, not received**
>
> `full_underwriting_signal_dictionary.xlsx` — the authoritative signal contract — was **never received**. It appears only as an embedded thumbnail on page 3 of the process-guide PDF, with zero extractable attachments. `CLEAN_MAIN_STREAM_FILE.sql` was likewise never received. So the 60-signal registry is **reconstructed from the nine prompts**, and that is the correct thing to say to the customer rather than presenting it as the signal set. Open questions.

## Fan-out

`agents/fanout.py` runs `asyncio.gather` over **eight freshly constructed Strands `Agent` instances**, one per domain, each on its own configured model. End to end is therefore *slowest specialist + synthesis*, not the sum of eight.




Fan-out and degradation paths

Three design constraints behind that, none of them stylistic:

| Constraint | Consequence if ignored |
|---|---|
| A Strands `Agent` cannot be invoked concurrently | `ConcurrencyException`. A fresh `Agent` per invocation is mandatory, never module-level |
| `strands.multiagent.GraphBuilder` is the wrong primitive | Rejected on four grounds, including fail-fast on any node error where this pipeline needs 7-of-8 degradation |
| `BedrockModel` defaults to `max_pool_connections=10` | A hard ceiling on an eight-way fan-out; and supplying `boto_client_config` silently drops Strands' `read_timeout=120` back to botocore's 60 |

Full failure modes: Strands Agents SDK.

**Graceful degradation is designed in.** A failing specialist does not fail the adjudication, and the 42-run benchmark contains a live demonstration: APP-1011 repetition 3 lost both `synthetic` and `rings` to `server_5xx`, finished in **21.2 s** — a 5xx fails fast, so it costs a fraud dimension rather than time — and **still validated all 21 keys**.

## Synthesizer

`agents/synthesize.py` substitutes the eight analyses into the customer's verbatim master prompt's own placeholders, in her input order — `IDENTITY, DEALER, STRAW, EMPLOYMENT, INCOME, SYNTHETIC, BUSTOUT, RINGS`, deliberately **not** alphabetical (`prompts.loader.DOMAINS`) — and emits the 21-key `Adjudication` via Strands' `structured_output_model=` kwarg with one bounded repair attempt.

Across the 42 benchmark runs, `repair_attempts` totalled **0**.

It runs on **GPT-5.6 Luna** via the `bedrock-mantle` endpoint: the one place a non-Claude model earned its slot on a measurement, and that measurement has its own page.

# Two runtime entrypoints

| Entrypoint | Path | Shape |
|---|---|---|
| Batch underwriting | `app/underwriter/main.py` | Async-generator `@app.entrypoint` → SSE. Fan-out then synthesis, streaming each specialist as it completes. |
| Interactive router ("Francis") | `app/francis/main.py` | Eight specialists as `@tool` wrappers, with the customer's **maximum-2-tools-per-query** cap. |

Francis carries her own behavioural contract, all of it from `master_agent_orchestration_062026` — a different artifact from `master_agent_prompt.txt`, the former being the interactive router and the latter the batch synthesizer.

| Rule | Where it comes from |
|---|---|
| Never identify as Claude; always refer to herself as Francis | The prompt's opening lines |
| Deflect RETRO questions to `RETRO_FRANCIS` | Named explicitly in the prompt |
| **Call a MAXIMUM of 2 tools per query**, picking the 2 most relevant | The prompt's `SPEED OPTIMIZATION` clause |
| Response shaped `## EXECUTIVE SUMMARY` / `## ANALYSIS` / conditional `###RECOMMENDATIONS` | The same document's `instructions.response` block |
| No unsolicited loan recommendations | The prompt's closing line, and separately General Rule 12 |

> ⚠ **Three runtime gotchas that produce silent or confusing failures**
>
> `@app.entrypoint`'s second parameter must be **literally named** `context` or it is not injected and every request raises `TypeError`. An async-generator entrypoint returns SSE automatically — never hand-format it. And exceptions raised mid-stream arrive as a terminal `data: {"error": ...}` frame **after HTTP 200**, so a client checking only the status code reports success on a failed run. Full list: **Runtime contract**.

## Model assignment as it stands

From `agents/models.py:SPECIALIST_MODELS` and `DEFAULT_SYNTHESIZER_MODEL`, read from the module:

| Role | Model | max_tokens |
|---|---|---|
| dealer, straw | `global.anthropic.claude-haiku-4-5-20251001-v1:0` | 3072 |
| identity, employment, income, synthetic, bustout, rings | `global.anthropic.claude-sonnet-5` | 4096 |
| master synthesizer | `openai.gpt-5.6-luna` (`bedrock-mantle`) | 4096 |

**No Opus 5 agent remains.** `synthetic` and `rings` were on Opus 5 until 2026-07-28, because a reading of the customer's prompts said fraud rings needs heavy multi-signal reasoning. Measurement contradicted that on both, and the correction is recorded rather than quietly applied: Tier re-derivation.

One consequence to state before anyone quotes a figure: **the 42-run pipeline benchmark ran under the previous assignment**, recording `synthetic` and `rings` on `global.anthropic.claude-opus-5`. Its latency and cost figures describe that older assignment. The new assignment's end-to-end cost is a projection, labelled as one, until `evals/bench.py --pipeline` is re-run.

## Repository map

```
prompts/verbatim/        byte-identical customer prompts (9) + orchestration docs (9)
prompts/loader.py        sha256-guarded loading; raises PromptIntegrityError on drift
signal_layer/
  registry.py            60 signals: owner domain(s), the customer's verbatim threshol
  schema.py              SignalPayload / DomainView / AlertFinding / render_for_agent
  alert_categories.py    the ten alert-ID categories (166 entries, 148 distinct IDs)
  bands.py               fraud-score and dealer-score bands, PRODUCT decode, credit va
  rules/                 one module per numbered General Rule + catalog.py coverage ta
agents/
  models.py              per-agent model assignment, rationales, recorded usage
  pricing.py             real rate card: global vs us. prefix, cache and batch rates
  mantle.py              GPT-5.6 on bedrock-mantle; the measured latency matrix
  fanout.py              asyncio.gather over 8 fresh Agents, per-agent real usage capt
  contracts.py           Adjudication: exactly 21 required fields, the customer's orde
  output_contract.py     which output rule applies to which agent, derived from the pr
  synthesize.py          structured-output synthesis off the verbatim master prompt
app/underwriter/main.py  AgentCore Runtime entrypoint — batch fan-out + synthesis (SSE
app/francis/main.py      AgentCore Runtime entrypoint — interactive router, 8 agents-a
fixtures/                14 applications with full signal coverage, _context, alerts_f
evals/bench.py           the ONLY code allowed to publish latency or cost
evals/results/           the artifacts every number on this site is quoted from
ui/                      Vite + React + Tailwind + shadcn/ui customer-facing demo
agentcore/               agentcore.json + aws-targets.json for the real CLI
```

**1,238 offline tests** are collected by `MOCK_MODE=1 python3 -m pytest tests/`.

## Still open at the component level

- `app/runtime_support.py::build_signal_payload` reads fixtures. Replacing its body with a Cortex or Aurora read is the whole data-layer migration; nothing above it changes.
- No Gateway or Lambda resource exists yet — `agentcore/aws-targets.json` ships as `[]`.
- `income`, `employment`, `dealer` and `straw` carry **no judged quality evidence** in either study. Open questions.

‹ Executive summary
For the account and executive audience — ...

Measured results ›
The headline benchmark — 42 live adjudica...

# Measured results

The headline benchmark — 42 live adjudications, 336 specialist calls, $6.47 of Bedrock spend. Every figure cited to its artifact.

*For both audiences. Unless a sentence names another file, every figure here comes from `evals/results/pipeline_14x3_us-east-1.json`, with its human-readable sidecar at `evals/results/pipeline_14x3_us-east-1.md`.*

**31.45 s p50, 46.55 s p95, 40 of 42 runs under the 60-second target, $0.1440 p50 per adjudication.** Everything below is the derivation of those four numbers and the caveats attached to each.

# What was run

| Field | Value |
|---|---|
| Schema | `pointpredictive.agentcore.pipeline-bench/1` |
| Mode | `pipeline-live` (`workload.mock = false`) |
| Timestamp | `2026-07-28T07:42:42+00:00` |
| Git | `9bb569bac801884f9c7c5f379520cf3ddbf4a270`, branch `feat/1to1-agentcore-replacement`, `git_dirty: true` |
| Region | `us-east-1` (synthesizer also `us-east-1`) |
| Host | Python 3.12.9, `macOS-26.5.2-arm64`, 11 CPUs |
| Packages | `strands-agents 1.26.0`, `bedrock-agentcore 1.14.0`, `boto3 1.43.54`, `pydantic 2.13.4` |
| Workload | 14 applications x 3 repetitions, 9 model calls per adjudication |
| Execution order | Round-robin: repetition 1 over every application, then repetition 2, and so on. **One application in flight at a time.** |
| Totals | **42 runs**, **336 specialist calls**, 41 complete, 1 degraded, **42 validated adjudications** |
| Measured spend | **$6.46591995** |

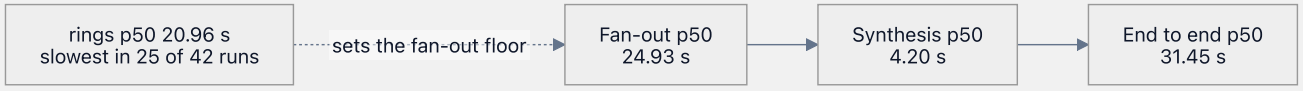
> ⚠ **This benchmark ran under the previous model assignment**
>
> `run.model_ids` records `synthetic` and `rings` on `global.anthropic.claude-opus-5`. Both moved to Sonnet 5 on 2026-07-28 *after* this run, on the strength of the tier re-derivation. Every latency and cost figure below therefore describes the **old** assignment. Its replacement's end-to-end result is a projection and is labelled one; it becomes a measurement only when `python -m evals.bench --pipeline --repetitions 3` is re-run.

# Headline distribution

Percentile method, quoted from the artifact: *"nearest-rank, inclusive (no interpolation); ceil(p*n)-th of the sorted samples."*

| Stage | n | min | p50 | p95 | max | mean |
|---|---|---|---|---|---|---|
| **End to end** | 42 | 18.93 s | **31.45 s** | **46.55 s** | 74.97 s | 32.80 s |
| Fan-out (8 specialists in parallel) | 42 | 15.37 s | 24.93 s | 41.05 s | 70.62 s | 28.31 s |
| Master synthesis | 42 | 3.08 s | 4.20 s | 5.59 s | 15.21 s | 4.49 s |
| Sum of eight (sequential counterfactual) | 42 | 65.52 s | 85.62 s | 157.14 s | 173.00 s | 101.09 s |

Every p99 on this page is a refusal rather than a number. Percentiles below a documented sample count are refused outright — p50 needs 5, p95 needs 20, p99 needs 100 — and all four rows above carry the same recorded string: `"p99": "refused: 42 sample(s), need >= 100. A p99 over 42 sample(s) is not a p99."` That string is in the artifact, and nothing on this site launders it into a figure.




Where the p50 goes: 24.93 s fan-out + 4.20 s synthesis

Fan-out p50 and synthesis p50 do not add to end-to-end p50, and that is expected: each column is an independent percentile over its own 42 samples, so the medians come from different runs.

What the last row is and is not, quoted from the artifact's own caveat: *"sum_of_eight_ms is the SEQUENTIAL COUNTERFACTUAL, the sum of the eight specialists' own measured wall clocks in this run. It is what eight calls cost one after another only if each would take the same time alone; the terms are measured, the arrangement is hypothetical."*

## Against the 60-second target

| | Value |
|---|---|
| Target | 60.0 s ( `run.target_e2e_seconds` ) |
| Runs under target | **40 of 42** |
| Applications under target on **every** repetition | **12 of 14** |
| Applications under target on **at least one** repetition | **14 of 14** |
| Applications **never** under target | **0** |
| Applications straddling the target | 2 |

A count is not an SLA, and the artifact says so: *"A count of runs under a threshold is not a percentile and not an SLA. It is the only form of a 'sub-minute per application' claim this sample size can support."* Both runs over target — APP-1007 rep 2 at 75.0 s and APP-1001 rep 2 at 70.2 s — are explained on Latency tail.

## Parallel speed-up

Fan-out against the sequential counterfactual, one ratio per run, n=42:

| min | p50 | p95 | max | mean |
|---|---|---|---|---|
| 1.85x | **3.64x** | 4.99x | 5.11x | 3.73x |

Reporting a distribution rather than a single number is the artifact's own insistence: *"One speed-up per run, not one number for the port. The spread is the finding: the fan-out is bounded by its slowest dimension, so the ratio moves with which specialist ran long."*

## Cost per adjudication

n=41 fully priced runs; 1 unpriced (the degraded run) is excluded.

| min | p50 | p95 | max | mean |
|---|---|---|---|---|
| $0.1177 | **$0.1440** | $0.2271 | $0.2536 | $0.1562 |

Spread is **$0.1359**, and because rates are fixed per model, token volume is the only driver. Both endpoints are recorded so the driver is identifiable rather than asserted:

| | Cheapest run | Most expensive run |
|---|---|---|
| Application / rep | APP-1001 rep 3 | APP-1004 rep 3 |
| Total | $0.117721175 | $0.253649875 |
| Specialist input tokens | 20,237 | 21,657 |
| **Specialist output tokens** | **4,098** | **13,735** |
| Synthesis input / output | 2 / 640 | 2 / 887 |
| Synthesis cache-write tokens | 3,433 | 6,757 |
| Specialist cost | $0.1087746 | $0.2385026 |
| Synthesis cost | $0.008946575 | $0.015147275 |

Specialist input tokens differ by 7%; specialist **output** tokens differ by **3.35x**. Cost variance on this workload is an output-length story — the same mechanism that drives the latency floor.

## Per specialist

Percentiles run over successful calls only, so `n calls` is the attempts and the percentile `n` is `n calls − n failed`. `Slowest in N runs` counts the runs in which that specialist owned the critical path.

| Specialist | Model | n calls | n failed | p50 | p95 | max | SDK-reported p50 | mean $/call | Slowest in N runs |
|---|---|---|---|---|---|---|---|---|---|
| `identity` | `global.anthropic.claude-sonnet-5` | 42 | 0 | 12.56 s | 33.40 s | 41.04 s | 11.16 s | $0.01727 | 4 |
| `dealer` | `global.anthropic.claude-haiku-4-5-...` | 42 | 0 | 6.56 s | 14.56 s | 18.85 s | 5.46 s | $0.00541 | 0 |
| `straw` | `global.anthropic.claude-haiku-4-5-...` | 42 | 0 | 4.40 s | 8.76 s | 12.52 s | 3.39 s | $0.00352 | 0 |
| `employment` | `global.anthropic.claude-sonnet-5` | 42 | 0 | 7.48 s | 17.64 s | 20.00 s | 6.67 s | $0.01022 | 0 |
| `income` | `global.anthropic.` | 42 | 0 | 8.09 s | 31.44 s | 66.01 s | 7.37 s | $0.01537 | 5 |

| Specialist | Model | n calls | n failed | p50 | p95 | max | SDK-reported p50 | mean $/call | Slowest in N runs |
|---|---|---|---|---|---|---|---|---|---|
| | `claude_sonnet-5` | | | | | | | | |
| `synthetic` | `global.anthropic.claude-opus-5` | 42 | 1 | 8.52 s | 20.82 s | 26.70 s | 7.54 s | $0.02869 | 1 |
| `bustout` | `global.anthropic.claude_sonnet-5` | 42 | 0 | 14.46 s | 35.71 s | **69.78 s** | 13.86 s | $0.01699 | 7 |
| `rings` | `global.anthropic.claude-opus-5` | 42 | 1 | **20.96 s** | 32.25 s | 36.17 s | 20.49 s | $0.04765 | **25** |

Two readings matter here.

- `rings` **owns the critical path in 25 of 42 runs but is not the source of the tail.** The two runs over 60 s were owned by `bustout` (69.78 s) and `income` (66.01 s). Median-setter and tail-producer are different agents. Critical path.
- **Wall clock and SDK-reported latency barely diverge** — p50 0.73 s, p95 1.69 s over n=334 (`evals/results/tail_analysis.json`). So the tail sits inside the model's own reported latency, not in the client.

### Per-specialist token totals

Sums over all 42 calls per specialist, and the mean per call:

| Specialist | Input (total / mean) | Output (total / mean) | Cache read (total / mean) | Cost total |
|---|---|---|---|---|
| `identity` | 130,620 / 3,110.0 | 46,419 / 1,105.2 | 0 / 0 | $0.72543 |
| `dealer` | 147,741 / 3,517.6 | 15,929 / 379.3 | 0 / 0 | $0.227386 |
| `straw` | 100,797 / 2,399.9 | 9,424 / 224.4 | 0 / 0 | $0.147917 |
| `employment` | 99,252 / 2,363.1 | 22,147 / 527.3 | 45,822 / 1,091.0 | $0.4291384 |
| `income` | 133,710 / 3,183.6 | 37,818 / 900.4 | 0 / 0 | $0.6456 |
| `synthetic` | 111,944 / 2,730.3 | 23,840 / 581.5 | 41,041 / 1,001.0 | $1.1762405 |
| `bustout` | 74,997 / 1,785.6 | 55,289 / 1,316.4 | 54,474 / 1,297.0 | $0.7137788 |
| `rings` | 64,295 / 1,568.2 | 64,320 / 1,568.8 | 48,175 / 1,175.0 | $1.9535625 |

`rings` and `bustout` write 1,568.8 and 1,316.4 output tokens per call against `straw`'s 224.4 — the direct consequence of **their prompts carrying no character cap**.

Note the cache-read column: four specialists record non-zero `cacheReadInputTokens` and four record zero. That is **not** evidence that prompt caching works as a cost lever. Prompts sit far below the minimum cacheable prefix for their models and the large per-application payload differs every time, so no entry is ever reused. See **what does not work**.

## Synthesis, in detail

Model `openai.gpt-5.6-luna` on `bedrock-mantle`, 42 calls, **0 failed**.

| | n | min | p50 | p95 | max | mean |
|---|---|---|---|---|---|---|
| Wall clock | 42 | 3.08 s | 4.20 s | 5.59 s | 15.21 s | 4.49 s |
| Adapter-reported latency | 42 | 3.05 s | 3.90 s | 5.55 s | 15.19 s | 4.45 s |

| Token class | Total | Mean per call |
|---|---|---|
| Input | 84 | 2.0 |
| Output | 30,066 | 715.9 |
| Cache read | 0 | 0.0 |
| **Cache write** | **180,610** | **4,300.2** |

Total synthesis cost $0.44686675, mean **$0.010640/call**. `repair_attempts` totalled **0** across all 42 runs, and `runs_needing_a_repair` was 0.

**Two input tokens is why cache-write pricing matters**

The synthesizer reports 2 input tokens and 4,300 cache-**write** tokens per call, because the whole ~31K-character prompt lands in the cache-write class. GPT-5.6 bills cache-write at **1.25x input**. Omitting that term under-reported a measured Luna call by **62%**. See cost levers.

## Per application

Each application ran three repetitions of byte-identical input.

| Application | e2e per repetition (s) | Reps under 60 s | Pinned band | Observed band(s) |
|---|---|---|---|---|
| APP-1001 | 24.3 / 70.2 / 18.9 | 2 / 3 | LOW RISK | LOW RISK |
| APP-1002 | 23.2 / 22.9 / 22.3 | 3 / 3 | LOW RISK | LOW RISK |
| APP-1003 | 31.7 / 34.4 / 34.3 | 3 / 3 | MEDIUM RISK | **LOW RISK (0 / 3 match)** |
| APP-1004 | 37.8 / 36.2 / 39.2 | 3 / 3 | HIGH RISK | HIGH RISK |
| APP-1005 | 46.5 / 42.0 / 43.9 | 3 / 3 | MEDIUM RISK | **HIGH / MEDIUM (2 / 3 match)** |
| APP-1006 | 22.3 / 23.8 / 33.7 | 3 / 3 | LOW RISK | LOW RISK |
| APP-1007 | 34.6 / **75.0** / 23.5 | 2 / 3 | LOW RISK | LOW RISK |
| APP-1008 | 25.5 / 31.4 / 28.5 | 3 / 3 | LOW RISK | LOW RISK |
| APP-1009 | 25.7 / 25.8 / 23.8 | 3 / 3 | LOW RISK | LOW RISK |
| APP-1010 | 28.6 / 28.4 / 28.0 | 3 / 3 | LOW RISK | LOW RISK |
| APP-1011 | 22.4 / 22.8 / 21.2 | 3 / 3 | LOW RISK | LOW RISK |
| APP-1012 | 42.7 / 36.6 / 38.2 | 3 / 3 | HIGH RISK | **HIGH / MEDIUM (1 / 3 match)** |
| APP-1013 | 34.0 / 35.2 / 39.8 | 3 / 3 | MEDIUM RISK | MEDIUM RISK |
| APP-1014 | 24.8 / 41.9 / 31.6 | 3 / 3 | MEDIUM RISK | MEDIUM RISK |

Pinned bands are the fixture expectations from `fixtures/applications/*.json`; observed bands come from `verdict_stability.per_application`.

# Verdict stability

| | Value |
|---|---|
| Applications returning the same `overall_risk_decision` on every repetition | 12 of 14 |
| Unstable applications | APP-1005, APP-1012 |
| Applications disagreeing with the pinned expectation | APP-1003, APP-1005, APP-1012 |
| Applications **never** matching the pinned expectation | APP-1003 |

- **APP-1005** — pinned MEDIUM RISK; observed HIGH RISK and MEDIUM RISK across 3 runs; matched 2 of 3. Its `recommendation_decision` moved with the band, between DECLINE and REVIEW AND APPLY STIPULATIONS.
- **APP-1012** — pinned HIGH RISK; observed HIGH RISK and MEDIUM RISK; matched 1 of 3.
- **APP-1003** — pinned MEDIUM RISK; observed LOW RISK on all three runs, stably. Matched 0 of 3.

Whether the pin or the model is wrong is not adjudicated here, and the artifact says why: *"Whether the pin or the model is wrong is the customer's call; run the calibration judge in `evals/evaluators/` before drawing an accuracy conclusion."* APP-1003 is a self-employed-income-variance fixture pinned at MEDIUM by its author, and three stable LOW readings is as likely to be a pin question as a model question.

A separate and stronger finding — two fixtures that **every model in both families** gets wrong the same way — is on Open questions.

## One degraded run

| Field | Value |
|---|---|
| Run | APP-1011, repetition 3 |
| Failed domains | `synthetic`, `rings` |
| Failure kinds | `synthetic: server_5xx`, `rings: server_5xx` |
| Underlying error (both) | `InternalServerException ... (reached max retries: 2)` on `ConverseStream` |
| Synthesis error | none |
| **Adjudication validated** | **true — all 21 keys** |
| End to end | **21.215 s** |

Three things this single run demonstrates, all of them design intent working:

1. **7-of-8 graceful degradation is real.** Two specialists were lost and the adjudication still validated its full 21-key contract.
2. **A 5xx fails fast.** 21.2 s is among the quickest runs in the whole sample. A server error here costs a *fraud dimension*, not latency.
3. **Degraded runs are reported, never averaged away.** Quoted from the artifact: *"It stays in the latency distribution and leaves the cost distribution."* Hence end-to-end n=42 and cost n=41.

## Throttling: zero

Across all 336 specialist calls, `ModelThrottledException` occurred **0** times. Retry ladder in force was `[1, 2, 4]s` over 4 attempts, and that strategy retries on `ModelThrottledException` and nothing else. Source: `evals/results/tail_analysis.json`.

That matters because it removes the most common explanation for a tail. See **why the tail is irreducible**.

## What this benchmark did not measure

Reproduced from the artifact's own `caveats` array, because the exclusions are as load-bearing as the figures.

- **AgentCore Runtime overhead.** *"These calls go to Bedrock (and to bedrock-mantle for the synthesizer) directly from the host named in run.host, so container cold start, session setup and SSE framing are NOT in the end-to-end figures."*
- **Signal generation.** *"build_payload runs OUTSIDE the timed window because the customer's architecture precomputes signals; an end-to-end figure here is agent latency, not pipeline-including-ETL latency."*
- **Throughput.** *"Applications were run ONE AT A TIME, so nothing here says what happens when N adjudications overlap."*
- **Per-application percentiles.** Computed over 3 repetitions each and *"therefore mostly suppressed by design."*

One process note from the same array: aggregates in the file were recomputed from `raw_runs` by `evals.bench.reaggregate_pipeline_report` at `2026-07-28T07:46:30+00:00`. Measurements are unchanged and no model was called to produce that recomputation.

## Where the earlier single-run figure fits

`README.md` and `CHANGELOG.md` carry an earlier headline: **37.5 s end-to-end at $0.245** on APP-1004, one live run.

**The 42-run benchmark is authoritative**: 31.45 s p50 / 46.55 s p95, $0.1440 p50. The 37.5 s figure is one run on one application and the repo says so in place (*"one measured run on one application, which is enough to show the target is reachable and not enough to call it a benchmark"*). Neither contradicts the other — APP-1004's three benchmarked repetitions

ran 37.8 / 36.2 / 39.2 s, which brackets 37.5 s neatly — but only one of them is a distribution. Quote the p50 with its n.

A recording of that single run is committed at `media/demo.mp4` (16.6 MB, 1920×1080), with `media/demo-poster.png` as its poster frame.

‹ Component detail
Each piece of the port — the eight specialis…

Open questions ›
Five things that need a customer answer — …

# Open questions

Five things that need a customer answer — the volume figure that decides the cost case, the enum conflict, the calibration failures no model fixes, the missing xlsx, and the two prompts with no character cap.

*Mixed audience. Question 1 is for the executive conversation; 2, 3 and 5 are for the customer's engineer; 4 is for both.*

Five items cannot be closed from our side. None is a gap in the port — each is a decision that belongs to the customer, or a document that never arrived. They are ordered by how much rests on the answer.

| # | Question | Blocks | Who decides |
|---|---|---|---|
| 1 | Applications per day through the full eight-agent path | **The entire cost comparison** | Customer, one number |
| 2 | Which artifact defines the authoritative enums | Downstream routing and the output DDL | Customer |
| 3 | May we draft an alert-counting sentence for bustout and rings | Two calibration failures no model fixes | Customer owns the prompt |
| 4 | The three documents never received | Signal set, output DDL, acceptance criteria | Customer, one email |
| 5 | Should the 4,000-character cap be added to two prompts | Fan-out latency floor and 24 invented titles | Customer owns the prompt |

# 1. How many applications per day go through the full eight-agent path?

> **Single most important open question in the engagement**
>
> Without this one number, the cost comparison against the customer's reported **$700K–$1M** Snowflake AI spend is **unresolvable**. With it, the comparison is decided in a sentence.

Measured cost is **$0.1440 per adjudication** (p50, n=41 priced runs, `evals/results/pipeline_14x3_us-east-1.json`). Annualising it requires a volume:

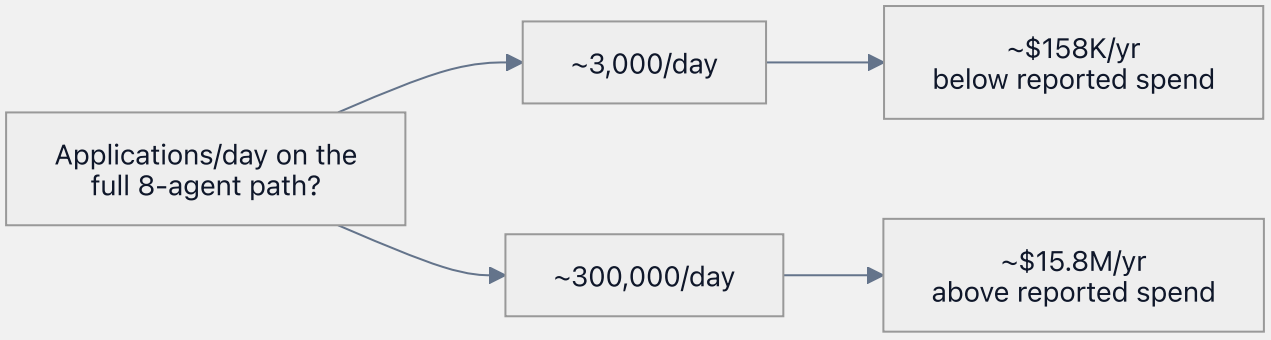
| Applications/day through the full 8-agent forensic path | Annualised at measured p50 |
|---|---|
| ~3,000 | ~$158K |
| ~300,000 | ~$15.8M |

Arithmetic: volume x 365 x $0.14395847. The per-adjudication p50 is measured; the two volumes are illustrative brackets chosen to show the range, **not** the customer's figures. At the low end the port sits comfortably below the incumbent spend; at the high end it is an order of magnitude above it.

**Exact ask.** Not "how many applications do you process" — consortium volume is much larger than forensic-underwriting volume, and mixing them is how this question gets answered wrongly. Ask: *of the applications you ingest, how many per day receive the full eight-agent forensic underwriting treatment, as opposed to a score-only or single-product path?*

Two secondary figures worth asking for at the same time, because they change the arithmetic rather than just the total:

- What fraction of that volume is re-run (repeat submissions, retro backfills)?
- Is the eight-agent path applied to every application, or only above a score threshold?




One input, two opposite conclusions

Full detail, including what the $700K–$1M does and does not cover: Snowflake comparison.

## 2. Which artifact defines the authoritative enums?

Her own two artifacts disagree, and a third vocabulary sits between them. Recorded in `agents/contracts.py:ENUM_CONFLICT`.

| | `docs/master_agent_prompt.txt` — implemented | **Process-guide PDF §5.1–5.2 — not implemented** | **The eight specialist prompts** |
|---|---|---|---|
| Mid band | `MEDIUM RISK` | `POSSIBLE RISK` | `POSSIBLE RISK` |
| Recommendation values | 3: `APPROVE`, `REVIEW AND APPLY STIPULATIONS`, `DECLINE` | 4: `APPROVE`, `APPROVE WITH STIPULATIONS`, `MANUAL REVIEW`, `DECLINE` | n/a — specialists classify, they do not recommend |
| `top_risk_factors`, `*_flag` columns | Required | **Absent** | n/a |
| Status | The string the production agent runs today, so it wins here | Narrative documentation | Their `CALIBRATION` blocks |
| **Verdict** | **Implemented.** Executable artifact beats narrative | Exposed via `PDF_TO_PROMPT_ENUM_MAP`, never silently adopted | Recorded: the specialists agree with the **PDF**, not the master prompt |

So a band rename happens between the analysis layer and the adjudication layer — by design or by drift, and only the customer knows which.

**What is mechanically mappable and what is not.** `PDF_TO_PROMPT_ENUM_MAP` handles `POSSIBLE RISK` → `MEDIUM RISK` and `APPROVE WITH STIPULATIONS` → `REVIEW AND APPLY STIPULATIONS`. `MANUAL REVIEW` **is deliberately absent** and `map_pdf_enum` raises `EnumConflictError` for it, because it is not a rename of any prompt value — it is a fourth outcome that does not exist in the executable artifact. Inventing a mapping there would be a business decision disguised as a default.

Blast radius, quoted from `ENUM_CONFLICT`: *"the enum strings are persisted values in `UNDERWRITING_REPORT_OUTPUT` and are read by downstream workflow routing."* Anything keying off the band string — a queue router, a dashboard, an audit trail — is wrong for one of the two artifacts. A real DDL question, not a documentation tidy-up.

**Exact ask.** *Which of the two is canonical for* `UNDERWRITING_REPORT_OUTPUT`*, and if the PDF is, what does* `MANUAL REVIEW` *map to in the routing that consumes it? Separately: is the specialist-to-master band rename intentional?*

One related item recorded but not enforced: PDF section 5.3 lists a controlled stipulation vocabulary (proof of income, employment verification, proof of residence, driver's licence validation, and so on). It is captured in `contracts.STIPULATION_VOCABULARY` and **not** enforced, because the executable prompt places no constraint on that string. If downstream routing keys off those exact phrases, that is a third thing to settle.

## 3. Calibration failures no model choice fixes

Two fixtures are mis-banded by **all five models across both families**, and every one is wrong the *same way*. That is the signature of a shared prompt defect rather than five independent mistakes, and a bigger model does not buy the fix.

Source: `evals/results/set_comparison.json` (`known_failure_reproduction`) and `evals/results/tier_quality.json`.

| Fixture / agent | Pinned | Every model said | Requirement |
|---|---|---|---|
| APP-1014 / rings | `LOW RISK` | `POSSIBLE RISK` — Claude Opus 5, Claude Sonnet 5, GPT-5.6 Luna, Terra, Sol. **0 of 5 correct.** | `RNG-020` |
| APP-1004 / bustout | `POSSIBLE RISK` | `HIGH RISK` — same five models. **0 of 5 correct.** | `BST-020` |

Reproduction is recorded explicitly in the artifact: *"GPT reproduced it 3/3; Claude 2/2. Correct band: GPT 0/3, Claude 0/2."*

**The fault the judges named in every case is the same:** a second alert firing off the *same underlying behaviour*, treated as independent corroboration.

Sharpest form of the finding, and worth checking against her text rather than taking on trust: both `bustout_agent_prompt.txt` and `rings_agent_prompt.txt` already carry a sentence

making exactly this point — that alert volume alone does not necessarily indicate material fraud risk, because several alerts may share one underlying behaviour or one weak heuristic. Her wording is not reproduced here; see the file.

So the principle **is** stated, in the exact terms of the failure, in both of the prompts that fail — and five models across two families still treat co-originating alerts as independent corroboration. Which means the fix is not "state the principle"; it is to state it as an operational counting rule rather than as guidance. Which sentence achieves that is the thing worth drafting with her.

Six of the eight prompts additionally state that LOW should be the most common outcome. Neither bustout nor rings contains that sentence — worth noting, since over-escalation is exactly what both failures are. The Claude-only tier study reached the same conclusion at three tiers; the cross-family study upgrades it from *"no tier fixes it"* to **"no model choice fixes it."**

**Exact ask.** *May we draft a sentence for the bustout and rings prompts stating that alerts arising from one underlying behaviour count as one signal, for your review?* We can draft it; she owns the artifact, and modifying a verbatim customer prompt is not our call.

**NOT MEASURED:** whether adding that sentence fixes either band. It requires a live run against modified prompt text, which is her artifact. It is the recommendation, not a result.

## 4. Artifacts never received

Three documents would settle several of these questions at once, and none of them arrived.

| Artifact | Why it matters | Current state |
|---|---|---|
| `full_underwriting_signal_dictionary.xlsx` | The authoritative signal contract | Appears only as an **embedded thumbnail on page 3** of the process-guide PDF, with **zero extractable attachments**. The 60-signal registry in `signal_layer/registry.py` is **reconstructed from the nine prompts.** |
| `CLEAN_MAIN_STREAM_FILE.sql` | The output DDL and the real column set | Never received |
| `PP_Forensic_{Hi,M,L}...pdf` (page 10 attachments) | Almost certainly gold-standard example outputs per risk band — i.e. the actual acceptance criteria | Never received |

**What "reconstructed" means for the signal registry.** It derives from the SIGNAL INTERPRETATION sections of the nine prompts. That is a defensible reconstruction — every threshold in it is the customer's verbatim sentence, read out of `prompts/verbatim/` at import rather than retyped — but it is **not** the same as having her contract. Two specific consequences:

- Signals she uses that no prompt names would be **missing**, and we cannot know how many.
- Near-duplicate spellings across prompts are both materialised rather than aliased (`employer_other_ssns_count` vs `employer_distinct_other_ssns_count`, plus two other pairs), because collapsing them would silently degrade one agent. Her dictionary would tell us whether they are genuinely two signals or one signal spelled two ways.

A discrepancy in our own documents, noted so nobody trips on it: `CLAUDE.md` says *"~75-signal registry"* while the module contains **60** entries in `SIGNAL_OWNER` (55 single-owner plus 5 shared). The module is authoritative; the `CLAUDE.md` line is stale.

**Exact ask.** *Can you send* `full_underwriting_signal_dictionary.xlsx`, `CLEAN_MAIN_STREAM_FILE.sql`, *and the three* `PP_Forensic_*` *example outputs?* Between them they settle the signal set, the output DDL, every enum conflict, and the acceptance criteria — four open items for one email.

One more platform-side question in the same family: **is her Cortex Analyst backed by a semantic VIEW or by a stage-based semantic model YAML?** Snowflake's managed MCP server supports views only, so this single answer gates the whole "keep the semantic layer in place" option. See Semantic layer.

## 5. Two prompts carry no character cap

Six of the eight specialist prompts cap output at 4,000 characters. `bustout_agent_prompt.txt` and `rings_agent_prompt.txt` close with a one-line instruction to return only the analysis — no length rule, no title, no emoji ban, no alert-number ban. Their two closing sentences differ from each other by one word, which is why `agents/output_contract.py` derives each agent's rules from its own prompt text instead of matching one string across all eight.

Measured consequence, from `evals/results/tail_analysis.json` over n=334 calls:

| | Domains | n calls | Output tokens p50 | Analyses over 4,000 chars |
|---|---|---|---|---|
| Prompt caps at 4,000 chars | identity, dealer, straw, employment, income, synthetic | 251 | 381 | 9 (3.6%) |
| **No cap in prompt** | **bustout, rings** | 83 | **1,331** | **25 (30.1%)** |

3.5x the output tokens, and since wall clock is linear in output tokens at $r^2$=0.9531, that is the structural owner of the fan-out floor. Those same two agents are also the **only** two ever measured to truncate, and the source of all 24 invented titles.

**Exact ask.** *Should the "max 4000 characters" sentence be added to the bustout and rings prompts?* A one-line change to two verbatim customer artifacts — her decision, not ours.

**NOT MEASURED:** whether adding the sentence actually shortens the output, because this work did not modify her verbatim prompts. It is the recommended fix and it is unproven.

> ⚠ **Do not reach for max_tokens instead**

> That obvious engineering substitute was measured and it **failed all three criteria**: no latency gain, 14 of 30 cells truncated, and only 13 of 30 kept the same band. A truncated call is the worst outcome available — full wall clock, fully billed, text discarded, fraud dimension lost. The measurement.

## Also open, lower stakes

Recorded so nothing is hidden; none of these blocks Friday.

| Item | Status |
|---|---|
| Bedrock quota headroom at high volume | At 300,000 apps/day x 9 calls x ~2,250 tokens the workload approaches 6.1B tokens/day. Sonnet 5's daily ceiling is 8.64B and is **not adjustable** — an account-team escalation, not a self-serve ticket, and it should be raised **before** any load test. Depends entirely on the answer to question 1. |
| Does her Snowflake account enforce network policies? | AWS publishes no stable Gateway egress IPs, so a "keep the semantic layer" path may need PrivateLink. |
| `income` and `employment` have no quality evidence at all | Never swept in either study, within-family or cross-family. `income` deserves it most: its p95/p50 spread (31.44 s / 8.09 s) is the second widest of the eight. |
| `dealer` and `straw` have no quality evidence either | Their LIGHT assignment rests on n=42 latency and cost plus their own prompts' scope, not on a judged comparison. |
| Whether a grounding instruction closes GPT's `_context` gap | The single experiment that would unlock a projected 47.8% cost reduction. Unmeasured. Detail. |
| End-to-end result of the current model assignment | The 42-run benchmark ran under the previous assignment. One command re-measures it: `AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --pipeline --repetitions 3`, roughly $5 of spend. |

# The one-page ask

If only one message goes to the customer before Friday, these five sentences are it.

1. Of the applications you ingest, how many per day get the **full eight-agent forensic underwriting**, as opposed to a score-only path?
2. Is `master_agent_prompt.txt` (`MEDIUM RISK`, three recommendations) or the process-guide PDF (`POSSIBLE RISK`, four including `MANUAL REVIEW`) canonical for `UNDERWRITING_REPORT_OUTPUT`?
3. May we draft the "alerts from one underlying behaviour count as one signal" sentence for bustout and rings? Every model we tested, in both families, gets APP-1014 and APP-1004 wrong the same way without it.
4. Can you send `full_underwriting_signal_dictionary.xlsx`, `CLEAN_MAIN_STREAM_FILE.sql`, and the three `PP_Forensic_*` examples?
5. Should the "max 4000 characters" sentence be added to the bustout and rings prompts, which currently write 3.5x the output of the other six?

‹ Measured results
The headline benchmark — 42 live adjudica...

Prompt fidelity ›
Nine customer prompts as byte-identical co...

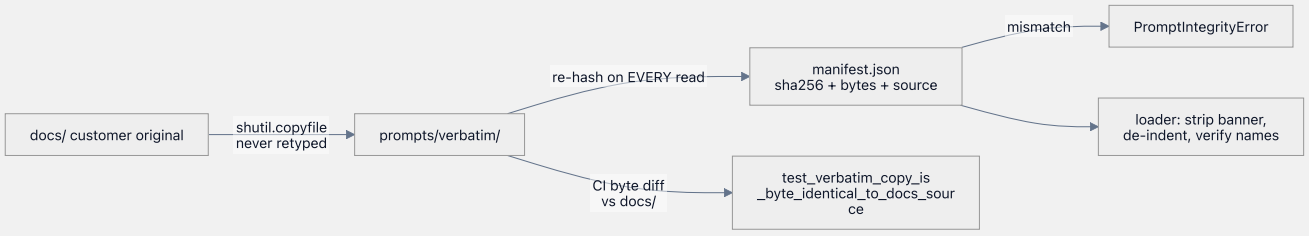
# Prompt fidelity

Nine customer prompts as byte-identical copies under a sha256 manifest, CI-diffed against source — three independent layers, and five mutation attempts all caught including regenerating the manifest.

*For the customer's engineer. This page is the mechanism, not a claim: what is loaded, what is hashed, what fails the build, and what was deliberately broken to prove each guard fires. No prompt text is reproduced anywhere on it.*

**Eighteen customer documents are copied byte for byte, sha256-verified on every read, and separately diffed in CI against her originals.** Paraphrase was made impossible to land rather than discouraged, because the person reviewing this port wrote all nine production agents and a paraphrase silently changes what an agent detects.

## Three layers, and the hole each one closes




Loading and verification chain

| Layer | What it does | What it alone cannot catch |
|---|---|---|
| 1. Byte-for-byte copy | `shutil.copyfile` from `docs/` into `prompts/verbatim/`. No transcription step exists, so a typo has nowhere to enter | Someone editing the copy afterwards |
| 2. sha256 on every read | `prompts/loader.py:read_verbatim` re-reads and re-hashes the file on each call and compares against `prompts/manifest.json`; a mismatch raises `PromptIntegrityError` naming the file, the expected digest and the found digest | Someone editing the copy **and** regenerating the manifest |
| 3. CI diff against her source | `test_verbatim_copy_is_byte_identical_to_docs_source` reads both files as **bytes** and asserts equality, parametrised over all 18 names. `SOURCES` in the loader maps each packaged name to its `docs/` origin | Nothing in this chain. It is what makes the manifest non-bypassable |

The loader deliberately does not cache. Quoting its own reasoning, caching would mean *"a file edited after import keeps serving stale bytes, which is exactly the silent drift the manifest exists to prevent."* Files are small — the largest in the manifest is `identity_orchestration_062026.txt` at 17,929 bytes — and are read a couple of dozen times at import only.

## What is loaded

Eighteen customer documents, all read from disk, none present as a Python string literal anywhere in the repository.

| Class | Count | Files |
|---|---|---|
| Specialist agent prompts | 8 | `<domain>_agent_prompt.txt` for the eight fraud domains |
| Master synthesizer prompt | 1 | `master_agent_prompt.txt` |
| Per-domain orchestration documents | 8 | one per fraud domain |
| Master / Francis orchestration document | 1 | `master_agent_orchestration_062026` |
| **Total under the manifest** | **18** | `prompts/manifest.json` |

Counts and file list come from `prompts/manifest.json` (18 entries) and `tests/test_prompt_fidelity.py::test_all_eighteen_customer_documents_are_packaged`.

A fresh clone cannot import prompts, and that is intended. Every one of those files is marked confidential by the customer, so both her originals under `docs/` and the working copies under `prompts/verbatim/` are **git-ignored**. A clean checkout therefore has `prompts/loader.py`, `prompts/manifest.json` and the tests but no prompt text, and importing `prompts` raises `PromptIntegrityError` naming the first missing file. The alternative — a fallback to bundled paraphrased text — is precisely the defect this package exists to prevent. Restoration instructions are in `prompts/README.md`.

**What this replaced, and why it was dangerous**

The prior revision shipped a module at `agents/prompts.py` — **deleted in commit `9bb569b`, so it is not in the tree you are reading** — which hand-rewrote all nine prompts. Its own docstring conceded the position: *"preserved verbatim in intent."* It also invented an output format the customer never wrote, `ROLE:` / `RISK:` / `FINDINGS:` lines.

What makes that more than a fidelity complaint is the **parser** reading the invented format: when it could not find a `RISK:` line, it **defaulted to `LOW RISK`**. Feed that parser the customer's real prompts — which produce titled prose and never a `RISK:` line — and every application clears. A fraud detection system whose failure mode is "no fraud found" is worse than one that crashes. Source: `CHANGELOG.md`, 2026-07-27 entry.

## Manifest contents

`prompts/manifest.json` carries one entry per file with three fields — `sha256`, `bytes` and `source`. Its own `_comment` states the rule: regenerate **only** when the customer's document changes.

A sample of the recorded provenance, showing that the two document classes come from different places (orchestration documents are RTF exports converted to text under `docs/.converted/`):

| Packaged file | Recorded source | bytes |
|---|---|---|
| `identity_agent_prompt.txt` | `docs/identity_agent_prompt.txt` | 2,412 |
| `master_agent_prompt.txt` | `docs/master_agent_prompt.txt` | 5,067 |
| `bustout_agent_prompt.txt` | `docs/bustout_agent_prompt.txt` | 4,048 |
| `rings_agent_prompt.txt` | `docs/rings_agent_prompt.txt` | 3,758 |
| `identity_orchestration_062026.txt` | `docs/.converted/identity_orchestration_062026.txt` | 17,929 |
| `master_agent_orchestration_062026.txt` | `docs/.converted/master_agent_orchestration_062026.txt` | 2,638 |

Sizes are the manifest's own `bytes` field, which `test_manifest_covers_exactly_the_packaged_files` asserts against `stat().st_size`.

## Five mutation attempts, all caught

A guard never observed to fire is an assumption. Five deliberate mutations were run against these layers and every one was caught. Recorded in `prompts/README.md` (*"Five mutation attempts, including that one, were verified to fail"*) and `CLAUDE.md`.

| Attempt | Caught by | Pinned as |
|---|---|---|
| 1. Edit a packaged prompt in place | Layer 2, the sha256 check at import. Error names the file, the expected hash and the found hash | `test_loader_raises_naming_the_file_on_drift` — which rewrites the persona sentence into `ROLE: Identity Fraud Detection Agent`, mutating the file *toward* the old paraphrased format |
| 2. Edit the prompt **and** regenerate the manifest to cover it | **Layer 3.** Regenerating the manifest makes the hash consistent with the *edit*, not with the customer; the `docs/` byte diff has no dependency on the manifest | `test_verbatim_copy_is_byte_identical_to_docs_source`. The most important of the five, and why the CI diff exists as a separate layer |
| 3. Delete a packaged prompt | `read_verbatim`, which raises before hashing and names both the missing file and the `docs/` path to re-copy from | `test_loader_raises_on_missing_file`. Prevents a silent fallback to substituted text |
| 4. Rename a persona identifier — `ISAAC_IDENTITY` to something plausible | `loader._verify_agent_names()` at import, cross-checking all eight persona ids against the `# <NAME> Agent Prompt` header line inside her own file | `test_loader_rejects_a_persona_id_absent_from_the_customer_text`, monkeypatching in `IVAN_IDENTITY`. These ids are the `RESULT_TABLE.AGENT_NAME` values downstream queries filter on |
| 5. Upper-case Francis to `FRANCIS`, which looks like tidying up | An explicit import-time guard. Her only all-caps spelling appears inside `RETRO_FRANCIS`, the separate retro system Francis must deflect to, so an all-caps id collides with the one name Francis must never answer as | `test_loader_rejects_an_uppercased_francis`, plus a test asserting every `FRANCIS` token in her orchestration document belongs to `RETRO_FRANCIS` |

The header those persona ids come from is stripped before the prompt is used, so `_verify_agent_names()` is the only place the two representations can be reconciled at all.

## What the loader is allowed to change

Exactly two things, both mechanical, both tested.

1. **Strip the extraction banner** — the confidentiality line, the `====` rule lines and the `# <NAME> Agent Prompt` / `# Extracted from ...` comment lines the extraction tool prepended. The match anchors to the top of the file and stops at the first real content line, so an interior `#` is never touched. `test_stripping_removes_only_the_header` asserts that **every non-header line of the source survives, in order, unchanged**.
2. **De-indent the YAML block scalars** in the orchestration documents — remove the longest whitespace prefix common to all non-blank lines, and nothing else.

> **Irregular whitespace in the customer's documents is content, not formatting**
>
> Orchestration documents are RTF exports of Snowflake Cortex Agent YAML, so they are not valid YAML: key indentation is inconsistent (0, 1 and 2 spaces all occur) and block bodies are sometimes less indented than their key. A real YAML parser rejects several of them.
>
> More to the point, `straw_orchestration_instructions_062026` contains doubled and tripled spaces inside its rule text, and `test_straw_doc_keeps_the_customers_doubled_spaces` pins them. Tests flex — matching loosely on whitespace — rather than the prompt being normalised. Filename inconsistency is preserved the same way: her names are variously `..._orchestration_062026`, `..._orchestration_instructions_062026` and `bustout_fraudbot_...`, and the loader carries that table rather than regularising it.

## What is pinned beyond the hashes

A sha256 proves the bytes did not move. It says nothing about whether the load-bearing strings survived the banner strip and reached the model, so 215 tests in `tests/test_prompt_fidelity.py` assert the contract strings individually.

| Pinned | Count | Note |
|---|---|---|
| Persona sentences | 8 | One per specialist, asserted to be **line 1** of the loaded prompt |
| Exact output titles | 6 | The six agents whose prompts define one — see **Output contracts** |
| The two "return only …" rules | 2 | bustout and rings; **both spellings pinned separately** |
| Calibration bands per specialist | 8 | `LOW RISK` / `POSSIBLE RISK` / `HIGH RISK` |
| The 4,000-character cap | 6 | Asserted present on six and **absent on bustout and rings** |
| The `_context` validation rule | 8 | The rule the cross-family study later measured compliance with |
| Master prompt placeholders | 8 | In the customer's input order, under bare uppercase headings |
| Master JSON keys | 21 | Read out of the prompt's own `REQUIRED JSON FORMAT` block |
| Francis's behavioural rules | — | Identity rule, RETRO deflection, the 2-tool budget, the response shape, the routing table |

`tests/test_prompt_fidelity.py` collects **215 tests** (`MOCK_MODE=1 python3 -m pytest tests/test_prompt_fidelity.py -q --collect-only`).

Two details worth showing her directly. **bustout and rings differ from each other by one word** — one says *"the final analysis"*, the other *"final analysis"* — and both spellings are pinned separately (`test_return_only_wording_differs_between_bustout_and_rings`), which is also why `agents/output_contract.py` derives each agent's rules from *its own* prompt text rather than matching one string across all eight. And **a customer-source quirk is pinned rather than fixed**: the `General Rules` heading is absent from the employment document even though the rules themselves are present, so `test_general_rules_heading_is_absent_from_the_employment_doc` records that and nobody later "corrects" the copy.

## Anti-regression canary

Three tests exist so the invented format cannot come back, and one of them exists so the other two are not vacuous.

- `test_invented_marker_regex_actually_matches_the_old_format` feeds the regex a synthetic sample of the *old* `ROLE:` / `RISK:` / `FINDINGS:` format and asserts it matches all three markers. Without it, an over-narrow regex would make the absence assertions below pass on any input.
- `test_no_loaded_prompt_uses_the_invented_format` asserts no loaded prompt contains those markers.
- `test_risk_colon_appears_only_as_a_calibration_band_label` is the subtle one. `RISK:` is legal in her text, but only as `LOW RISK:`, `POSSIBLE RISK:`, `HIGH RISK:`, `MEDIUM RISK:` or `OVERALL RISK:`. Every occurrence is checked for one of those prefixes, so a bare `RISK:` marker anywhere fails.

## Where fidelity ends and interpretation begins

Two honest limits, stated because the engineer will look for them.

**Two derived names are not the customer's strings.** `AGENT_NAMES` holds ten entries; eight are literal customer strings quoted from her file headers, two are not, and the loader records the distinction in place so nobody quotes them back to her as hers. `MASTER_FRAUD_AGENT` is this port's identifier, derived by underscore-joining the words of her master prompt header, and `_verify_agent_names()` pins that derivation rather than trusting it. `Francis` is her exact casing from the master orchestration document, deliberately not upper-cased for the reason in mutation 5.

**The General Rules are extracted from one document of eight.** All 29 numbered rules are read from the **identity** orchestration document, which carries the longest form of them. The other seven differ: rule 6 is absent from some, and rules 11, 13, 17, 18, 20 and 24 are worded differently or truncated per agent. `signal_layer.rules._source.variants_of()` reports the differences and `test_rule_variants_across_documents` pins them, so a re-import surfaces

any change. Source: `docs/general_rules_coverage.md`, "Known gaps". Detail: General Rules coverage.

## How to verify this yourself

```
# with the customer's documents restored under docs/ and prompts/verbatim/
MOCK_MODE=1 python3 -m pytest tests/test_prompt_fidelity.py -q     # 215 tests

# the integrity check on its own, outside pytest
python3 -c "from prompts.loader import verify_integrity; verify_integrity()"
```

`verify_integrity()` walks every manifest entry and raises on the first drift. It is the same code path the package's own import runs.

**Output contracts**
The 21-key adjudication, its enums and cross-field rules, and the eight specialist contracts.

**General Rules coverage**
29 numbered rules: 27 enforced as tested code, 2 by the prompt text.

**Testing**
Every suite, what it guards, and which guards were mutation-tested.

‹ Open questions
Five things that need a customer answer — ...

Output contracts ›
A 21-key adjudication object in the custome...

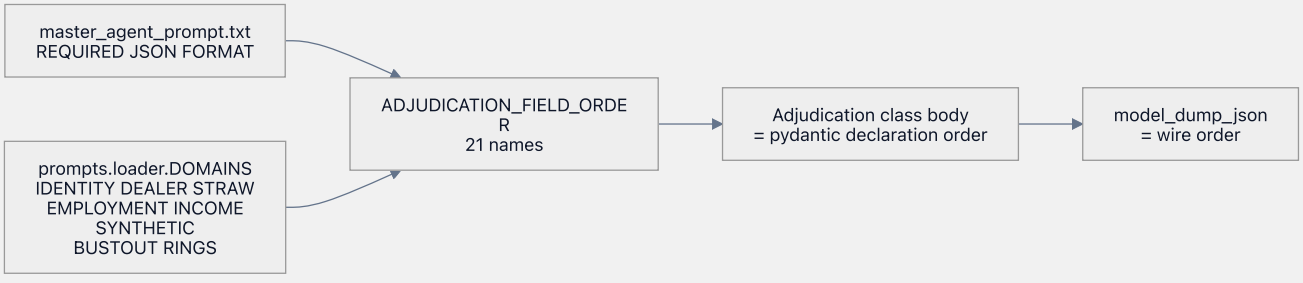
# Output contracts

A 21-key adjudication object in the customer's order with its exact enums and cross-field rules, the PDF's conflicting shape side by side, the six per-agent titles, and the two agents whose prompts define no title at all.

*For the customer's engineer. This is the wire contract: every key, every enum, every length rule, and where each one is asserted in code. Rule ids of the form `MSTR-0xx` are this port's identifiers for lines in `master_agent_prompt.txt`, carried so a review can run line by line.*

**Exactly 21 keys, in her order,** `master_summary` **first, and no** `application_id`. All 42 benchmark runs produced a validated 21-key adjudication with `repair_attempts` totalling **0**, including the one degraded run that lost two specialists (`evals/results/pipeline_14x3_us-east-1.json`).

## The 21 keys




Key order is generated, not typed

`master_agent_prompt.txt` ends in a `REQUIRED JSON FORMAT` block. That block is a wire contract, not a suggestion: downstream, these keys become columns in `SNOWFLAKE_INTELLIGENCE.PUBLIC.UNDERWRITING_REPORT_OUTPUT`. Table read from `agents/contracts.py:ADJUDICATION_FIELD_ORDER`.

| # | Key | Type | Rule |
|---|---|---|---|
| 1 | `master_summary` | string, < 1200 chars | MSTR-011 |
| 2 | `overall_risk_decision` | `LOW RISK` / `MEDIUM RISK` / `HIGH RISK` | MSTR-014 |
| 3 | `recommendation_decision` | `APPROVE` / `REVIEW AND APPLY STIPULATIONS` / `DECLINE` | MSTR-015 |
| 4 | `recommended_stipulations` | string, populated for exactly one outcome | MSTR-036 |
| 5 | `top_risk_factors` | string, band-conditional | MSTR-037 / MSTR-038 |
| 6 | `identity_summary` | string, < 1200 chars | MSTR-011 |
| 7 | `identity_flag` | integer `0` or `1` | MSTR-016 |
| 8 | `dealer_summary` | string, < 1200 chars | |
| 9 | `dealer_flag` | integer `0` or `1` | |
| 10 | `straw_summary` | string, < 1200 chars | |
| 11 | `straw_flag` | integer `0` or `1` | |
| 12 | `employment_summary` | string, < 1200 chars | |
| 13 | `employment_flag` | integer `0` or `1` | |
| 14 | `income_summary` | string, < 1200 chars | |
| 15 | `income_flag` | integer `0` or `1` | |
| 16 | `synthetic_summary` | string, < 1200 chars | |
| 17 | `synthetic_flag` | integer `0` or `1` | |
| 18 | `bustout_summary` | string, < 1200 chars | |

| # | Key | Type | Rule |
|---|---|---|---|
| 19 | `bustout_flag` | integer `0` or `1` | |
| 20 | `rings_summary` | string, < 1200 chars | |
| 21 | `rings_flag` | integer `0` or `1` | |

**No application_id key — a correction, not an omission**

The prior revision emitted **22 keys with `application_id` first**. It is not in the customer's contract. It belongs to the transport envelope carrying the adjudication (`agents.synthesize.SynthesisResult` and the AgentCore entrypoint), so the model sets `extra="forbid"` and an explicit import-time guard raises if `application_id` ever appears in `Adjudication.model_fields`.

Two further import-time guards sit beside it: one asserting the **declared field order** equals `ADJUDICATION_FIELD_ORDER`, one asserting the length is exactly 21. Both raise `AdjudicationContractError` at import rather than emitting a silently wrong report row.

## How the key order is guaranteed

MSTR-013 fixes the order with `master_summary` first. Pydantic emits fields in **declaration order** from both `model_dump()` and `model_dump_json()`, so the class body *is* the wire order.

Keys 6 to 21 are not typed out by hand. They are generated from `prompts.loader.DOMAINS` — the customer's master-prompt input order, `IDENTITY, DEALER, STRAW, EMPLOYMENT, INCOME, SYNTHETIC, BUSTOUT, RINGS`, which is deliberately **not alphabetical**. One definition of that order serves both the prompt substitution and the key order, so the two cannot drift apart.

No field is `Optional`, and that is a structured-output requirement rather than a style choice. The JSON schema generated from this model constrains the model at inference time via Strands' `structured_output_model=` kwarg, and a non-required field drops out of the schema's `required` array — so **one `Optional` would silently make one of the 21 keys omissible.**

## Cross-field rules, all enforced

These are the rules a plain schema cannot express. Each is a validator in `agents/contracts.py:Adjudication` and each raises `AdjudicationContractError` naming the rule.

| Rule | What it enforces |
|---|---|
| MSTR-010 | **No nulls anywhere.** Empty strings instead. Runs `mode="before"`, so a `null` reads as a contract breach rather than a generic type error |
| MSTR-016 | The eight flags are **integers**, not booleans. Also `mode="before"`, because `Literal[0, 1]` would otherwise accept JSON `true` / `false` as 1/0 |
| MSTR-020 | No emojis, in any field |
| MSTR-036 | `recommended_stipulations` is populated **only** for `REVIEW AND APPLY STIPULATIONS`, and must be non-empty for it. Both directions checked |
| MSTR-033 | `REVIEW AND APPLY STIPULATIONS` applies **only** to `MEDIUM RISK` |
| MSTR-037 | `MEDIUM` / `HIGH RISK` carries a numbered list of **3 to 4** factors, each matching `^\d+\.\s+\S` |
| MSTR-038 | `LOW RISK` carries an **empty** `top_risk_factors` |
| MSTR-011 | Every one of the nine summary fields is **strictly under** 1200 characters, so the declarative ceiling is 1199 |

The emoji regex is deliberately narrow — pictograph and dingbat blocks only — so it catches an emoji but never an en dash, ellipsis or arrow, all of which appear in ordinary underwriting prose.

## Prompt shape versus PDF shape

Three vocabularies exist across the customer's artifacts, and this port does not pick quietly. Side by side, with what each artifact is:

| Field | `docs/master_agent_prompt.txt` — **the executable artifact** | `docs/Agent_Underwriting_Process_Guide (1).pdf` **§5.1/5.2 — narrative documentation** | **Implemented** |
|---|---|---|---|
| `overall_risk_decision` | `LOW RISK` / `MEDIUM RISK` / `HIGH RISK` | `LOW RISK` / `POSSIBLE RISK` / `HIGH RISK` | **The prompt.** It is the string the production agent runs today |
| `recommendation_decision` | 3 values: `APPROVE` / `REVIEW AND APPLY STIPULATIONS` / `DECLINE` | **4** values: `APPROVE` / `APPROVE WITH STIPULATIONS` / `MANUAL REVIEW` / `DECLINE` | **The prompt**, with `PDF_TO_PROMPT_ENUM_MAP` for the mechanical renames |
| `top_risk_factors` | Required, band-conditional | **Absent** from the PDF's shape | **The prompt** |
| `*_flag` columns | Eight required `0` / `1` integers | **Absent** from the PDF's shape | **The prompt** |
| Specialist bands | — | `LOW RISK` / `POSSIBLE RISK` / `HIGH RISK` | The eight specialist `CALIBRATION` blocks **agree with the PDF, not with the master prompt** |
| Stipulation wording | Unconstrained string | A controlled vocabulary in §5.3 | Captured as `contracts.STIPULATION_VOCABULARY` and **not** enforced |

A band rename therefore happens between the analysis layer and the adjudication layer, by design or by drift. `PDF_TO_PROMPT_ENUM_MAP` handles the mechanical part — `POSSIBLE RISK` → `MEDIUM RISK`, `APPROVE WITH STIPULATIONS` → `REVIEW AND APPLY STIPULATIONS`. `MANUAL REVIEW` **is deliberately absent** and `map_pdf_enum` raises `EnumConflictError` for it, because it

is not a rename of any prompt value — it is a fourth outcome that does not exist in the executable artifact. Inventing a mapping there would be a business decision disguised as a default.

Section 5.3's stipulation vocabulary (proof of income, employment verification, proof of residence, driver's licence validation, and so on) is recorded and not enforced, because the executable prompt places no constraint on that string. If downstream routing keys off those exact phrases, that is a separate thing to settle. Full ask: Open questions.

## Eight specialist output contracts

Specialist output is **titled prose**, not key/value. This is where the paraphrase-era `RISK:` / `FINDINGS:` format did the most damage — see Prompt fidelity.

Every cell below is derived from the verbatim prompt text at import by `agents/output_contract.py:_derive_rules`, not hard-coded, so a customer prompt update changes the contract automatically rather than leaving a stale table.

| Agent | Exact title | Char cap | No emoji | No alert numbers | No risk labels in headers |
|---|---|---|---|---|---|
| identity | `Identity Fraud Risk Analysis` | 4000 | yes | yes | **yes** |
| dealer | `Dealer Fraud Risk Analysis` | 4000 | yes | yes | **yes** |
| straw | `Straw Borrower Fraud Risk Analysis` | 4000 | yes | yes | **yes** |
| employment | `Employment Fraud Risk Analysis` | 4000 | yes | yes | **yes** |
| income | `Income Fraud Risk Analysis` | 4000 | yes | yes | **no** |
| synthetic | `Synthetic Identity Fraud Risk Analysis` | 4000 | yes | yes | **no** |
| **bustout** | **NONE** | none | no | no | no |
| **rings** | **NONE** | none | no | no | no |

Cross-checked against `evals/results/tier_quality.json:contract_scope`, which records the same scope with the customer's own example prompt line for each rule and was produced by an independent sweep: `no_alert_numbers` 6 agents, `no_emoji` 6, `no_risk_label_in_headers` 4, `exact_title` 6, `max_4000_chars` 6, `low_risk_3_to_5_sentences` 6, `no_coborrower_2_to_3_sentences` 1 (straw only).

bustout and rings define no title, and none is invented for them. Their prompts say only *"Return only the final analysis."* (bustout) and *"Return only final analysis."* (rings) — one word apart — so `agents/fanout.py:ANALYSIS_TITLES` and `agents/output_contract.py:TITLES` both map them to `None`, a test asserts the two tables agree, and `test_bustout_and_rings_are_unconstrained_on_every_rule` asserts that **none** of the five rules applies to either. Every model tested invents a title anyway, which is a measured defect with its own page: Output-contract defects.

# Length rules

| Case | Rule | Applies to |
|---|---|---|
| LOW RISK | 3–5 sentences max; state the clearing conclusion and the key mitigating factor | the six capped agents |
| POSSIBLE / HIGH RISK | detailed analysis proportional to severity, **max 4000 characters** | the six capped agents |
| straw with no co-borrower | 2–3 sentences — straw risk is inherently minimal | straw only |
| bustout, rings | **no length rule of any kind** | bustout, rings |

Those two missing caps have a measured cost. From `evals/results/tail_analysis.json` over n=334 specialist calls:

| | Domains | n calls | Output tokens p50 | Analyses over 4,000 chars |
|---|---|---|---|---|
| Prompt caps at 4,000 chars | identity, dealer, straw, employment, income, synthetic | 251 | **381** | 9 (3.6%) |
| **No cap in prompt** | **bustout, rings** | 83 | **1,331** | **25 (30.1%)** |

3.5x the output tokens. Since wall clock is linear in output tokens at $r^2$=0.9531 (n=332 calls), those two agents are the structural owner of the fan-out floor. Detail: Latency tail.

**A character cap and a max_tokens cap are not the same constraint**

Measured chars-per-output-token medians differ 2.9-fold across agents — `dealer` 4.51, `identity` 1.55, with an observed minimum of 0.33 on one identity call that spent 3,582 output tokens to produce 1,191 characters. So a `max_tokens` value chosen from the 4,000-character contract truncates far earlier than the contract implies. Source: `evals/results/tail_analysis.json`.

The measured attempt to use `max_tokens` as the instrument failed all three of its criteria: what does not work.

## Francis: the interactive-path contract

`master_agent_orchestration_062026` is a **different artifact** from `master_agent_prompt.txt`. The former is the interactive router; the latter is the batch synthesizer. Francis's rules are hers, and each is pinned by a test in `tests/test_prompt_fidelity.py`.

| Rule | Where it comes from |
|---|---|
| Never identify as Claude; always Francis | The prompt's opening lines |
| Deflect RETRO questions to `RETRO_FRANCIS` | Named explicitly in the prompt |
| Call a **maximum of 2 tools per query**, picking the 2 most relevant | The `SPEED OPTIMIZATION` clause |
| Response shaped `## EXECUTIVE SUMMARY` / `## ANALYSIS` / conditional `###RECOMMENDATIONS` | The same document's `response` block |
| No unsolicited loan recommendations | The prompt's closing line, and separately General Rule 12 |

## Where each rule is asserted

| Layer | File | Tests collected |
|---|---|---|
| The 21-key model, enums, cross-field rules | `agents/contracts.py` | `tests/test_contracts.py` — 199 |
| Per-agent output rules, derived from prompt text | `agents/output_contract.py` | `tests/test_output_contract.py` — 25 |
| The contract strings surviving the loader | `prompts/loader.py` | `tests/test_prompt_fidelity.py` — 215 |

Counts from `MOCK_MODE=1 python3 -m pytest tests/<file> -q --collect-only`.

# Still unknown

- **Which artifact defines the authoritative enums.** The prompt is implemented and the PDF's shape is exposed as a map, but the real DDL of `UNDERWRITING_REPORT_OUTPUT` is not in our hands.
- **Whether downstream routing keys off the §5.3 stipulation vocabulary.** Recorded, not enforced.
- **Whether the specialist-to-master band rename is intended.** Specialists say `POSSIBLE RISK`, the master prompt says `MEDIUM RISK`, and the port implements the rename mechanically.


**Output-contract defects**
24 of 24 bustout and rings analyses invented a title, in 15 spellings. GPT invents none.

**Prompt fidelity**
Where these strings come from, and the five mutation attempts that were caught.

**Open questions**
The enum conflict, the artifacts never received, and the calibration failures.

‹ Prompt fidelity
Nine customer prompts as byte-identical co…

General Rules coverage ›
29 numbered General Rules, rule by rule — …

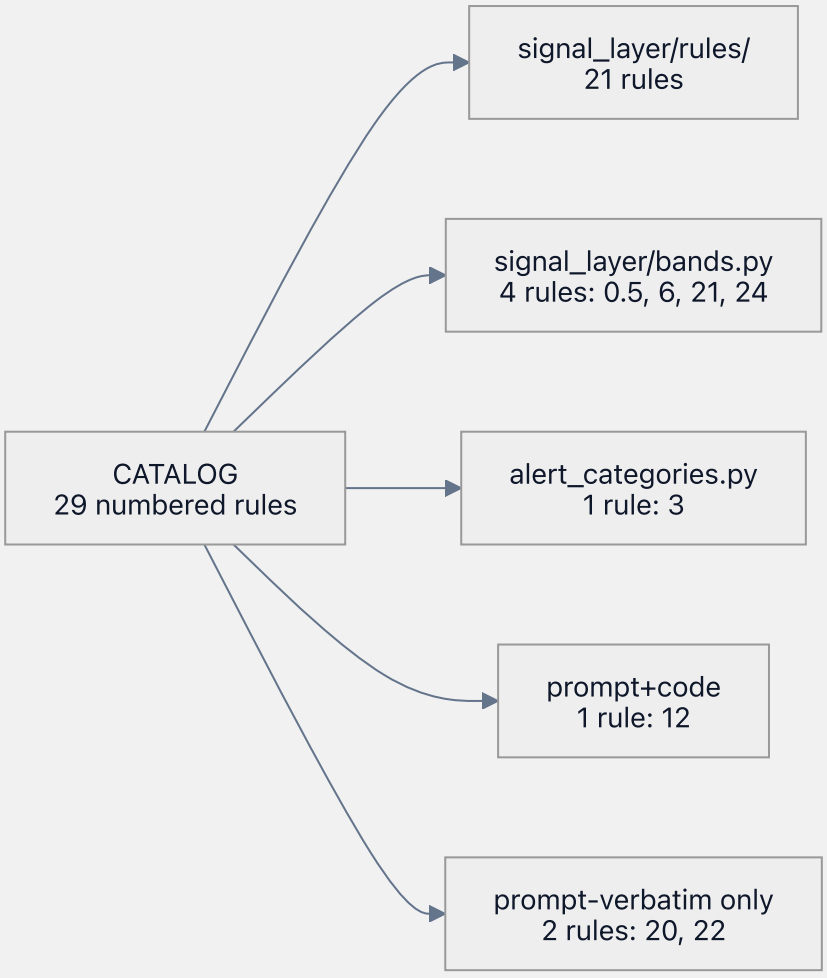
# General Rules coverage

29 numbered General Rules, rule by rule — 27 enforced as tested deterministic code, 2 by the verbatim prompt text, with the enforcement layer and test name for each.

*For the customer's engineer. This page answers, rule by rule, the risk she named herself. Every count is read from* `signal_layer/rules/catalog.py:CATALOG` *at import and matches the generated table in* `docs/general_rules_coverage.md`*.*

**29 numbered rules: 27 enforced by deterministic tested code, 2 by the verbatim prompt text.** Replicating the semantic layer's guardrails was named as the biggest risk in the migration — in her words, *"replicating the SQL guardrails built into snowflake cortex by the semantic layer"* — so the rules became code with one module per rule, not prose in a prompt.

# Where each rule is enforced




Four enforcement layers, 29 rules

| Layer | Count | Rules | Meaning |
|---|---|---|---|
| `signal-layer-code` | 21 | 1, 1.5, 2, 4, 5, 7, 8, 9, 10, 11, 13, 14, 15, 16, 17, 18, 19, 23, 25, 26, 27 | A deterministic function in `signal_layer/rules/` runs it |
| `bands` | 4 | 0.5, 6, 21, 24 | Already implemented in `signal_layer/bands.py`; imported, not duplicated |
| `alert-categories` | 1 | 3 | Already implemented in `signal_layer/alert_categories.py` |
| `prompt+code` | 1 | 12 | Interpretive in the prompt, with a code-side validator as well |
| `prompt-verbatim` | 2 | 20, 22 | Interpretive guidance enforced by the verbatim prompt text; a test asserts the text is present in the loaded prompts |

Summing the first four gives the 27 enforced in code. `tests/test_rules.py` collects **67 tests** (`MOCK_MODE=1 python3 -m pytest tests/test_rules.py -q --collect-only`).

Why exactly two rules are prompt-enforced, and why that is correct rather than a gap. **Rule 20** is score *interpretation* — what an IncomePass score means, and that income overstatement in isolation is not correlated to early payment default while overstatement combined with other misrepresentation is. That belongs in the model's context, not in a function; `signal_layer.bands.INCOMEPASS_SEMANTICS` carries the verbatim semantics and `test_rule_20_incomepass_semantics_in_prompts` asserts the text reached the loaded prompts. **Rule 22** is a **ban on a statistic**, not a computation to perform: do not average fraud scores, because they are standard-scaled. Nothing exists to implement, so what is asserted instead is that nothing in this repository computes `AVG(fraud_score)`, plus that the caveat text is present in the prompts. A function that "enforced" either one would be theatre.

> **Why the rules had to become code**

> The prior revision represented the entire rule set as **six prose display strings**. A rule expressed only as prose in a prompt is enforced by the model's attention budget; a rule expressed as a tested function is enforced by the process.
>
> Two of these rules make the difference concrete. Rule 17 is a null-safe exclusion filter whose naive form silently deletes every legitimate application with no email on file. Rule 23 is an absolute prohibition on raw SSN values.

## Rule by rule

Titles and enforcement layers are read from `signal_layer/rules/catalog.py`. The full table *including her verbatim rule text for every row* is in `docs/general_rules_coverage.md` — that column is read out of the verbatim orchestration document at generation time rather than retyped, so it cannot drift from her document. It is not reproduced here because it is confidential.

| Rule | Title | Layer | Test |
|---|---|---|---|
| 0.5 | Fraud score bands and the meaning of 999 | `bands` | `test_rule_0_5_fraud_score_bands` |
| 1 | INNER join to `delink_lender_table` on `hash_lender_id`, DISTINCT, excluding TEST lenders | `signal-layer-code` | `test_rule_1_lender_join` |
| 1.5 | Lender name normalisation and the 12-row / 14-nickname mapping table | `signal-layer-code` | `test_rule_1_5_lender_nicknames` |
| 2 | `UPPER(TRIM(x))` on borrower strings, and the `123_WALN_92101` address key | `signal-layer-code` | `test_rule_2_normalisation_and_address_key` |
| 3 | The ten alert-ID categories | `alert-categories` | `test_rule_3_alert_categories` |
| 4 | Join `tbl_afa_alerts_clean` on `application_id` AND `hash_lender_id` | `signal-layer-code` | `test_rule_4_alert_join_composite_key` |
| 5 | All application retrieval joins on `application_id` plus a lender identifier | `signal-layer-code` | `test_rule_5_application_grain` |
| 6 | Fraud score bands (restatement of 0.5) | `bands` | `test_rule_6_restates_rule_0_5` |
| 7 | Exclude pure chargeoffs from default calculations | `signal-layer-code` | `test_rule_7_chargeoff_exclusion` |
| 8 | Deprecated all-records tables must not be queried | `signal-layer-code` | `test_rule_8_deprecated_tables` |

| Rule | Title | Layer | Test |
|---|---|---|---|
| 9 | Co-borrower data from `tbl_auto_consortium` first, then all-records | `signal-layer-code` | `test_rule_9_coborrower_source_precedence` |
| 10 | Only active dealers (`consortium_number_of_apps_last_1_year > 0`) | `signal-layer-code` | `test_rule_10_active_dealers` |
| 11 | `submitted_to_prod_at` only for "submitted"; never `<= CURRENT_TIMESTAMP()` | `signal-layer-code` | `test_rule_11_date_field_routing` |
| 12 | No unsolicited recommendations; never auto-decline; never criminal penalty | `prompt+code` | `test_rule_12_recommendation_constraints` |
| 13 | Retro: never filter by default, null-safe predicate, no `submitted_to_prod_at` | `signal-layer-code` | `test_rule_13_retro_handling` |
| 14 | `ppi_dealer_id` + `dealer_name` together identify a dealer | `signal-layer-code` | `test_rule_14_dealer_identity_key` |
| 15 | NULL `borr_ssn` is not a distinct SSN | `signal-layer-code` | `test_rule_15_null_ssn_not_distinct` |
| 16 | `PRIME` and `SUBPRIME` are both Stellantis lenders | `signal-layer-code` | `test_rule_16_prime_subprime_stellantis` |
| 17 | Null-safe fake-record exclusion (fake emails, common VINs, common phones) | `signal-layer-code` | `test_rule_17_fake_record_exclusion` |
| 18 | Check `TBL_AUTO_CONSORTIUM` when all-records returns nothing | `signal-layer-code` | `test_rule_18_source_fallback` |

| Rule | Title | Layer | Test |
|---|---|---|---|
| 19 | Crescent Bank is `CRESCENTBANK`, never `FCRACRESCENTBANK` | `signal-layer-code` | `test_rule_19_crescent_bank` |
| 20 | IncomePass ( `income_misrep_score` ) is distinct from fraud score | `prompt-verbatim` | `test_rule_20_incomepass_semantics_in_prompts` |
| 21 | `dealer_risk_score` (lender-scoped) vs `dealer_consortium_score` (consortium-scoped) | `bands` | `test_rule_21_dealer_score_binding` |
| 22 | Do not average fraud scores — they are standard-scaled | `prompt-verbatim` | `test_rule_22_no_fraud_score_averaging` |
| 23 | NEVER include raw SSN values or any parts of them | `signal-layer-code` | `test_rule_23_no_raw_ssn` |
| 24 | `PRODUCT` concatenates PPI acronyms; match with wildcards on both sides | `bands` | `test_rule_24_product_decode` |
| 25 | Ignore invalid credit data: time in file ≤ 0 or 9999, oldest tradeline null/zero | `signal-layer-code` | `test_rule_25_invalid_credit_data` |
| 26 | 11-digit and 10-digit forms of a phone are the same phone | `signal-layer-code` | `test_rule_26_phone_equivalence` |
| 27 | Max 5 SQL calls per request; batches > 5 applications summarise incrementally | `signal-layer-code` | `test_rule_27_query_budget` |

## Rules where the naive reading is a real defect

Twenty-nine rules do not carry equal weight. These are the ones where implementing the obvious version produces a wrong answer, with the failure each one causes. Notes taken

from `docs/general_rules_coverage.md`.

| Rule | Naive implementation | What it produces | Implemented by |
|---|---|---|---|
| 17 | `<> ALL (...)` without the `or ... is null` disjunct she explicitly writes | **Silently deletes every legitimate application with no email on file** — in SQL, `NULL <> ALL (...)` is NULL, not true. Highest-risk rule in the set | `exclude_fake_records`, `phone_excluded`, `validate_exclusion_sql` in `signal_layer/rules/r17_fake_records.py` |
| 15 | Count NULL `borr_ssn` as a distinct SSN | Inflates every reuse count by one and manufactures a shared-identifier cluster out of a missing value — a **false positive** against the anti-false-positive discipline the whole prompt set is built around | `count_distinct_ssns`, `is_present_ssn` |
| 26 | Compare phone strings without normalising to the last 10 digits | The opposite failure, a **false negative**: `14724933572` and `4724933572` are the same phone, so reuse counts undercount and a real shared-phone ring is missed. Also required upstream of rule 17's phone exclusion, so getting it wrong corrupts two rules | `norm_phone`, `same_phone` |
| 7 | Blanket `AND chargeoff_flag = false` | **Under-counts defaults.** The rule excludes *pure* chargeoffs — the flag set with **neither** EPD flag. The full chargeoff × (epd, epd6) truth table is tested | `is_default`, `is_pure_chargeoff`, `default_rate` |
| 9 | Two-valued co-borrower resolution | Turns a data gap into a finding. PRESENT / ABSENT / UNKNOWN must stay three-valued because the identity agent escalates on "no co-borrower" but not on "not sourced" | `resolve_coborrower` |
| 10 | Treat NULL activity as active | The customer does not state the NULL branch. `NULL` is not `> 0`, so unknown activity is excluded, and | — |

| Rule | Naive implementation | What it produces | Implemented by |
|---|---|---|---|
| | | the decision is recorded as `ACTIVE_DEALER_NULL_NOTE` rather than left implicit | |
| 19 | `%CRESCENT%` wildcard | **Double-counts two institutions.** Rule 1.5 itself supplies `CRESCENT` as a nickname, which matches both Crescent Bank and `FCRACRESCENTBANK`. Rules 1.5, 16 and 19 are resolved together by one lender resolver for that reason | one lender resolver; rule 16 is the mirror case — Stellantis resolves to **two** names, so `LenderResolution.lender_names` is a tuple and `.lender_name` raises on a multi-name result |
| 27 | Trust a prompt instruction to "limit to 5" | A prompt instruction is a request. `QueryBudget` is a ceiling: the budget is spent **before** a call is issued, so the sixth call cannot be made. Shared with rules 13 and 18, since a rule-18 retry costs a rule-27 call | `QueryBudget` |

Rules 0.5 and 6 are the same rule under two ids, and both appear deliberately. Rule 6 is byte-identical to rule 0.5 in the identity document, and one implementation serves both. Quoting the coverage doc: *"deduplicating in code is correct, deduplicating in the inventory is not"* — so both ids are listed, and the count of 29 is the customer's count rather than a tidied one.

## Rule 23 and the one hard compliance boundary

Rule 23 is the only rule she **capitalises**: never include raw SSN values or any parts of them, only hashed values. It is enforced as a **recursive gate** over the payload — `assert_no_raw_ssn` / `find_raw_ssn` in `signal_layer/rules/r23_no_raw_ssn.py` — rather than as a prompt instruction, because a prompt instruction is not a compliance control.

> ✗ **Known gap: rules 12 and 23 both want a Bedrock Guardrail, and none is configured**

> Quoting `docs/general_rules_coverage.md` directly: *"No `guardrailIdentifier` or `apply_guardrail` call exists anywhere in it."*
>
> Rule 23 is enforced by the recursive payload gate and rule 12 by a post-generation validator. Both are real and both are tested. Neither is the platform-level block the customer's compliance posture should ultimately have, **because a validator in the same process can be bypassed by a code path that forgets to call it.**
>
> Recorded as a gap in `r23_no_raw_ssn.ENFORCEMENT_LAYERS` and in the coverage document, and `bedrock-guardrail` is deliberately never used as the *sole* layer for any rule. It is a platform configuration item for the migration, not a code change.

## What "enforced in code" does and does not mean

Three honest boundaries, stated because a review will look for them.

1. **Rules are extracted from one document of eight.** The identity orchestration document carries the longest form of all 29. The other seven differ — rule 6 is absent from some, and rules 11, 13, 17, 18, 20 and 24 are worded differently or truncated per agent. `signal_layer.rules._source.variants_of()` reports the differences and `test_rule_variants_across_documents` pins them.
2. **Several rules govern SQL this repository does not execute.** Rules 1, 4, 5, 8, 11 and 13 are about query construction. What exists for them is a **resolver plus a SQL validator** — `validate_lender_join_sql`, `validate_join_sql`, `validate_date_sql`, `validate_retro_sql`, `validate_no_deprecated_tables` — so each rule is checkable against whatever generates the query, whether that is this signal layer or the customer's existing pipeline. That is a genuine difference from "the rule runs in production", and it should be said in the room.
3. **Rule 1.5 does not guess.** The rule's own words are *"don't hallucinate."* An unresolved lender input returns `resolved=False` rather than a best-effort match.

## Regenerating the coverage document

```
python -m signal_layer.rules.catalog > docs/general_rules_coverage.md
MOCK_MODE=1 python3 -m pytest tests/test_rules.py -q          # 67 tests
```

The second command asserts the generated file matches the catalog, so the document cannot be edited into agreement with a claim the code does not support.

## Still unknown

- **Whether the per-agent rule variants matter.** Seven documents differ from the identity document in six rules, and which wording the customer considers authoritative is her call.
- **Whether a Bedrock Guardrail is wanted.** Rules 12 and 23 both point at one; none is configured.
- **Whether the SQL validators match her production query generator.** They are checkable against it; they have not been run against it.


**Alert categories**
General Rule 3 as data: ten categories, 166 entries, 148 distinct IDs, 18 in more than one.

**Signal layer**
The 60 signals, the DomainView projection, and why a specialist has no data tool.

**Semantic layer**
What replaces Cortex Analyst, and the Lambda Gateway design behind SIGNAL_MODE.

‹ Output contracts
A 21-key adjudication object in the custome...

Alert categories ›
General Rule 3 as data rather than prose — ...

# Alert categories

General Rule 3 as data rather than prose — ten categories, 166 list entries, 148 distinct IDs, 18 in more than one category, and the 31-39 disregard rule that belongs to none of them.

*For the customer's engineer. Every count here is read from* `signal_layer/alert_categories.py` *at import. The alert IDs themselves are her numbers from General Rule 3 and are shown, because the routing they drive is the thing being verified; her surrounding prose is not reproduced.*

**Ten categories, 166 list entries, 148 distinct IDs, 18 of them in more than one category.** The gap between 166 and 148 is the whole finding: **the categories are not a partition**, so `category_of(alert_id)` returns a list and the reverse index is many-to-many by construction.

## Why category routing decides what a specialist can see

Alerts reach a specialist only through category filtering

A separate rule makes that filtering load-bearing: **all eight specialists are forbidden from citing alert numbers** — *"Reference specific findings from alerts, not alert numbers."* An agent that cannot name an alert number cannot reason its way to the right subset, so the only way a specialist ever sees the correct alerts is for the signal layer to filter them **by category before the payload is built**.

Quoting the module's own docstring on what a transcription error costs: *"A single wrong digit here silently removes a detection or attributes it to the wrong specialist."* No error, no warning — just a fraud dimension the specialist was never shown. The prior revision of this repository did not carry these IDs at all; the whole rule set existed as six prose display strings.

## Counts

Read from `signal_layer/alert_categories.py` at import:

| | Value |
|---|---|
| Categories | 10 |
| Total list entries across all ten categories | 166 |
| Distinct alert IDs | 148 |
| IDs appearing in more than one category | 18 |
| Lowest / highest ID named | 3 / 169 |

## The ten categories

Labels are her own headings with the trailing "Alerts"/"Alert" removed; her exact source headings are preserved in `SOURCE_HEADINGS`.

| Category | Source heading | n IDs |
|---|---|---|
| Identity | `Identity-related Alerts` | 60 |
| Income | `Income Alerts` | 12 |
| Employment | `Employment Alerts` | 21 |
| Employment Deviation | `Employment Deviation Alerts` | 10 |
| Straw Borrower | `Straw Borrower Alerts` | 10 |
| Vehicle Risk | `Vehicle Risk Alerts` | 3 |
| Default | `Default Alerts` | 33 |
| Dealer Risk | `Dealer Risk Alerts` | 9 |
| Collateral | `Collateral Alert` | 6 |
| Bust Out Risk | `Bust Out Risk Alert` | 2 |

ID order inside each tuple is the customer's, not sorted. Her Employment list is deliberately not ascending: it names `76, 78, 116, 117, 132, 140, 142, 143, 144, 148, 156` first and then the ten Employment Deviation IDs `68, 69, 70, 71, 73, 74, 75, 77, 79, 114`. The tuple preserves that because the ordering carries her grouping intent, while membership tests go through `CATEGORY_ALERTS`, which is order-free.

## The 18 multi-category IDs

`MULTI_CATEGORY_ALERTS` is **derived** from the transcribed sets rather than retyped, and a test asserts the module docstring and the data agree, so this list cannot drift.

| Alert ID | Categories |
|---:|---|
| 40 | Income + Default |
| 42 | Income + Default |
| 68, 69, 70, 71, 73, 74, 75, 77, 79 | Employment + Employment Deviation |
| 89 | Income + Default |
| 97 | Vehicle Risk + Collateral |
| 113 | Income + Default |
| 114 | Employment + Employment Deviation |
| 116 | Employment + Default |
| 117 | Employment + Default |
| 164 | Income + Bust Out Risk |

Eighteen IDs in total, ten of them the Employment / Employment Deviation overlap.

**A single-label mapping loses a category for 18 IDs**

Quoting `docs/general_rules_coverage.md` on rule 3: *"a single-label mapping would silently lose a category for 18 ids."*

Concretely: build a `dict[int, str]` from these lists and alert **164** stops being a Bust Out Risk alert, so the bustout specialist never sees it. Alert **97** stops being a Collateral alert. Alerts 116 and 117 stop being Default alerts. That is why `category_of(alert_id)` returns a list.

## The 31-39 disregard rule

Immediately after the Income list, General Rule 3 carves out the range **31–39**: she rates those nine ids as weak fraud indicators and tells the agent it can set them aside for the most part. Her exact sentence is held in code as `DISREGARD_RULE` — extracted from the orchestration document rather than retyped — with `DISREGARDED_ALERTS = frozenset(range(31, 40))`.

None of 31–39 appears in any of the ten categories, and that is verified: `DISREGARDED_ALERTS & ALL_CATEGORISED_ALERTS` is **empty**. So this is not an instruction to strip nine IDs out of a category; it is an instruction about **uncategorised** alerts arriving in `alerts_fired`. The implementation consequence, recorded in the module, is that they **must not be dropped from the record** but **must not drive risk** — deleting them would be a different rule from the one she wrote, since hers says "for the most part", which is interpretive, and a hard delete is not.

`category_of()` returns `[]` for any ID she never categorised. In the 1–169 range that is **21 IDs**: the nine covered by the disregard rule, plus **12 others** — 1, 2, 108, 109, 118, 121, 122, 124, 125, 126, 127, 149. Those twelve are uncategorised in her rule and are carried on the unfiltered record rather than guessed into a category.

## Which specialist sees which categories

Only **five** of the eight fraud domains have a same-named category in General Rule 3. For the other three the mapping is derived, and `PROVENANCE` records for every domain whether the mapping is **DOC-QUOTED** from a customer document or **INFERRED** — so she can correct the inferred ones without archaeology.

| Domain | Categories | Extra IDs from its own doc | Total IDs visible | Provenance | Reasoning offered |
|---|---|---|---|---|---|
| identity | Identity | 4 | 60 | DOC-QUOTED | — |
| dealer | Dealer Risk | 0 | 9 | DOC-QUOTED | — |
| straw | Straw Borrower | 0 | 10 | DOC-QUOTED | — |
| employment | Employment + Employment Deviation | 12 | 23 | DOC-QUOTED | — |
| income | Income | 0 | 12 | DOC-QUOTED | — |
| synthetic | Identity | 18 | 62 | INFERRED | Her synthetic document names its own supplemental list of 18 IDs, and **16 of those 18 belong to the Identity-related set** (the other two, 138 and 163, are Default). Identity is the closest customer-defined container |
| bustout | Bust Out Risk + Default | 0 | 35 | INFERRED for Default | She defines bust-out as coordinated rapid credit acquisition followed by non-repayment and instructs the agent to look at EPD / EPD6 / FPD defaults. `Default` is the only place those live |
| rings | Identity | 4 | 60 | INFERRED | Her rings document names fraud-ring alerts 64, 65, 66 |

| Domain | Categories | Extra IDs from its own doc | Total IDs visible | Provenance | Reasoning offered |
|---|---|---|---|---|---|
| | | | | | and 67, and **all four belong to the Identity-related set** |

Totals are `len(alerts_for_domain(domain))`, computed from the module. Extra IDs come from `DOMAIN_EXTRA_ALERTS` — supplemental IDs named directly in that domain's own orchestration document, outside General Rule 3, each carried alongside the verbatim quote that named it in `DOMAIN_EXTRA_ALERT_QUOTES` so the numbers are auditable. Each inferred row is tagged `INFERRED` in `PROVENANCE` with an explicit "Confirm with the customer", and those three rows are the ones to put in front of her.

Two categories are deliberately assigned to no domain. `Vehicle Risk` and `Collateral` sit in `UNASSIGNED_CATEGORIES` because no customer document routes them to any of the eight specialists, so they are not injected into a specialist's view on a guess. They stay reachable through `CATEGORY_ALERTS` and are carried on the unfiltered record. The alternative — attaching them to whichever specialist seemed closest — would put nine alert IDs in front of an agent whose prompt has no interpretation for them, which is the same class of error as handing a specialist a signal its prompt never mentions.

## Provenance of the transcription

IDs were transcribed character by character from `docs/.converted/identity_orchestration_062026.txt` lines 72–93, and the identical lists were cross-checked against **the other seven** converted orchestration documents — bustout, dealer, employment, income, rings, straw, synthetic. All eight agree exactly, **including ID order**. The master orchestration document contains no General Rules at all.

`alerts_for_domain()` raises `KeyError` for an unknown domain rather than returning an empty set, because a typo'd domain name must not silently produce a specialist with zero

alerts.

## Verifying it

```
MOCK_MODE=1 python3 -m pytest tests/test_rules.py -q -k alert_categories
```

`test_rule_3_alert_categories` is the coverage entry for this module in `docs/general_rules_coverage.md`, and separate tests assert that `MULTI_CATEGORY_ALERTS` agrees with the docstring list and that the 31–39 set is disjoint from every category.

## Still unknown

- **The three inferred mappings.** synthetic → Identity, rings → Identity, and bustout → Default are reasoned, not quoted. Confirmation is a one-line answer from her.
- **Where `Vehicle Risk` and `Collateral` belong.** Nine IDs across two categories are routed to no specialist because no document routes them.
- **Whether the 12 uncategorised IDs outside 31–39 are intentional.** They are carried unfiltered rather than guessed.


**General Rules coverage**
All 29 rules, their enforcement layer, and the ones whose naive reading is a defect.

**Signal layer**
How a DomainView is built, and what each specialist can and cannot see.

**Fixtures**
The 14 applications, including the anti-false-positive cases.

‹ General Rules coverage
29 numbered General Rules, rule by rule — ...

Output-contract defects ›
All 24 bustout and rings analyses invented ...

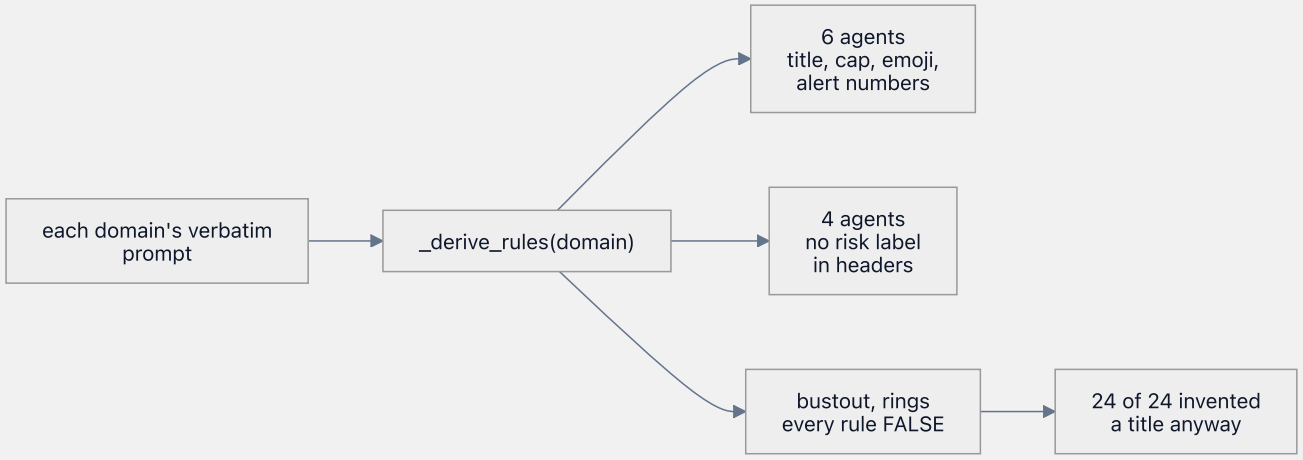
# Output-contract defects

All 24 bustout and rings analyses invented a title their prompt never defines, in 15 spellings, across every Claude tier — GPT invents none — plus the wrong assumption that deriving rules from prompt text corrected.

*For the customer's engineer. This page reports a divergence between what two of her prompts ask for and what models actually do, and it deliberately does not repair the prose. Rewriting would hide the divergence, and the divergence is a decision for the prompt's author.*

**24 of 24 bustout and rings analyses opened with a title their prompt never defines, in 15 distinct spellings, on all three Claude tiers.** GPT-5.6 invents none, on 36 of 36 analyses. Sources: `evals/results/tier_quality.json:per_analysis` (48 analyses, $4.9088 spend) and `evals/results/set_comparison.json:per_analysis` (120 analyses, $16.8869 spend).

## What the two prompts specify




Five rules derived per agent from that agent's own prompt text

`bustout_agent_prompt.txt` and `rings_agent_prompt.txt` define **no title**. They end with *"Return only the final analysis."* and *"Return only final analysis."* respectively — one word apart — and that is the entire output specification. No title, no character cap, no emoji ban, no ban on citing alert numbers, no ban on risk labels in headers. The other six prompts define all five.

| | Value | Source |
|---|---|---|
| bustout + rings analyses in the tier study | 24 | `evals/results/tier_quality.json:per_analysis` |
| That invented a title | **24 (100%)** | same, `contract_findings` |
| Distinct title strings observed | **15** | same, derived from the finding details |
| Distinct after normalising case, markdown level and suffix | **6** | same |
| Models this held across | **3 of 3** — Haiku 4.5, Sonnet 5, Opus 5 | `run.candidate_models` |

Per-cell split: rings 15 analyses (5 per model), bustout 9 (3 per model). Zero clean.

## The 15 spellings

Every string below is the real opening line of a measured analysis, extracted from the `invented_title` findings in `evals/results/tier_quality.json`. None is an example.

| Count | Opening line |
|---|---|
| 4 | `# Fraud Ring Risk Analysis` |
| 3 | `# BUSTOUT FRAUD RISK ANALYSIS` |
| 2 | `# FRAUD RING DETECTION ANALYSIS` |
| 2 | `# Bustout Fraud Risk Analysis` |
| 2 | `# BUST-OUT FRAUD RISK ANALYSIS` |
| 2 | `# Fraud Ring Detection Analysis` |
| 1 | `# Bust-Out Fraud Risk Analysis` |
| 1 | `# RINGS FRAUD RISK ANALYSIS` |
| 1 | `# FRAUD RINGS RISK ANALYSIS` |
| 1 | `## Fraud Ring Risk Analysis: HIGH RISK` |
| 1 | `## Fraud Ring Risk Analysis — POSSIBLE RISK` |
| 1 | `## Bustout Fraud Risk Analysis — Nadia Kowalski` |
| 1 | `# Fraud Ring Risk Analysis — Application (Devon Cross / hash_ssn_ff02aa)` |
| 1 | `# Fraud Ring Risk Analysis — Application (Callum Reyes / hash_ssn_9e4a26)` |
| 1 | `## Fraud Ring Risk Analysis — Application (hash_ssn_5c8b17)` |

Eleven of the fifteen are generic; **four embed application-specific data** — a borrower name, a hashed SSN, or both.

Spread matters more than the count. A consistent invented title would be cosmetic: strip one string in the display layer and move on. Fifteen strings, at two markdown levels (`#` on 20 of 24, `##` on 4), with inconsistent hyphenation of "Bust-Out" and three different nouns for the same domain ("Fraud Ring", "Fraud Rings", "Rings"), is not something a fixed-string filter removes. It is also a **contract divergence in the persisted output**: whatever consumes

`rings_summary` downstream is receiving a heading its schema never specified, in a shape that changes between runs.

Two of the fifteen carry a **risk label**, three embed a **hashed SSN**, and one a borrower's full name — and none of that is a rule violation for these two agents, which is the awkward part. `agents/output_contract.py` reports the risk-label rule only for the four agents whose prompts state it, so a risk label in a rings heading violates nothing she wrote. What it is, is a divergence from the intent of the other four prompts applied to an agent whose prompt happens to be silent. Reporting it as a violation would fabricate a rule; not mentioning it would hide a real inconsistency. So the module records the identifier-in-heading case explicitly as *"(observation, not a stated prompt rule)"* and notes that it is **not** a Rule 23 breach — the SSN value is hashed, never raw — but that it does put an identifier in a title.

> **Where the 'eleven spellings' figure in the code comes from**
>
> `agents/output_contract.py`'s docstring says *"eleven different spellings"*. That is the count of the **generic** strings, i.e. the fifteen distinct opening lines minus the four varying only because they embed an application's own name or hashed SSN. Both numbers are correct readings of the same 24 analyses: **15 is the number of distinct opening lines**, 11 the number of distinct title *forms*. If a slide needs one number, use 15 and say what it counts.

## GPT invents none

The cross-family study swept the same two agents across five models and re-ran the same contract checker. Source: `evals/results/set_comparison.json:per_analysis`, 120 analyses of which 60 are bustout or rings.

| Model | Set | bustout+rings analyses | Invented a title | Verdict |
|---|---|---|---|---|
| Claude Opus 5 | claude | 12 | 12 | Every analysis |
| Claude Sonnet 5 | claude | 12 | 11 | All but one |
| GPT-5.6 Luna | gpt | 12 | 0 | Clean |
| GPT-5.6 Terra | gpt | 12 | 0 | Clean |
| GPT-5.6 Sol | gpt | 12 | 0 | Clean |

Across all four swept domains and all 120 analyses, output-contract cleanliness was **Claude 17 of 48, GPT 72 of 72.** All 23 invented titles in that study belong to the two Claude models.

> **Model-sensitive, and the fix is still the prompt**
>
> This is one of the few findings in the engagement a model swap genuinely does fix. It is still not a reason to swap, and the reason is on the cross-family study page: the specialists are held on Claude on `_context` grounding — GPT 0.357 against Claude 0.642, lower on 4 of 4 agents, deterministic, no judge involved.
>
> So the recommended fix remains a sentence in the customer's two prompts, not a family change. It is the counterpart to the missing character cap, and both are hers to decide.

## Deriving the rules corrected a wrong assumption

Most useful thing this sweep produced was not the title count. It was catching a wrong assumption in the checker itself.

An earlier revision of `agents/output_contract.py` declared the rule set as a hard-coded table and assumed the risk-label rule applied to the same **six** agents as the title, the character cap, the emoji ban and the alert-number ban. It does not. Reading the prompt text found that sentence in **four** — identity, dealer, straw, employment — while `income` and `synthetic` never received it. Confirmed independently from a second artifact: `evals/results/tier_quality.json:contract_scope` records `no_risk_label_in_headers` with `count: 4` and lists exactly those four, while every other shared rule carries `count: 6`.

Enforcing the six-agent contract on all eight would have reported violations **the customer never asked for** — two agents flagged for something their prompts permit. On a fidelity review that is the same class of error as missing a rule: it means the checker is not reading her document.

The fix was structural rather than a corrected constant. `_derive_rules(domain)` reads each rule out of that domain's own verbatim prompt text at import:

| Rule | Derived by looking for | Agents it applies to |
|---|---|---|
| `title` | whether `TITLES[domain]` is not `None` | 6 |
| `char_cap` | `"max 4000 characters"` | 6 |
| `no_emoji` | `"no emojis"` | 6 |
| `no_alert_numbers` | `"not alert numbers"` | 6 |
| `no_risk_labels_in_headers` | `"risk classification labels"` | 4 |

So a customer prompt update changes the contract automatically instead of leaving the table stale, and the same mechanism is what makes the four-versus-six question answerable at all. `tests/test_output_contract.py` collects **25 tests**, including one asserting that `RULES["bustout"]` and `RULES["rings"]` have **every** value false, and one asserting that `output_contract.TITLES` and `fanout.ANALYSIS_TITLES` agree.

## Model-sensitive versus not

Two defects landed on these two agents. They look similar and they are not.

| | Invented titles | The calibration failures |
|---|---|---|
| Affects | bustout, rings | APP-1014 rings, APP-1004 bustout |
| Rate | 24 of 24 Claude analyses; 23 of 23 in the cross-family study | 0 of 5 models correct, each |
| Model-sensitive? | **Yes** — GPT-5.6 invents none, on 36 of 36 analyses | **No** — wrong on all five models across both families |
| Same mistake each time? | No — 15 spellings | **Yes** — every model wrong the same way |
| Fix | a title line in the prompt, or accept and strip in display | an operational counting rule in the prompt |
| Measured to work? | The GPT result is measured; the prompt fix is **unmeasured** | **Unmeasured** |

Detail on the second column: Open questions. Both prompts already state the principle in the exact terms of the failure, which is why the recommendation is to state it as a counting rule rather than as guidance.

## Two prompts with no character cap

Same two agents, same root cause — an output specification consisting of one sentence. Measured over n=334 specialist calls in `evals/results/tail_analysis.json`:

| | Domains | n calls | Output tokens p50 | Analysis chars p50 | Over 4,000 chars |
|---|---|---|---|---|---|
| Prompt caps at 4,000 chars | identity, dealer, straw, employment, income, synthetic | 251 | **381** | 1,224 | 9 (3.6%) |
| **No cap in prompt** | **bustout, rings** | 83 | **1,331** | **3,519** | **25 (30.1%)** |

3.5x the output tokens and 2.9x the characters. Since wall clock is linear in output tokens at $r^2$=0.9531 over n=332 calls, these two agents are the structural owner of the fan-out floor, and `rings` alone was the slowest specialist in **25 of 42** pipeline runs.

They are also the **only** two agents ever measured to truncate: `evals/results/tail_analysis.json:live_cap_sweep` records `MaxTokensReachedException` on both at caps of 1024, 1536 and 2048, non-monotonically.

Reaching for `max_tokens` instead of the prompt was measured and failed. Over 30 live calls: no latency gain (5 of 10 comparable cells faster than control, 5 slower, mean delta −0.23 s), **14 of 30 cells truncated**, and only **13 of 30** kept the same band. A truncated call spends the full wall clock, is fully billed, and returns no analysis, so the fraud dimension is lost. The measurement.

## How to reproduce the check

```
MOCK_MODE=1 python3 -m pytest tests/test_output_contract.py -q     # 25 tests

python3 -c "
from agents.output_contract import RULES, TITLES, check
print(RULES['rings'])                       # every value False
print(TITLES['bustout'], TITLES['rings'])   # None None
print(check('rings', '# Fraud Ring Risk Analysis\n\nBody.').violations)
"
```

`check()` reports; it never repairs. `strip_invented_title()` exists for a display surface that wants to honour the contract the prompt states, and the violation must still be recorded — because what the customer is owed here is the finding, not a tidied output.

## Still unknown

- **Whether adding a title line to those two prompts stops the invention.** Unmeasured; her artifact to change.
- **Whether adding the character-cap sentence shortens their output.** Also unmeasured, and the larger of the two effects.

- **Whether her downstream consumer already tolerates the heading.** If `rings_summary` is rendered rather than parsed, this is cosmetic; if it is parsed, it is not.


**Output contracts**
What the eight specialist contracts actually say, and the 21-key adjudication.

**Latency tail**
What the missing character cap costs in latency, and why max_tokens is the wrong fix.

**Cross-family study**
Why the specialists stay on Claude despite GPT's clean contract compliance.

‹ Alert categories
General Rule 3 as data rather than prose — ...

Benchmark method ›
How evals/bench.py measures — the timed ...

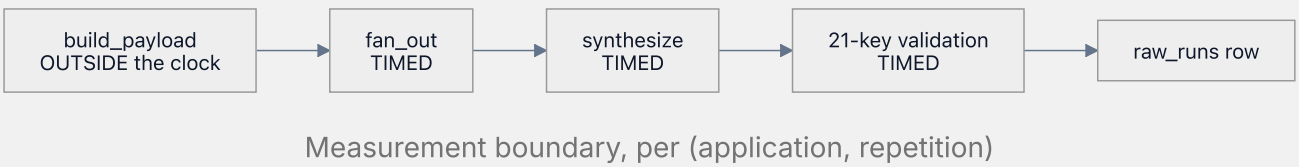
# Benchmark method

How evals/bench.py measures — the timed window's exact boundary, three modes of which only one may publish a headline, nearest-rank percentiles with sample-count refusals, and four mutation-tested anti-fabrication guards.

*For the customer's engineer, and for anyone who has to defend a figure in a meeting. This page describes the instrument; the readings are on **Pipeline results**.*

**One module, `evals/bench.py`, is the only code in this repository permitted to emit a latency or a cost figure.** Everything else — the UI, the README, this site — quotes an artifact that module wrote. That restriction has a specific history, and the rest of this page is the mechanism enforcing it.

## What sits inside the timed window




Measurement boundary, per (application, repetition)

Signal projection runs before `perf_counter` starts, because the customer's platform precomputes signals rather than querying per application. Consequence, stated in the artifact's own `caveats` array: every end-to-end number on this site is **agent latency, not pipeline-including-ETL latency**.

Three further exclusions belong to the same boundary. AgentCore Runtime overhead is absent — benchmarked calls went to Bedrock directly from the host recorded in `run.host`, so container cold start, session setup and SSE framing appear in no figure. Throughput is absent, since applications ran one at a time. And a failed synthesis is captured as data

rather than crashing the harness: exception type and message are recorded, the run continues.

## Three modes, and which one may publish

Mixing readings across these modes is the easiest available route to a fabricated number here, so each mode carries an explicit publication verdict.

| Mode | What it calls | Model latency measured | Cost fields | May publish a headline |
|---|---|---|---|---|
| `--pipeline` | `agents.fanout.fan_out` and `agents.synthesize.synthesize` — the shipped functions the Runtime entrypoint calls | Yes, real Bedrock and `bedrock-mantle` | Priced from each call's own reported usage | **Yes. Every headline on this site came from here** |
| `--offline` | Same functions, no model | **No** | `None`, never `0.0` | **No.** Orchestration overhead only |
| `--live` | Re-implements the fan-out inside `bench.py` | Yes, but not through the shipped path | Priced | **No.** Legacy; see below |

`--pipeline` defaults to 3 repetitions per application, and that default comes from **cost, not statistics**: 14 applications × 3 is roughly $10 of inference, and 42 samples is the smallest set clearing the p95 minimum of 20 for the **pooled** figure. Per-application p95, at 3 samples each, is suppressed rather than printed.

```
python -m evals.bench --pipeline --mock --repetitions 1              # shape only, no
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --pipeline --repetitions 3
python -m evals.bench --offline --concurrency 1,4,16 --iterations 40
```

Disqualifying `--live` is the module's own decision, in its own terms: it bypasses `agents.fanout`'s semaphore, widened executor and shared botocore pool, bypasses `agents.synthesize`'s structured-output validation and bounded repair, and cannot reach

the mantle synthesizer at all because GPT-5.x is not on Converse. Quoting it: *"A latency published from it would be a latency for a different program."*

Offline mode goes further than refusing to publish. Cost fields return `None` rather than `0.0` — nothing was tokenized, so a zero would be a claim — and `test_offline_report_has_no_cost_figures_anywhere` asserts it. Its caveat appears in the JSON `not_measured` array, in the table header and in each row label, three places because one is skippable: *"Do not present any offline number to the customer as an application-processing time."*

> **What the harness replaced**
>
> Pre-port, the headline claim *"26 s sequential → 8.5 s parallel"* came from a hand-written `NOMINAL_LATENCY_S` dictionary, printed beside `time.sleep(nominal * 0.05)` labelled *"measured wall-clock"*. Per-agent cost came from a hard-coded `NOMINAL_TOKENS` table. Every one of those figures was invented, and all of the nominal tables are now deleted.
>
> Quoting the module's docstring: *"AnyCompany's own engineer will ask where a number came from, so the answer has to be 'this run measured it, here is the sample count, the git sha, the region and the model id' — or 'we did not measure that'."*

## Percentile policy

Two constants, both written into every report so a figure stays reproducible:

```
PERCENTILE_MIN_SAMPLES = {"p50": 5, "p95": 20, "p99": 100}
PERCENTILE_METHOD = "nearest-rank, inclusive (no interpolation); ceil(p*n)-th of the s
```

Minimums come from **1/(1−p)** — the smallest sample in which the requested tail can actually be observed. A 1-in-100 event cannot appear in 20 samples, so a "p99" over 20 samples is the maximum wearing a percentile's name. p50 keeps a practical floor of 5 rather than the arithmetic 2, because a three-sample median is not worth showing a customer either.

Nearest-rank with no interpolation means every reported value is one the run actually observed, asserted by `test_percentile_values_are_observed_samples_not_interpolated`. At the small sample sizes this harness runs at, an interpolated p95 can sit between two real measurements and belong to neither.

## Refusals, and what one looks like

Where a sample cannot support a percentile, the field is `None` and a sentence goes into a `suppressed` map. Callers render the sentence, not a number. All four headline rows in `evals/results/pipeline_14x3_us-east-1.json` carry this one:

> “`"p99": "refused: 42 sample(s), need >= 100. A p99 over 42 sample(s) is not a p99."`”

Exactly one p99 in the whole engagement clears its minimum: healthy output tokens per second in `evals/results/tail_analysis.json`, n=332 calls, **89.78 tok/s**, whose `suppressed` map is empty. No end-to-end, cost or per-specialist p99 exists at any n reached here, and none is inferred. That contrast is the point — the policy is a threshold, not a blanket refusal.

A site-level guard makes the same rule mechanical: `tests/test_docs_site.py::test_no_page_claims_a_percentile_without_a_sample_count` fails the build if a page quotes a percentile and states no `n` anywhere in it.

Thirteen of the collected tests in `tests/test_bench.py` cover the percentile policy alone, including `test_single_sample_yields_no_percentiles_at_all` — described in its own docstring as *"the exact failure the old demo shipped: a p99 from one measurement"* — `test_percentiles_appear_exactly_at_their_minimum_sample_count`, `test_markdown_summary_refuses_a_suppressed_percentile_out_loud`, and two that re-read the **committed** artifact and assert its percentiles obey the minimums.

## Derived quantities and their stated limits

| Quantity | Definition | Stated limit |
|---|---|---|
| `sum_of_eight_ms` | Sum of the eight specialists' own measured wall clocks in this run | **Sequential counterfactual.** *"The terms are measured, the arrangement is hypothetical"* |
| `parallel_speedup` | fan-out ÷ sum-of-eight, one ratio per run | Reported as a **distribution**, never one number: a fan-out is bounded by its slowest member, so the ratio moves with which specialist ran long |
| `runs_under_target` | Count of runs under 60.0 s | *"A count of runs under a threshold is not a percentile and not an SLA. It is the only form of a 'sub-minute per application' claim this sample size can support"* |
| `cost_usd` | Priced from each call's **own** reported token usage | A run with any failed call is excluded, so a 7-of-8 adjudication cannot deflate per-application cost |

`_under_target_accounting` asks three questions about the 60-second target rather than one: `runs_under_target`, `applications_all_reps_under_target` (the strict reading, and the one to quote) and `applications_any_rep_under_target`. Whatever gap separates the last two **is** the set of applications straddling the line, and naming it is the point — a p50 under 60 s is not a promise of 60 s.

## Live gate and the pre-run estimate

Live spend is **double-gated**: the `--live` flag (or `--pipeline` without `--mock`) *and* `AGENTCORE_BENCH_ALLOW_LIVE=1` in the environment. Refusal message states the reason: *"two gates, deliberately: `--live` alone is one keystroke away from a billed run in someone's account."*

Before calling anything, live mode prints every model id, its rate card with the profile (`global.*` versus `us.*`), any rate note, and an upper-bound spend estimate.

> The pre-run spend estimate is partial on purpose, and says which parts

> Estimation uses Bedrock `CountTokens`, a free metering call. Two measured behaviours shape it.
>
> **An inference-profile prefix is rejected.** Both `global.anthropic.claude-haiku-4-5-...` and `us.anthropic....` fail with `ValidationException: The provided model doesn't support counting tokens`; the bare `anthropic.claude-haiku-4-5-...` succeeds. So the prefix is stripped before the call — a token count is a property of the model, not of the routing profile.
>
> **Not every model offers it.** Measured in us-east-1 on 2026-07-28, `anthropic.claude-haiku-4-5-20251001-v1:0` returns a count while `anthropic.claude-sonnet-5` and `anthropic.claude-opus-5` both raise the same `ValidationException`. Estimates are therefore exact for the Haiku specialists and `None` — explicitly unknown — for the rest. Quoting the module: *"a partial estimate that says which parts are missing beats a complete-looking one built on a char/4 guess."*

## Provenance recorded in every report

`run_environment()` writes all of this into every JSON artifact:

| Field | Value in the headline run |
|---|---|
| `mode` | `pipeline-live` |
| `timestamp_utc` | `2026-07-28T07:42:42+00:00` |
| `git_sha` / `git_branch` / `git_dirty` | `9bb569bac801884f9c7c5f379520cf3ddbf4a270`, `feat/1to1-agentcore-replacement`, `true` |
| `aws_region`, `synthesizer_region` | `us-east-1`, `us-east-1` |
| `model_ids` | all nine, by role |
| `host` | Python 3.12.9, `macOS-26.5.2-arm64`, 11 CPUs |
| `packages` | `strands-agents 1.26.0`, `bedrock-agentcore 1.14.0`, `boto3 1.43.54`, `pydantic 2.13.4` |
| `target_e2e_seconds` | `60.0` |

`git_dirty: true` is recorded rather than hidden. A dirty tree at measurement time is a real caveat about reproducibility, and a reader is entitled to see it.

## Anti-fabrication guards

Four guards, each verified by mutation — by breaking the thing it protects and confirming the test fails.

| Guard | Test | What it prevents |
|---|---|---|
| Missing usage yields `None`, never `0.0` | `test_missing_usage_yields_none_not_a_fabricated_figure` | A truncated or 5xx call has unknown cost, and unknown is not zero. Keeps the one degraded run out of the cost distribution instead of pulling the mean down |
| An unpriced model yields `None`, never a borrowed price | `test_unknown_model_yields_none_rather_than_borrowing_another_price` | Opus 4.7 has no verified Bedrock rate here, so it prices as `None` rather than inheriting Opus 5's card |
| Rate card cannot fork | `test_rate_card_delegates_to_agents_pricing_and_matches_it` | `agents/pricing.py` owns published rates; the report records which source it used, so a stale fallback cannot masquerade as the real table |
| A temporary rate is flagged temporary | `test_sonnet_5_introductory_rate_is_flagged_as_temporary` | Sonnet 5's $2/$10 per MTok reverts to $3/$15 on 2026-09-01; both pairs are carried, so a cost slide cannot present a promotional rate as permanent |

Cache minimums follow the same delegation pattern (`test_cache_minimums_defer_to_agents_models_and_cannot_diverge`). `tests/test_bench.py` collects 133 tests (`MOCK_MODE=1 python3 -m pytest tests/test_bench.py -q --collect-only`).

## Re-aggregation without re-measuring

`reaggregate_pipeline_report` rebuilds every derived statistic from `raw_runs` with no model call. The headline artifact records that this was done at `2026-07-28T07:46:30+00:00`, with the caveat *"The measurements are unchanged and still belong to the run recorded in `run`; only the derived statistics were rebuilt. No model was called to produce this file."*

```
python -m evals.bench --reaggregate evals/results/pipeline_14x3_us-east-1.json \
  --markdown evals/results/pipeline_14x3_us-east-1.md
```

Separating measurement from derivation lets a statistics bug be fixed without spending $6.4659 again, and lets the fix be audited: `raw_runs` is the measurement, everything above it is derivation.

## Four sibling harnesses

Same policy, same refusal behaviour, different questions. All of them import `evals.bench.summarize_values` rather than computing their own percentiles.

| Harness | Question | Artifact |
|---|---|---|
| `evals/bench.py --pipeline` | End-to-end latency and cost of the shipped path | `evals/results/pipeline_14x3_us-east-1.json` |
| `evals/tier_quality.py` | Does the cheaper Claude tier analyse as well? | `evals/results/tier_quality.json` |
| `evals/set_comparison.py` | Should the specialists move to GPT-5.6? | `evals/results/set_comparison.json` |
| `evals/tail_analysis.py` | Where does the 75-second run come from? | `evals/results/tail_analysis.json` |
| `evals/judge_sol.py` | GPT-side judge for the cross-family study | feeds `set_comparison` |

# Still unknown

- **AgentCore Runtime overhead.** Absent from every figure on this site.
- **Signal generation cost and latency.** `build_payload` runs outside the clock by design.
- **Throughput.** One application in flight at a time, so nothing here says what happens when N adjudications overlap.
- **Per-application percentiles.** Three repetitions each, therefore mostly suppressed.
- **Any p99** on an end-to-end, cost or per-specialist axis, at any n reached here.


**Pipeline results**
All 42 runs individually, per specialist, and every exclusion.

**Latency tail**
Two of 336 calls, four candidate causes tested, three ruled out.

**Artifact index**
Every committed artifact with its schema, n and spend.

‹ Output-contract defects
All 24 bustout and rings analyses invented ...

Pipeline results ›
All 42 measured runs individually — per-sp...

# Pipeline results

All 42 measured runs individually — per-specialist latency and cost in one table with a critical-path column, the speed-up distribution, the 60-second accounting, the one degraded run, and every exclusion.

*For the customer's engineer. Unless a sentence names another file, every figure here comes from `evals/results/pipeline_14x3_us-east-1.json`, with its human-readable sidecar at `evals/results/pipeline_14x3_us-east-1.md`.*

**42 runs, 336 specialist calls, 31.45 s p50 end to end, $6.46591995 of measured spend.** This is the row-level cut. The summary version with fewer tables is Measured results; the instrument is described on Benchmark method.

# Run provenance

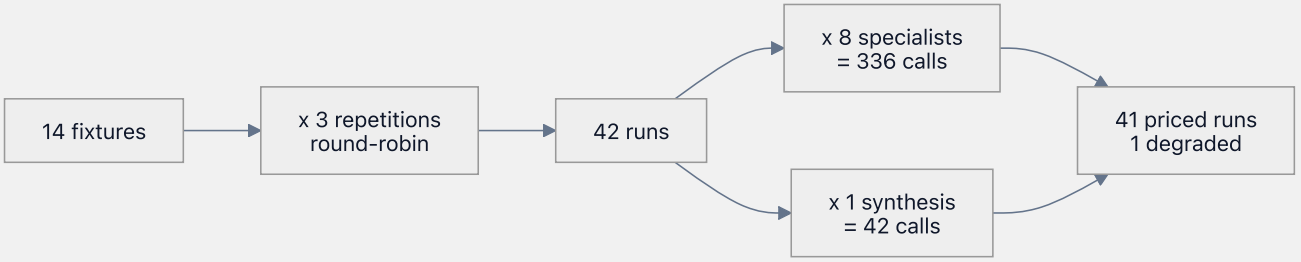
| Field | Value |
|---|---|
| Schema | `pointpredictive.agentcore.pipeline-bench/1` |
| Mode | `pipeline-live` (`workload.mock = false`) |
| Timestamp | `2026-07-28T07:42:42+00:00` |
| Git | `9bb569bac801884f9c7c5f379520cf3ddbf4a270`, branch `feat/1to1-agentcore-replacement`, `git_dirty: true` |
| Region | `us-east-1`; synthesizer `us-east-1` |
| Host | Python 3.12.9, `macOS-26.5.2-arm64`, 11 CPUs |
| Packages | `strands-agents 1.26.0`, `bedrock-agentcore 1.14.0`, `boto3 1.43.54`, `pydantic 2.13.4` |
| Application source | `fixtures.loader`, 14 applications |
| Repetitions | 3 per application, 9 model calls per adjudication |
| Execution order | Round-robin: repetition 1 over every application, then 2, then 3. **One application in flight at a time**, so an application's repetitions are spread across the wall-clock window rather than measured back to back |
| Runs | **42** of 42 expected — 41 complete, **1 degraded**, **42 validated adjudications** |
| Specialist calls | **336** |
| Measured spend | **$6.46591995** |

**This run used the previous model assignment**

`run.model_ids` records `synthetic` and `rings` on `global.anthropic.claude-opus-5`. Both moved to Sonnet 5 on 2026-07-28 **after** this run, on the strength of the tier re-derivation. Every latency and cost figure below therefore describes the **old** assignment, and the current

> assignment's end-to-end result is **not measured**. One command re-measures it, at roughly $5 of spend: `AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --pipeline --repetitions 3`.

## Headline distribution




Where the 336 + 42 calls came from

Percentile method, quoted from the artifact: *"nearest-rank, inclusive (no interpolation); ceil(p*n)-th of the sorted samples."*

| Stage | n | min | p50 | p95 | max | mean |
|---|---|---|---|---|---|---|
| **End to end** | 42 | 18.93 s | **31.45 s** | **46.55 s** | 74.97 s | 32.80 s |
| Fan-out (8 specialists in parallel) | 42 | 15.37 s | 24.93 s | 41.05 s | 70.62 s | 28.31 s |
| Master synthesis | 42 | 3.08 s | 4.20 s | 5.59 s | 15.21 s | 4.49 s |
| Sum of eight (sequential counterfactual) | 42 | 65.52 s | 85.62 s | 157.14 s | 173.00 s | 101.09 s |

All four rows carry the identical p99 refusal:

> “`"p99": "refused: 42 sample(s), need >= 100. A p99 over 42 sample(s) is not a p99."`”

## Every run, individually

Forty-two rows from `raw_runs`. `slowest` names the specialist that owned the critical path in that run; `speed-up` is fan-out ÷ sum-of-eight for that run alone.

| Application | Rep | e2e | fan-out | synthesis | slowest | speed-up | cost | band | < 60 s |
|---|---|---|---|---|---|---|---|---|---|
| APP-1001 | 1 | 24.25 s | 19.51 s | 4.74 s | rings 16.79 s | 4.43x | $0.1258 | LOW | yes |
| APP-1001 | 2 | **70.24 s** | 66.51 s | 3.73 s | **income 66.01 s** | 1.85x | $0.1195 | LOW | **no** |
| APP-1001 | 3 | 18.93 s | 15.37 s | 3.56 s | rings 14.47 s | 4.26x | $0.1177 | LOW | yes |
| APP-1002 | 1 | 23.18 s | 19.37 s | 3.80 s | rings 17.66 s | 4.39x | $0.1337 | LOW | yes |
| APP-1002 | 2 | 22.92 s | 19.39 s | 3.53 s | rings 17.52 s | 4.42x | $0.1364 | LOW | yes |
| APP-1002 | 3 | 22.32 s | 19.01 s | 3.31 s | rings 18.22 s | 4.26x | $0.1364 | LOW | yes |
| APP-1003 | 1 | 31.68 s | 28.08 s | 3.60 s | income 27.30 s | 3.64x | $0.1489 | LOW | yes |
| APP-1003 | 2 | 34.41 s | 30.87 s | 3.54 s | income 30.38 s | 3.55x | $0.1569 | LOW | yes |
| APP-1003 | 3 | 34.30 s | 30.81 s | 3.49 s | income 30.42 s | 3.28x | $0.1534 | LOW | yes |
| APP-1004 | 1 | 37.79 s | 32.63 s | 5.16 s | rings 31.80 s | 5.11x | $0.2479 | HIGH | yes |
| APP-1004 | 2 | 36.25 s | 31.47 s | 4.78 s | rings 29.84 s | 4.99x | $0.2271 | HIGH | yes |
| APP-1004 | 3 | 39.23 s | 33.85 s | 5.37 s | income 33.41 s | 5.11x | **$0.2536** | HIGH | yes |
| APP-1005 | 1 | 46.55 s | 41.05 s | 5.50 s | identity 41.04 s | 3.10x | $0.1897 | MEDIUM | yes |

| Application | Rep | e2e | fan-out | synthesis | slowest | speed-up | cost | band | < 60 s |
|---|---|---|---|---|---|---|---|---|---|
| APP-1005 | 2 | 42.01 s | 36.42 s | 5.59 s | bustout 35.71 s | 3.30x | $0.1702 | HIGH | yes |
| APP-1005 | 3 | 43.91 s | 39.69 s | 4.22 s | identity 39.69 s | 3.04x | $0.1727 | MEDIUM | yes |
| APP-1006 | 1 | 22.31 s | 18.85 s | 3.47 s | rings 17.79 s | 3.60x | $0.1231 | LOW | yes |
| APP-1006 | 2 | 23.77 s | 19.57 s | 4.20 s | rings 17.42 s | 3.95x | $0.1181 | LOW | yes |
| APP-1006 | 3 | 33.67 s | 18.46 s | **15.21 s** | rings 17.51 s | 4.16x | $0.1256 | LOW | yes |
| APP-1007 | 1 | 34.58 s | 31.51 s | 3.08 s | rings 30.79 s | 2.51x | $0.1284 | LOW | yes |
| APP-1007 | 2 | **74.97 s** | 70.62 s | 4.35 s | **bustout 69.78 s** | 1.98x | $0.1368 | LOW | **no** |
| APP-1007 | 3 | 23.52 s | 20.11 s | 3.41 s | rings 19.43 s | 3.26x | $0.1195 | LOW | yes |
| APP-1008 | 1 | 25.50 s | 21.72 s | 3.78 s | rings 20.38 s | 3.75x | $0.1356 | LOW | yes |
| APP-1008 | 2 | 31.45 s | 26.95 s | 4.50 s | rings 25.83 s | 3.28x | $0.1473 | LOW | yes |
| APP-1008 | 3 | 28.46 s | 24.93 s | 3.52 s | rings 24.03 s | 3.07x | $0.1387 | LOW | yes |
| APP-1009 | 1 | 25.68 s | 21.89 s | 3.79 s | rings 21.10 s | 3.34x | $0.1366 | LOW | yes |
| APP-1009 | 2 | 25.80 s | 21.88 s | 3.92 s | rings 20.83 s | 3.49x | $0.1358 | LOW | yes |

| Application | Rep | e2e | fan-out | synthesis | slowest | speed-up | cost | band | < 60 s |
|---|---|---|---|---|---|---|---|---|---|
| APP-1009 | 3 | 23.84 s | 20.08 s | 3.76 s | rings 19.28 s | 3.63x | $0.1325 | LOW | yes |
| APP-1010 | 1 | 28.59 s | 23.93 s | 4.66 s | rings 22.99 s | 3.45x | $0.1440 | LOW | yes |
| APP-1010 | 2 | 28.38 s | 23.85 s | 4.54 s | rings 22.68 s | 3.37x | $0.1379 | LOW | yes |
| APP-1010 | 3 | 27.95 s | 24.07 s | 3.89 s | rings 23.17 s | 3.32x | $0.1478 | LOW | yes |
| APP-1011 | 1 | 22.40 s | 18.85 s | 3.55 s | bustout 17.20 s | 4.33x | $0.1288 | LOW | yes |
| APP-1011 | 2 | 22.79 s | 19.06 s | 3.72 s | bustout 18.28 s | 3.90x | $0.1239 | LOW | yes |
| APP-1011 | 3 | 21.21 s | 18.09 s | 3.13 s | bustout 17.42 s | 3.68x | unpriced | LOW | yes |
| APP-1012 | 1 | 42.72 s | 37.46 s | 5.26 s | bustout 36.73 s | 3.79x | $0.1934 | HIGH | yes |
| APP-1012 | 2 | 36.58 s | 30.70 s | 5.88 s | identity 30.69 s | 4.18x | $0.1844 | MEDIUM | yes |
| APP-1012 | 3 | 38.22 s | 33.40 s | 4.82 s | identity 33.40 s | 4.02x | $0.1751 | MEDIUM | yes |
| APP-1013 | 1 | 34.01 s | 29.72 s | 4.30 s | bustout 28.42 s | 4.42x | $0.1854 | MEDIUM | yes |
| APP-1013 | 2 | 35.19 s | 29.88 s | 5.31 s | rings 28.46 s | 4.18x | $0.1854 | MEDIUM | yes |
| APP-1013 | 3 | 39.77 s | 34.57 s | 5.20 s | rings 33.80 s | 3.53x | $0.1841 | MEDIUM | yes |

| Application | Rep | e2e | fan-out | synthesis | slowest | speed-up | cost | band | < 60 s |
|---|---|---|---|---|---|---|---|---|---|
| APP-1014 | 1 | 24.75 s | 20.34 s | 4.41 s | rings 19.50 s | 4.07x | $0.1454 | MEDIUM | yes |
| APP-1014 | 2 | 41.90 s | 37.16 s | 4.74 s | rings 36.17 s | 3.12x | $0.2017 | MEDIUM | yes |
| APP-1014 | 3 | 31.63 s | 27.27 s | 4.36 s | synthetic 26.70 s | 4.35x | $0.1973 | MEDIUM | yes |

Two rows repay attention beyond the two over-target runs. **APP-1006 rep 3** is the only run where *synthesis* is the outlier — 15.21 s against a 4.20 s p50, on a fan-out of 18.46 s that is otherwise the fastest of that application's three; the synthesizer's max is 15.21 s and its p95 is 5.59 s, so this single call is the entire gap between them. **APP-1004 rep 3** is the most expensive run in the sample and its critical path is `income`, not `rings`, the only APP-1004 repetition where that holds.

## Against the 60-second target

| Question | Answer |
|---|---|
| Target ( `run.target_e2e_seconds` ) | 60.0 s |
| Runs under target | **40 of 42** |
| Applications under target on **every** repetition | **12 of 14** |
| Applications under target on **at least one** repetition | **14 of 14** |
| Applications **never** under target | **0** |
| Applications straddling the target | **2** — APP-1001, APP-1007 |

A count is not an SLA, and the artifact says so: *"A count of runs under a threshold is not a percentile and not an SLA. It is the only form of a 'sub-minute per application' claim this*

*sample size can support."* Both over-target runs are single-call events explained on Latency tail, and both of those applications came in at 23.5 s and 18.9 s on other repetitions.

## Parallel speed-up

Fan-out against the sequential counterfactual, one ratio per run, n=42:

| min | p50 | p95 | max | mean |
|---|---|---|---|---|
| 1.85x | **3.64x** | 4.99x | 5.11x | 3.73x |

Reporting a distribution rather than a single number is the artifact's own insistence: *"One speed-up per run, not one number for the port. The spread is the finding: the fan-out is bounded by its slowest dimension, so the ratio moves with which specialist ran long."*

Both extremes make that concrete. **1.85x** is APP-1001 rep 2, where `income` stalled at 66.01 s and dominated a 123.28 s sum-of-eight. **5.11x** is APP-1004 rep 3, where the eight specialists were more evenly loaded across a 173.00 s sum-of-eight, the largest total work in the sample.

What the counterfactual is, quoted: *"sum_of_eight_ms is the SEQUENTIAL COUNTERFACTUAL, the sum of the eight specialists' own measured wall clocks in this run. It is what eight calls cost one after another only if each would take the same time alone; the terms are measured, the arrangement is hypothetical."* The one thing it cannot account for is that a sequential run would issue its eight calls at eight different moments, and this sample shows Bedrock's per-call generation speed varying enough to matter.

## Per specialist: latency, cost and critical-path share

One table, because latency and cost rank the eight agents differently and reading them apart invites the wrong conclusion. Wall clock covers **successful calls only**, so the percentile n is `n_calls` minus `n_failed`. `On critical path` counts the 42 runs where that specialist was the slowest member of its fan-out.

| Specialist | Model | tier | n calls | n failed | p50 | p95 | max | SDK p50 | $/call | On criti path |
|---|---|---|---|---|---|---|---|---|---|---|
| `rings` | Opus 5 | heavy | 42 | **1** | **20.96 s** | 32.25 s | 36.17 s | 20.49 s | **$0.04765** | Yes 25 o 42 |
| `bustout` | Sonnet 5 | mid | 42 | 0 | 14.46 s | 35.71 s | **69.78 s** | 13.86 s | $0.01699 | Yes 7 of 42 |
| `income` | Sonnet 5 | mid | 42 | 0 | 8.09 s | 31.44 s | **66.01 s** | 7.37 s | $0.01537 | Yes 5 of 42 |
| `identity` | Sonnet 5 | mid | 42 | 0 | 12.56 s | 33.40 s | 41.04 s | 11.16 s | $0.01727 | Yes 4 of 42 |
| `synthetic` | Opus 5 | heavy | 42 | **1** | 8.52 s | 20.82 s | 26.70 s | 7.54 s | $0.02869 | Yes 1 of |
| `employment` | Sonnet 5 | mid | 42 | 0 | 7.48 s | 17.64 s | 20.00 s | 6.67 s | $0.01022 | Neve |
| `dealer` | Haiku 4.5 | light | 42 | 0 | 6.56 s | 14.56 s | 18.85 s | 5.46 s | $0.00541 | Neve |
| `straw` | Haiku 4.5 | light | 42 | 0 | **4.40 s** | 8.76 s | 12.52 s | 3.39 s | $0.00352 | Neve |

Model ids in full: `global.anthropic.claude-sonnet-5`, `global.anthropic.claude-haiku-4-5-20251001-v1:0`, `global.anthropic.claude-opus-5`. Minimums, omitted above for width: identity 5.87 s, dealer 4.31 s, straw 2.49 s, employment 5.89 s, income 4.33 s, synthetic 5.89 s, bustout 9.41 s, rings 14.47 s.

Three readings follow from that table. `rings` **sets the median while** `income` **and** `bustout` **produce the tail** — rings owns the critical path in 25 of 42 runs, yet its own max of 36.17 s is the *lowest* max of the three mid/heavy agents that ever led, so the agent setting the

median and the agent producing the tail are different agents (Critical path). `rings` **has the highest floor of any agent**: its minimum of 14.47 s sits above `dealer`'s p95 and above `straw`'s max, so no cheap run of rings exists. And **wall clock tracks SDK-reported latency closely** — the gap over n=334 calls is p50 0.73 s, p95 1.69 s (`evals/results/tail_analysis.json`), so almost none of the variance is client-side.

## Tokens behind the cost column

Sums over all calls, and the mean per call.

| Specialist | Input total / mean | Output total / mean | Cache read total / mean | Cost total | Mean $/call |
|---|---|---|---|---|---|
| `identity` | 130,620 / 3,110.0 | 46,419 / 1,105.2 | 0 / 0 | $0.725430 | $0.01727 |
| `dealer` | 147,741 / 3,517.6 | 15,929 / 379.3 | 0 / 0 | $0.227386 | $0.00541 |
| `straw` | 100,797 / 2,399.9 | 9,424 / 224.4 | 0 / 0 | $0.147917 | $0.00352 |
| `employment` | 99,252 / 2,363.1 | 22,147 / 527.3 | 45,822 / 1,091.0 | $0.429138 | $0.01022 |
| `income` | 133,710 / 3,183.6 | 37,818 / 900.4 | 0 / 0 | $0.645600 | $0.01537 |
| `synthetic` | 111,944 / 2,730.3 | 23,840 / 581.5 | 41,041 / 1,001.0 | $1.176241 | $0.02869 |
| `bustout` | 74,997 / 1,785.6 | 55,289 / 1,316.4 | 54,474 / 1,297.0 | $0.713779 | $0.01699 |
| `rings` | 64,295 / 1,568.2 | 64,320 / 1,568.8 | 48,175 / 1,175.0 | **$1.953563** | **$0.04765** |

Token means for `synthetic` and `rings` run over their 41 priced calls; the failed call recorded no usage and contributes nothing rather than a zero.

`rings` inverts the usual ratio, and that explains its cost better than its tier does. It is the only specialist whose **output** total exceeds its **input** total — 64,320 against 64,295 — where every other agent is input-dominated, `dealer` by 9.3x. Output prices at 5x input on every Claude model in this run, and rings' prompt carries no character cap, so it writes 1,568.8 output tokens per call against `straw`'s 224.4. Detail.

**Cache reads with no cache writes are not a working cache lever**

`employment`, `synthetic`, `bustout` and `rings` show non-zero `cacheReadInputTokens`; `identity`, `dealer`, `straw` and `income` show exactly zero. **No specialist recorded a single cache *write* token** — that column is 0 for all eight.

Reads with no writes in the same run means those reads hit entries this run did not create, and no per-application prefix is long enough to be reused anyway: prompts run 547–956 estimated tokens against per-model minimums of 512 / 1024 / 4096, and the per-application payload differs every time. What does not work.

## Cost per adjudication

n=41 fully priced runs; the one degraded run is excluded.

| min | p50 | p95 | max | mean |
|---|---|---|---|---|
| $0.1177 | **$0.1440** | $0.2271 | $0.2536 | $0.1562 |

Spread is $0.1359. Rates are fixed per model, so token volume is the only driver, and both endpoints are recorded so that driver is identifiable rather than asserted:

| | Cheapest run | Most expensive run |
|---|---|---|
| Application / rep | APP-1001 rep 3 | APP-1004 rep 3 |
| Total | $0.117721175 | $0.253649875 |
| Specialist input tokens | 20,237 | 21,657 |
| **Specialist output tokens** | **4,098** | **13,735** |
| Synthesis input / output | 2 / 640 | 2 / 887 |
| Synthesis cache-write tokens | 3,433 | 6,757 |
| Specialist cost | $0.1087746 | $0.2385026 |
| Synthesis cost | $0.008946575 | $0.015147275 |

Specialist **input** tokens differ by 7%; specialist **output** tokens differ by **3.35x**. Cost variance on this workload is entirely an output-length story, the same mechanism that sets the latency floor.

## Synthesis, in detail

Model `openai.gpt-5.6-luna` on `bedrock-mantle`, 42 calls, **0 failed**, 0 repair attempts.

| | n | min | p50 | p95 | max | mean |
|---|---|---|---|---|---|---|
| Wall clock | 42 | 3.08 s | 4.20 s | 5.59 s | 15.21 s | 4.49 s |
| Adapter-reported latency | 42 | 3.05 s | 3.90 s | 5.55 s | 15.19 s | 4.45 s |

| Token class | Total | Mean per call |
|---|---|---|
| Input | 84 | 2.0 |
| Output | 30,066 | 715.9 |
| Cache read | 0 | 0.0 |
| **Cache write** | **180,610** | **4,300.2** |

Total $0.44686675, mean **$0.010640** per call — about **7%** of a median adjudication.

Two input tokens per call is not a measurement error, and it is why cache-write pricing matters here. The whole ~31K-character synthesis prompt lands in the **cache-write** class on this transport, and GPT-5.6 bills cache-write at **1.25x input**; omitting that term under-reported a measured Luna call by **62%**. Cost levers.

`repair_attempts` totalled **0** across all 42 runs and `runs_needing_a_repair` was 0, so the 21-key structured output validated first time every time — including on the degraded run where two of the eight narratives were missing.

## Verdict stability

| | Value |
|---|---|
| Applications returning the same `overall_risk_decision` on every repetition | **12 of 14** |
| Unstable applications | APP-1005, APP-1012 |
| Applications disagreeing with the pinned expectation | APP-1003, APP-1005, APP-1012 |
| Applications **never** matching the pinned expectation | APP-1003 |

- **APP-1005** — pinned MEDIUM RISK; observed HIGH RISK and MEDIUM RISK; matched 2 of 3. Its `recommendation_decision` moved with the band, between DECLINE and REVIEW

AND APPLY STIPULATIONS, which is the contract behaving correctly (MSTR-033 ties stipulations to the mid-band) on an unstable input.

- **APP-1012** — pinned HIGH RISK; observed HIGH RISK and MEDIUM RISK; matched 1 of 3.
- **APP-1003** — pinned MEDIUM RISK; observed LOW RISK on all three runs, **stably**. Matched 0 of 3.

Whether a pin or a model is wrong is not decided here. Quoting the artifact: *"Whether the pin or the model is wrong is the customer's call; run the calibration judge in* `evals/evaluators/` *before drawing an accuracy conclusion."* APP-1003 is a self-employed-income-variance fixture pinned MEDIUM by the fixture author, and three **stable** LOW readings is as likely to be a pin question as a model question. A separate and much stronger finding — two fixtures every model in both families gets wrong the same way — is on Open questions.

## One degraded run

| Field | Value |
|---|---|
| Run | APP-1011, repetition 3 |
| Failed domains | `synthetic`, `rings` |
| Failure kinds | `synthetic: server_5xx`, `rings: server_5xx` |
| Underlying error (both) | `InternalServerException ... (reached max retries: 2)` on `ConverseStream` |
| Time each failed call spent before failing | synthetic 8.57 s, rings 8.02 s |
| Synthesis error | none |
| **Adjudication validated** | **true — all 21 keys** |
| End to end | **21.215 s** |

Three things this single run demonstrates, all of them design intent working:

1. **7-of-8 graceful degradation is real.** Two specialists were lost and the adjudication still validated its full 21-key contract with zero repair attempts. That is the specific behaviour `strands.multiagent.GraphBuilder` cannot provide, being fail-fast on any node error — why GraphBuilder was rejected.
2. **A 5xx fails fast.** 21.2 s is among the quickest runs in the sample, so a server error here costs a **fraud dimension**, not latency — the opposite shape from the stall, which costs latency and loses nothing.
3. **Degraded runs are reported, never averaged away.** Quoted: *"It stays in the latency distribution and leaves the cost distribution."* Which is why end-to-end is n=42 and cost is n=41.

## Throttling: zero

Across all 336 specialist calls, `ModelThrottledException` occurred **0** times. The retry ladder in force was `[1, 2, 4]s` over 4 attempts, and that strategy retries on `ModelThrottledException` and nothing else. Source: `evals/results/tail_analysis.json`.

`stop_reasons` over the 336 calls: `end_turn` **334**, `None` **2** — the two 5xx failures. No call in the sample was truncated.

## What this benchmark did not measure

Reproduced from the artifact's own `caveats` array, because the exclusions are as load-bearing as the figures:

- **AgentCore Runtime overhead.** *"These calls go to Bedrock (and to bedrock-mantle for the synthesizer) directly from the host named in run.host, so container cold start, session setup and SSE framing are NOT in the end-to-end figures."*
- **Signal generation.** *"build_payload runs OUTSIDE the timed window because the customer's architecture precomputes signals; an end-to-end figure here is agent latency, not pipeline-including-ETL latency."*
- **Throughput.** *"Applications were run ONE AT A TIME, so nothing here says what happens when N adjudications overlap."*

- **Per-application percentiles.** Computed over 3 repetitions each and *"therefore mostly suppressed by design."*
- **Any p99.**

One process note from the same array: aggregates were recomputed from `raw_runs` by `evals.bench.reaggregate_pipeline_report` at `2026-07-28T07:46:30+00:00`. *"The measurements are unchanged... No model was called to produce this file."*

## Where the earlier single-run figure fits

`README.md` and `CHANGELOG.md` carry an earlier headline: **37.5 s end-to-end at $0.245** on APP-1004, one live run, recorded at `9bb569b`. Authoritative is the **42-run benchmark**: 31.45 s p50 / 46.55 s p95, $0.1440 p50.

Those two do not conflict. APP-1004's three benchmarked repetitions ran 37.79 / 36.25 / 39.23 s, which brackets 37.5 s, and its three costs were $0.2479 / $0.2271 / $0.2536, which bracket $0.245. Only one of them is a distribution, though, so **quote the p50 with its n**. A recording of that single run is at `media/demo.mp4`.


**Latency tail**
The two over-target runs, four candidate causes, and the max_tokens fix that failed.

**Critical path**
Which agent owns the floor, and the measured ceiling on fixing it.

**Benchmark method**
The instrument: timed window, percentile policy, anti-fabrication guards.

‹ Benchmark method
How evals/bench.py measures — the timed ...

Latency tail ›
Two of 336 calls generated at 16.1 and 4.7 o...

# Latency tail

Two of 336 calls generated at 16.1 and 4.7 output tok/s against a 65.50 tok/s median, inside Bedrock's own reported latency, with zero throttles — and the max_tokens fix that was measured and failed all three of its criteria.

*For the customer's engineer. Every figure comes from `evals/results/tail_analysis.json`, derived from `evals/results/pipeline_14x3_us-east-1.json`, with a human-readable sidecar at `evals/results/tail_analysis.md`.*

**Two calls out of 336 caused both over-target runs, and neither is fixable from the client.** They generated at **16.1** and **4.7** output tokens per second against a healthy median of **65.50 tok/s** (n=332), entirely inside the model's own reported latency, with zero throttles and `stop_reason=end_turn`. This is the page where a measurement was allowed to say no: four candidate causes were tested, three were ruled out, and the obvious engineering fix was then measured and made things worse.

## Four questions, asked in order

Elimination sequence, and what survived it

Two mechanisms come out of that sequence, and only one of them belongs to us. Verbosity sets the **floor** and is addressable through the prompt. Server-side generation speed sets the **tail** and is not addressable at all.

## Where the tail sits, and who owns it

Critical path — the slowest specialist — across all 42 runs, then split by run speed:

| Specialist | Slowest in all 42 runs | In the 10 slowest runs | In the 10 fastest runs |
|---|---|---|---|
| `rings` | **25** | 2 | **7** |
| `bustout` | 7 | 3 | 3 |
| `income` | 5 | 2 | 0 |
| `identity` | 4 | 3 | 0 |
| `synthetic` | 1 | 0 | 0 |

`same_owner_for_median_and_tail: false` is recorded in the artifact, and this table is why: `rings` owns the median and appears in the fastest ten more than three times as often as in the slowest ten. Tail ownership belongs to `bustout` and `income`.

The six slowest runs, with the critical path's share of its own fan-out:

| Application | Rep | e2e | fan-out | synthesis | Critical path | Its share of fan-out |
|---|---|---|---|---|---|---|
| APP-1007 | 2 | **75.0 s** | 70.6 s | 4.3 s | bustout 69.8 s | **98.8%** |
| APP-1001 | 2 | **70.2 s** | 66.5 s | 3.7 s | income 66.0 s | **99.3%** |
| APP-1005 | 1 | 46.5 s | 41.0 s | 5.5 s | identity 41.0 s | 100.0% |
| APP-1005 | 3 | 43.9 s | 39.7 s | 4.2 s | identity 39.7 s | 100.0% |
| APP-1012 | 1 | 42.7 s | 37.5 s | 5.3 s | bustout 36.7 s | 98.0% |
| APP-1005 | 2 | 42.0 s | 36.4 s | 5.6 s | bustout 35.7 s | 98.1% |

One call held APP-1007 rep 2 open for 40 extra seconds on its own. Its other seven specialists finished in 29.03 s or less, and six of them in under 10 s.

## 61% of the variance is within an application

Here is the decisive framing. Every application ran three repetitions of **byte-identical** input — fixtures are deterministic and `build_payload` runs outside the timed window — so any spread within a row is service-side, not workload.

| | Share of total variance |
|---|---|
| Between applications | 39.0% |
| **Within an application, across repetitions of identical input** | **61.0%** |

Method, quoted: *"one-way sum-of-squares split on application id; reported as a variance SHARE, not an F test — n=3 per cell cannot support a significance claim."* Interpretation rule, also quoted: *"within-application share above 50% means most of the spread is service-side variance on identical input, so a verbosity or model change cannot remove it."*

Per-application spread, widest first, over 14 applications:

| Application | Repetitions (s) | Spread | As multiple of min |
|---|---|---|---|
| APP-1007 | 23.52, 34.58, **74.97** | **51.45 s** | 3.19x |
| APP-1001 | 18.93, 24.25, **70.24** | **51.32 s** | 3.71x |
| APP-1014 | 24.75, 31.63, 41.90 | 17.15 s | 1.69x |
| APP-1006 | 22.31, 23.76, 33.67 | 11.35 s | 1.51x |
| APP-1012 | 36.58, 38.22, 42.72 | 6.14 s | 1.17x |
| APP-1008 | 25.50, 28.46, 31.45 | 5.94 s | 1.23x |
| APP-1013 | 34.01, 35.19, 39.77 | 5.75 s | 1.17x |
| APP-1005 | 42.01, 43.91, 46.55 | 4.54 s | 1.11x |
| APP-1004 | 36.25, 37.79, 39.23 | 2.98 s | 1.08x |
| APP-1003 | 31.68, 34.30, 34.41 | 2.73 s | 1.09x |
| APP-1009 | 23.84, 25.67, 25.80 | 1.95 s | 1.08x |
| APP-1011 | 21.22, 22.40, 22.79 | 1.57 s | 1.07x |
| APP-1002 | 22.32, 22.92, 23.18 | 0.86 s | 1.04x |
| APP-1010 | 27.96, 28.38, 28.59 | **0.64 s** | 1.02x |

Twelve of the fourteen sit within 17 s of themselves; two sit within 51 s of themselves. So the distribution is not "some applications are slow" — it is "any application can draw a slow call."

Repetition index is a wall-clock window, not a warm-up state. Per-repetition means ran 30.29 s / **37.62 s** / 30.50 s, and both over-target runs landed in repetition 2, but quoting the artifact, *"each agent is a fresh Strands Agent on a shared botocore client, so there is no per-agent cache to warm."* Repetition 2 is simply the window the two stalls happened to fall in, and two events cannot establish a time-of-day effect either. The study does not claim one.

# Latency is linear in output tokens — except for two calls

Pooled regression of wall clock against output tokens:

| Fit | n | r | $r^2$ | ms per output token | Fixed overhead | Marginal rate |
|---|---|---|---|---|---|---|
| All calls | 334 | 0.8514 | 0.7249 | 11.2419 | 3.40 s | 89.0 tok/s |
| **Excluding the 2 stalls** | **332** | **0.9763** | **0.9531** | 11.3379 | 2.98 s | 88.2 tok/s |

Removing two calls out of 334 lifts $r^2$ from 0.72 to **0.95**. So `wall_ms = 2980 + 11.34 × output_tokens` describes 332 calls, and two calls obey a different mechanism entirely.

## The two stalls

| Application | Rep | Agent | Wall | Model-reported latency | Unaccounted | Output tokens | tok/s | stop_reason |
|---|---|---|---|---|---|---|---|---|
| APP-1007 | 2 | bustout | 69.78 s | 69.45 s | 0.33 s | 1,120 | **16.1** | `end_turn` |
| APP-1001 | 2 | income | 66.01 s | 65.01 s | 1.00 s | 306 | **4.7** | `end_turn` |

Against the healthy population, n=332 calls under the 60 s threshold:

| Output tok/s | n | min | p50 | p95 | p99 | max | mean |
|---|---|---|---|---|---|---|---|
| Healthy calls | 332 | 20.93 | **65.50** | 86.63 | **89.78** | 95.88 | 65.93 |

That `p99 = 89.78 tok/s` is the **one p99 in the whole engagement that clears its minimum** — n=332 exceeds the 100-sample requirement, and its `suppressed` map is empty. Every other percentile on this site sits at n=42 or below on the p99 axis and carries a refusal

instead, which is worth naming because it shows the policy is a threshold rather than a blanket refusal.

Sharper of the two is `income`: **306 output tokens in 66 seconds**. At the fitted marginal rate of 88.2 tok/s those tokens should have taken about 6.5 s including overhead, and `income`'s p50 in the same benchmark is 8.09 s. It ran **14x** slower than the median call in its own population while writing a *below-median* amount of text.

Diagnosis rule, stated in the artifact before being applied: *"A stall whose tok/s is an order of magnitude below the healthy median, whose unaccounted time is near zero and whose stop_reason is end_turn was slow INSIDE the model. It is not a retry, not a truncation, and not verbosity — it is server-side generation speed, which the client cannot configure away."*

## Verbosity is real, and it is a different problem

Verbosity does not explain the tail. It does explain the floor.

| | Domains | n calls | Output tok p50 | Output tok max | Analysis chars p50 | Over 4,000 chars |
|---|---|---|---|---|---|---|
| Prompt caps at 4,000 chars | identity, dealer, straw, employment, income, synthetic | 251 | **381** | 3,582 | 1,224 | 9 (3.6%) |
| **No cap in prompt** | **bustout, rings** | 83 | **1,331** | 3,257 | 3,519 | **25 (30.1%)** |

3.5x the output tokens. Latency is linear in output tokens at $r^2=0.9531$, so those two agents buy the floor one token at a time. They are also the two whose prompts carry no character cap at all.

Per-domain fits show *where* the two stalls live:

| Agent | Prompt cap | n | $r^2$ all | $r^2$ excl. stalls | ms/output token | Fixed overhead |
|---|---|---|---|---|---|---|
| identity | 4000 | 42 | 0.9834 | 0.9834 | 10.34 | 4.20 s |
| dealer | 4000 | 42 | 0.9107 | 0.9107 | 10.37 | 3.70 s |
| straw | 4000 | 42 | 0.8439 | 0.8439 | 10.08 | 2.48 s |
| employment | 4000 | 42 | 0.9669 | 0.9669 | 10.71 | 3.17 s |
| **income** | 4000 | 42 | **0.3855** | **0.9619** | 10.63 | 3.18 s |
| synthetic | 4000 | 41 | 0.8931 | 0.8931 | 12.90 | 2.48 s |
| **bustout** | **NONE** | 42 | **0.3077** | **0.9810** | 10.83 | 2.16 s |
| rings | **NONE** | 41 | 0.8437 | 0.8437 | 13.80 | 1.10 s |

Only `income` and `bustout` move at all when the stalls are removed — 0.39 → 0.96 and 0.31 → 0.98 — because they are the two agents that stalled. One outlier in 42 destroys the fit for that agent and leaves the other six untouched, which is a much stronger form of "two events, one mechanism" than a pooled number.

Note also that a character cap and a `max_tokens` cap are not the same constraint. Measured chars-per-output-token medians: dealer 4.51, straw 4.22, synthetic 3.10, employment 3.03, rings 2.81, income 2.72, bustout 2.67, **identity 1.55** — with an observed minimum of **0.33** on an identity call that spent 3,582 output tokens to produce 1,191 characters of analysis. Quoting the artifact: *"Where this ratio collapses, the model is billed for output the analysis text does not contain, so a cap chosen from the character contract truncates earlier than the contract implies."* A `max_tokens` derived from "4,000 characters" is a different and unpredictably tighter rule.

## Not retries, and not throttling

This is the strongest single piece of evidence, because it removes the explanation everyone reaches for first.

| | Value |
|---|---|
| Retry strategy in force | `max_attempts=4`, `initial_delay=1`, `max_delay=8` → ladder **[1, 2, 4] s**, worst case +7 s |
| What it retries on | `ModelThrottledException` **only** (`strands.event_loop._retry` 1.26.0) |
| `ThrottlingException` / `ModelThrottledException` recorded on 336 calls | **0** |
| Failure kinds actually recorded | `synthetic: server_5xx` ×1, `rings: server_5xx` ×1 |
| Synthesis repair attempts | **0** |
| `stop_reason` distribution | `end_turn` 334, `None` 2 (the two 5xx) |

Wall clock minus the model's own `latencyMs`, over the 334 calls reporting both clocks, then bucketed against the backoff ladder:

| n | min | p50 | p95 | p99 | max | mean |
|---|---|---|---|---|---|---|
| 334 | 0.16 s | **0.73 s** | 1.69 s | 2.59 s | 7.66 s | 0.81 s |

| Bucket | n calls |
|---|---|
| Below the first backoff step — **cannot** be a retry | 246 |
| One step, consistent with at most one retry | 85 |
| Two steps or more | 3 |
| At or above the full ladder | 1 |

> ✕ **Verdict, quoted in full**

> *""RULED OUT as the cause of the tail. No ThrottlingException was recorded on any of 336 calls, and ModelThrottledException is the only exception this strategy retries. The two stalled calls carry 1.00s, 0.33s of unaccounted wall clock; even attributing ALL of it to backoff explains at most 1.8% of their excess over the median call. The tail is inside the model's own reported latency, not in the client.""*
>
> That 1.8% is the load-bearing number. Not "we found no evidence of retries" but "even if every unaccounted millisecond were backoff, it explains 1.8% of the problem." Connection-pool queueing falls with it, since queueing would also appear as unaccounted client-side wall clock.

A 5xx has the opposite shape, and this sample contains one. The degraded run, APP-1011 rep 3, lost both `synthetic` and `rings` to `InternalServerException` and finished in **21.2 s**, among the quickest in the sample; the two failed calls spent 8.57 s and 8.02 s before giving up. So a 5xx **fails fast** and costs a fraud dimension rather than time, while a stall costs time and loses nothing. Conflating those two signatures is how a tail investigation goes wrong.

## Floor: fan-out cannot beat its slowest agent

| Layer | n | p50 | p95 | max |
|---|---|---|---|---|
| Fan-out floor (slowest agent's own wall clock) | 42 | **24.03 s** | 41.04 s | 69.78 s |
| Orchestration above the floor | 42 | **0.83 s** | 1.87 s | 2.71 s |
| Synthesis + validation | 42 | 4.20 s | 5.59 s | 15.21 s |

| Share | Median |
|---|---|
| Floor as a share of fan-out | **96.3%** |
| Fan-out as a share of end-to-end | 85.6% |
| Post-fan-out as a share of end-to-end | 14.4% |

Quoting the reading: *"Fan-out cannot return before its slowest member, so the floor distribution IS the fan-out latency distribution up to ~1s of gathering. Any fix that does not lower the slowest agent lowers nothing."* Orchestration costs 0.83 s at the median and

never more than 2.71 s across 42 runs, so a perfect orchestration layer buys under a second.

Slowest minus second-slowest, per run, bounds any single-agent fix — the most it can remove before another agent becomes the floor:

| n | min | p50 | p95 | max | mean |
|---|---|---|---|---|---|
| 42 | 0.014 s | **6.34 s** | 20.56 s | 50.13 s | 8.53 s |

Making the critical-path agent instantaneous recovers 6.34 s at the median. Detail: Critical path.

## max_tokens sweep

"The floor is bought one output token at a time" invites capping output tokens. That was tested rather than assumed: **30 live Bedrock calls** in us-east-1 on 2026-07-28, caps 1024 / 1536 / 2048 / 3072 / 4096 across bustout and rings on APP-1004, APP-1012 and APP-1013. **$0.627827 measured** on the 16 cells that returned an analysis, plus a bounded **$0.437793** on the 14 that truncated. The control is cap 4096 — the shipped value for these tiers — **re-run in the same session**, so a cap delta is measured against a same-session control rather than against a run from another day.

Gate stated before the sweep ran: *"A cap is only a fix if it (a) lowers wall clock, (b) truncates nothing, and (c) leaves every specialist's risk band unchanged. Failing (b) or (c) makes a latency win a lost fraud dimension."*

| Criterion | Measured result | Pass/fail |
|---|---|---|
| **(a)** Lowers wall clock | 10 comparable cells: 5 faster, 5 slower. Delta min −10.39 s / p50 −1.52 s / mean **−0.23 s** / max **+17.84 s** | FAILED |
| **(b)** Truncates nothing | **14 of 30 cells truncated.** 263.2 s of wall clock burned for no analysis (mean 18.8 s each, max 29.0 s) | FAILED |
| **(c)** Leaves every band unchanged | **13 of 30** agreed with the same-session control band. 17 disagreed, of which 14 lost the band entirely | FAILED |

Only cells that returned an analysis at **both** their cap and the control can support a latency claim, and the 14 truncated cells are excluded from criterion (a) for that reason. Quoting: *"A cell that truncated finished early because it stopped, not because it was faster, and counting it would turn a lost fraud dimension into a latency win."* Including them would have produced a headline like "lower caps are 25% faster," which would have been arithmetic on nothing.

Mechanism behind criterion (a) is visible in the cell table: at a given cap the model writes what it wants to write, and the cap either does not bind or kills the call. **It never produces a shorter analysis.**

**A truncated call is the worst outcome available.** Truncation here is `MaxTokensReachedException` from Strands, which **fails the whole specialist** rather than shortening it. Quoting the artifact: *"The output is not a briefer analysis; the dimension is gone and fan-out degrades to seven of eight."* Full wall clock, fully billed, text discarded, fraud dimension lost — so **a cap low enough to shorten these agents is a cap low enough to delete them.**

That $0.437793 basis is stated per cell rather than estimated: output tokens assumed equal to the cap, which is what raised the exception, and input tokens taken from that same cell's completed sibling at another cap. It is labelled *bounded*, not measured, because a truncated call reports no usage.

## The failure boundary is not monotonic

| Agent | Caps tested | Truncated at | Completed at | Lowest cap where every cell completed |
|---|---|---|---|---|
| bustout | 1024, 1536, 2048, 3072, 4096 | **1024, 1536, 2048** | 1536, 2048, 3072, 4096 | **3072** |
| rings | 1024, 1536, 2048, 3072, 4096 | **1024, 1536, 2048** | 1536, 2048, 3072, 4096 | **3072** |

Note the overlap: both agents **both truncated and completed** at 1536 and at 2048, depending on the application and the draw. One cell — APP-1012 rings — completed at 1536 and then **truncated at 2048**, a higher cap. No safe cap below the shipped 4096 exists for these two agents on these three applications, and 3072 is only "lowest that completed every cell" at one draw each, which is not enough to call it safe.

Three of the 17 band disagreements were **not** truncations and still moved a fraud band:

| Cell | Control band | At the cap |
|---|---|---|
| APP-1012 bustout @ 3072 | `HIGH RISK` | `POSSIBLE RISK` |
| APP-1012 rings @ 1536 | `POSSIBLE RISK` | `HIGH RISK` |
| APP-1013 rings @ 3072 | `HIGH RISK` | `POSSIBLE RISK` |

Whether the cap **caused** those moves is **NOT ESTABLISHED**, and the artifact refuses to claim it: *"whether those band moves are caused by the cap or are the same run-to-run instability the 42-run benchmark already records on APP-1005 and APP-1012. One draw per cell cannot separate the two, and this sweep does not claim to."* The source benchmark independently records band instability on APP-1005 and APP-1012 with no cap change at all, so the honest statement is "evidence of instability at that cap."

## Which caps would bind on observed traffic

Separately from the live sweep, a census of the 334 successful pipeline calls — which ones already wrote more than a given cap, i.e. which would have raised

`MaxTokensReachedException`:

| max_tokens | Calls already over it | % | Domains affected |
|---|---|---|---|
| 1024 | 105 / 334 | **31.4%** | rings 41, bustout 24, identity 14, income 13, employment 5, synthetic 5, dealer 3 |
| 1536 | 45 / 334 | 13.5% | rings 18, income 9, bustout 8, identity 7, employment 2, synthetic 1 |
| 2048 | 21 / 334 | 6.3% | identity 7, income 5, bustout 5, rings 4 |
| 3072 | 3 / 334 | 0.9% | identity 2, bustout 1 |
| **4096** | **0 / 334** | **0.0%** | — |

Shipped caps by tier: light 3072, mid 4096, heavy 4096. Census labelling is precise: *"MEASURED: which observed calls already wrote more than the cap... NOT MEASURED here: the latency or cost of running with the cap, because a cap changes what the model writes."* At 1024 the binding is not confined to the two verbose agents — **seven of eight** domains are affected, `dealer` on Haiku among them.

## Two mechanisms, one conclusion

| | Floor | Tail |
|---|---|---|
| Cause | Output length. $r^2$=0.9531 over n=332, so wall clock is linear in output tokens | Server-side generation speed: 2 of 336 calls (0.6%) at 16.1 and 4.7 tok/s |
| Evidence | bustout and rings write 1,331 output tokens p50 against 381 for the six capped agents; rings owned the critical path in 25 of 42 runs | Inside the model's own reported latency. 0 throttles, no retry, `stop_reason=end_turn`. 61.0% of e2e variance is within-application |
| Right instrument | The **customer's own prompt text** — the "max 4000 characters" sentence bustout and rings are missing | None available. Quoted: *"This is irreducible run-to-run Bedrock variance and no client-side change removes it"* |
| Wrong instrument | `max_tokens`, which does not shorten the analysis but deletes it — measured above, failed all three criteria | — |
| Whose call | Hers. A one-line change to two verbatim customer artifacts, **asked as a question** | — |

**NOT MEASURED:** whether adding the missing character-cap sentence shortens the output. This work did not modify the customer's verbatim prompts.

## What this analysis did not establish

Refusals, quoted from the artifact's own closing section. This list is the reason the rest of the page can be trusted.

- **Why** a Bedrock call generates at 4.7 tok/s. *"Only that it does, that it did so on 0.6% of 336 calls, and that it is not throttling, not retries and not output length."*
- **Whether the stall rate is stable.** *"Two events over 336 calls cannot support a rate; the 0.6% figure is a count divided by a denominator, not a predicted frequency."*
- Whether adding the missing character cap to those prompts shortens their output.
- Whether the band moves in the sweep were **caused** by the cap.

- **Any p99** on the end-to-end axis. *"n=42 runs and n=42 fan-outs; the p99 stays suppressed, which matters here precisely because the tail is the question — a 0.6%-per-call event is exactly what a sample this size cannot pin down."*
- AgentCore runtime overhead, signal generation, and concurrent throughput, all of which the source benchmark also excludes.

Most important of those is the second one. A rare-event question needs a rare-event sample, and n=42 runs is not one, so the honest position is that **the tail is characterised, not bounded**. We can say what caused these two events and what did not. We cannot say how often it happens, and this site does not.

## Mitigation actually available

Not a code change: run selection plus a floor reduction.

| Step | What it is | Status |
|---|---|---|
| Demo on a measured-stable application | APP-1010 (0.64 s spread), APP-1002 (0.86 s), APP-1011 (1.57 s), each over 3 repetitions of identical input — against APP-1007 and APP-1001, whose spreads exceed 51 s | Available now. n=3 makes it a low-variance **observation**, not a guarantee: the stall rate is 0.6% **per call** and a run makes eight |
| Move `rings` off Opus 5 | rings owned the critical path in 25 of 42 runs and the floor is 96.3% of fan-out, so this is the one change that moves the median. `evals/results/tier_quality.json`, n=5 applications per cell: rings p50 **22.05 s on Sonnet 5 against 32.81 s on Opus 5**, at 2,026 against 2,448 output tokens p50, both reaching the pinned band 4 of 5 with the same one judge-confirmed false positive | **Applied** to `agents/models.py`. End-to-end effect **NOT MEASURED** — the 42-run benchmark predates it. Tier re-derivation |
| Lower `max_tokens` | Measured above | **Rejected.** 14 of 30 cells truncated, non-monotonically, with no latency gain |

One caution on the rings figure: `evals/results/tail_analysis.md` also quotes a smaller separate draw — *"n=3 applications, one draw each"* — at 16.22 s versus 32.81 s, but that draw's source is `/tmp/bench/tiering.json`, a scratch file that is not a committed artifact. Quote the 22.05 s figure.

# Reproducing this

```
# re-derive the whole analysis from the committed benchmark, no model calls
python -m evals.tail_analysis --source evals/results/pipeline_14x3_us-east-1.json \
  --json evals/results/tail_analysis.json --markdown evals/results/tail_analysis.md

# the live cap sweep (this one spends money)
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.tail_analysis --live \
  --caps 1024,1536,2048,3072,4096 --domains bustout,rings \
  --applications APP-1004,APP-1012,APP-1013

MOCK_MODE=1 python3 -m pytest tests/test_tail_analysis.py -q     # 50 tests
```

The sweep's provenance string in the artifact is that exact command line, and it records that the committed summary was *"re-summarised here through the same functions without re-calling any model; the cell measurements are unchanged."*

**Critical path**
Who owns the floor, the 6.34 s ceiling on fixing it, and which fixtures to demo on.

**Pipeline results**
The 42-run source benchmark, row by row.

**Output-contract defects**
The two prompts with no character cap, and the titles every Claude model invents.

‹ Pipeline results
All 42 measured runs individually — per-sp...

Critical path ›
rings owned the critical path in 25 of 42 run...

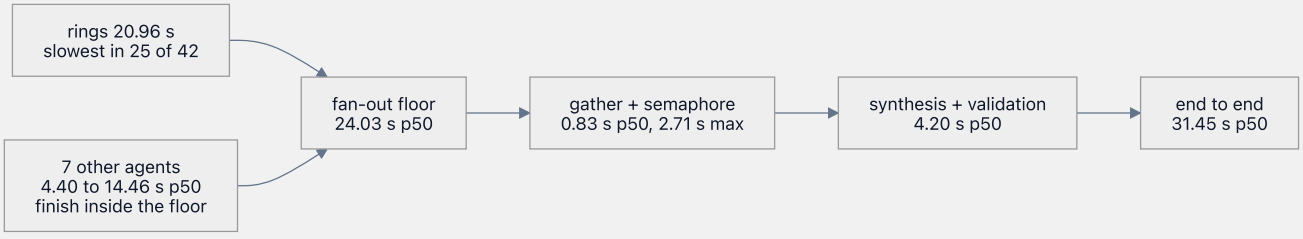
# Critical path

rings owned the critical path in 25 of 42 runs, the floor is 96.3% of fan-out, and the measured ceiling on any single-agent fix is 6.34 s at p50 — plus which fixtures to demo on.

*For the customer's engineer and for whoever plans the demo. Every figure comes from* `evals/results/tail_analysis.json` *(question 4) and* `evals/results/pipeline_14x3_us-east-1.json`*, n=42 runs / 336 specialist calls.*

**One agent,** `rings`**, was the slowest member of its fan-out in 25 of 42 runs, and the slowest member accounts for 96.3% of fan-out at the median.** Making that agent instantaneous would buy **6.34 s** at p50 and nothing at all in the two over-target runs. Both of those numbers are measured, and together they say where a latency fix can and cannot land.

## Where an end-to-end second actually goes




Fan-out timeline, p50 figures, n=42

| Layer | n | p50 | p95 | max | Median share of e2e |
|---|---|---|---|---|---|
| **Fan-out floor** (slowest agent's own wall clock) | 42 | **24.03 s** | 41.04 s | 69.78 s | — |
| Orchestration above the floor | 42 | **0.83 s** | 1.87 s | 2.71 s | — |
| Fan-out total | 42 | 24.93 s | 41.05 s | 70.62 s | **85.6%** |
| Synthesis + validation | 42 | 4.20 s | 5.59 s | 15.21 s | 14.4% |
| Floor as a share of fan-out | 42 | **96.3%** | — | — | — |

Quoting the artifact's reading: *"Fan-out cannot return before its slowest member, so the floor distribution IS the fan-out latency distribution up to ~1s of gathering. Any fix that does not lower the slowest agent lowers nothing."*

Nothing is available to win in the orchestration layer. 0.83 s at the median and **never more than 2.71 s** across 42 runs, covering `asyncio.gather` scheduling, the admission semaphore and result collection for eight concurrent Bedrock streams. Stating that explicitly matters because "optimise the orchestration" is the first instinct on a multi-agent latency problem, and here it is measurably the wrong place to look.

## Who owns the critical path

Slowest specialist in each of the 42 runs:

| Specialist | Model in this run | Runs where it was slowest | Its own p50 | Its own max |
|---|---|---|---|---|
| `rings` | Opus 5 | **25** (59.5%) | **20.96 s** | 36.17 s |
| `bustout` | Sonnet 5 | 7 | 14.46 s | **69.78 s** |
| `income` | Sonnet 5 | 5 | 8.09 s | **66.01 s** |
| `identity` | Sonnet 5 | 4 | 12.56 s | 41.04 s |
| `synthetic` | Opus 5 | 1 | 8.52 s | 26.70 s |
| `dealer` | Haiku 4.5 | 0 | 6.56 s | 18.85 s |
| `straw` | Haiku 4.5 | 0 | 4.40 s | 12.52 s |
| `employment` | Sonnet 5 | 0 | 7.48 s | 20.00 s |

Three agents **never** led, which makes `dealer`, `straw` and `employment` structurally irrelevant to latency: `employment`'s p95 of 17.64 s sits below `rings`'s p50 of 20.96 s.

`rings` leads because its floor is high, not because it spikes. Its **minimum** is 14.47 s — above `straw`'s max of 12.52 s, and above `dealer`'s p50 of 6.56 s by more than 2x — so no cheap run of rings exists. Meanwhile its max of 36.17 s is the **lowest max** of the five agents that ever led. It is the most consistently slow agent and the least explosive one.

**The agent that sets the median and the agent that produces the tail are different agents.** Split the 42 runs by speed:

| Subset | Critical-path owners |
|---|---|
| The **10 slowest** runs | bustout 3, identity 3, income 2, rings 2 |
| The **10 fastest** runs | **rings 7**, bustout 3 |

`rings` appears in the fastest ten more than three times as often as in the slowest ten, and the artifact records `same_owner_for_median_and_tail: false`. Practical consequence: **fixing rings moves the p50 and does nothing for the p95 or the max.** Both over-target runs

belonged to `bustout` (69.78 s) and `income` (66.01 s), and those were **server-side stalls, not slow agents** — two separate problems that a single "make the slow agent faster" framing merges into one.

## Ceiling on any single-agent fix

Slowest minus second-slowest, per run. This is the most any single-agent improvement can remove from fan-out before another agent becomes the floor:

| n | min | p50 | p95 | max | mean |
|---|---|---|---|---|---|
| 42 | **0.014 s** | **6.34 s** | 20.56 s | 50.13 s | 8.53 s |

Definition, quoted from the artifact: *"slowest specialist wall clock minus second slowest, per run: the most any single-agent fix can remove from fan-out before another agent becomes the floor."*

So making the critical-path agent instantaneous buys **6.34 s at p50**. Not 20 s, not "half the latency" — from a 31.45 s end-to-end p50 to roughly 25 s if nothing else changed, about a 20% reduction, and it does not touch either over-target run. Sharpest form of the point is the minimum of 0.014 s: in at least one run the two slowest agents finished **14 milliseconds apart**, so removing the leader bought nothing at all. The p95 of 20.56 s is the opposite case, the runs where one agent genuinely dominated, and those are precisely the stalled ones that no tier change prevents.

## What was done about it

`rings` moved from Opus 5 to Sonnet 5 on 2026-07-28, on measurement rather than on this latency observation alone. From `evals/results/tier_quality.json`, n=5 applications per cell, paired so only same-payload analyses are compared:

| | Opus 5 | Sonnet 5 | Verdict |
|---|---|---|---|
| Band agreement with the pinned expectation | 4 / 5 | **4 / 5** | Tie |
| Judge-confirmed false positives | 1 | **1** | Tie |
| Judge calibration mean | 4.20 | 4.00 | Opus by +0.20, below the judge's own +0.31 self-preference |
| Wall clock p50 | 32.81 s | **22.05 s** | **Sonnet, 1.49x faster** |
| $/call p50 | $0.0696 | **$0.0241** | **Sonnet, 2.9x cheaper** |
| Output tokens p50 | 2,448 | 2,026 | Sonnet writes less |

Same band, same false positive, 1.49x faster, 2.9x cheaper. Opus's only remaining edge is +0.20 judge calibration, which is **smaller than the judge's own measured +0.31 self-preference for its family** and therefore not distinguishable from bias. Full reasoning: tier re-derivation and judge methodology.

**NOT MEASURED: the end-to-end effect of that change.** The 42-run benchmark predates the move — `run.model_ids` still records rings and synthetic on Opus 5 — so no measured end-to-end p50 exists for the current assignment. Applying the 6.34 s ceiling to it would be arithmetic on a projection, and this site does not publish that. One command produces the measurement, at roughly $5 of spend and about 25 minutes of wall clock:

```
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --pipeline --repetitions 3
```

## Where the remaining floor sits

If `rings` drops to roughly its Sonnet-5 p50 of 22.05 s, the next candidates on measured p50 are `bustout` at 14.46 s and `identity` at 12.56 s. Both already sit on Sonnet 5, so no tier step remains on either. That leaves exactly two levers on the floor, and only one of them is ours:

| Lever | Evidence | Verdict |
|---|---|---|
| The **missing character cap** in the bustout and rings prompts | Those two write 1,331 output tokens p50 against 381 for the six capped agents, and latency is linear in output tokens at $r^2$=0.9531 (n=332) | **Recommended, unmeasured.** The customer's artifact to change. The ask |
| Lowering `max_tokens` | 14 of 30 live cells truncated, mean delta −0.23 s, 13 of 30 band agreement | **Measured and rejected.** The measurement |

One further lever exists that this engagement did not test. Moving the four cross-family-swept agents to GPT-5.6 Luna would drop the fan-out **floor lower bound** from **22.05 s** (rings, the swept agent that currently binds) to **8.09 s** (`income`, the fastest unswept agent that would then bind), with a projected **47.8%** cost reduction, $0.1247 → $0.0651 per adjudication. Both are **projections**, labelled as such in `docs/model-selection.md` §"the price of holding": the terms are measured — 22.05 s from `evals/results/tier_quality.json` (n=5) and 8.09 s from `evals/results/pipeline_14x3_us-east-1.json` (n=42) — but the arrangement is not, and `python -m evals.bench --pipeline` is what would confirm either. It is rejected on `_context` grounding rather than on latency, and one reason is a critical-path reason: a **partial** migration would leave `income` — which has no quality evidence at all, in either study — as the binding fan-out agent. Cross-family study.

## Which fixtures to demo on

Predicting a single live run defensibly requires an application that ran three repetitions of byte-identical input inside a narrow band. Three qualify.

| Application | Repetitions (s) | Spread | Critical path each rep | Pinned band | Observed |
|---|---|---|---|---|---|
| APP-1010 | 27.96, 28.38, 28.59 | **0.64 s** | rings, rings, rings | LOW RISK | LOW RISK, 3/3 |
| APP-1002 | 22.32, 22.92, 23.18 | **0.86 s** | rings, rings, rings | LOW RISK | LOW RISK, 3/3 |
| APP-1011 | 21.22, 22.40, 22.79 | **1.57 s** | bustout, bustout, bustout | LOW RISK | LOW RISK, 3/3 |

All three are **verdict-stable** as well as latency-stable, and all three had the **same** agent on the critical path in every repetition, which is a second independent sign of a well-behaved cell. Sources: `evals/results/tail_analysis.json` (spread) and `evals/results/pipeline_14x3_us-east-1.json` (`verdict_stability`, `raw_runs`).

Two of them also demonstrate something on substance rather than just on latency. **APP-1010** is an **anti-false-positive** fixture — eleven fired alerts, every one from a single shared household phone, pinned LOW — and it came back LOW on all three runs. That is the customer's own quality bar, *"LOW should be the most common outcome"*, being met on the hardest case for it, and it demos far better than a HIGH-risk application, which any system can flag. **APP-1011** has no co-borrower and no alerts at all, which is the `straw` prompt's "inherently minimal, 2-3 sentences" branch; it is also the run that lost two specialists to `server_5xx` on repetition 3 and **still validated all 21 keys in 21.2 s**, so if a 5xx happens live, that behaviour has been seen.

## What to avoid, and why

| Application | Repetitions (s) | Why not |
|---|---|---|
| **APP-1007** | 23.52, 34.58, **74.97** | 51.45 s spread on identical input. The 75 s run |
| **APP-1001** | 18.93, 24.25, **70.24** | 51.32 s spread. The 66 s `income` stall |
| APP-1005 | 42.01, 43.91, 46.55 | Tight, but **verdict-unstable** (HIGH / MEDIUM across reps) and the slowest stable application |
| APP-1012 | 36.58, 38.22, 42.72 | Verdict-unstable, matched its pin 1 of 3 |
| APP-1003 | 31.68, 34.30, 34.41 | Latency-stable but returned LOW on all three runs against a MEDIUM pin |

⚠ **n=3 is an observation, not a guarantee**

Stated plainly because it is the part a demo plan gets wrong: the stall rate measured in this engagement is **0.6% per call**, and one adjudication makes **eight** calls. Three clean repetitions of APP-1010 is evidence that its workload is well-behaved; it is not protection against a draw.

And that 0.6% is itself *"a count divided by a denominator, not a predicted frequency"* — two events over 336 calls cannot support a rate. If a live demo draws a stall, the correct response is the tail page: inside Bedrock's own reported latency, zero throttles, and 40 of 42 measured runs finished under target.

## Summary for a slide

| Claim | Figure | n | Artifact |
|---|---|---|---|
| One agent owns the median | `rings`, **25 of 42** runs | 42 | `tail_analysis.json` |
| Fan-out is bounded by that agent | floor is **96.3%** of fan-out (median) | 42 | same |
| Orchestration is not the problem | **0.83 s** p50, 2.71 s max | 42 | same |
| A perfect single-agent fix buys | **6.34 s** at p50 | 42 | same |
| The tail is a different problem | 2 of 336 calls, server-side, 0 throttles | 336 | same |
| The fix applied | rings Opus 5 → Sonnet 5, same band 4/5 | 5 | `tier_quality.json` |
| Its end-to-end effect | NOT MEASURED | — | — |

## Still unknown

- **End-to-end latency under the current model assignment.** Every figure here predates the rings move.
- **Whether a shorter bustout/rings prompt lowers the floor.** The customer's prompts were not modified.
- **Throughput.** One application in flight at a time, so nothing here says which agent owns the critical path when adjudications overlap.

**Latency tail**
Why the two over-target runs are not a critical-path problem.

**Model assignment**
Which model each of the nine agents runs on, and where each choice came from.

**Pipeline results**
All 42 runs, per specialist, with cost beside latency.

‹ Latency tail
Two of 336 calls generated at 16.1 and 4.7 o...

GPT-5.6 on this platform ›
Where OpenAI models are used, where the...

# GPT-5.6 on this platform

Where OpenAI models are used, where they were tested and declined, and every measurement taken on Luna, Terra and Sol — 72 blind-judged analyses, an 8-cell region matrix, and the one production component running on GPT today.

*For anyone asking "is this a Claude system or a mixed one". It is mixed, and both halves of that were decided by measurement.*

**GPT-5.6 Luna runs the master synthesizer in production. GPT-5.6 was tested for all eight specialists and declined.** Those two sentences are the whole summary, and neither was a preference: the first replaced a 55.2 s Claude synthesis with a **4.20 s p50 over 42 runs**, and the second overruled a projected **47.8% cost reduction** on one deterministic compliance measurement.

## Where GPT runs, and where it does not

Nine agents: eight on Claude, one on GPT-5.6 Luna

| Role | Model | Endpoint | Why this model | Evidence |
|---|---|---|---|---|
| 8 specialists | Claude Sonnet 5 / Haiku 4.5 | Bedrock Converse | GPT was faster and cheaper and lost on `_context` grounding | Cross-family study |
| **Master synthesizer** | **GPT-5.6 Luna** | `bedrock-mantle` | 55.2 s → 4.20 s p50 for the same verdict | Synthesizer selection |
| Second judge (eval only) | GPT-5.5 | `bedrock-mantle` | A Claude judge cannot referee a Claude-vs-GPT comparison | Judge methodology |

The judge row matters more than it looks. The cross-family comparison is scored by **two judges biased in opposite directions**, one of them OpenAI's, so a finding only counts where both agree. GPT is therefore in this repo three times: as a shipped component, as a rejected candidate, and as an instrument.

## Everything measured on GPT-5.6

| Measurement | Scale | Result | Artifact |
|---|---|---|---|
| Blind-judged specialist analyses | **72** (Luna, Terra, Sol x 4 agents x 6 applications) | Band comparable, grounding lower on 4 of 4 agents | `evals/results/set_comparison.json` |
| Synthesis in production | **42 calls, 0 failed** | **4.20 s p50**, 5.59 s p95, 3.08 s min | `evals/results/pipeline_14x3_us-east-1.json` |
| Region x variant latency | **8 cells**, n=3 each | Luna `us-east-1` fastest at 4.61 s p50 | `agents/mantle.py::MEASURED_SYNTHESIS_LATENCY` |
| Output-contract compliance | 72 analyses | **0 violations** against Claude's 34 | `agents/output_contract.py` |
| Cache-write pricing | 1 measured call | `cacheWriteInputTokens=6740` = **62% of a $0.01499 call** | `agents/mantle.py::CACHE_WRITE_MULTIPLIER` |

Detail per variant, including why price does not predict quality on this task: GPT-5.6 variants.

## What GPT-5.6 won on, undisputed

Pooled across the four swept agents, against the Claude incumbents:

| Axis | Claude | GPT-5.6 | Margin |
|---|---|---|---|
| Wall clock p50 | 18.88 s | **4.73 s** | **4.0x faster** |
| Cost per call p50 | $0.03125 | **$0.01066** | **2.9x cheaper** |
| Output-contract violations | 34 | **0** | GPT invents no titles at all |
| Judged substance | — | **higher on BOTH judges** | +0.398 / +0.684 |

The Claude judge scored GPT higher on substance — that is, **against its own family** — which is much harder to dismiss than a single-judge result. And the length difference is padding rather than lost evidence: paired on identical payloads, Claude writes 2.95x the characters while naming 1.08x the payload's signals, agreeing on band in 23 of 24 pairs. Length versus evidence.

## The one axis that decided against it

> ⚠ **A compliance measurement, not a quality opinion**
>
> Every specialist prompt requires signals to be validated against their paired `_context` columns. `grounded_fraction` counts how often that actually happens, deterministically, with **no judge involved**: GPT **0.357** (n=72) against Claude **0.642** (n=48), lower on **4 of 4** agents. On `synthetic` it falls to **0.150**.
>
> A model can land the right band without doing the validation her prompt asks for. Band agreement cannot see that; this can.

Declining it has a price, and the site states it rather than hiding it: moving the four swept agents to Luna projects **$0.0651 per adjudication against $0.1247 — a 47.8% reduction** — and would drop the fan-out floor from 22.05 s to 8.09 s. Both are labelled projections of a rejected counterfactual in `agents/models.py`. Cost levers.

## What was never tested on GPT

**Four of the eight specialists — `dealer`, `straw`, `employment`, `income` — never ran on GPT at all.** Three of the five anti-false-positive fixtures (APP-1006, APP-1007, APP-1008) target

exactly those domains, so the hardest calibration cases have no GPT evidence in either direction. `income` is also the current latency bottleneck, which makes it the first place a further sweep would pay.

Nothing here is an argument that GPT-5.6 is worse at fraud analysis. It is an argument that on the four agents measured, it complied less often with a specific instruction in the customer's own prompts, and that four agents were not measured at all.


**GPT-5.6 variants**
Luna, Terra and Sol per agent and per region — including why the price ordering does not predict the quality ordering here.

**bedrock-mantle**
The endpoint, the base-URL trap that returns a misleading 400, bearer-token auth, and the IAM action.

**Synthesizer selection**
Why the synthesizer moved to Luna and why the specialists did not follow it.

**Cross-family study**
The full 120-analysis design, both judges, and the per-agent verdict grid.

‹ Critical path
rings owned the critical path in 25 of 42 run...

GPT-5.6 variants ›
Luna, Terra and Sol measured against each ...

# GPT-5.6 variants

Luna, Terra and Sol measured against each other on the same work — per agent, per region, per dollar. On this task the price ordering does not predict the quality ordering, and the mechanism is explainable.

*For whoever has to pick a variant. Every cell is measured; nothing is inferred from the price list. n=6 per cell, which is this repo's floor for a band claim rather than a comfortable margin.*

**On this task, price does not predict quality.** Luna is the cheapest of the three and scored highest on band agreement (20 of 24 against Sol's 15); Sol costs 5x Luna's output rate and scored lowest. That is a surprising sentence, so the rest of this page is mostly the mechanism and the caveats — because "the expensive model is worse" is the kind of claim that is usually a measurement error.

## Why that is not as strange as it sounds

Three things explain it, and none of them is "Sol is a weaker model".

**1. Price here is a rate card, not a quality tier.** Sol's premium buys reasoning capacity, and this task barely uses it. The specialist prompts are explicit instruction sets — apply these numbered rules to this payload, in this format — not open problems. Output volume is nearly identical across the three (306 median output tokens for Sol against 348 for Luna), so the 5x cost is almost entirely the per-token rate:

| Variant | Output $/MTok | Median output tokens | Output-only cost | Measured median cost/call |
|---|---|---|---|---|
| `luna` | 6.60 | 348 | $0.00230 | **$0.0048** |
| `terra` | 16.50 | 256 | $0.00423 | **$0.0107** |
| `sol` | 33.00 | 306 | $0.01010 | **$0.0232** |

Sol writes **fewer** sentences than Luna (7.5 median against 10.0) and charges 5x per token for them. There is no volume story here at all.

**2. More reasoning pushes toward escalation on this task, and escalation is usually wrong here.** Nine of the fourteen `LOW RISK` fixtures in the study are anti-false-positive cases — records built to *look* bad and resolve LOW anyway. A model that reasons harder about suspicious-looking signals finds more to say about them. Sol escalated **7 of 24** analyses against Luna's 3, and over-escalated **4 of 14** `LOW RISK` fixtures against Luna's 2. Its errors are directional, not random.

**3. It is also wrong in the other direction.** On `identity`/APP-1014 (pinned `POSSIBLE RISK`) Sol declared `LOW RISK` with grounding **0.0** — it cited none of the seven signals in its view against their `_context` evidence. So this is not simple over-caution that a threshold could correct; the calibration is loose in both directions.

> ⚠ **What this does and does not license**
>
> It licenses: **do not pick a GPT-5.6 variant by price for this workload.** Test it.
>
> It does not license "Sol is a bad model" or "reasoning effort hurts". Sol is the **judge** in `evals/judge_sol.py` precisely because it measured the highest reasoning effort of the three, and judging is exactly the open-ended task that rewards it. The finding is about *fit*, on 24 analyses per variant.

## Price versus what it bought

| Variant | Input $/MTok | Output $/MTok | Synthesis p50 (us-east-1, n=3) | Band, 4 agents x 6 apps | Grounding median | Verdict |
|---|---|---|---|---|---|---|
| `openai.gpt-5.6-luna` | **1.10** | **6.60** | **4.61 s** | 4/6, 6/6, 5/6, 5/6 | 0.2860–0.5000 | **Shipped** as synthesizer |
| `openai.gpt-5.6-terra` | 2.75 | 16.50 | 6.72 s | 4/6, 4/5, 5/6, 5/6 | 0.3500–0.6250 | Best grounding of the three, 2.5x Luna's price |
| `openai.gpt-5.6-sol` | 5.50 | 33.00 | 11.19 s | 4/6, **2/6**, 4/6, 5/6 | **0.1000–0.5000** | Lowest band agreement, highest price |

Rates from `agents/mantle.py::MANTLE_RATES_PER_MTOK`; latency from `MEASURED_SYNTHESIS_LATENCY`; band and grounding from `evals/results/set_comparison.json`.

## Band agreement across all five models

Sol being last is only meaningful next to the others, so here is every model in the study over its own 24 analyses. Note that **Luna leads the field**, above both Claude incumbents, on band alone:

| Model | Correct band | Escalated | Over-escalated on a `LOW RISK` fixture |
|---|---|---|---|
| `luna` | **20 / 24** | 3 | 2 / 14 |
| `opus-5` | 19 / 24 | 4 | 3 / 14 |
| `terra` | 18 / 24 | 4 | 3 / 14 |
| `sonnet-5` | 17 / 24 | 4 | 2 / 14 |
| `sol` | **15 / 24** | **7** | **4 / 14** |

The spread from 15 to 20 over 24 analyses is **five cases**. That is a real ordering on this sample and a thin one: one differently-judged application moves any model by a full point. It is reported as a ranking on 24 analyses, not as a general capability claim — and it is *not* what the specialist decision rests on. That rests on `_context` grounding, where every GPT variant including Luna sits below both Claude incumbents. Cross-family study.

Sol's `identity` cell is the worst single result in the study and worth reading in full, because it shows all three failure directions at once over six applications:

| Application | Pinned | Sol declared | Grounding | Reading |
|---|---|---|---|---|
| APP-1004 | `HIGH RISK` | `HIGH RISK` | 0.571 | Correct, and its best-grounded analysis |
| APP-1009 | `LOW RISK` | `POSSIBLE RISK` | 0.143 | **Over-escalated an anti-FP fixture** |
| APP-1010 | `LOW RISK` | `POSSIBLE RISK` | 0.143 | **Over-escalated an anti-FP fixture** |
| APP-1012 | `POSSIBLE RISK` | `HIGH RISK` | 0.429 | Over-escalated |
| APP-1013 | `LOW RISK` | `LOW RISK` | 0.143 | Correct band, band read from a lead sentence |
| APP-1014 | `POSSIBLE RISK` | `LOW RISK` | **0.0** | **Under-escalated, citing no `_context` at all** |

Two of six correct, and the two anti-false-positive fixtures both escalated — the only cell in the study to do that. APP-1014 is the reason this is not fixable by tightening a threshold: the same model that over-escalates two benign records clears a genuinely risky one while grounding nothing.

## Per agent, all three variants

Grounding is `grounded_fraction_median` over 6 applications per cell. `DISQ` marks a cell the study disqualified on its own recorded rule.

| Agent | Variant | Band | Grounding | Contract violations | Wall p50 | $/call p50 | |
|---|---|---|---|---|---|---|---|
| `bustout` | luna | 4/6 | 0.2860 | 0 | 3.21 s | $0.0053 | |
| `bustout` | terra | 4/6 | 0.3575 | 0 | 7.90 s | $0.0103 | |
| `bustout` | sol | 4/6 | 0.2145 | 0 | 6.06 s | $0.0231 | DISQ |
| `identity` | luna | **6/6** | 0.3575 | 0 | **2.60 s** | **$0.0045** | |
| `identity` | terra | 4/5 | 0.3575 | 0 | 8.92 s | $0.0118 | DISQ |
| `identity` | sol | **2/6** | 0.1430 | 0 | 5.50 s | $0.0269 | DISQ |
| `rings` | luna | 5/6 | 0.5000 | 0 | 3.20 s | $0.0050 | |
| `rings` | terra | 5/6 | **0.6250** | 0 | 8.04 s | $0.0105 | |
| `rings` | sol | 4/6 | 0.5000 | 0 | 4.86 s | $0.0206 | DISQ |
| `synthetic` | luna | 5/6 | **0.1500** | 0 | **2.28 s** | $0.0047 | DISQ |
| `synthetic` | terra | 5/6 | 0.3500 | 0 | 7.05 s | $0.0106 | |
| `synthetic` | sol | 5/6 | 0.1000 | 0 | 2.82 s | $0.0210 | DISQ |

Two readings worth taking from that grid.

**Every GPT cell has zero contract violations.** All 72 analyses, across all three variants. The Claude incumbents recorded 34. Whatever else the comparison decided, the invented-title defect is a Claude-side problem. Output-contract defects.

**Luna's weakness is specific, not general.** It is the strongest variant on three agents and collapses on `synthetic` alone — 0.1500 grounding, less than a third of Opus's 0.517 on the same payloads. A variant-per-agent assignment is therefore arguable on this data; it is not taken because `synthetic`'s incumbent is measured and the alternative's n is 6.

# Region matters more than variant choice

Same prompt (31,343 characters), `effort=low`, n=3 per cell, measured 2026-07-28:

| Variant | Region | Total p50 | TTFT p50 | Output tok/s | Output tokens p50 |
|---|---|---|---|---|---|
| **luna** | **us-east-1** | **4.61 s** | 0.79 s | 176.0 | 812 |
| luna | us-west-2 | 5.02 s | 1.19 s | 180.0 | 903 |
| luna | us-east-2 | 6.79 s | 1.63 s | 115.8 | 878 |
| terra | us-east-2 | 6.29 s | 0.63 s | 126.4 | 825 |
| terra | us-east-1 | 6.72 s | 0.69 s | 122.3 | 785 |
| terra | us-west-2 | 7.63 s | 0.89 s | 95.1 | 790 |
| sol | us-east-1 | 11.19 s | 4.19 s | 91.1 | 1,020 |
| sol | us-east-2 | **30.66 s** | **22.07 s** | 31.6 | 900 |

Sol in `us-east-2` is **6.6x slower than Luna in** `us-east-1` on identical work. Every cell returned the same verdict (`HIGH RISK/DECLINE`), so the spread is throughput, not different answers.

There is **no cross-region inference on this endpoint** — no `global.*` or `us.*` prefix — so the region is a deployment decision, not an inference-profile one. Sol is served in `us-east-1` and `us-east-2` only. Region selection.

**A third-party probe ranks these regions differently, and it is not wrong**

A public latency dashboard put Luna `us-west-2` fastest at 531 ms and `us-east-2` at 2,703 ms — a different ordering from the table above, which measured `us-east-1` fastest.

Both are correct and they measure different things. A short-prompt probe is dominated by **time to first token**; a 31,000-character prompt producing 800+ tokens is dominated by **throughput**. Our workload is the second kind, so our own measurement governs the deployment and the probe is recorded as context rather than as a tiebreak.

## Cost accounting that differs from Converse

Two details will under-report a GPT bill if missed, both now priced and regression-tested:

| Term | Behaviour | Consequence if ignored |
|---|---|---|
| `cacheWriteInputTokens` | Billed at **1.25x** input on 5.6 (30-min cache), implicit caching **on by default** | A measured Luna synthesis reported 6,740 cache-write tokens = **$0.00927 of a $0.01499 call, 62%** |
| `input_tokens` | Responses API counts the **total** prompt including cached; Converse counts only non-cached | Double-counting cached tokens at the full rate |

GPT-5.4 and GPT-5.5 cache with **no** write charge, so their multiplier is `0.0` — recorded in `CACHE_WRITE_MULTIPLIER` rather than assumed. `rates_for()` raises on an unpriced model id instead of defaulting, so a new variant cannot enter a cost-reported path silently. What does not reduce cost.

## Still unknown

- **Four agents were never run on any GPT variant** — `dealer`, `straw`, `employment`, `income` — and with them three of the five anti-false-positive fixtures.
- **n=6 per cell** on band, and n=3 per cell on latency. A band claim needs 5 by this repo's own rule, so these are at the floor rather than comfortably above it.
- **No GPT variant was measured as a specialist in production.** Every specialist figure here comes from the study, not from a pipeline run.

‹ GPT-5.6 on this platform
Where OpenAI models are used, where the...

Model assignment ›
Which model each of the nine agents runs ...

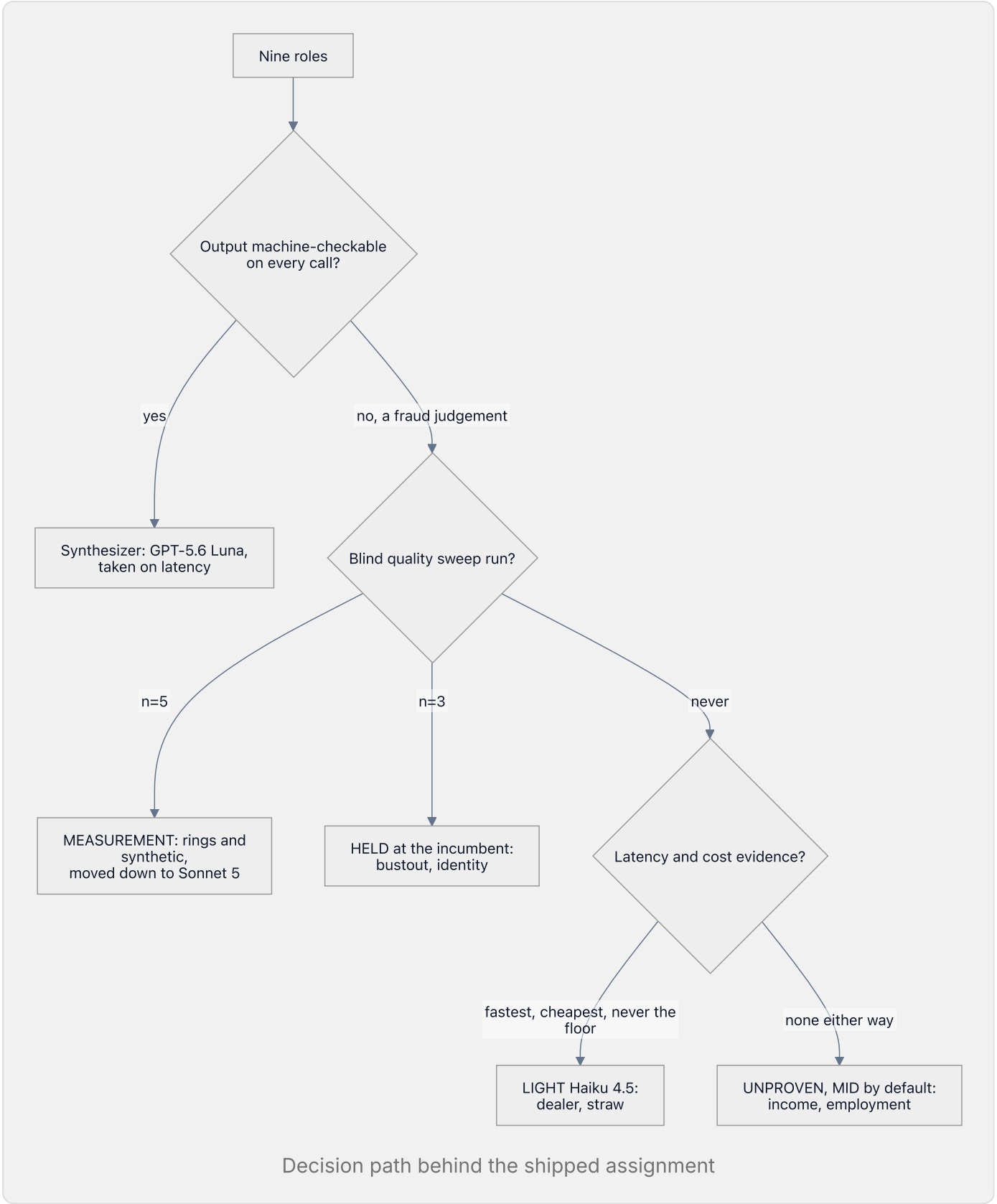
# Model assignment

Which model each of the nine agents runs on, the measurement behind each choice with its n and its artifact, and the four agents for which no quality evidence exists in either direction.

*For the customer's engineer, and for anyone who has to defend a model choice in a room. Every row names its evidence grade, including the grade that means "nothing was measured".*

Eight Claude specialists, one GPT-5.6 synthesizer, and **four of the eight specialists have never had their analysis quality compared against anything.** That last fact is the one most model-selection tables hide, so it is graded in the code (`agents/models.py:QUALITY_SWEPT_AGENTS`) rather than left to prose.

# How a role reached its model




Decision path behind the shipped assignment

## Shipped assignment

| Role | Model | Endpoint | max_tokens | Evidence grade |
|---|---|---|---|---|
| `identity` | Claude Sonnet 5 | Bedrock Converse | 4096 | **HELD** |
| `dealer` | Claude Haiku 4.5 | Bedrock Converse | 3072 | MEASUREMENT (latency/cost only) |
| `straw` | Claude Haiku 4.5 | Bedrock Converse | 3072 | MEASUREMENT (latency/cost only) |
| `employment` | Claude Sonnet 5 | Bedrock Converse | 4096 | **UNPROVEN** |
| `income` | Claude Sonnet 5 | Bedrock Converse | 4096 | **UNPROVEN** |
| `synthetic` | Claude Sonnet 5 | Bedrock Converse | 4096 | MEASUREMENT |
| `bustout` | Claude Sonnet 5 | Bedrock Converse | 4096 | **HELD** |
| `rings` | Claude Sonnet 5 | Bedrock Converse | 4096 | MEASUREMENT |
| master synthesizer | **GPT-5.6 Luna** | `bedrock-mantle` | 4096 | MEASUREMENT |

Two structural facts sit inside that table. **No agent runs on Opus 5** — `synthetic` and `rings` were there until 2026-07-28 and measurement moved both down (tier re-derivation). And **the one non-Claude model is the synthesizer**, placed there on a latency measurement rather than on price (synthesizer selection).

Source of truth in code: `agents/models.py` (`SPECIALIST_MODELS`, `TIER_RATIONALE`, `QUALITY_SWEPT_AGENTS`, `MAX_TOKENS_BY_AGENT`, `DEFAULT_SYNTHESIZER_MODEL`), asserted by

`tests/test_models.py`. Narrative version at `docs/model-selection.md`.

**The 42-run benchmark predates two of those rows.** `evals/results/pipeline_14x3_us-east-1.json` records `synthetic` and `rings` on `global.anthropic.claude-opus-5`, so the measured end-to-end describes the previous assignment and this one's end-to-end cost is a projection that labels itself one. One command converts it, at roughly $5 of spend:

```
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --pipeline --repetitions 3
```

## What the three evidence grades mean

`agents/models.py:evidence_grade` reads the grade out of the rationale text rather than re-deciding it, so a rendered table and its prose cannot disagree.

| Grade | What it asserts | Agents |
|---|---|---|
| MEASUREMENT | A measurement on this agent decided the assignment. Latency, cost, the quality verdict and n are all quoted in `TIER_RATIONALE`. | `rings`, `synthetic`, `dealer`, `straw`, synthesizer |
| HELD | A comparison ran and was deliberately not acted on, because n was too small to move a fraud dimension. The cheaper candidate and what it would save are recorded. | `bustout`, `identity` |
| UNPROVEN | No quality comparison exists for this agent at all. The entry says so instead of borrowing a neighbour's result. | `income`, `employment` |

**A latency grade is not a quality grade, and the two are tracked separately**

`dealer` and `straw` are graded **MEASUREMENT** — on n=42 latency and cost — and appear in `QUALITY_SWEPT_AGENTS` **not at all**. No blind judge has ever compared their analyses against a larger model. Conflating "we measured it" with "we measured its quality" is how a cost win gets presented as an accuracy result, so the code keeps two separate structures:

```
QUALITY_SWEPT_AGENTS = {"rings": 5, "synthetic": 5, "bustout": 3, "identity": 3
MIN_APPLICATIONS_FOR_A_BAND_CLAIM = 5
```

# Every rationale, with its n and its artifact

Three artifacts do all the work below.

| Artifact | What it holds | Measured spend |
|---|---|---|
| `evals/results/tier_quality.json` | 48 blind-judged analyses, judge Claude Opus 5, identifiers stripped and order shuffled under seed `20260728`, scored against each agent's own verbatim calibration text and the fixture's pinned band | **$4.9088** (specialists $1.3586, judge $3.5501) |
| `evals/results/pipeline_14x3_us-east-1.json` | 42 live pipeline runs, 336 specialist calls — source of every figure quoted at n=42 | **$6.4659** |
| `evals/results/set_comparison.json` | Cross-family study, 120 analyses, 240 verdicts, two judges, four agents only | **$16.8869** |

## `rings` — MID, moved down from HEAVY

| | Opus 5 | Sonnet 5 |
|---|---|---|
| Band agreement vs pinned expectation (n=5) | 4/5 | **4/5** |
| Judge-confirmed false positives | 1 | **1** |
| Judge calibration mean | 4.20 | 4.00 |
| Wall clock p50 | 32.81 s | **22.05 s** |
| $/call p50 | $0.0696 | **$0.0241** |
| Output tokens p50 | 2448 | 2026 |

Opus keeps one edge, **+0.20 judge calibration**, and the judge is Opus 5 scoring its own family. On its own 16 rows that judge scored calibration **+0.31** and substance **+0.78** versus the other 32, while the band-agreement margin it cannot influence moved **−0.026**. A +0.20 lead inside a +0.31 self-preference margin is refused as indistinguishable from bias.

Why this row is worth more than any other: `rings` was the slowest agent in **25 of 42** pipeline runs and the fan-out floor is **96.3%** of fan-out time (`evals/results/tail_analysis.json`). Ceiling on the fix, also measured — slowest-minus-second-slowest is **6.34 s p50**, so even an instantaneous `rings` leaves the next agent as the floor. Critical path.

## `synthetic` — MID, moved down from HEAVY, accepting a latency loss

| | Opus 5 | Sonnet 5 | Haiku 4.5 |
|---|---|---|---|
| Band agreement (n=5) | 4/5 | **5/5** | **5/5** |
| Judge calibration mean | 4.20 | **4.80** | 4.40 |
| False negatives | **1** | 0 | 0 |
| Output-contract violations | 5 | 5 | **8** |
| Wall clock p50 | **8.51 s** | 13.54 s | 9.56 s |
| $/call p50 | $0.0293 | $0.0194 | **$0.0059** |

Sonnet 5 is **slower** here — slower on 4 of 5 paired applications — and it is taken anyway, because Opus produced the sweep's only **false negative**: on APP-1014 it declared LOW RISK against a pinned POSSIBLE RISK (`SYN-008`) and the judge scored it "dismisses".

Two things this row proves about the table as a whole. **It is not cost-driven** — a cost-driven table takes Haiku here at 5/5 band, $0.0059 and 9.56 s, and Haiku was rejected on the customer's own output contract: 8 violations against Sonnet's 5, including citing numeric alert IDs on 3 of 5 calls, which her `synthetic` prompt forbids. And **it is the one cell with no bias caveat available**, because the margin runs *against* the judge's own family.

## `bustout` and `identity` — HELD at n=3

n=3 cannot support a band claim; one application moves the rate 33 points. On that thin sample Haiku 4.5 lost on no axis:

| Agent | Incumbent Sonnet 5 | Haiku 4.5 | Verdict |
|---|---|---|---|
| `bustout` | band 1/3, calibration 2.67, 2 judge-confirmed false positives, $0.0254, 25.31 s | band 2/3, calibration 2.67, 1 false positive, **$0.0087, 13.65 s** | **HELD** — n too small |
| `identity` | band 2/2 readable, calibration 5.00, 2 contract violations, $0.0171, 13.65 s | band 3/3, calibration 4.00, **7 contract violations**, **$0.0051, 8.04 s** | **HELD** — n too small, and a fidelity cost |

Moving either would save roughly **$0.017/application** and ~11 s of critical path. Three applications do not buy that decision. `identity`'s Haiku rows also carry 7 output-contract violations against Sonnet's 2 — alert IDs its prompt forbids, plus a risk label in a section header on all three calls — so the swap has a fidelity cost as well as a statistical one.

`bustout` is the one most worth resolving: at n=42 it is 14.46 s p50 / 35.71 s p95 at $0.01699 and was the slowest agent in **7 of 42** runs, second only to `rings`.

## `dealer` and `straw` — LIGHT, corroborated on latency and cost only

| Agent | p50 | p95 | $/call | Slowest in N of 42 | Output tokens mean |
|---|---|---|---|---|---|
| `straw` | 4.40 s | 8.76 s | $0.00352 | 0 | 224 |
| `dealer` | 6.56 s | 14.56 s | $0.00541 | 0 | 379 |

Two fastest and two cheapest agents, and neither ever set the fan-out floor. Their case is n=42 latency and cost plus the scope their own prompts define — `straw` keeps a no-co-borrower analysis to two or three sentences, which measurement corroborates at 672 chars p50. **Moving either up would need a quality argument nobody has measured. So would keeping them down, strictly speaking.**

## `income` and `employment` — UNPROVEN

Never quality-swept, within family or across families. Their rationales say exactly that instead of inferring from a neighbour.

| Agent | p50 | p95 | $/call | Slowest in N of 42 | Note |
|---|---|---|---|---|---|
| `income` | 8.09 s | **31.44 s** | $0.01537 | 5 | Owned one of the two 60 s+ tail runs (APP-1001 rep 2: 66.0 s of a 66.5 s fan-out, generating at 4.7 output tok/s against a 66 tok/s median). Second-widest p95/p50 spread of the eight. |
| `employment` | 7.48 s | 17.64 s | $0.01022 | 0 | Cheapest and fastest of the six MID agents, so a plausible LIGHT candidate on cost — which is exactly why it needs a judged sweep rather than a guess. A LIGHT move saves ~$0.005/app. |

A coverage gap sits behind that table and it matters more than it looks. Three of the customer's five anti-false-positive fixtures — **APP-1006, APP-1007, APP-1008** — target exactly `employment`, `dealer` and `income`. Those are the three agents with the least evidence, and the fixtures built to catch their specific failure mode were never run against an alternative model in either study.

### `synthesizer` — GPT-5.6 Luna on `bedrock-mantle`

Not a Claude tier, so LIGHT/MID/HEAVY do not apply. Measured at n=42 live runs: **4.20 s p50 / 5.59 s p95** at **$0.010640/call**, 715.9 output tokens mean, **0 failures and 0 repair attempts** across 42 adjudications — 14.4% of end-to-end against fan-out's 85.6%. Full derivation, including the one-run comparison that decided it: synthesizer selection.

## Projected cost of this assignment

`agents/models.py:projected_cost_per_adjudication` sums nine per-agent costs, each priced at the token counts **that model actually produced**. It is a **PROJECTION** and every return value says so.

| Role | Model | Recorded tokens in / out / cacheRead | n | $/call |
|---|---|---|---|---|
| `identity` | Sonnet 5 | 3110 / 1105 / 0 | 42 | $0.01727 |
| `dealer` | Haiku 4.5 | 3518 / 379 / 0 | 42 | $0.00541 |
| `straw` | Haiku 4.5 | 2400 / 224 / 0 | 42 | $0.00352 |
| `employment` | Sonnet 5 | 2363 / 527 / 1091 | 42 | $0.01021 |
| `income` | Sonnet 5 | 3184 / 900 / 0 | 42 | $0.01537 |
| `bustout` | Sonnet 5 | 1786 / 1316 / 1297 | 42 | $0.01699 |
| `synthetic` | **Sonnet 5** | 3793 / 1518 / 0 | **5** | $0.02276 |
| `rings` | **Sonnet 5** | 1621 / 1863 / 705 | **5** | $0.02259 |
| `synthesizer` | GPT-5.6 Luna | 2 / 716 / 0 (+4300 cache write) | 42 | $0.01063 |
| **Projected total** | | | **smallest n = 5** | **$0.12475** |

Specialists $0.11412, synthesizer $0.01063.

> **Why token counts cannot be held constant across a model swap**
>
> A different model writes a different amount. On the same five applications, `rings` on Opus wrote **1568.8** output tokens mean; on Sonnet, **1862.6**. `synthetic` on Opus wrote **581.5**; on Sonnet, **1517.8**. Pricing one model's rates against the other's token counts is fiction, so `RECORDED_USAGE_PER_CALL` and `RECORDED_USAGE_PER_CALL_PRIOR_ASSIGNMENT` each carry their own model's measured tokens.

Against the prior assignment, with both sides repriced at the same date so the Sonnet 5 promotional rate cannot be charged to one side only:

| Priced as of | This assignment (projected) | Prior assignment (projected) | Delta | What to claim |
|---|---|---|---|---|
| 2026-07-28, promo live | $0.12475 | $0.15571 | **−$0.03096 (−19.9%)** | A latency and quality decision that is also cheaper today |
| 2026-09-01, promo lapsed | $0.17734 | $0.18563 | **−$0.00829 (−4.5%)** | A latency and quality decision, full stop |

**Do not put −20% on a slide dated after 2026-09-01.** Sonnet 5's $2/$10 is introductory and reverts to $3/$15; six of the eight specialists sit on it. Repricing only the new side while holding the old side at the fixed measured constant manufactures a fake cost *increase* of ~$0.021, and `test_the_post_expiry_comparison_reprices_both_sides_not_just_one` asserts against exactly that. Cost levers.

Prior assignment's own **measured** figures, for reference: **$0.14396 p50 / $0.15616 mean**, n=41 priced runs. A projection is a sum of per-agent means, so it is comparable to the **mean**, not the p50 — and `projected_cost_per_adjudication` returns that comparison as a string field rather than leaving it to the reader. Applying the identical arithmetic to the prior assignment's own recorded tokens reproduces the benchmark's measured mean to within **$0.0005** (`test_the_same_method_reproduces_the_measured_cost_under_the_old_assignment`). The *method* is sound; only this configuration's end-to-end result is unconfirmed.

## Output caps as a property of the agent

Caps moved from `MAX_TOKENS_BY_TIER` to `MAX_TOKENS_BY_AGENT` on 2026-07-28, because under the old table moving `rings` from HEAVY to MID would silently have changed which cap it inherited — on one of only two agents ever measured to truncate.

| Agent | Max output observed | Cap | Headroom | Its prompt says |
|---|---|---|---|---|
| `straw` | 989 | 3072 | 3.1x | max 4000 chars |
| `dealer` | 1464 | 3072 | 2.1x | max 4000 chars |
| `employment` | 1647 | 4096 | 2.5x | max 4000 chars |
| `synthetic` | 2737 | 4096 | 1.5x | max 4000 chars |
| `income` | 2783 | 4096 | 1.5x | max 4000 chars |
| `rings` | 2876 | 4096 | 1.4x | **nothing** |
| `bustout` | 3257 | 4096 | 1.3x | **nothing** |
| `identity` | 3582 | 4096 | 1.1x | max 4000 chars |
| synthesizer | 1005 | 4096 | 4.1x | ~1200 chars per summary |

"Max output observed" counts every live call on disk **at the model that agent is now assigned** — the 42-run benchmark, the tier sweep's Sonnet 5 cells, and the cap sweep. Opus 5 rows are excluded on purpose: including them would size `bustout`'s cap against 3385 tokens from a model this assignment does not use.

`identity`'s 1.1x is the thinnest margin and is deliberately **not** tightened. Its 3,582-token call carried only **1,191 characters** of analysis, so it is the agent whose token usage is least predictable from its visible output. A cap computed as 4000/4 = 1024 would truncate it at a third of its own contract.

No cap was lowered, and the sweep says why: `max_tokens` was measured as a brevity instrument and **failed all three criteria** — no latency gain, 14 of 30 cells truncated, 13 of 30 kept the same band. A truncated call is the worst outcome available: full wall clock, fully billed, text discarded, fraud dimension lost. What does not reduce cost.

## Calibration failures no assignment fixed

Two failures survive every model in both families, and every model gets them wrong the same way.

| Fixture / agent | Pinned | Every model said | Requirement |
|---|---|---|---|
| APP-1014 / `rings` | `LOW RISK` | `POSSIBLE RISK` — Opus 5, Sonnet 5, Luna, Terra, Sol. **0 of 5.** | `RNG-020` |
| APP-1004 / `bustout` | `POSSIBLE RISK` | `HIGH RISK` — same five. **0 of 5.** | `BST-020` |

Judges named the same fault every time: alerts arising from **one underlying behaviour** treated as independent corroboration. That is a prompt-and-guardrail problem, not a model-selection one, and buying a bigger model does not buy the fix. Open questions.

Also untouched by any assignment: **the latency tail.** 61.0% of end-to-end variance is *within*-application, across repetitions of byte-identical input, and the two runs over 60 s were single calls generating at 16.1 and 4.7 output tok/s against a 66 tok/s median — inside Bedrock's own reported latency, zero throttles. Latency tail.

## Still unknown

Ordered by how likely the customer's engineer is to ask.

| Open item | Why it is open | What would close it |
| --- | --- | --- |
| End-to-end latency and cost of this exact assignment | Projected, not measured | `evals.bench --pipeline --repetitions 3`, ~$5 |
| `bustout` and `identity` at n≥5 | Haiku lost on no axis at n=3 and is ~3x cheaper | Extend the tier sweep; real money sits on it |
| `income` and `employment` quality | Never swept in either study, no evidence in either direction | A judged sweep including APP-1006/1007/1008 |
| `dealer` and `straw` quality | LIGHT rests on n=42 latency and cost plus their prompts' own scope | Same |
| Luna's fraud *reasoning* vs a Claude synthesizer | 42 runs validated 21 keys and 12 of 14 applications were verdict-stable, but no blind judge scored the two | `evals/evaluators/pp_calibration_judge.json` across the fixture set on both |
| Whether a grounding instruction closes GPT's `_context` gap | One prompt experiment stands between the evidence and a projected 47.8% specialist cost reduction | Cross-family study |

**Tier re-derivation**
How reading the prompts put two agents on Opus 5, and how measurement contradicted it on both.

**Cross-family study**
Claude set versus GPT set, per agent, with the verdict column and the metric that decided it.

**Cost levers**
What this assignment costs, and the three levers that survive arithmetic.

‹ GPT-5.6 variants
Luna, Terra and Sol measured against each ...

Tier re-derivation ›
Reading the customer's prompts put fraud r...

# Tier re-derivation

Reading the customer's prompts put fraud rings and synthetic identity on the largest model. Measurement contradicted it on both — and the padding finding is why.

*For the customer's engineer, and for any reviewer asking how much of this model table is reasoning and how much is measurement.*

Reasoning was wrong on both agents it touched. A first version of the per-agent model table was authored by **reading** the customer's prompts, which put `rings` and `synthetic` on Claude Opus 5; a blind sweep of 48 analyses then contradicted the reading on both, and the correction is the point of this page.

## Two moves, and what decided each

Reading said HEAVY, measurement said MID

Neither correction is subtle. On `rings` the cheaper model reached the identical band with the identical false positive; on `synthetic` the *expensive* model produced the sweep's only false negative.

**Why a wrong first guess belongs in a customer-facing document**

"This dimension is harder, so it needs the bigger model" is the reasoning a customer will apply to the next agent. It is intuitive, it is cheap to say, and it was measurably wrong twice out of two.

## What was measured

`evals/results/tier_quality.json`, generated `2026-07-28T17:29:48+00:00`, region `us-east-1`.

| | Value |
|---|---|
| Candidate models | `global.anthropic.claude-haiku-4-5-20251001-v1:0`, `global.anthropic.claude-sonnet-5`, `global.anthropic.claude-opus-5` |
| Agents swept | `rings`, `synthetic` (n=5 applications each); `bustout`, `identity` (n=3 each) |
| Judged analyses | 48 |
| Judge | `global.anthropic.claude-opus-5`, blind |
| Blinding | model/provider identifiers stripped from the analysis text; judge order shuffled under seed `20260728` |
| Measured spend | **$4.9088** — specialists $1.3586, judge $3.5501 |
| Judge calls in this report | 0 made; **48 verdicts reused from cache** across three earlier sweeps |

Each band is read **twice** — once by regex, once by the blind judge — and rows where the two readers disagree are not counted as agreement.

> ⚠ **The judge is also a candidate, and that conflict is quantified rather than waved away**
>
> Opus 5 judges, and Opus 5 is one of the three candidates. On its own 16 rows it scored substance **+0.78** and calibration **+0.31** versus the other 32, while the deterministic band-agreement margin — which it cannot influence — moved only **−0.026**.
>
> Quoted from the artifact: *"Treat any Opus-vs-cheaper margin below the substance margin as unsupported."* No conclusion on this page rests on such a margin. Judge methodology.

## `rings`: HEAVY → MID

n=5 applications per cell (APP-1004, APP-1009, APP-1012, APP-1013, APP-1014), paired so only same-payload analyses are compared.

| | Opus 5 | Sonnet 5 | Haiku 4.5 | Verdict |
|---|---|---|---|---|
| Band agreement vs pinned | 4/5 | **4/5** | **4/5** | tie |
| Judge-confirmed false positives | 1 | **1** | **1** | tie |
| Judge calibration mean / min | 4.20 / 2 | 4.00 / 2 | 3.40 / 1 | Opus +0.20, inside the bias floor |
| Judge substance | 4.0 | 4.0 | 3.0 | Haiku loses |
| Output-contract violations | 5 | 5 | 5 | tie |
| Wall clock p50 | 32.81 s | **22.05 s** | 14.65 s | Sonnet −10.76 s |
| $/call p50 | $0.0696 | **$0.0241** | $0.0081 | Sonnet −65% |
| Output tokens p50 | 2448 | 2026 | 1195 | — |

Report's own verdict for this cell: **"DEFENSIBLE on this sample: haiku-4-5-20251001-v1:0, sonnet-5"**.

Opus keeps one surviving advantage, **+0.20 judge calibration**, and +0.20 sits inside the judge's own measured **+0.31** self-preference for its family. The artifact says so in the cell itself: *"that +0.20 lead is smaller than the judge's measured +0.31 self-preference for its own family, so it is not distinguishable from bias."*

Sonnet 5 was taken rather than Haiku, which is the conservative half of the decision: Haiku lost on no band but scored 0.60 lower on calibration and 1.0 lower on substance, and this is the agent that owns the critical path.

## Why `rings` was worth more than any other row

| | Value | Source |
|---|---|---|
| Slowest agent in N of 42 runs | 25 | `pipeline_14x3_us-east-1.json` |
| Fan-out floor as a share of fan-out | 96.3% | `tail_analysis.json` |
| Slowest-minus-second-slowest p50 | 6.34 s | `tail_analysis.json` |

Fan-out cannot finish before its slowest member, and `rings` was that member in 25 of 42 runs. Row three is the ceiling on the whole idea: **even an instantaneous `rings` leaves the next agent as the floor, 6.34 s later.** A median improvement, not a tail fix. Critical path.

## `synthetic`: HEAVY → MID on quality, accepting a latency loss

| | Opus 5 | Sonnet 5 | Haiku 4.5 | Verdict |
|---|---|---|---|---|
| Band agreement (n=5) | 4/5 | **5/5** | **5/5** | Opus loses |
| Judge calibration mean / min | 4.20 / 1 | **4.80 / 4** | 4.40 / 3 | Sonnet best, against the judge's own family |
| Judge substance | 4.8 | 3.8 | 3.4 | Opus best |
| Judge-confirmed false positives | 0 | 0 | 0 | tie |
| **False negatives** | **1** | 0 | 0 | **Opus disqualified** |
| Output-contract violations | 5 | 5 | **8** | Haiku loses |
| Wall clock p50 | **8.51 s** | 13.54 s | 9.56 s | Sonnet +5.03 s, accepted |
| $/call p50 | $0.0293 | $0.0194 | **$0.0059** | — |
| Output tokens p50 | 451 | 1123 | 654 | — |

Report's verdict: **"DEFENSIBLE on this sample: haiku-4-5-20251001-v1:0, sonnet-5"**.

Deciding line sits in the band-disagreement list: `synthetic` on `opus-5`, application APP-1014, declared **LOW RISK** against a pinned **POSSIBLE RISK** (`SYN-008`). That is a false **negative** on a synthetic-identity dimension — the more dangerous error direction on a fraud product — and it is the sweep's only one. The judge scored it "dismisses".

**This row costs latency and is taken anyway.** Sonnet 5 runs 13.54 s against 8.51 s p50 and is slower on 4 of 5 paired applications, so roughly 5 s of median latency bought back a fraud dimension the faster, more expensive model missed. On a fraud product that is the right side to err on, and it is the clearest single evidence that this table is not cost-driven.

Two further observations from the same cell. **A cost-driven table takes Haiku** — it banded 5/5 at $0.0059 and 9.56 s, cheaper *and* faster than the Sonnet now assigned, and was rejected on the customer's own output contract: 8 violations against Sonnet's 5, including citing numeric alert IDs on 3 of 5 calls, which her `synthetic` prompt forbids. And **no bias caveat is available here**, because the margin runs *against* the judge's own family.

## Padding finding: extra length is not extra evidence

Mechanism behind both moves, and the finding worth keeping. Paired by (application, domain), so only analyses of the **same payload** are compared. Distinct evidence items are distinct numbers plus distinct snake_case signal names. Source: `tier_quality.json → verbosity_vs_substance`.

| Scope | Longer | Shorter | Paired | chars ratio | distinct-evidence ratio | Judge substance delta | Pairs where only the longer escalates |
|---|---|---|---|---|---|---|---|
| **ALL POOLED** | opus-5 | sonnet-5 | 16 | **1.31x** | **1.60x** | +0.0 | **0** |
| `bustout` | opus-5 | sonnet-5 | 3 | 1.98x | 1.94x | +0 | 0 |
| `identity` | opus-5 | sonnet-5 | 3 | 1.31x | 1.08x | +1 | 0 |
| `rings` | opus-5 | sonnet-5 | 5 | **1.76x** | **1.71x** | **+0** | **0** |
| `synthetic` | opus-5 | sonnet-5 | 5 | 0.89x | 1.30x | +1 | 0 |

That `rings` row is the one quoted everywhere else in this engagement: **1.76x the characters carrying 1.71x the distinct evidence, for a judge substance delta of +0.** An extra 76% of text bought 71% more evidence items and zero measured substance — length, not analysis.

Reading rule, quoted from the artifact: *"A chars ratio above the distinct-evidence ratio means the extra length is not carrying proportionally more evidence."* Pooled, Opus is 1.31x longer than Sonnet for 1.60x the evidence — so **pooled, Opus is slightly denser**, and the finding is agent-specific rather than global. Pooling four agents can cancel out the one that motivated the question, which is why the artifact reports both.

### Verbosity does not drift toward over-escalation

An obvious worry about a shorter analysis is that it under-escalates, and about a longer one that it talks itself into a higher band. Neither happened: the last column is **0 in every scope, in all three pairings** — across 48 paired comparisons there is **no** pair where the longer analysis escalated and the shorter one did not.

So the case against Opus on `rings` is cost and latency. It is **not** a false-positive risk argument, and stating it as one would be unsupported.

### Haiku is not simply "the short one"

| Scope | Longer | Shorter | Paired | chars ratio | evidence ratio | substance delta |
|---|---|---|---|---|---|---|
| ALL POOLED | opus-5 | haiku-4-5 | 16 | 1.0x | 1.17x | **+1.0** |
| ALL POOLED | **haiku-4-5** | sonnet-5 | 16 | **1.33x** | 1.32x | **−1.0** |

Haiku writes **more** characters than Sonnet 5 pooled (1.33x) and scores **1.0 lower** on substance. Length and quality are close to orthogonal on this workload, which is why neither substitutes for the other in a model decision. Length versus evidence.

## What the sweep could not settle

`bustout` and `identity` were swept at **n=3**, and the study's own bar for a band claim is **5** (`MIN_APPLICATIONS_FOR_A_BAND_CLAIM`). Both cells return the same mechanical verdict:

> *"n TOO SMALL TO TELL — 3 applications per cell. One case moves the band rate by 33 points; haiku-4-5-20251001-v1:0 did not lose on any axis here"*

That verdict is generated mechanically on purpose. Quoted from `evals/set_comparison.py`, which shares the machinery: *"the honest answer to several of these questions is 'n is too small', and that answer must not depend on the author's mood."* Both were **HELD**. Detail and the money sitting on the unrun eval: model assignment.

## Eight band disagreements, in full

Every row where a model's declared band differed from the fixture's pinned expectation:

| Agent | Model | Application | Declared | Pinned | Requirement |
|---|---|---|---|---|---|
| `bustout` | haiku-4-5 | APP-1004 | HIGH RISK | POSSIBLE RISK | `BST-020` |
| `bustout` | opus-5 | APP-1004 | HIGH RISK | POSSIBLE RISK | `BST-020` |
| `bustout` | sonnet-5 | APP-1004 | HIGH RISK | POSSIBLE RISK | `BST-020` |
| `bustout` | sonnet-5 | APP-1012 | HIGH RISK | POSSIBLE RISK | `BST-020` |
| `rings` | haiku-4-5 | APP-1014 | POSSIBLE RISK | LOW RISK | `RNG-020` |
| `rings` | opus-5 | APP-1014 | POSSIBLE RISK | LOW RISK | `RNG-020` |
| `rings` | sonnet-5 | APP-1014 | POSSIBLE RISK | LOW RISK | `RNG-020` |
| `synthetic` | opus-5 | APP-1014 | LOW RISK | POSSIBLE RISK | `SYN-008` |

**Seven of those eight are two fixtures.** APP-1004 `bustout` is wrong on all three tiers; APP-1014 `rings` is wrong on all three tiers. The cross-family study later reproduced both on GPT-5.6 Luna, Terra and Sol as well — **0 of 5 models correct on either, across two families, every one wrong the same way.** That is the signature of a shared prompt defect, not of a model tier. Open questions.

## What the re-derivation changed in the code

Beyond the two model ids:

1. `MAX_TOKENS_BY_TIER` → `MAX_TOKENS_BY_AGENT`. Under the old table `rings` drew its cap from `tier_label(model)`, so this very re-assignment would have silently changed the truncation risk of one of only two agents ever measured to truncate. Both values happen to be 4096 today, so nothing changed in effect — a future move to LIGHT would have dropped it to 3072.
   `test_per_agent_caps_preserve_the_shipped_effective_values` pins that no effective cap moved.
2. `TIER_RATIONALE` **entries now open with a grade.** `MEASUREMENT` / `HELD` / `UNPROVEN`, and `evidence_grade()` reads the grade out of the text rather than re-deciding, so a rendered table cannot disagree with the prose. An entry stating no grade returns `UNRECORDED`, treated as a defect rather than a fourth category.
3. `QUALITY_SWEPT_AGENTS` **was added**, separately from the grade, because `dealer` and `straw` are graded MEASUREMENT on latency and cost and have never had their analysis quality compared to anything.

## Still unknown

| Not done by this re-derivation | Consequence |
|---|---|
| Pipeline was not re-measured | The 42-run benchmark ran under the *previous* assignment; everything downstream of these two moves is a projection until `evals.bench --pipeline --repetitions 3` runs |
| Four of the eight agents untouched | `dealer`, `straw`, `employment` and `income` were never in this sweep |
| Two calibration failures unfixed | No tier fixes either, and neither does the other family |
| No new end-to-end p50 | Fan-out is bounded by a per-run *maximum*, which per-agent medians cannot reconstruct |

What *is* claimable is per-agent and paired: on the same five applications `rings` ran **22.05 s p50 on Sonnet 5 against 32.81 s on Opus 5**, faster on 3 of 5 paired draws, while `synthetic`

ran **13.54 s against 8.51 s** — slower on 4 of 5. Both are medians of five and no p95 is offered for either, because five samples do not support one.


**Model assignment**
The shipped table, every rationale, and the four agents with no quality evidence.

**Judge methodology**
Why a +0.20 margin inside a +0.31 self-preference is refused.

‹ Model assignment
Which model each of the nine agents runs ...

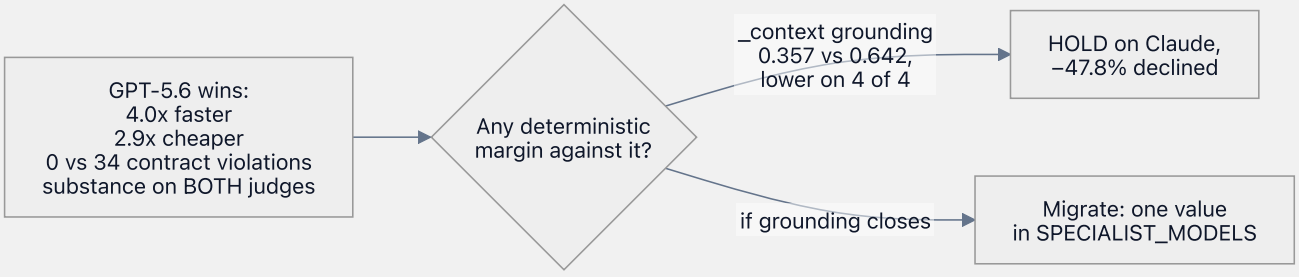
Cross-family study ›
4 agents x 6 applications x 5 models = 120 ...

# Cross-family study

4 agents x 6 applications x 5 models = 120 blind-judged analyses, 240 verdicts, two oppositely-biased judges, $16.89 measured. Latency and cost favoured GPT overwhelmingly and were overruled by one deterministic compliance measurement.

*For the customer's engineer first, and for the account team second — because the headline is a measured 47.8% projected cost reduction being declined.*

**Eight specialists stay on Claude.** GPT-5.6 measured 4x faster and 3x cheaper pooled, wrote a fifth of the words, and produced **zero** output-contract violations against Claude's 34. It was held anyway, on one deterministic measurement in which no judge takes part: `_context` grounding, **0.642 for Claude (n=48) against 0.357 for GPT (n=72)**, lower on **4 of 4** agents.




Six axes, one of them decisive

## Claude set versus GPT set, per agent

Every cell below is a field of `per_set_verdict` in `evals/results/set_comparison.json`, reproduced rather than recomputed. Grounded fraction is `grounded_fraction_median`; contract violations come from `agents.output_contract.check`; wall clock and cost are per-cell medians over 6 applications.

| Agent | Model | Set | Band | Grounded | Contract viol | Wall p50 | $/call p50 | Verdict |
|---|---|---|---|---|---|---|---|---|
| `rings` | `opus-5` | Claude | 5/6 | **0.875** | 6 | 26.22 s | $0.0583 | incumbent |
| `rings` | `luna` | GPT | 5/6 | 0.500 | **0** | **3.20 s** | **$0.0050** | **AS GOOD ON THIS SAMPLE** |
| `rings` | `terra` | GPT | 5/6 | 0.625 | **0** | 8.04 s | $0.0105 | **AS GOOD ON THIS SAMPLE** |
| `rings` | `sol` | GPT | 4/6 | 0.500 | **0** | 4.86 s | $0.0206 | NOT AS GOOD — band 4/6 below 5/6 |
| `bust out` | `sonnet-5` | Claude | 3/6 | **0.571** | 6 | 15.90 s | $0.0173 | incumbent |
| `bust out` | `luna` | GPT | 4/6 | 0.286 | **0** | **3.21 s** | **$0.0053** | **AS GOOD ON THIS SAMPLE** |
| `bust out` | `terra` | GPT | 4/6 | 0.3575 | **0** | 7.90 s | $0.0103 | **AS GOOD ON THIS SAMPLE** |
| `bust out` | `sol` | GPT | 4/6 | 0.2145 | **0** | 6.06 s | $0.0231 | NOT AS GOOD — grounding below half the incumbent's |
| `identity` | `sonnet-5` | Claude | 3/3 (3 of 6 unreadable) | **0.429** | 0 | 13.49 s | $0.0164 | incumbent |

| Agent | Model | Set | Band | Grounded | Contract viol | Wall p50 | $/call p50 | Verdict |
|---|---|---|---|---|---|---|---|---|
| `identity` | `luna` | GPT | 6/6 | 0.3575 | **0** | **2.60 s** | **$0.0045** | **AS GOOD ON THIS SAMPLE** |
| `identity` | `terra` | GPT | 4/5 | 0.3575 | **0** | 8.92 s | $0.0118 | NOT AS GOOD — band 4/5, one analysis declared no band |
| `identity` | `sol` | GPT | 2/6 | 0.143 | **0** | 5.50 s | $0.0269 | NOT AS GOOD — 2 anti-FP false positives, 1 dismissal, band 2/6, grounding |
| `synthetic` | `opus-5` | Claude | 5/6 | **0.550** | 2 | 8.59 s | $0.0318 | incumbent (1 false negative of 2) |
| `synthetic` | `terra` | GPT | 5/6 | 0.350 | **0** | 7.05 s | $0.0106 | **AS GOOD ON THIS SAMPLE** |
| `synthetic` | `luna` | GPT | 5/6 | **0.150** | **0** | **2.28 s** | **$0.0047** | NOT AS GOOD — grounding below half the incumbent's |

| Agent | Model | Set | Band | Grounded | Contract viol | Wall p50 | $/call p50 | Verdict |
|---|---|---|---|---|---|---|---|---|
| `synthetic` | `sol` | GPT | 5/6 | 0.100 | 0 | 2.82 s | $0.0210 | NOT AS GOOD — grounding below half the incumbent's |

Read that table one column at a time and the decision falls out. GPT wins **contract violations 0 against 34**, wins **latency on every row**, wins **cost on every row**, and matches or beats the incumbent's band on 9 of 12 candidate rows. It loses **grounding on 12 of 12**. Grounding is the only axis with no judge in it and the only one the customer's prompts state as a requirement, so grounding decides.

> ⚠️ **What this study did NOT find**
>
> It did **not** find that GPT-5.6 is worse at the fraud judgement. On band agreement the two families sit within one case of each other on every swept agent except `identity`/Sol — and at n=6, one case is 17 points. The hold rests on a measured compliance gap plus half the fan-out being unswept, **not** on a finding that GPT reasons less well.

## Pooled, over 120 analyses

| Axis | Claude (n=48) | GPT (n=72) | Direction |
|---|---|---|---|
| `_context` grounding, mean of per-analysis values | 0.642 | 0.357 | Lower on **4 of 4** agents |
| Wall clock p50 | 18.88 s | **4.73 s** | **4.0x faster** |
| $/call p50 | $0.03125 | **$0.01066** | **2.9x cheaper** |
| Output tokens p50 | 1422.5 | 288.0 | GPT writes a fifth of the words |
| Output-contract violations, total | **34** | **0** | deterministic, `agents.output_contract.check` |
| Output-contract clean rows | 17 / 48 | **72 / 72** | |
| Judged substance margin (GPT − Claude) | — | **+0.398 / +0.684** | **better on BOTH judges** |

Widest single cell: `rings` Luna at **3.201 s / $0.0050** against the Opus 5 incumbent's **26.2175 s / $0.0583 — 8.2x faster and 11.7x cheaper** for the same band agreement, 5/6 each.

## Design of the study

`evals/results/set_comparison.json`, generated `2026-07-28T21:32:24+00:00`, `aws_region` `us-east-1`, `mantle_region` `us-east-1`.

| | Value |
|---|---|
| Agents ( `run.domains` ) | `rings` , `synthetic` , `bustout` , `identity` — **4 of 8** |
| Applications ( `run.applications` ) | APP-1004, APP-1009, APP-1010, APP-1012, APP-1013, APP-1014 — **6** |
| Anti-false-positive fixtures in the sample | **APP-1009, APP-1010** — 2 |
| Claude set | `global.anthropic.claude-opus-5` , `global.anthropic.claude-sonnet-5` |
| GPT set | `openai.gpt-5.6-luna` , `openai.gpt-5.6-terra` , `openai.gpt-5.6-sol` |
| Cells / analyses / verdicts | 20 / **120** / **240** |
| `specialist_max_output_tokens` | 4096 |
| Shuffle seed | `20260728` |
| Judges | `global.anthropic.claude-opus-5` (claude family, **also a candidate**) and `openai.gpt-5.5` (gpt family, not a candidate) |
| Measured spend | specialists **$2.7526**, judges **$14.1343**, total **$16.8869** |
| `live_calls` in this report | 0 specialist calls, 0 judge calls — all reused from the committed sidecars |

Blinding, quoted from `run.blinding` : *"model/provider identifiers stripped from the analysis text before judging ( `evals.tier_quality.anonymise` ); judge order shuffled under the fixed seed; both judges see the same anonymised text and the same rubric."*

Judging cost **5.1x** the analyses it judged. That is what two judges at high reasoning effort over 120 analyses costs, and it is why this study exists once rather than per sprint. Sidecars `set_comparison_analyses.json` (120 analyses) and `set_comparison_verdicts.json` (240 verdicts) make the aggregate re-derivable at zero spend:

```
PYTHONPATH=. python3 -m evals.set_comparison --rescore
```

Two judges biased in opposite directions is the whole instrument, and the combination rule is on its own page: judge methodology.

## Deciding metric: `_context` grounding

Mean `grounded_fraction` per (agent, model), computed over the 120 `per_analysis` rows. **No judge is involved in any cell.**

| Agent | Opus 5 | Sonnet 5 | Luna | Terra | Sol |
|---|---|---|---|---|---|
| `rings` | 0.875 | 0.792 | 0.625 | 0.625 | 0.583 |
| `identity` | 0.738 | 0.476 | 0.357 | 0.381 | 0.238 |
| `bustout` | 0.833 | 0.524 | 0.286 | 0.310 | 0.214 |
| `synthetic` | 0.517 | 0.383 | **0.150** | 0.350 | 0.167 |
| Pooled mean | Claude **0.642** (n=48) | | GPT **0.357** (n=72) | | |

Three properties make this the deciding axis rather than one more column. GPT is lower on **4 of 4** agents against both Claude incumbents, so it is not one bad cell dragging an average. Scoring is **deterministic** — `grounded_fraction` counts how many of the signals in an agent's own domain view appear in the analysis alongside their paired `_context` evidence, and no model scores it. And every specialist prompt states the requirement: the `rings` prompt tells the model it must validate signals using the columns whose names end in `_context`. A model that meets that less often is less compliant with her contract, whatever its band agreement says.

> **Mean here, median in the verdict cells**
>
> Cells above are the **mean** of the six `per_analysis.grounding.grounded_fraction` values in each cell. Verdict rows quote `measured.grounded_fraction_median` from the same artifact, and on some cells the two differ — `identity`/Sonnet 5 is 0.476 mean against 0.429 median, `synthetic`/Opus 5 is 0.517 mean against 0.550 median. Both are in the file; neither is

recomputed into the other. The study's disqualification rule ("less than half the incumbent's") is evaluated on the **medians**, which is what the verdict strings quote.

`deterministic_cross_check`, GPT minus Claude, computed with no judge at all:

| Metric | Margin | Reading |
|---|---|---|
| Band agreement | −0.054 | inside this sample's resolution |
| Contract violations | −0.756 | GPT better |
| Grounded fraction | −0.284 | GPT worse — the only deterministic margin against it |

Quoted from the artifact: *"A judge-scored margin unaccompanied by a deterministic margin in the same direction is more likely preference than quality."* The judge-scored substance margin (+0.398 / +0.684 for GPT) has **no** deterministic margin behind it in the same direction, so grounding is the reading.

**The Claude judge scored GPT higher on substance, against its own family**

`judge_self_preference.per_judge`: `global.anthropic.claude-opus-5` scored its **own** family **+0.226** on calibration and **−0.398** on substance; `openai.gpt-5.5` scored its own family **−0.239** on calibration and **+0.684** on substance.

So on substance *both* judges preferred GPT — including the one whose family that penalises. That is much harder to dismiss than a single-judge win, and it is why "GPT writes a fifth of the words, so it must be thinner" does not survive this study.

## Verdict grid, condensed

n=6 clears the study's own bar for a band claim (`MIN_APPS_FOR_A_BAND_CLAIM` = 5), so these verdicts *are* claimable. "As good" means the candidate lost on **none** of: band agreement, escalation on an anti-false-positive fixture, dismissal of a corroborated application, or grounded fraction below half the incumbent's.

| Agent | Luna | Terra | Sol |
|---|---|---|---|
| `rings` | **as good** | **as good** | not — band 4/6 vs 5/6 |
| `bustout` | **as good** | **as good** | not — grounding |
| `identity` | **as good** | not — band 4/5, one analysis declared no band | not — 2 anti-FP false positives, 1 false negative, band 2/6 |
| `synthetic` | not — grounded 0.150 vs 0.550 | **as good** | not — grounded 0.100 vs 0.550 |

Terra is the one GPT variant graded AS GOOD on **all four** swept agents. Luna is "as good" on 3 of 4 and is still not taken — the per-agent evidence would support moving `rings`, `bustout` and `identity` to Luna today, and it is not being done because the grounding gap is real on all three and because moving three of eight leaves a split fan-out whose *remaining* agents have **less** evidence than the ones that moved.

Two incumbent rows deserve naming while the grid is on screen. `bustout`/Sonnet 5 is the **worst** band cell in the whole study at 3/6, and all three GPT variants reached 4/6. And `identity`/Sonnet 5 has a band denominator of **3, not 6** — three of its analyses had no band the regex reader and the judge agreed on, which is a fidelity defect of the incumbent's own: *"6 application(s) answered per cell, 3 with a readable band; a band claim needs 5."*

## Sol: worst variant on quality at 4x Luna's price

Worth stating plainly because the naming does not suggest it and because Sol is the variant a buyer would reach for.

| | Luna | Terra | Sol |
|---|---|---|---|
| Published in-region rate, input / cache-read / output per 1M | $1.10 / $0.11 / $6.60 | $2.75 / $0.28 / $16.50 | **$5.50 / $0.55 / $33.00** |
| Agents graded AS GOOD | 3 / 4 | **4 / 4** | **0 / 4** |
| Anti-FP false positives on `identity` | 0/2 | 0/2 | **2/2** |
| Dismissals of a corroborated application | 0 | 0 | **1 (APP-1014)** |

Sol on `identity` is this study's live demonstration of what an unswept agent could hide: it escalated **both** anti-false-positive fixtures against the incumbent's zero, dismissed a corroborated application, and banded 2/6 against Sonnet's 3/3 — at $0.0269/call, 4x Luna's price. Paying more did not buy calibration.

Sol is nevertheless the right GPT **judge** in the sibling study, precisely because it is the only 5.6 variant that is not a candidate for a specialist slot. See `evals/judge_sol.py` and judge methodology.

## Coverage gap is half the fan-out

**4 of 8 agents were swept.** `dealer`, `straw`, `employment` and `income` were never run on GPT at all. Worse for the false-positive question, quoted from the artifact's caveats: *"Only 2 of the 5 anti-false-positive fixtures (APP-1009, APP-1010) exercise the four agents swept here; APP-1006/1007/1008 target employment, dealer and income, which were not swept. The false-positive finding is therefore narrow and a wider anti-FP sweep is the obvious next measurement."*

So the three fixtures designed to catch a specific false-positive failure mode target exactly the three agents with the least evidence.

## Price of holding

A hold with an unstated price is not a decision. Both figures are **PROJECTIONS** from the measured per-agent tokens, not benchmarks.

| | Hold on Claude | Move the four swept agents to Luna | Recommendation |
|---|---|---|---|
| $/adjudication, projected at 2026-07-28 rates | $0.1247 | **$0.0651** | hold |
| Delta | — | **−$0.0596 (−47.8%)** | forgone, and stated as forgone |
| Fan-out floor lower bound | 22.05 s (`rings`) | **8.09 s (`income`)** | hold |

Two things the second column exposes. A projected **47.8%** cost reduction and roughly **14 s** of fan-out floor are being declined. And after such a move the binding agent becomes `income` — **which has no quality evidence of any kind**, so the agent with the least evidence would set the pipeline's latency. That is an argument for sweeping it, and a second argument against a partial migration.

Floor figures are **lower bounds**, not predicted end-to-ends: fan-out is bounded by a per-run *maximum*, which per-agent medians cannot reconstruct.

`agents/models.py:projected_cost_if_gpt_specialists` computes the counterfactual and labels itself `PROJECTION OF A REJECTED COUNTERFACTUAL` — a weaker object than an ordinary projection, because **no benchmark run would confirm it without changing the assignment first.** It also *understates* the GPT case deliberately: the other four agents are priced at Claude in that figure, because no measured GPT token count exists for them. A full-fan-out GPT figure would be lower and is **not offered**.

## Two calibration failures reproduce across families

`known_failure_reproduction`:

| Fixture / agent | Pinned | Opus 5 | Sonnet 5 | Luna | Terra | Sol |
|---|---|---|---|---|---|---|
| APP-1014 / `rings` (`RNG-020`) | LOW RISK | POSSIBLE | POSSIBLE | POSSIBLE | POSSIBLE | POSSIBLE |
| APP-1004 / `bustout` (`BST-020`) | POSSIBLE RISK | HIGH | HIGH | HIGH | HIGH | HIGH |

Quoted verbatim: *"GPT reproduced it 3/3; Claude 2/2. Correct band: GPT 0/3, Claude 0/2."*

This is the study's most consequential *negative* result: it upgrades the finding from "no *tier* fixes it" to **"no model choice fixes it."** Five models, two families, every one wrong the same way — a second alert firing off the same underlying behaviour treated as independent corroboration. That is the signature of a shared prompt defect. Open questions.

## Still unknown

Every caveat the artifact carries, reproduced rather than summarised, because the study's credibility is in its limits.

| Limit | Detail |
| --- | --- |
| One judge cannot settle this | Nothing is "bias-corrected" — subtracting a bias estimate would produce a number that looks adjusted and was never measured. Judge methodology |
| Recommendation rests on the deterministic layers only | Band agreement, anti-FP escalation, `_context` grounding, `agents.output_contract.check`. Judged calibration and substance are subjective, reported separately, and an axis where the judges disagree yields no finding |
| 6 applications per cell | A band-agreement difference of one case is **17 points**, so a one-case gap is inside this sample's resolution by construction |
| `signals_cited` is style-sensitive | Pooled, Claude cites **4.56** signals per analysis to GPT's **2.86**; a looser stemming matcher moves GPT to **3.60** and Claude only to **4.65**, so about **0.7 of the ~1.7 gap is paraphrasing**. The looser matcher was rejected because it cannot keep `ssn_apps_last_7d` and `ssn_apps_last_30d` distinct. Length versus evidence |
| Only 2 of 5 anti-FP fixtures exercise the swept four | APP-1006/1007/1008 target the unswept agents |
| No p95 anywhere | `evals.bench.summarize_values` refuses a p95 below 20 samples and every cell records the refusal string rather than a number |
| An unremovable asymmetry | GPT specialists receive the verbatim prompt **concatenated with the user turn** (the mantle Responses shape); Claude specialists receive it as a **cached system prompt**. That cannot be removed while GPT-5.x is off Bedrock Converse |
| Latency and cost here are single serial calls | The end-to-end consequence of a swap belongs to the pipeline benchmark |

# What would change the answer

| Experiment | Cost | What it settles |
|---|---|---|
| One prompt-engineering attempt on grounding — instruct the model to quote the paired `_context` row it validated against, then re-measure `grounded_fraction` | small | The only thing standing between this evidence and a migration |
| Extend the sweep to the other four agents, including APP-1006 / 1007 / 1008 | a second $17-scale study | Whether the anti-false-positive result covers the whole fan-out rather than half of it |

**If grounding closes, the latency and cost case is already made and this assignment should change.** Plumbing is not the blocker: `agents/fanout.py` routes a GPT specialist today (`transport_for('openai.gpt-5.6-luna') == 'bedrock-mantle'`), so reversing the decision is one value in `SPECIALIST_MODELS`. The hold is a measurement result, not a missing code path.

**Judge methodology**
Two judges biased in opposite directions, and NOT ESTABLISHED as a real outcome.

**Length versus evidence**
Whether a fifth of the words is a fifth of the analysis. Mostly not — except on bustout.

‹ Tier re-derivation
Reading the customer's prompts put fraud r...

Judge methodology ›
Two judges biased in opposite directions, s...

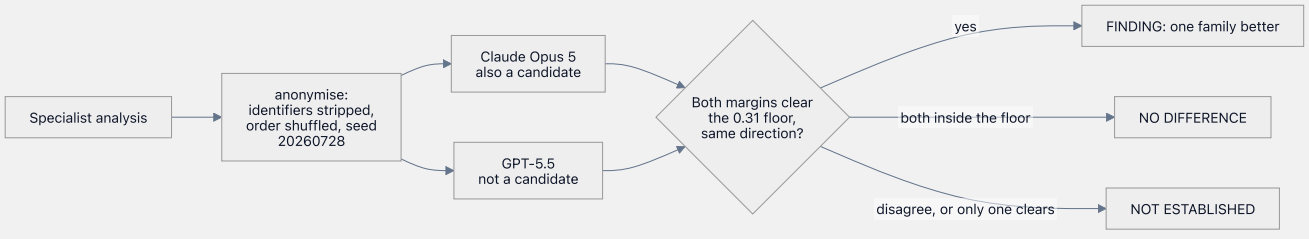
# Judge methodology

Two judges biased in opposite directions, self-preference quantified rather than assumed away, agreement as the only signal, and NOT ESTABLISHED as a real outcome the design is built to produce.

*For the customer's engineer, and for anyone deciding whether to believe the two model studies at all.*

Bias was measured, not assumed away. An LLM asked "which of these two fraud analyses is better?" has a family, a house style, and a measurable preference for text resembling its own output — so the first study made that conflict maximal on purpose, then quantified it: the judge rated its own family **+0.78** on substance while the axis it could not touch moved **−0.026**. If this page does not hold up, neither **tier re-derivation** nor **cross-family study** does.

## How a verdict is produced




One analysis, two judges, three possible outcomes

Code: `evals/tier_quality.py` (the Claude judge, the rubric, the anonymiser), `evals/set_comparison.py` (the two-judge combination rule), `evals/judge_sol.py` (the GPT judge). Artifacts: `evals/results/tier_quality.json` and `evals/results/set_comparison.json`,

with cached verdicts in `tier_quality_verdicts.json` (48) and `set_comparison_verdicts.json` (240).

## Self-preference, quantified

From `tier_quality.json → judge_self_preference`:

| | Value |
|---|---|
| Judge | `global.anthropic.claude-opus-5` |
| Its own rows | 16 |
| Other rows | 32 |
| Judge-scored calibration margin on its own family | +0.31 |
| Judge-scored substance margin on its own family | +0.78 |
| Deterministic band-agreement margin (the judge cannot influence this) | −0.026 |
| Deterministic contract-violation margin | −0.12 |

Reading, quoted from the artifact:

> *"On its own 16 rows the judge scored substance +0.78 and calibration +0.31 versus the other 32, while band agreement moved −0.026. Treat any Opus-vs-cheaper margin below the substance margin as unsupported."*

**What that pair of numbers actually shows**

A judge rated its own family nearly a **full point** higher on substance while the axis it could not touch — whether the declared band matched the fixture's pinned expectation — moved **−0.026**, i.e. essentially nothing, and slightly *against* its own family.

So the judge-scored margin was mostly **preference**, not quality. That single comparison is the whole justification for everything else on this page.

It is also what killed the last remaining argument for keeping `rings` on Opus 5. Opus's only surviving edge there was **+0.20** judge calibration, which sits **inside** the judge's own measured +0.31 self-preference and is therefore not distinguishable from bias. Detail.

## Two judges, biased in opposite directions

A harder question follows in the cross-family study — is the GPT set as good? — where a single judge from either family would decide the answer by its own identity. So every one of the 120 analyses is judged **twice**.

| Judge | Family | Also a candidate? | Why that choice | Effort | Transport |
|---|---|---|---|---|---|
| `global.anthropic.claude-opus-5` | claude | **Yes** | The strongest possible form of the conflict: if a finding survives a judge scoring its own output, it survives | Strands structured output (forced tool schema) | Bedrock Converse |
| `openai.gpt-5.5` | gpt | **No** | Carries the mirror bias without scoring a sibling it might be deployed beside — the candidates are the 5.6 variants | `medium` reasoning effort | `bedrock-mantle` Responses |

Sol is picked by the same logic in `evals/judge_sol.py`, the standalone GPT judge module. Quoted from its docstring: *"Sol is deliberately the GPT judge rather than Luna or Terra: it is the only GPT-5.6 variant NOT a candidate for any specialist assignment (Luna is the leading candidate and the incumbent synthesizer, Terra is the middle candidate), so it is the one GPT model that is not scoring a sibling it might be assigned alongside. It also measured the highest reasoning effort of the three, which is what a judge should be."*

## What is held identical between the two judges

If the rubric drifted between judges, their agreement would mean nothing. So `judge_sol.py` and `set_comparison.py` import the shared pieces from `evals/tier_quality.py` rather than restating them:

```
from evals.tier_quality import (
    JUDGE_MAX_TOKENS, AnalysisRow, _JUDGE_INSTRUCTIONS,
    _judge_verdict_model, anonymise,
)
```

| Held identical | Differs |
|---|---|
| Judge instruction text | The model |
| Anonymisation | Transport (Converse vs Responses) |
| Verdict field names, enums and score anchors | How the JSON shape is enforced |
| Pinned expectation from `fixtures.loader.expected_for` | Reasoning effort |

A test asserts the two judges use the same instruction text.

> ⚠ **The one place the transport forces a difference**
>
> Strands structured output carries the Claude judge's verdict — the pydantic model becomes a forced tool schema, so shape is enforced by the API. The mantle Responses path has no equivalent here, so the GPT judge is asked for a bare JSON object and the reply is parsed and validated by `_parse_gpt_verdict` against the same ten fields.
>
> Critically: **a malformed or out-of-range verdict is an ERROR, not a clamped value.** Quoted from the code's comment on `_VERDICT_FIELDS`: *"silently clamping a score would invent agreement between the two judges, which is the one thing this design must not do."*

## Verdict schema

Ten fields, identical for both judges:

| Field | Type | What it is |
|---|---|---|
| `band_read_from_analysis` | `LOW RISK` / `POSSIBLE RISK` / `HIGH RISK` / `UNCLEAR` | Band the judge reads out of the prose. Compared against the regex reader; disagreements are not counted as agreement |
| `band_matches_expectation` | bool | Against the fixture's pinned band |
| `calibration_score` | int 1–5 | How well the band follows the customer's own calibration text |
| `false_positive_verdict` | `Pass` / `PassWithNote` / `FalsePositive` / `FalseNegative` | Against her anti-false-positive rules |
| `escalation_drift` | `aligned` / `escalates` / `dismisses` | |
| `substance_score` | int 1–5 | **Density, explicitly not length** |
| `padding_observed` | bool | |
| `cites_context_fields` | bool | Whether it validates signals against the paired `_context` |
| `contract_notes` | string | Output-contract deviations |
| `justification` | string, capped | Citation for the scores |

Anchors are stated in the prompt rather than left to the model. `substance_score`: *"1 = mostly restatement, scaffolding or repetition of the same finding; 3 = mixed; 5 = every section adds a distinct fact or a distinct inference. Judge density, NOT length: a short analysis that carries its evidence scores high, and a long one that repeats itself scores low."*

A `calibration_score` of 5 requires the model to *"explicitly weigh the benign explanation the prompt names for this signal before concluding, and treat alerts from one behaviour as one signal"* — the anchor encodes the customer's own anti-false-positive principle.

Blinding, from `run.blinding`: model and provider identifiers stripped from the analysis text before judging (`evals.tier_quality.anonymise`), judge order shuffled under the fixed seed, both judges seeing the same anonymised text and the same rubric. Shuffle seed `20260728` in both studies, so ordering is reproducible rather than merely random.

## Combination rule

`evals/set_comparison.py:two_judge_agreement`. Margins are **GPT minus Claude**; positive favours GPT. `JUDGE_MARGIN_FLOOR = 0.31`.

| Both margins | Verdict |
|---|---|
| favour GPT by more than the floor | **GPT better on both judges** |
| favour Claude by more than the floor | **CLAUDE better on both judges** |
| inside the floor | **NO DIFFERENCE — both margins inside the measured bias floor** |
| point opposite ways, or only one clears the floor | **NOT ESTABLISHED** |
| at least one judge produced no verdict | **NOT ESTABLISHED** |

0.31 is a measured number, not a confidence interval. Quoted from the code: *"It is not a confidence interval — the sample is far too small for one — it is the prior study's MEASURED self-preference on calibration (+0.31), used as the floor below which a judge's opinion cannot be distinguished from its own identity."* Note what the third row is: **"no difference" is a real finding, and the likeliest one — it is what "as good at the fraud analysis" looks like when it is true.**

## NOT ESTABLISHED is an outcome, not an error

Two of the ten axes in the cross-family study returned it, and no recommendation rests on either.

| Axis | Scope | Claude judge | GPT judge | Verdict |
|---|---|---|---|---|
| `calibration_score` | ALL POOLED | −0.226 | −0.239 | NO DIFFERENCE |
| `calibration_score` | `bustout` | +0.222 | +0.111 | NO DIFFERENCE |
| `calibration_score` | `identity` | **−0.791** | **−0.850** | **CLAUDE better on both judges** |
| `calibration_score` | `rings` | −0.083 | −0.139 | NO DIFFERENCE |
| `calibration_score` | `synthetic` | −0.361 | −0.222 | **NOT ESTABLISHED** — disagree in magnitude |
| `substance_score` | ALL POOLED | +0.398 | +0.684 | **GPT better on both judges** |
| `substance_score` | `bustout` | +0.361 | +0.861 | **GPT better on both judges** |
| `substance_score` | `identity` | +0.248 | +0.431 | **NOT ESTABLISHED** — disagree in magnitude |
| `substance_score` | `rings` | +0.528 | +0.806 | **GPT better on both judges** |
| `substance_score` | `synthetic` | +0.417 | +0.583 | **GPT better on both judges** |

Read the two NOT ESTABLISHED rows carefully, because they are the mechanism working. On `synthetic` calibration the Claude judge's −0.361 clears the floor and the GPT judge's

−0.222 does not, so the axis yields **nothing**. On `identity` substance it is the mirror: +0.431 clears, +0.248 does not. Neither is reported as a small effect, and neither is averaged into a single number.

Nothing is bias-corrected, and that is deliberate. Quoted from `JUDGE_CONFLICT_NOTE`: *"Neither margin is 'corrected' by subtracting a bias estimate: that would produce a number that looks adjusted and was never measured."* A "bias-corrected" score is a fabricated score with a plausible name on it. This design's answer to bias is redundancy, not arithmetic.

## Each judge's measured preference for its own family

From `set_comparison.json → judge_self_preference.per_judge`:

| Judge | Family | Own-family calibration margin | Own-family substance margin | Behaved as predicted? |
|---|---|---|---|---|
| `global.anthropic.claude-opus-5` | claude | +0.226 | −0.398 | On calibration yes; on substance **no** |
| `openai.gpt-5.5` | gpt | −0.239 | +0.684 | On substance yes; on calibration **no** |

GPT's analyses were rated higher by 0.398 on substance by the Claude judge. That is the most useful single number in the study, because it is a preference running the wrong way for the judge's own identity.

## Layers no judge touches

Both studies are explicit that the judges do not decide. Quoted from the caveats:

> *"The recommendation rests on the DETERMINISTIC layers — band agreement against the pinned fixture, escalation on the anti-false-positive fixtures, `_context` grounding, and `agents.output_contract.check`. The judged calibration and substance scores are*

> *subjective and are reported separately, and an axis where the two judges disagree yields no finding at all."*

| Layer | Computed by | Judge involved? |
|---|---|---|
| Band agreement vs the pinned expectation | `fixtures.loader.expected_for` + regex, cross-checked by the judge's own band read | No |
| False positives on the anti-FP fixtures | fixture metadata | No |
| False negatives | fixture metadata | No |
| `_context` grounding | `evals.tier_quality.context_grounding` | No |
| Output-contract violations | `agents.output_contract.check` | No |
| Calibration score | judge | Yes |
| Substance score | judge | Yes |
| Padding observed | judge | Yes |

Policing the boundary between those two groups is the `deterministic_cross_check` block's job: band agreement **−0.054**, contract violations **−0.756**, grounded fraction **−0.284**, all GPT minus Claude. *"A judge-scored margin unaccompanied by a deterministic margin in the same direction is more likely preference than quality."* The cross-family recommendation therefore rests on **grounding**, which has no judge in it, and not on the substance margin, which does.

## Reading the band twice

Small, and it matters. From `tier_quality.json` caveats: *"The band is read twice — by regex and by the blind judge. Rows where the two readers disagree are listed per cell and are not counted as agreement."*

That is how `identity`/Sonnet 5 ends up with a band denominator of **3, not 6** in the cross-family study: three of its analyses had no band the two readers agreed on, and the sample note says so — *"6 application(s) answered per cell, 3 with a readable band; a band claim needs 5."* A design taking the judge's read alone would have reported 6/6 and hidden a real fidelity defect.

## What judging cost

| Study | Analyses | Verdicts | Specialist spend | Judge spend | Total | Judge multiple |
|---|---|---|---|---|---|---|
| Tier quality | 48 | 48 | $1.3586 | **$3.5501** | **$4.9088** | **2.6x** |
| Cross-family | 120 | 240 | $2.7526 | **$14.1343** | **$16.8869** | **5.1x** |

That is the real price of the methodology, measured rather than estimated, and it is why verdicts are cached in sidecar files (`tier_quality_verdicts.json`, `set_comparison_verdicts.json`) keyed by `domain|model|application` so a failed judge call can be retried without re-paying the successes. Both reports regenerate from the cached verdicts at **zero** spend:

```
PYTHONPATH=. python3 -m evals.set_comparison --rescore
```

`live_calls` in the committed cross-family report records `judge_calls_made: 0` and `judge_verdicts_reused_from_cache: 240`, so the published aggregate is reproducible without calling a model.

# Still unknown

| Limit | Why two judges do not fix it |
|---|---|
| Small n | Six applications per cell in the cross-family study, three to five in the tier study. A one-case band difference is 17 points at n=6, and two judges do not add samples |
| A shared blind spot | If both judges are wrong the same way, agreement will not catch it — and there is a live example on this site: five models across two families all mis-band APP-1014 and APP-1004, and the judges named the same fault rather than disputing the band. Agreement is evidence about the *analyses*, not about the *rubric* |
| Subjectivity in the anchors | "Every section adds a distinct fact" is a human sentence. Both judges interpret it, and the interpretation is not measured |
| Transport asymmetry | The Claude judge's verdict shape is API-enforced; the GPT judge's is parsed. Both validate against the same ten fields, but the failure modes are not identical |


**Cross-family study**
Where this machinery was applied: 120 analyses, 240 verdicts, one deciding metric.

**Tier re-derivation**
Where the +0.31 floor came from, and the two moves it enabled.

‹ Cross-family study
4 agents x 6 applications x 5 models = 120 ...

Length versus evidence ›
"GPT writes a fifth of the words" was the int...

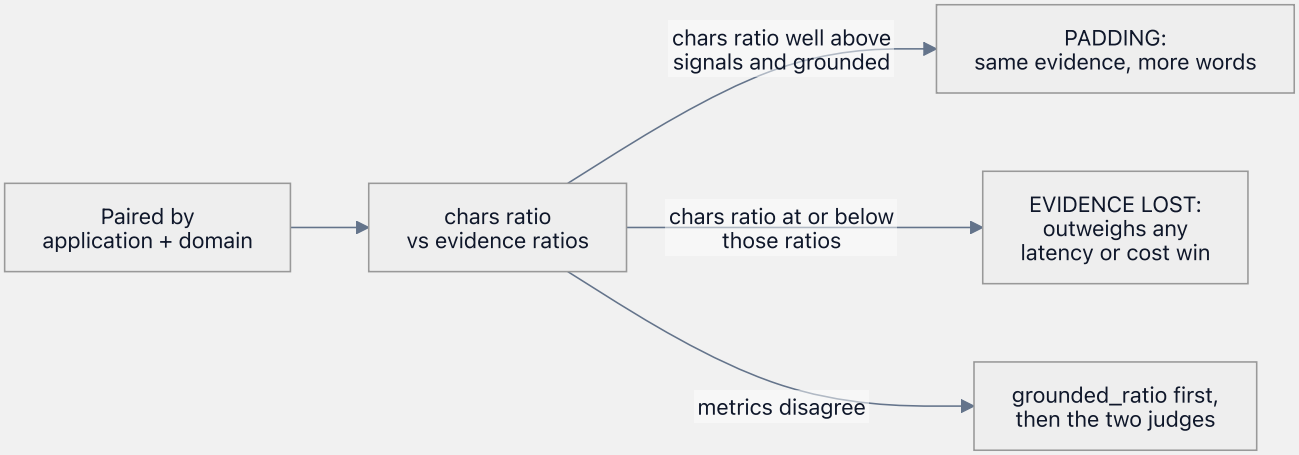
# Length versus evidence

"GPT writes a fifth of the words" was the intuitive objection to a cheaper specialist. Paired on identical payloads it is mostly padding — except on bustout, where it really is evidence lost.

*For the customer's engineer. This page tests the objection an experienced fraud reviewer raises first, and it is the one page where the answer comes out against the cheaper model on a specific agent.*

**Mostly padding, and on `bustout` a real loss.** Pooled and paired on identical payloads, the longer Claude analysis is 2.95x the length for 1.08x the signals named, reaching the same band in 23 of 24 comparisons. On `bustout` alone the shorter analysis names proportionally fewer signals than its length predicts, and that is the one agent where the objection fires.

## How the four ratios read




Which ratio decides, fixed before the numbers were read

Pairing is by **(application, domain)**, so only analyses of the *same payload* are compared and a length difference cannot be an artifact of one model drawing easier applications. Artifacts: `evals/results/set_comparison.json → length_gap` (30 paired comparisons, 6 model pairings x 5 scopes) and `evals/results/tier_quality.json → verbosity_vs_substance` (12 paired comparisons within Claude). Code: `evals/set_comparison.py:length_gap`.

| Ratio, all Claude ÷ GPT, all medians | What it measures | Weight |
|---|---|---|
| `chars_ratio` | Raw character count | The denominator for the question; says nothing alone |
| `evidence_ratio` | Distinct numeric facts plus signal identifiers | **Style-sensitive** — a backticked identifier table inflates it. Reported, not decisive |
| `signals_cited_ratio` | How many of the payload's OWN signals each names | **Also style-sensitive**, and **quantified below** |
| `grounded_ratio` | `_context` fields actually validated | **The style-free one.** `context_grounding` counts only tokens unique to the `_context` row, which cannot be produced by restating the signals block. **Decisive** |

Plus `band_justified_both`, so a shorter analysis cannot pass by declaring a band with no reason attached.

Reading rule, quoted verbatim from `length_gap.reading`:

> *"`chars_ratio` well above `signals_cited_ratio` and `grounded_ratio` means the longer (Claude) analysis is padding: same evidence, more words. `chars_ratio` at or below those ratios means the shorter (GPT) analysis really is dropping evidence, which outweighs any latency or cost win."*

Which ratio wins when they disagree was decided before the numbers were read. Quoted from the docstring: *"READ `grounded_ratio` FIRST WHERE THE TWO DISAGREE. `signals_cited_ratio` partly measures whether a model writes identifiers or prose ... When it and `grounded_ratio` point different ways, the honest reading is that the metrics disagree*

*about a paraphrase, and the judges' substance scores are the tiebreak — not whichever number happens to support the conclusion already preferred."*

## The objection, measured

Pooled over the 120 cross-family analyses:

| | Claude (n=48) | GPT (n=72) |
|---|---|---|
| Output tokens p50 | 1422.5 | 288.0 |

GPT writes roughly a fifth of the words. On a fraud product that reads as "a fifth of the analysis", and that intuition is why the study was designed to test it rather than assert either way.

Her own master prompt says the opposite. Rule `MSTR-018` in it states — in her words, kept in `prompts/verbatim/master_agent_prompt.txt` rather than reproduced here — that how much a specialist *writes* is not a signal of how severe the fraud is. That rule is why this comparison exists: she had already ruled out length as evidence, so a port picking a model for writing more would be contradicting her.

## Headline: padding, pooled and on 3 of 4 agents

`opus-5` vs `luna`, the widest pairing in the study:

| Scope | Paired | chars ratio | evidence ratio | signals-cited ratio | grounded ratio | Same band | Finding |
|---|---|---|---|---|---|---|---|
| ALL POOLED | 24 | **2.95x** | 5.08x | **1.08x** | 2.25x | **23/24** | **PADDING** |
| `bustout` | 6 | 4.62x | 4.95x | **7.0x** | 3.0x | 6/6 | **EVIDENCE LOST** |
| `identity` | 6 | 3.44x | 3.12x | 1.17x | 1.88x | 5/6 | PADDING |
| `rings` | 6 | **2.83x** | 7.75x | **1.00x** | 1.5x | 6/6 | **PADDING** |
| `synthetic` | 6 | 2.02x | 10.0x | 0.67x | 2.75x | 6/6 | length and evidence scale together |

That `rings` row is the cleanest statement of the finding: **Opus writes 2.83x Luna's characters while naming 1.00x the payload's signals — exactly the same signals — and reaches the same band in 6 of 6 paired comparisons.** An extra 183% of text names no additional signal at all.

On `synthetic` the ratio inverts: Luna names **more** of the payload's signals than Opus does (0.67x means Opus names two thirds of what Luna names), in a third of the characters.

This reproduces the within-Claude result at a smaller magnitude. On `rings`, Opus wrote **1.76x** Sonnet's characters carrying **1.71x** the distinct evidence for a judge substance delta of **+0** (n=5 paired). So verbosity-without-substance is not a cross-family artifact — it is a property of the larger model on this workload. Tier re-derivation.

## Same pairing against the actual incumbent

`rings` and `synthetic` had Opus 5 as incumbent; `bustout` and `identity` had Sonnet 5. Against Sonnet 5 the gap narrows and the shape holds:

| Scope | Paired | chars ratio | evidence ratio | signals-cited ratio | grounded ratio | Same band | Finding |
|---|---|---|---|---|---|---|---|
| ALL POOLED | 21 | 2.24x | 3.00x | 1.33x | 1.67x | 19/21 | PADDING |
| `bustout` | 6 | 2.29x | 2.55x | **4.5x** | 2.0x | 5/6 | **EVIDENCE LOST** |
| `identity` | 3 | 2.24x | 2.33x | 1.33x | 0.75x | 3/3 | PADDING |
| `rings` | 6 | 2.06x | 5.17x | 1.00x | 1.25x | 6/6 | PADDING |
| `synthetic` | 6 | 2.33x | 4.33x | 1.00x | 2.25x | 5/6 | length and evidence scale together |

## Where the objection is right: `bustout`

`bustout` **returns EVIDENCE LOST against every Claude model in every GPT pairing.** It is the only agent that does, and it is unanimous across six pairings:

| Claude | GPT | Paired | chars ratio | signals-cited ratio | Finding string from the artifact |
|---|---|---|---|---|---|
| opus-5 | luna | 6 | 4.62x | **7.0x** | *"the shorter GPT analysis names 0.14x the signals for 0.22x the length"* |
| opus-5 | terra | 6 | 4.57x | 5.25x | *"names 0.19x the signals for 0.22x the length"* |
| opus-5 | sol | 6 | 5.38x | **7.0x** | *"names 0.14x the signals for 0.19x the length"* |
| sonnet-5 | luna | 6 | 2.29x | 4.5x | *"names 0.22x the signals for 0.44x the length"* |
| sonnet-5 | terra | 6 | 2.26x | 2.75x | *"names 0.36x the signals for 0.44x the length"* |
| sonnet-5 | sol | 6 | 2.96x | 3.5x | *"names 0.29x the signals for 0.34x the length"* |

Arithmetic in the top row: Luna's `bustout` analysis is **22% of Opus's length** but names **14% of the signals** — proportionally *fewer* signals than its length would predict. That is the definition the method set in advance, and it fires.

Underlying pairs make the mechanism stark. From `length_gap.comparisons` → `opus-5` vs `luna` → pooled `pairs`, the `signals_cited` counts on `bustout`:

| Application | opus-5 | luna |
|---|---|---|
| APP-1004 | 7 | 0 |
| APP-1009 | 7 | 1 |
| APP-1010 | 7 | 1 |
| APP-1012 | 5 | 1 |
| APP-1013 | 5 | 0 |
| APP-1014 | 7 | 1 |

Luna names **zero or one** of `bustout`'s signals by name on every application. Opus names five to seven.

> ⚠️ **Read the paraphrase correction before concluding on bustout**
>
> `bustout` is precisely the agent the artifact's own caveat singles out as most affected by the identifier-vs-prose measurement problem. `grounded_ratio` — the style-free metric — is 3.0x on this pairing, i.e. Opus validates three times as many `_context` rows, a real gap but a much smaller one than 7.0x. And the judges scored **GPT better on substance on `bustout` on both judges** (+0.361 / +0.861).
>
> So the honest reading is that **the metrics disagree, and they disagree about paraphrasing.** `signals_cited` says evidence lost; `grounded_fraction` says a 3x gap; the two judges say GPT is denser. The study's own tiebreak puts `grounded_fraction` and the judges first — which still leaves a grounding gap, and that is why `bustout` stays on Claude anyway.

## Paraphrase correction: 0.7 of a 1.7-signal gap

Quoted verbatim from the artifact's caveats, because this is the single most self-critical measurement in the study:

> *"`signals_cited` is STYLE-SENSITIVE and partly measures whether a model writes signal identifiers or paraphrases them. Opus writes a backticked table of every identifier; Luna writes 'two SSN applications in both the last 7 and 30 days', which cites the same two velocity signals without containing the token 'apps'. Measured: pooled*

> *over these 120 analyses Claude cites 4.56 signals per analysis to GPT's 2.86, and a looser stemming matcher moves GPT to 3.60 while moving Claude only to 4.65 — so about 0.7 of the ~1.7 gap is paraphrasing rather than missing evidence. The looser matcher was rejected because it cannot keep* `ssn_apps_last_7d` *and* `ssn_apps_last_30d` *distinct."*

| Matcher | Claude signals/analysis | GPT signals/analysis | Gap | Shipped? |
|---|---|---|---|---|
| Strict | 4.56 | 2.86 | **1.70** | **Yes** — keeps 7-day and 30-day velocity distinct |
| Looser stemming | 4.65 | 3.60 | **1.05** | No — blurs `ssn_apps_last_7d` into `ssn_apps_last_30d` |

**About 41% of the measured citation gap is a style difference, not a missing-evidence difference.** The looser matcher was nevertheless rejected, for a fraud-domain reason: it cannot distinguish a 7-day velocity signal from a 30-day one, which is exactly the distinction a bust-out analysis turns on. A metric that improves the score by blurring two signals into one is worse than a metric with a known bias.

## Band agreement survives the length gap

Strongest single argument that brevity is not costing fraud judgement:

| Pairing | Paired | Same band | Both bands justified |
|---|---|---|---|
| opus-5 vs luna | 24 | **23/24** | 23 |
| opus-5 vs terra | 23 | **23/23** | 23 |
| opus-5 vs sol | 24 | 18/24 | 24 |
| sonnet-5 vs luna | 21 | 19/21 | 21 |
| sonnet-5 vs terra | 20 | 18/20 | 20 |
| sonnet-5 vs sol | 21 | 15/21 | 21 |

**Terra reached the same band as Opus 5 in 23 of 23 paired comparisons**, in roughly 30% of the characters. Luna missed one of 24.

Both low rows are Sol, and the collapse is localised: `identity`/Sol agrees with Opus 5 in **1 of 6** and with Sonnet 5 in **0 of 3**. That is the same cell that escalated both anti-false-positive fixtures. Sol's problem is not brevity — its per-cell output medians run 126–370 tokens, in the same range as Luna's 206–421 — it is calibration, at 4x Luna's price. Cross-family study.

## Within Claude, length is not monotonic in the tier

From `tier_quality.json → verbosity_vs_substance`, pooled over 16 paired applications:

| Longer | Shorter | Paired | chars ratio | distinct-evidence ratio | Judge substance delta | Pairs where only the longer escalates |
|---|---|---|---|---|---|---|
| opus-5 | sonnet-5 | 16 | 1.31x | 1.60x | +0.0 | 0 |
| opus-5 | haiku-4-5 | 16 | 1.00x | 1.17x | +1.0 | 0 |
| **haiku-4-5** | sonnet-5 | 16 | **1.33x** | 1.32x | **−1.0** | 0 |

**Haiku 4.5 writes more characters than Sonnet 5 (1.33x) and scores a full point lower on substance.** The cheapest model in the study is not the shortest, and the longest is not the best. Length and quality are close to orthogonal on this workload, which is why neither substitutes for the other in a model decision.

Every row of every pairing shows **0 in the last column** — across 48 paired comparisons in the tier study, no pair exists where the longer analysis escalated the band and the shorter one did not. So verbosity does not drift toward over-escalation either.

# Conclusions

| Claim | Verdict on the evidence |
| --- | --- |
| "GPT writes a fifth of the words" is, by itself, an objection | **No.** Pooled and paired on identical payloads, the longer Claude analysis is 2.95x the length for 1.08x the signals named, and the same band 23 of 24 times |
| On `bustout` it is a real objection | **Yes, on one metric.** `signals_cited` says 7.0x; `grounded_ratio` says 3.0x; both judges say GPT is denser. The metrics disagree about paraphrasing and the study says so rather than picking the convenient one |
| Brevity is why the specialists stayed on Claude | **No.** The deciding measurement was `_context` grounding — 0.642 vs 0.357, lower on 4 of 4 agents, no judge involved. The length gap was tested, and it mostly exonerates GPT. Cross-family study |
| Verbosity that does matter is `bustout`'s and `rings`'s own | **Yes.** Those two prompts carry no character cap and write **1,331 output tokens p50** against **381** for the six capped agents, with **30.1%** of their calls exceeding 4,000 characters against 3.6%. A prompt question for the customer, not a model question. Open questions |

# Still unknown

Every ratio here is a **median of six paired comparisons per scope**, so no p95 is offered and none is computable. `signals_cited`'s residual bias is bounded but not removed — 0.7 of the 1.7-signal gap is measured as paraphrasing and the remaining 1.0 is not attributed either way. And the four unswept agents (`dealer`, `straw`, `employment`, `income`) have no length comparison at all, in either family.

Cross-family study
The full Claude-versus-GPT table, per agent, with the verdict column.

What does not reduce cost
Why capping max_tokens to shorten these two agents failed all three of its criteria.

‹ Judge methodology
Two judges biased in opposite directions, s...

Synthesizer selection ›
GPT-5.6 Luna on bedrock-mantle, chosen o...

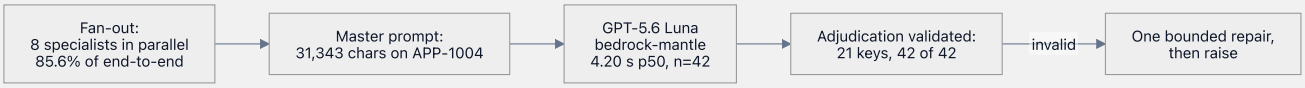
# Synthesizer selection

GPT-5.6 Luna on bedrock-mantle, chosen on a 55.2 s → 6.1 s measurement, validated at 4.20 s p50 over 42 live adjudications with zero repair attempts — and why the specialists did NOT follow it.

*For both audiences. The executive reading is that one substitution makes the sub-minute target reachable; the engineering reading is that the specialists were offered the same measurement and declined it.*

**GPT-5.6 Luna, and it is the only model choice in this repo made on a cross-family measurement.** Substituting it for Claude Sonnet 5 on the master adjudication took a measured 55.2 s call to 6.1 s on the same eight analyses, and 42 live runs then measured **4.20 s p50 / 5.59 s p95** at **$0.010640/call** with **0 failures and 0 repair attempts** (`evals/results/pipeline_14x3_us-east-1.json → synthesis_detail`).

## Where the synthesizer sits




One serial call after eight parallel ones

A master adjudicator receives **all eight specialist narratives** substituted into the customer's verbatim `master_agent_prompt.txt` and emits the **21-key adjudication object**. On a real application that prompt is **31,343 characters** — measured on APP-1004, with the eight narratives in place — and the reply is ~800–1,000 tokens of JSON. Single largest call in the pipeline, and serial: nothing can start until all eight specialists finish.

Code: `agents/mantle.py`, `agents/synthesize.py`, `agents/models.py:SYNTHESIZER_MODEL`.

## Measurement that decided it

Measured on APP-1004, against **the same eight analyses, n=1 each**:

| Synthesizer | Wall clock | Cost | Result | Verdict |
|---|---|---|---|---|
| Claude Sonnet 5 | **55.2 s** | $0.11731 | 21/21 keys, HIGH RISK / DECLINE | rejected on latency |
| GPT-5.6 Luna | **6.1 s** | $0.01488 | 21/21 keys, HIGH RISK / DECLINE | **shipped** |

Identical verdict, ~9x faster, ~87% cheaper on that call. Claude synthesis alone consumed **55.2 s of a 101.1 s end-to-end run** (28,567 input tokens for 6,018 output tokens); the substitution took that measured end-to-end to **37.5 s**.

**n=1, and this page says so.** One application, one call each. Enough to separate 55.2 s from 6.1 s — a 9x gap is not a sampling artifact — and **not** a distribution. `agents/mantle.py` records a slightly different pair for the same substitution measured on a different draw (55.2 s / $0.11731 against **4.4 s / $0.01314**), which is the run-to-run spread expected at n=1. Neither number is a benchmark; the 42-run figure below is.

## What 42 live adjudications then measured

`evals/results/pipeline_14x3_us-east-1.json → synthesis_detail`, model `openai.gpt-5.6-luna`, **42 calls, 0 failed**:

| | n | min | p50 | p95 | max | mean |
|---|---|---|---|---|---|---|
| Wall clock | 42 | 3.08 s | **4.20 s** | **5.59 s** | 15.21 s | 4.49 s |
| Adapter-reported latency | 42 | 3.05 s | 3.90 s | 5.55 s | 15.19 s | 4.45 s |

| | Value |
|---|---|
| Share of end-to-end | **14.4%** (against fan-out's 85.6%) |
| Total cost | $0.44686675 |
| Mean per call | **$0.010640** |
| `repair_attempts` total | **0** |
| `runs_needing_a_repair` | **0** |
| Errors | none |
| p99 | *"refused: 42 sample(s), need >= 100. A p99 over 42 sample(s) is not a p99."* |

**4.20 s p50 at n=42 is the figure to quote.** The 6.1 s (or 4.4 s) figure is the one-run comparison that motivated the change.

## Token profile, and why it looks strange

| Token class | Total over 42 calls | Mean per call |
|---|---:|---:|
| Input | 84 | **2.0** |
| Output | 30,066 | 715.9 |
| Cache read | 0 | 0.0 |
| **Cache write** | **180,610** | **4,300.2** |

Two input tokens per call. The entire ~31K-character prompt lands in the **cache-write** class, because implicit caching is on by default for GPT-5.6 and every application's prompt is new.

> **This is why the cache-write term is load-bearing**

> GPT-5.6 bills cache-write at **1.25x the uncached input rate**. Omitting that term under-reported a measured Luna call by **62%** — `cacheWriteInputTokens=6740` was **$0.00927 of a $0.01499 call**. `agents/mantle.py:CACHE_WRITE_MULTIPLIER` prices it, and GPT-5.4/5.5 carry a multiplier of **0.0** because they cache automatically with no write charge. Cost levers.

Note also that `agents/mantle.py:_usage` normalises the Responses convention onto the Bedrock Converse one. Responses `usage.input_tokens` is the **total** prompt size and already includes cached tokens; Converse reports only the non-cached count. Subtracting keeps a cached token billed once, at the right rate, for both families — otherwise `agents/pricing.py` would double-count.

## The 21-key contract is validated, not assumed

A synthesizer's whole job is a contract, so the routing validates rather than trusts:

- `structured_output_model=Adjudication` on the Converse path — Strands converts the pydantic model to a forced tool spec and hands the model back its own validation errors mid-loop.
- On the mantle path there is no equivalent tool, so `agents/synthesize.py:_invoke_master_mantle` parses the reply and validates it against the same `Adjudication` model, feeding the same bounded-repair loop. Exactly **one** repair invocation is allowed; after that it **raises**.
- **No code path emits an unvalidated adjudication.**

Measured over the 42 runs: **42 of 42 validated**, including the one degraded run that lost two specialists to `InternalServerException`. **0 repairs needed.**

`agents/models.py:SYNTHESIZER_MAX_TOKENS` is 4096, and the reason is in its comment: the customer caps each of nine summaries at "under 1200 characters", which is ~2,850 tokens plus JSON scaffolding. Measured, the largest synthesis wrote **1,005** output tokens and the median **666** — so 4096 is 4.1x the observed ceiling and has never bound.

Truncation here destroys the object rather than shortening it: a truncated synthesis fails the 21-key contract outright, a *failed* adjudication rather than a cheaper one. The ceiling matters in the other direction too — an earlier pass measured that at `max_tokens=16384` a

live synthesis took **84 s of a 113 s end-to-end run** and emitted **9,244** output tokens, far past the contract it was asked to honour.

## Which variant, and it is not the ordering anyone expects

Measured on the real 31,343-character prompt, streamed Responses calls, `effort=low`, **n=3 per cell**, 2026-07-28. All 24 calls returned a valid 21-key `Adjudication` and all 24 adjudicated **HIGH RISK / DECLINE**, so the latency spread is not bought with a shorter or worse answer.

| Variant | Best measured p50 | Input / cache-read / output per 1M | Verdict |
|---|---|---|---|
| **GPT-5.6 Luna** | **4.61 s** (us-east-1) | **$1.10 / $0.11 / $6.60** | **shipped — cheapest AND fastest** |
| GPT-5.6 Terra | 6.29 s (us-east-2) | $2.75 / $0.28 / $16.50 | 2.5x the price for 1.4x the latency |
| GPT-5.6 Sol | 11.19 s (us-east-1) | $5.50 / $0.55 / $33.00 | 5x the price for 2.4x the latency |

**5x the token price buys 2.4x more latency on this workload.** No speed-for-money trade exists to make, and Luna is not a compromise pick. Full matrix, region reasoning and the short-prompt trap: region selection.

Sol is also the worst GPT variant on **quality** in the cross-family study — 0 of 4 agents graded "as good", and the only cell that escalated both anti-false-positive fixtures. Cross-family study.

## Why the specialists did not follow

An obvious next question after moving the synthesizer is whether to move the eight agents. It was measured — 120 blind-judged analyses, two judges, $16.8869 — and the answer is **not yet**.

| | Synthesizer | Specialists | Consequence |
|---|---|---|---|
| What the model produces | A **21-key JSON object** whose shape is machine-validated on every call | **Titled prose** a human underwriter reads, whose quality is the product | Asymmetric burden of proof |
| How correctness is checked | Contract validation: 42/42, deterministic | Band agreement, anti-FP escalation, `_context` grounding, output contract | One is checkable per call; one is not |
| Prompt size | 31,343 chars | ~7,500 chars | Different latency regime |
| Measured decision | **GPT, on latency** | **Claude, on `_context` grounding: 0.642 vs 0.357, lower on 4 of 4 agents** | Same family measured, opposite answers |

That asymmetry is the point. **A synthesizer's output contract is checkable by a machine on every single call, so a latency win can be taken without a quality argument.** A specialist's output is a fraud judgement, and its quality axis — validating each signal against its paired `_context` evidence, as every specialist prompt requires — is measurably weaker on GPT.

That specialist question is not closed and the plumbing is not the blocker. `agents/fanout.py` routes a GPT specialist today (`transport_for('openai.gpt-5.6-luna') == 'bedrock-mantle'`), so reversing the decision is one value in `SPECIALIST_MODELS`. One prompt-engineering experiment on grounding stands between the hold and a projected **47.8%** specialist cost reduction. Cross-family study.

## Reversing it is one environment variable

```
BEDROCK_MODEL_SYNTHESIZER=global.anthropic.claude-sonnet-5
```

`agents/synthesize.py` routes on the **model id**, not on a flag: `agents.mantle.is_mantle_model` returns True for anything starting `openai.gpt-5`, and anything else goes through `build_bedrock_model` and Converse. So there is no second code path to maintain and no

feature flag to get wrong — the same bounded-repair loop and the same `Adjudication` validation apply to both families.

## Operational notes on the mantle path

Verified live, and each one cost time.

| Finding | Detail |
|---|---|
| GPT-5.x is not on Bedrock Converse or InvokeModel | `BedrockModel` rejects the id with `ValidationException: The provided model identifier is invalid` |
| Hostname is `bedrock-mantle.{region}.api.aws`, not `.amazonaws.com` | The `amazonaws.com` form does not resolve at all (NXDOMAIN), which looks exactly like a permissions problem and is not one |
| Path is `/openai/v1`, not `/v1`, and the two are mutually exclusive | Strands documentation shows `/v1` with `openai.gpt-oss-120b`, correct for that pair and **wrong for GPT-5.6**. Detail |
| Auth is a bearer token | From `aws_bedrock_token_generator.provide_token`, not SigV4 request signing. Tokens are short-lived, so one is minted per call. IAM needs `bedrock-mantle:CreateInference` (managed policy `AmazonBedrockMantleInferenceAccess`) — a different action prefix from both `bedrock:` and `bedrock-agentcore:` |
| These are reasoning models | `temperature` / `top_p` / `top_k` are ignored; use `reasoning={"effort": ...}`. Both `low` and `medium` returned a valid 21-key object; `low` is the default because the master prompt is an explicit instruction set, not a puzzle |
| No `global.*` or `us.*` prefix, no cross-region inference | The id is bare (`openai.gpt-5.6-luna`) and the call is in-region only — which is why the region comes from a measurement rather than from `AWS_REGION` |
| Regional availability differs per variant | Luna and Terra: us-east-1, us-east-2, us-west-2. Sol: us-east-1 and us-east-2 only |

## Still unknown

> ⚠ **Luna's fraud reasoning has never been blind-judged against a Claude synthesizer**
>
> Stated as `TIER_RATIONALE` states it: *"WHAT IS UNPROVEN: that Luna's fraud REASONING matches Sonnet 5's. The 42 runs validated 21 keys every time and the verdict-stability check found 12 of 14 applications stable, but no blind judge has scored Luna against a Claude synthesizer across the fixture set."*

What exists instead of that missing eval:

| Evidence | What it is | What it is not |
|---|---|---|
| 42 runs, 21 keys every time, 0 repairs | A **contract** result | A reasoning result |
| 12 of 14 applications verdict-stable across three repetitions of byte-identical input | Stability. APP-1005 and APP-1012 moved between HIGH RISK and MEDIUM RISK | Accuracy |
| 24 matrix calls unanimous on HIGH RISK / DECLINE across three variants and three regions | A **determinism** observation | An accuracy observation |
| Mild corroboration from the specialist cross-family study that Luna reasons acceptably on this fraud material | Adjacent evidence | A synthesizer result — the synthesis prompt is 31,343 chars against a specialist's ~7,500, and it emits a 21-key object rather than prose |

Closing it takes the calibration judge in `evals/evaluators/pp_calibration_judge.json`, run across the fixture set on both a Luna and a Claude synthesizer.

Also unproven: **that the variant or region ranking holds for any other prompt size.** It is a property of a ~31K-character prompt with an ~800–1000 token reply, and a short-prompt probe ranks the regions differently.

**Region selection**

The measured model x region matrix, and why a short-prompt probe picks the wrong region.

**GPT-5.6 on bedrock-mantle**

The transport itself: endpoint, auth, usage normalisation.

‹ Length versus evidence

"GPT writes a fifth of the words" was the int...

Region selection ›

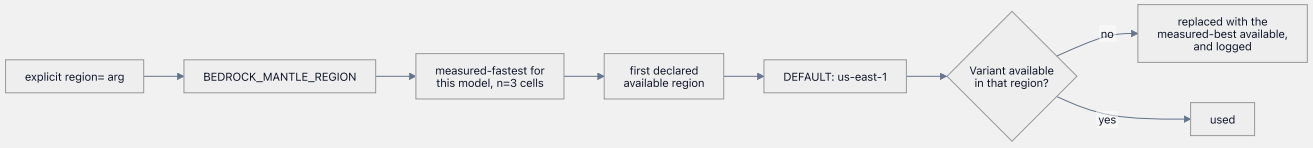
Measured model x region latency matrix tha...

# Region selection

Measured model x region latency matrix that picks the synthesizer's region, why AWS_REGION is deliberately not consulted, and why a short-prompt probe ranks the same regions differently.

*For the customer's engineer, and for anyone about to quote a vendor latency dashboard in a deployment decision.*

**us-east-1 for Luna, measured at 4.61 s p50 against us-west-2's 5.02 s (n=3 per cell, real 31,343-character prompt).** A `bedrock-mantle` call is in-region only — no `global.*` or `us.*` profile for GPT-5.x, no cross-region inference — so whichever region the client points at is the region that answers, and the synthesizer is the pipeline's serial bottleneck. The transferable finding is the last section: **a region ranking is a property of your reply length**, and ranking on someone else's probe picks the wrong region.




How region_choice resolves, in precedence order

Code: `agents/mantle.py` (`MEASURED_SYNTHESIS_LATENCY`, `THIRD_PARTY_SHORT_PROMPT_PROBE`, `region_choice`, `measure_matrix`). Raw probe output was `/tmp/bench/matrix.json` — scratch, uncommitted — and the numbers below are transcribed from it into the module, where `measure_matrix` can re-derive them.

# Measured matrix

Streamed Responses calls on the **real 31,343-character synthesis prompt** — the customer's verbatim master prompt with all eight specialist narratives substituted in, APP-1004 — `effort=low`, `max_output_tokens=8192`, **n=3 per cell, 24 calls total, 2026-07-28.**

**All 24 calls returned a valid 21-key `Adjudication` and all 24 adjudicated HIGH RISK / DECLINE.** So the latency spread below is not bought with a shorter or a worse answer.

| Model | Region | p50 total (n=3) | p50 TTFT | p50 tok/s | p50 out tok |
|---|---|---|---|---|---|
| **luna** | **us-east-1** | **4.61 s** | **0.79 s** | 176.0 | 812 |
| luna | us-west-2 | 5.02 s | 1.19 s | **180.0** | 903 |
| luna | us-east-2 | 6.79 s | 1.63 s | 115.8 | 878 |
| terra | us-east-2 | 6.29 s | 0.63 s | 126.4 | 825 |
| terra | us-east-1 | 6.72 s | 0.69 s | 122.3 | 785 |
| terra | us-west-2 | 7.63 s | 0.89 s | 95.1 | 790 |
| sol | us-east-1 | 11.19 s | 4.19 s | 91.1 | 1020 |
| sol | us-east-2 | **30.66 s** | **22.07 s** | 31.6 | 900 |

`openai.gpt-5.5` and `openai.gpt-5.4` are priced and region-mapped in `agents/mantle.py` but carry **no latency cell at all**, because they were never measured on this prompt. Absence is treated as "unknown", never as "slow": `measured_region_preference` returns `()` for an unmeasured model and the caller falls back to declared availability order.

Three ways to read that table honestly, from the module's own notes:

1. **n=3 is a median of three points, not a percentile.** It separates 4.61 s from 30.66 s. It does **not** defend 4.61 s against 5.02 s as a stable ranking, and the code says so: `MIN_TRIALS_FOR_A_RANKING = 3`, and every rendering prints its n.

2. **The four p50 columns are independent medians**, taken column-wise. `tok/s` is the median of the per-trial throughputs, **not** `out tok / total`. Those coincide exactly in **3 of the 8** cells and differ by up to **10.5%** elsewhere. Kept as measured rather than recomputed, because a recomputed number is not a measured one.
3. **Totals are streamed totals.** Streaming is the only way to observe TTFT; `agents/synthesize.py` calls the endpoint **non-streamed**, and the report says so rather than assuming the two are equal.

> **AWS_REGION is deliberately not consulted, and it measurably mattered**
>
> `resolve_mantle_region` does **not** read `AWS_REGION`. With `AWS_REGION=us-west-2` — this repo's own dev default — a previous revision sent every synthesis to us-west-2 at **5.02 s p50** instead of us-east-1 at **4.61 s p50** (n=3 each, real prompt). That is an **8.9% latency tax** on the pipeline's bottleneck, paid for no benefit.
>
> Co-locating the synthesizer with the Converse specialists buys nothing either, because the mantle call is in-region only with no cross-region profile to share.

## What the ranking is ordered on

`measured_region_preference` orders on **total** latency, because for this workload total is the number the customer feels — one adjudication, no streaming UI on the critical path. The module is unusually careful about how much that choice is worth, and the care is warranted:

| Ranking key | Luna's first place | Same as ordering on total? |
|---|---|---|
| Total p50 | us-east-1 (4.61 s) | — |
| TTFT p50 | us-east-1 (0.79 s) | **Yes** — coincidentally identical for all three variants |
| **Tokens/s p50** | **us-west-2 (180.0)** | **No** |

So "we rank on total, not TTFT" is a statement of intent that **changed nothing on this data**, and it is asserted as such in the tests rather than overstated. The axis that actually flips the answer is **throughput**, and `test_ranking_on_throughput_would_pick_a_different_region` pins that.

## Sol's regional gap is an availability question too

| Variant | Available in |
|---|---|
| `openai.gpt-5.6-luna` | us-east-1, us-east-2, us-west-2 |
| `openai.gpt-5.6-terra` | us-east-1, us-east-2, us-west-2 |
| `openai.gpt-5.6-sol` | **us-east-1, us-east-2 only** |

A stack deployed in us-west-2 that asks for Sol gets a confusing 404. `region_choice` therefore **replaces** an unavailable region with the measured-best *available* one and logs the reason: availability overrides both an explicit `region=` argument **and** the `BEDROCK_MANTLE_REGION` override, because the alternative is a call that cannot succeed.

Sol in us-east-2 measured **30.66 s p50 with 22.07 s TTFT** — 2.7x its own us-east-1 number and **6.7x Luna's** us-east-1 number. That one cell would blow the customer's sub-minute target on the synthesis call by itself.

## Precedence order

`agents/mantle.py:region_choice` returns `(region, why)`, and the `why` string is the point of the function — it lands in the log once per distinct choice rather than once per adjudication.

| Precedence | Source | Rationale |
|---|---|---|
| 1 | explicit `region=` argument | A caller that has already decided |
| 2 | `BEDROCK_MANTLE_REGION` | Operator override: data residency, a new region, or disagreeing with our measurement |
| 3 | measured-fastest region for that model, from `MEASURED_SYNTHESIS_LATENCY` | Our evidence |
| 4 | the model's first declared available region | For a variant nobody measured |
| 5 | `DEFAULT_MANTLE_REGION` = us-east-1 | The only region where every GPT-5.6 variant is available, and measured-fastest for the default synthesizer |

Steps 1 and 2 are still subject to the availability check.

## THIRD-PARTY corroboration — not our measurement

Customer-supplied third-party dashboard readings on a **~16–32 output-token** prompt. **No sample count, no prompt text, no verdict validation came with them.** Recorded in `THIRD_PARTY_SHORT_PROMPT_PROBE` behind a mandatory provenance label (`THIRD_PARTY_PROBE_SOURCE`), so a screenshot of it cannot be mistaken for one of ours, and it **never selects a region**.

| Model | Region | TTFT | tok/s |
|---|---|---|---|
| luna | us-west-2 | **531 ms** | **405** |
| luna | us-east-1 | 666 ms | 220 |
| luna | us-east-2 | 2703 ms | 73 |
| terra | us-west-2 | 796 ms | — |
| terra | us-east-1 | 914 ms | — |
| terra | us-east-2 | 1105 ms | — |
| sol | us-east-1 | 3896 ms | — |
| sol | us-east-2 | **15636 ms** | — |

A missing tok/s means the reading did not include one. It is recorded as `None`, not as zero.

## Where it agrees with ours, and where it does not

| Finding | THEIRS, reported, ~32-token reply | OURS, measured, ~800–1000 token reply | Agree? |
|---|---|---|---|
| Worst region for Luna | us-east-2, 2703 ms TTFT vs 666 ms in us-east-1 | us-east-2, 1.63 s vs 0.79 s | **Yes** |
| us-east-2 for Sol | 15636 ms TTFT | 22.07 s TTFT, 30.66 s total | **Yes** |
| Highest-TTFT variant | Sol, 3896 ms in us-east-1 | Sol, 4.19 s against Luna's 0.79 s | **Yes** |
| **Best region for Luna** | **us-west-2**, 531 ms / 405 tok/s | **us-east-1**, 4.61 s total vs us-west-2's 5.02 s | **No** |
| Luna's lowest TTFT | us-west-2 (531 ms) | **us-east-1 (0.79 s vs us-west-2's 1.19 s)** | **No — the TTFT ordering itself disagrees** |

Three independent agreements on a dataset gathered a different way is real corroboration, and it is why the table is kept at all. The disagreement is the lesson.

**Both readings are correct about their own workload.** On a ~32-token reply almost all the wall clock is time to first token: generating 32 tokens at any plausible throughput takes a fraction of a second, so the region with the lowest TTFT wins essentially by definition. On an ~800–1000 token reply **~83% of the wall clock is generation** — computed from our own numbers as (4.61 − 0.79) / 4.61 — so throughput decides the total and TTFT is a smaller additive term. And note the last row: the two datasets disagree about the **TTFT ordering itself**, not merely about which metric to rank on. That is a stronger divergence than a metric-choice disagreement.

> **The transferable finding**
>
> **Ranking on a short probe and then deploying your own workload picks the wrong region.**
>
> Which is why the table in the code is labelled as a measurement of *our* prompt, why every cell carries its own `prompt_chars`, and why a re-measurement on a materially different prompt size **refuses to diff itself** against the recorded table.

## Re-derivable, not merely assertable

```
python3 -m agents.mantle                          # print the recorded table; spends n

AGENTCORE_MANTLE_ALLOW_MEASURE=1 \
  python3 -m agents.mantle --measure --dry-run    # call plan + spend upper bound, the

AGENTCORE_MANTLE_ALLOW_MEASURE=1 \
  python3 -m agents.mantle --measure              # re-run the matrix and diff it
```

Re-measuring is **double-gated** on the flag *and* the env var, because it is 24 billed inference calls on a 31K-character prompt. Several guards are worth naming:

| Guard | Behaviour |
|---|---|
| `--dry-run` spend estimate | An explicit **UPPER BOUND** whose basis is stated: input estimated at chars/4 (there is no `CountTokens` API on the mantle endpoint), priced at the 1.25x cache-write rate, output priced at the **full** token cap that no measured call reached (recorded p50 output was 785–1020) |
| `--trials` below 3 | **Refused.** A cell below `MIN_TRIALS_FOR_A_RANKING` is dropped by `measured_region_preference`, so a smaller run would spend money and change nothing |
| An unpriced model id | **Refused.** `rates_for` raises rather than guessing a rate |
| `prompt_comparability` | A re-measurement more than **±25%** off 31,343 chars is labelled **NOT comparable** and reported as a new experiment rather than a diff — because the ranking depends on the TTFT-vs-throughput balance, which is a function of prompt and reply size |
| `DRIFT_TOLERANCE` | 25%, deliberately generous: n=3 medians are noisy and the recorded min/max spreads span 1.3x **within** a single cell |
| `STALE_AFTER_DAYS` | 90. Past that the renderer prints a staleness warning with the table's age in days |
| Regions enumerated from `MANTLE_REGIONS` | Not from the recorded matrix — so a newly available region shows up as a `new` row instead of being invisible |

Each cell is reported as `unchanged` / `faster` / `slower` / `new` / `failed` / `missing`, and the diff **separately** reports whether the *ranking* moved — an individual cell drifting 10% matters much less than us-east-1 losing first place. When the ranking changes, the printed advisory says to update `MEASURED_SYNTHESIS_LATENCY` **with the new n and date**, not to paper over it with `BEDROCK_MANTLE_REGION`.

`build_probe_prompt` runs the real eight-way fan-out and assembles today's synthesis prompt, because the narratives **are** the prompt: substituting stubs produces a few-hundred-character prompt whose ranking says nothing about production. Under `MOCK_MODE=1` the fan-out returns canned analyses, the prompt is much shorter, and

`prompt_comparability` catches it — the diff labels itself not comparable rather than silently comparing two different experiments.

## Specialist regions are a separate question

Everything above concerns the `bedrock-mantle` endpoint. The eight Claude specialists go through Bedrock Converse with a `global.*` inference profile, where region selection is a different problem:

- `agents/models.py:resolve_region` reads `AWS_REGION` (then `AWS_DEFAULT_REGION`, then us-east-1) and governs the Converse client only.
- The `global.*` prefix means those calls are **not** pinned to one region's capacity the way a mantle call is — and it costs exactly 1/1.10 of the `us.*` prefix for identical behaviour. Cost levers.
- **No model x region matrix was measured for the Claude specialists.** The 42-run benchmark ran entirely in us-east-1, and nothing on this site says what us-west-2 would have measured.

## Still unknown

One older, narrower measurement exists and is worth keeping only for the shape of its result: on a real fraud prompt with `maxTokens=120` in us-east-1, **Haiku 4.5 1848 ms, Sonnet 4.6 4481 ms, Opus 4.8 2990 ms, Sonnet 5 @effort=low 2393 ms, Opus 5 @effort=low 2924 ms**. Sonnet 4.6 is *slower* than Opus 4.8 — **latency does not track price.** That is a 120-token probe, so by this page's own argument it is a TTFT-dominated measurement and must not be used to predict a 1,500-token specialist analysis.

Nothing here measures region under **concurrency**, and nothing measures a Claude specialist in any region other than us-east-1.

**Synthesizer selection**
Why Luna, and what 42 live adjudications measured.

**GPT-5.6 on bedrock-mantle**
Endpoint, auth, and the usage normalisation this region choice sits on.

‹ Synthesizer selection
GPT-5.6 Luna on bedrock-mantle, chosen o...

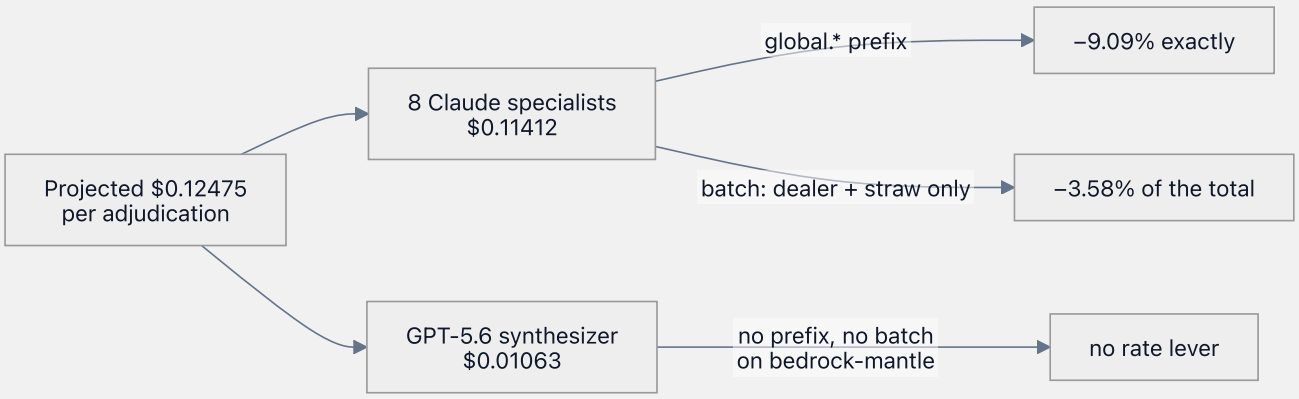
Cost levers ›
Levers that survive arithmetic — the global....

# Cost levers

Levers that survive arithmetic — the global.* inference profile at exactly −9.09%, batch inference at −50% with its exclusion list, and removing per-application Cortex Analyst calls.

*For both audiences, but the exclusions are the part that matters. Every figure here comes from `agents/pricing.py` on the published rate card or from a measured usage record.*

**Three levers survive arithmetic, and three more look like levers and are not.** The `global.*` inference profile is worth exactly **−9.09%** on the Claude calls, batch inference **−50%** where Bedrock offers it — which on this assignment is 3.58% of the bill — and removing per-application Cortex Analyst calls eliminates eight Analyst messages and eight warehouse executions per application.




Where $0.12475 of projected model spend goes, and what moves it

## Which lever applies to us

| Lever | Measured effect | Applies here? | Worth on this assignment |
|---|---|---|---|
| `global.*` instead of `us.*` inference profile | **exactly −9.09%** per rate category, identical API | **Yes**, on the 8 Claude calls | **−9.09%** on specialists, **−8.38%** blended |
| Batch inference | **−50%** where offered | **Partly** — `dealer` and `straw` only | **−3.58%** |
| Precomputed signals instead of per-application Cortex Analyst | 8 Analyst messages + 8 warehouse executions removed per application | **Yes**, structurally | **Not priced** — we lack their credit rate |
| Per-agent model tiering | **$0.00** structurally, by arithmetic | No | 0%. Detail |
| Prompt caching | Engages on **0 of 8** agents | No | 0%. Detail |
| Lower `max_tokens` | Failed all three of its own criteria | No | 0%, and it destroys analyses. Detail |

## Rate card these figures come from

`agents/pricing.py:BASE_RATES_PER_MTOK`, `global.*` on-demand, $ per 1M tokens:

| Model | Input | Output | Batch offered? |
|---|---|---|---|
| Claude Haiku 4.5 | $1.00 | $5.00 | Yes |
| Claude Sonnet 5 | **$2.00** | **$10.00** | No |
| Claude Sonnet 4.6 | $3.00 | $15.00 | Yes |
| Claude Opus 5 | $5.00 | $25.00 | Yes |
| Claude Opus 4.8 | $5.00 | $25.00 | No |
| GPT-5.6 Luna (`bedrock-mantle`) | $1.10 | $6.60 | No |
| GPT-5.6 Terra | $2.75 | $16.50 | No |
| GPT-5.6 Sol | $5.50 | $33.00 | No |

Cache rates are multipliers on the model's own **input** rate: read **0.10x**, 5-minute write **1.25x**, 1-hour write **2.00x**.

Two ids appear in `BATCH_SUPPORTED` but **not** in the rate table — `anthropic.claude-opus-4-7` and `anthropic.fable-5` — because their Bedrock rates were not verified for this port. Pricing either raises `UnknownModelRateError` rather than guessing.

**Sonnet 5's $2/$10 is introductory and reverts to $3/$15 on 2026-09-01**

Six of the eight specialists sit on Sonnet 5, so this date moves the whole bill. Repricing the current assignment at both dates (`agents/models.py:projected_cost_per_adjudication`):

| Priced as of | Projected $/adjudication |
|---|---|
| 2026-07-28 | **$0.12475** |
| 2026-09-01 | **$0.17734** |

A **42% increase** on the same tokens, from a calendar date and nothing else. Any TCO slide with a horizon past September needs the second row. `SONNET_5_STANDARD_RATES` and `SONNET_5_INTRO_EXPIRY` are in the code so the second figure is computable rather than remembered.

## Lever 1: `global.*` instead of `us.*` — exactly −9.09%

`us.*` and `global.*` cross-region inference profiles address the **same weights** through the **same API** with the same behaviour. `us.*` bills at exactly **1.10x** `global.*` on **every** rate category — input, output, cache read, cache write. So choosing `global.*` saves 1 − 1/1.10 = 9.0909…%.

Verified on the projected per-adjudication cost, both prefixes, same tokens, same date:

| Prefix | Specialists (8 Claude calls) | Synthesizer | Total | Saving |
|---|---|---|---|---|
| `global.*` | **$0.11412** | $0.01063 | **$0.12475** | — |
| `us.*` | $0.12553 | $0.01063 | $0.13616 | — |
| Delta | **−9.09% exactly** | — (no prefix exists on mantle) | **−8.38% blended** | |

Note which figure is which. **The 9.09% applies to the Claude specialist calls only.** The synthesizer is on `bedrock-mantle`, which has no `global.*` / `us.*` prefix and no cross-region inference — the model id is bare and the call is in-region. So the *blended* per-adjudication saving on this nine-call pipeline is **8.38%**, and quoting 9.09% as the end-to-end figure would be slightly wrong. Both are computable from the code; neither is rounded here.

`REGIONAL_MULTIPLIER = 1.10` lives next to the prefix in `agents/models.py`, and `agents/pricing.py` reads the multiplier from the **prefix carried by the model id** — `prefix_of(model_id)` — rather than from the process-wide default. So a model id built with an explicit prefix, or read back from a live response, is still priced correctly. Pricing a `us.*` call at global rates would under-report by 9.09% silently.

That prefix is env-overridable (`BEDROCK_MODEL_PREFIX`), because a customer with a data-residency requirement needs `us.*`. An unknown value **raises**: a silently-accepted typo

would route every agent to a model id that does not exist and produce cost math that cannot be trusted.

## Lever 2: batch inference — −50% with a real exclusion list

Batch inference is half price where Bedrock offers it. It does **not** offer it everywhere, and the exclusions are exactly the models this pipeline uses most.

| Model | Batch offered | Consequence for this pipeline |
|---|---|---|
| Claude Opus 5 | Yes | No agent runs on Opus any more |
| Claude Sonnet 4.6 | Yes | The source system's model, not ours |
| Claude Haiku 4.5 | Yes | `dealer` and `straw` — the two cheapest agents |
| **Claude Sonnet 5** | **No** | **Six of eight specialists** |
| Claude Opus 4.8 | No | |
| Claude Opus 4.7 / Fable 5 | No | |
| GPT-5.6 (any variant) | No | The synthesizer |

Priced on the recorded per-agent tokens, taking batch wherever it is offered and on-demand everywhere else:

| | Projected $/adjudication |
|---|---|
| All on-demand | $0.12475 |
| Batch where offered (`dealer` + `straw` only) | $0.12028 |
| **Saving** | −3.58% |

Because the two batch-eligible agents are also the two cheapest, a −50% discount on them is worth 3.58% of the bill. **A bulk-underwriting plan assuming −50% end-to-end is wrong by an order of magnitude on this assignment.** `rates_for(..., batch=True)` raises `UnsupportedBatchError` for a model with no batch price rather than silently applying the discount.

Two further constraints. **Batch and prompt caching are mutually exclusive** on the same request (`BATCH_AND_CACHING_ARE_EXCLUSIVE = True`), which costs nothing here because **prompt caching does not engage anyway**. And **batch is not real-time** — an offline queue, so it does not apply to the forward-flow underwriting path the customer's sub-minute target describes. It applies to a **backfill**, and the customer has a very specific one: their own reported figure is **4,500 applications taking over 48 hours**. That is where batch is worth having, and it is a different workload from the demo.

If the September rate change forces a re-look, note the shape it creates: **Sonnet 4.6 at $3/$15 with batch is $1.50/$7.50, cheaper than post-promotional Sonnet 5 at $3/$15 on-demand.** For a backfill the *older* model becomes the cheaper choice. That is arithmetic on the rate card, not a measurement, and no quality comparison of Sonnet 4.6 against Sonnet 5 exists in this repo.

## Lever 3: precomputed signals remove per-application Analyst calls

Largest structural saving, and the one that is not a Bedrock rate at all. The port keeps the customer's own two-layer separation: governed signals are generated **first** (`signal_layer/`), then handed to the agents for interpretation (`agents/`). The same agent never both queries and investigates.

| Per application | Incumbent path | Here |
|---|---|---|
| Cortex Analyst messages | 8 | 0 |
| Warehouse executions | 8 | 0 |
| LLM calls | 9 | 9 |

**This is a structural claim, not a priced one.** No dollar figure is put on the removed Analyst calls or warehouse executions anywhere on this site, because doing so would require the customer's Snowflake credit rate, their warehouse size, and their Analyst message pricing — none of which we have. What is claimable is the **count**. Their leadership often finds the count more persuasive than a latency number, precisely because they know what a warehouse-second costs and we do not.

A **latency** argument attaches to the same change, and it is why the architecture looks like this at all: a published production decomposition of a Cortex Agent call attributes **~27 s of a 55 s total** to Cortex Analyst SQL generation alone. For a fixed set of fraud signal groups the queries are known at design time, so regenerating them per application buys nothing. That figure is **not our measurement** — it is a published third-party decomposition, labelled as such wherever it appears. See `docs/semantic-layer-mcp.md` and the semantic layer.

## GPT-5.6 cache-write tokens bill at 1.25x input

Not a lever — a **correction**, and the largest single mis-pricing found in this engagement.

GPT-5.6 turns implicit caching on by default. A first call against a large prompt reports most of its input as `cacheWriteInputTokens`, and **cache-write bills at 1.25x the uncached input rate** (a 30-minute cache).

| Measured Luna synthesis call | Value |
|---|---|
| `cacheWriteInputTokens` | 6,740 |
| Cost of that term alone | $0.00927 |
| Total cost of the call | $0.01499 |
| **Share of the bill the term represents** | 62% |

Dropping the term under-reported that call by 62%. `agents/mantle.py:CACHE_WRITE_MULTIPLIER` prices it, with a regression test. Note the variant asymmetry: **GPT-5.4 and GPT-5.5 cache automatically with NO write charge**, so their

multiplier is **0.0**. A single hard-coded 1.25x across the GPT family would over-report those two.

Synthesis rows from the pipeline itself show why this is not an edge case:

| **Synthesizer token class, mean per call over 42 runs** | **Value** |
|---|---|
| Input | **2.0** |
| Output | 715.9 |
| Cache read | 0.0 |
| **Cache write** | **4,300.2** |

**Two** input tokens and **4,300** cache-write tokens. Essentially the entire ~31K-character prompt lands in the cache-write class on every application, because every application's prompt is new. Pricing the synthesizer without that term would report it as almost free.

## Where a cost figure is allowed to come from

| **Source** | **What it may publish** |
|---|---|
| `evals/bench.py` on a live run | Measured cost per adjudication, with n |
| `agents/pricing.py:cost_usd` on a real usage dict | The cost of one measured call |
| `agents/models.py:projected_cost_per_adjudication` | A **PROJECTION**, labelled as one in every return value |
| `agents/pricing.py:compare_tiered_vs_uniform` | A saving, **decomposed** into `structural_saving_usd` + `model_saving_usd` |
| Anything else | Nothing |

`cost_usd_or_none` exists so a caller with unknown usage renders an em dash rather than a fabricated figure. `agents/mantle.py:cost_usd` returns `None` for a missing usage dict for the

same reason, and `rates_for` raises for an unpriced model id, with the message `no verified rate card for {model_id}; refusing to price it.`

## Measured cost this all sits under

For the avoidance of doubt about which number is a measurement:

| | Value | Status |
|---|---|---|
| Cost per adjudication, prior assignment | **$0.1177 min / $0.1440 p50 / $0.2536 max** (n=41 priced runs) | MEASURED |
| Cost per adjudication, current assignment | $0.12475 | PROJECTION |
| Total measured Bedrock spend for the benchmark | **$6.46592** (42 runs, 336 specialist calls) | MEASURED |

Spread across those 41 runs is **$0.1359**, and because rates are fixed per model, **token volume is the only driver**. Between the cheapest and the most expensive run, specialist *input* tokens differ by 7% and specialist *output* tokens differ by **3.35x** (4,098 vs 13,735). Cost variance on this workload is an output-length story. Measured results.

## Still unknown

**Whether any of this is cheaper than the incumbent.** That is not a measurement gap on our side — it needs one number from the customer, and at the two ends of a plausible range the same architecture gives opposite answers. Also unmeasured: the end-to-end cost of the *current* assignment (a projection until `evals.bench --pipeline --repetitions 3` runs), and anything under concurrency, since applications were benchmarked one at a time.

**Snowflake comparison**
Our measured cost against their reported spend, and the one missing input that decides it.

**What does not reduce cost**
Tiering, prompt caching and max_tokens: believed, measured, removed from the cost story.

‹ Region selection
Measured model x region latency matrix tha...

What does not reduce cost ›
Three things that look like cost levers and a...

# What does not reduce cost

Three things that look like cost levers and are not — per-agent tiering saves $0.00 structurally, prompt caching never engages on this workload, and max_tokens failed all three of its own criteria.

*For both audiences, and this is the page to read before building a cost slide. Two of the three items below were on this repository's own README at some point.*

**Tiering saves $0.00 by its shape, prompt caching engages on 0 of 8 agents, and a lower `max_tokens` failed all three criteria set for it in advance.** Each was believed, then measured, then removed from the cost story — and each is now asserted against by a test so it cannot come back.

| Proposal | What it was believed to be worth | What it measured | Status |
|---|---|---|---|
| Per-agent model tiering | "The Bedrock differentiator" | **$0.00** structurally, by exact arithmetic | Removed; `test_two_opus_cancelling_two_haiku_reports_a_zero_saving` |
| Prompt caching | Lower input cost on a repeated system prompt | Engages on **0 of 8** agents; prompts are 547–956 tokens against 1024/4096 minimums | Left enabled, not counted; `cache_prefix_report` |
| Lower `max_tokens` | A latency fix and a cost fix | No latency gain, **14 of 30** cells truncated, **13 of 30** kept their band | Rejected; `MEASURED_TRUNCATION_BOUNDARY` |
| Orchestration overhead | Something to optimise | `asyncio` fan-out costs **0.83 s p50**, never more than **2.71 s** above its slowest member (n=42) | Nothing to win |
| Batch inference at −50% end-to-end | Half the bill | **3.58%**, because Bedrock offers no batch price for Sonnet 5 or GPT-5.6 | Detail |
| Moving the specialists to GPT-5.6 | −47.8% | Projected −47.8%, declined on a measured `_context` grounding gap | A real forgone saving, stated as one. Detail |
| A cheaper region | Regional arbitrage | No per-region Bedrock rate difference in the card | `us.*` vs `global.*` is a **profile** difference, and a real lever |

## Per-agent tiering saves $0.00 structurally

Intuitive claim: Snowflake Intelligence pins every Cortex Agent to one LLM class, Bedrock lets each of the nine agents run a right-sized model, therefore right-sizing saves money.

Arithmetic says the *shape* saves nothing.

A symmetric roster cancels to zero

**Haiku → Sonnet is a $2/$10-per-MTok step down. Sonnet → Opus is the identical step up.** So at equal token counts one LIGHT agent's discount is exactly one HEAVY agent's surcharge, and a symmetric 2-down / 2-up tiering cancels. Asserted directly in `tests/test_pricing.py`:

```
assert (baseline - light) == pytest.approx(heavy - baseline, rel=1e-12)
```

**The regression, reproduced to the cent**

A previous configuration ran 2 Haiku + 4 Sonnet 4.6 + 2 Opus 4.8 specialists plus a Sonnet 4.6 synthesizer, against an all-Sonnet-4.6 baseline. On a uniform token profile:

| | $/adjudication |
|---|---|
| Tiered | $0.12270 |
| Uniform baseline | $0.12270 |
| **Saving** | **$0.00** |

`test_two_opus_cancelling_two_haiku_reports_a_zero_saving` pins that identity to `abs=1e-9`. The README of the time called tiering "the Bedrock differentiator."

## What the current assignment's saving actually is

**No HEAVY agent is left** — `rings` and `synthetic` both moved to MID on measurement — so the two sides no longer cancel and `structural_saving_usd` is genuinely positive today. But note what that means: the shape is now "two cheap agents, six standard", and the saving is

simply **not paying Opus rates for output that measured no better**. A *model-choice* result, not a property of tiering.

`agents/pricing.py:compare_tiered_vs_uniform` splits any reported saving into three components from real usage, so they can never be conflated on a slide:

| Component | What it is | Expires? |
|---|---|---|
| `structural_saving_usd` | The tier **shape**: LIGHT and HEAVY assignments priced against the baseline with MID held at the baseline model. Exactly zero on a symmetric roster | n/a — it is near zero |
| `model_saving_usd` | What the MID **model generation** is worth. Mostly the Sonnet 5 promotional rate | **Yes, 2026-09-01** |
| `synthesizer_saving_usd` | The GPT-5.6 Luna **substitution**, broken out so nobody attributes it to tiering | No |

Computing `structural_saving_usd` deliberately holds the synthesizer at the baseline model: swapping it to a mantle model is a **substitution**, not a tier assignment, and letting it into the structural figure would credit the tier shape with a saving it did not produce. `test_the_whole_current_saving_expires_with_the_sonnet_5_promo` asserts that after the expiry the whole remaining saving is exactly `structural_saving_usd + synthesizer_saving_usd` — i.e. the MID tier collapses onto the baseline and contributes nothing.

**Say "right-sized models plus the levers that survive arithmetic", not "per-agent tiering is the Bedrock differentiator"** — and then name them: the `global.*` prefix, batch for backfills, and removing the per-application Cortex Analyst calls. Cost levers.

## Prompt caching does not engage

Bedrock prompt caching is real and it is enabled in this code. On **this workload** it never pays, for two independent reasons — either alone would be sufficient.

## Reason 1: the prompts are below the minimum cacheable prefix

`agents/models.py:MIN_CACHEABLE_PROMPT_TOKENS`, measured per model **by bisection**:

| Model | Minimum cacheable prefix |
|---|---|
| Claude Opus 5 | **512** |
| Claude Sonnet 5 | 1024 |
| Claude Sonnet 4.6 | 1024 |
| Claude Opus 4.8 | 1024 |
| **Claude Haiku 4.5** | **4096** |

**That table is not monotonic across generations, and that is the trap: Haiku 4.5 needs 8x the prefix Opus 5 does.**

Against the customer's actual prompts, at the model each agent now runs (`estimate_prompt_tokens`, a chars/4 estimate labelled as one everywhere it surfaces):

| Agent | Prompt chars | Est. tokens | Model | Minimum | Caches? |
|---|---|---|---|---|---|
| `identity` | 2,188 | 547 | Sonnet 5 | 1024 | **No** |
| `straw` | 2,373 | 593 | Haiku 4.5 | 4096 | **No** |
| `income` | 2,529 | 632 | Sonnet 5 | 1024 | **No** |
| `synthetic` | 2,697 | 674 | Sonnet 5 | 1024 | **No** |
| `employment` | 2,996 | 749 | Sonnet 5 | 1024 | **No** |
| `dealer` | 3,363 | 840 | Haiku 4.5 | 4096 | **No** |
| `rings` | 3,537 | 884 | Sonnet 5 | 1024 | **No** |
| `bustout` | 3,825 | 956 | Sonnet 5 | 1024 | **No** |

**Zero of eight.** Prompts run **547–956** estimated tokens; the nearest threshold is 1024.

Below the minimum, a `cachePoint` is accepted **silently**. No error, no warning; `cacheWriteInputTokens` comes back **0** and the caller pays full input price forever. That is the entire failure mode, and it is why `agents/models.py:cache_prefix_report` exists — it reports, per agent, whether the cache point will engage:

> *""prefix 956 < minimum 1024: cachePoint is accepted silently, cacheWriteInputTokens will be 0 and every call pays full input price""*

Note also that under the previous assignment the two Opus-tier agents (`rings`, `synthetic`) **did** clear their 512-token minimum, exactly the kind of partial success that makes a "caching is enabled" claim feel true. Both are now on Sonnet 5 at 1024, so **nothing caches on the prompt alone any more.** Extending the cached prefix with the signal layer's per-domain reference block — signal names, the customer's own bands, entitled alert ids — lifts five of eight past 1024 but still leaves **all eight** far below Haiku's 4096.

## Reason 2: it would not matter, because the payload differs every time

This is the reason that cannot be engineered around. The part of each request that is actually large is not the prompt — it is the **projected application payload**, 3.4–8.3 KB per domain. A cache entry is keyed on an **exact prefix**, so it can only be reused when the same prefix recurs, which means analysing the **same application** again. In a forward-flow underwriting stream, that essentially never happens.

| | **Cacheable?** |
|---|---|
| System prompt, identical across applications | Yes in principle — **too short** |
| Per-application payload, the large part | **Never reused** |

Padding the prefix to cross a threshold would **raise** input token count in order to make a cache-hit metric look good, which costs more than it saves.

> **So caching is left enabled and is not counted as a lever**

> A cache miss is free. `cached_system_prompt` places the cache point after the verbatim customer prompt — the only correct place for it — and `cache_prefix_report` reports the truth per agent. `evals/bench.py` records real `cacheReadInputTokens` rather than assuming any.
>
> That mixed picture is exactly what the 42-run benchmark shows: four specialists record non-zero `cacheReadInputTokens` (`employment` 1,091/call, `bustout` 1,297, `synthetic` 1,001, `rings` 1,175) and four record zero. That is **not** evidence of a working cost lever. Those are the two Opus agents plus two others under the previous assignment, and none of it recurs across applications.

GPT is not a counterexample either. `agents/mantle.py` notes that prompt caching is explicit for 5.6 and *"would not help here anyway — the eight analyses sit inside the prompt, so the prefix differs on every application."* And on GPT the cache term is not a discount but a **surcharge**: cache-write bills at **1.25x** input, which is **62% of a measured Luna call**.

## `max_tokens` failed all three criteria

`bustout` and `rings` write 3.5x the output of the other six agents, and wall clock is linear in output tokens ($r^2$=0.9531 over n=332 calls). So capping output tokens looks like both a latency fix and a cost fix. It was **measured**, against a quality gate set in advance.

That gate, quoted from the artifact: *"A cap is only a fix if it (a) lowers wall clock, (b) truncates nothing, and (c) leaves every specialist's risk band unchanged. Failing (b) or (c) makes a latency win a lost fraud dimension."*

**30 live calls** in `us-east-1` across `bustout` and `rings` at caps 1024 / 1536 / 2048 / 3072 / 4096, each against a **same-session** 4096 control, so a cap delta is measured against a control run in the same session rather than against a run from another day. `evals/results/tail_analysis.json → live_cap_sweep`. Measured cost **$0.627827** plus **$0.437793** bounded on the truncated calls.

| Criterion | Result | Verdict |
|---|---|---|
| (a) Lowers latency? | Of the 10 cells that returned an analysis at both their cap and the control, **5 were faster and 5 slower.** Mean delta **−0.23 s** (min −10.39 s, max **+17.84 s**) | Fail |
| (b) Truncates nothing? | **14 of 30** cells raised `MaxTokensReachedException`. 1024 truncated **6 of 6** | Fail |
| (c) Leaves bands unchanged? | **13 of 30** agreed with the same-session control; 14 lost the band entirely and 3 of the remaining disagreements were **genuine band moves** | Fail |

## Mechanism: the model writes what it intends to write

APP-1004 `rings` is the clean counter-example. The cap **did** bind — output fell from 2,876 tokens to 1,980 at cap 2048 — and the call still took **28.50 s** against the control's 38.90 s at a different draw. Meanwhile APP-1012 `bustout` took **33.70 s at cap 3072** and **15.87 s at 4096**, i.e. 17.84 s *slower* with a *tighter* cap. A cap either does not bind, or it raises and destroys the whole analysis. It does not gracefully shorten.

**A truncated call is the worst outcome available, not a cheap one**

`strands` **raises** rather than returning the partial text. So a truncated call:

| | Measured |
|---|---|
| Spends its full wall clock | **263.2 s** across the 14 truncated cells; **17.4 s p50**, up to **29.0 s** each |
| Is billed for every token it generated | bounded at **$0.437793** |
| Returns **nothing** | fan-out degrades to **seven of eight** fraud dimensions |

**A cap low enough to shorten these agents is a cap low enough to delete them.**

## Failure boundary is a probability, not a threshold

Which is a *stronger* reason not to ship a lower cap than a clean boundary would have been:

| Agent | Caps tested | Caps observed truncating | Caps observed completing | Lowest cap where every cell completed |
|---|---|---|---|---|
| `bustout` | 1024, 1536, 2048, 3072, 4096 | **1024, 1536, 2048** | **1536, 2048**, 3072, 4096 | 3072 |
| `rings` | 1024, 1536, 2048, 3072, 4096 | **1024, 1536, 2048** | **1536, 2048**, 3072, 4096 | 3072 |

Both agents truncated **and** completed at 1536 **and** at 2048, depending on the application and the draw. 3072 is the lowest cap where every cell completed — on **one draw each**, which is not enough to call it safe. Recorded in `agents/models.py:MEASURED_TRUNCATION_BOUNDARY` so nobody tightens the caps blindly, and the other six agents carry `None` there — **because nobody measured them, not because they are safe.**

## A character cap is not a token cap

Six of the eight prompts say "max 4000 characters". Deriving a token cap from that is a trap, because measured chars-per-output-token runs from **4.51** (`dealer`) down to **1.55** (`identity`), and `identity`'s worst calls hit **0.33 — 3,582 billed output tokens behind 1,191 characters of analysis**, on its slowest calls. A cap computed as 4000/4 = 1024 would truncate `identity` at a third of its own contract, and 1024 truncated every cell in the sweep.

**The customer's character rule belongs in her prompt text and in the output-contract tests, not in a token cap.**

## What the measurement points at instead

Wall clock is linear in output tokens: `wall_ms = 2980 + 11.34 x output_tokens`, $r^2$=0.953 over n=332 calls (excluding two server-side stalls). And the driver of output tokens is the prompt:

| | Domains | n calls | Output tokens p50 | Analyses over 4,000 chars |
|---|---|---|---|---|
| Prompt caps at 4,000 chars | identity, dealer, straw, employment, income, synthetic | 251 | **381** | 9 (3.6%) |
| **No cap in prompt** | **bustout, rings** | 83 | **1,331** | **25 (30.1%)** |

So the lever is **the customer's own prompt text** — adding the missing length sentence to two of her artifacts. That is a one-line change to two verbatim customer files, it is **her decision, not ours**, and it is **NOT MEASURED**: nobody has run it. It is the recommendation, not a result. Open questions.

## Still unknown

Nothing here measures the six agents that were never in the cap sweep, so `MEASURED_TRUNCATION_BOUNDARY` carries `None` for each of them rather than a safe-looking number. No prompt-caching variant with a payload-prefix cache point has been measured either — moving the cache point after the system prompt *plus* the stable prefix of the signal payload is the untried idea, and quoting a caching benefit before that measurement would be exactly the failure this page documents.

## Pattern all three share

Each was plausible, each was checked, and each is now asserted against by a test so it cannot re-enter the cost story:

| Claim | The test that prevents its return |
| --- | --- |
| Tiering saves money by its shape | `test_two_opus_cancelling_two_haiku_reports_a_zero_saving`, `test_a_tier_step_down_and_a_tier_step_up_are_worth_the_same_per_token` |
| A −20% saving holds past the promo expiry | `test_the_post_expiry_comparison_reprices_both_sides_not_just_one` |
| The current caps can be tightened | `test_per_agent_caps_preserve_the_shipped_effective_values`, plus `MEASURED_TRUNCATION_BOUNDARY` |
| Prompt caching is engaging | `cache_prefix_report` reports per agent; `evals/bench.py` records real `cacheReadInputTokens` |

**Cost levers**
The three that do survive arithmetic, each with its measured effect.

**Snowflake comparison**
What our measured cost is being compared against, and why it is undecided.

‹ Cost levers
Levers that survive arithmetic — the global....

Snowflake comparison ›
Our measured $0.14396 per adjudication ag...

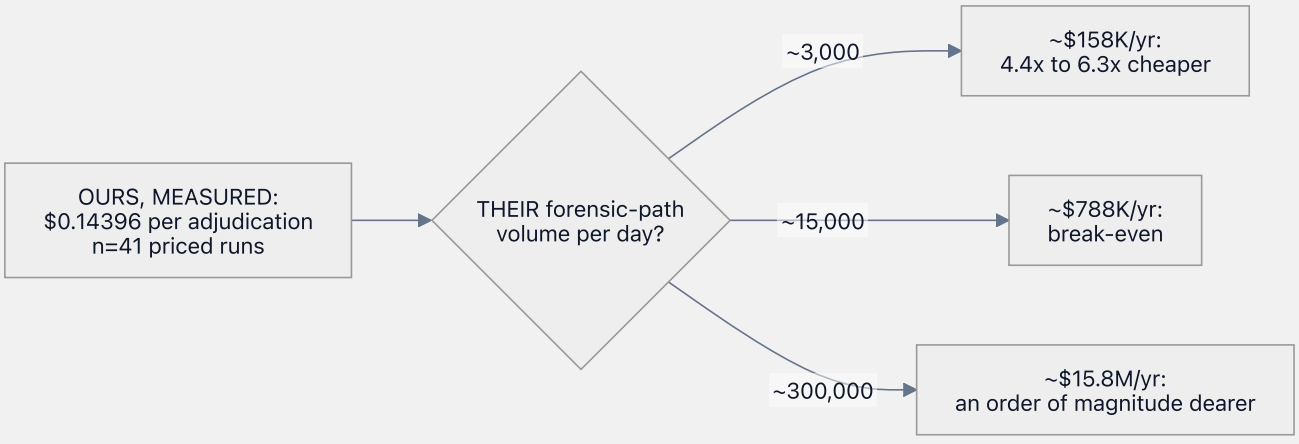
# Snowflake comparison

Our measured $0.14396 per adjudication against the customer's reported $700K-$1M annual AI spend. The comparison cannot be made without one number they have not given us — and at the two ends of a plausible range it points in opposite directions.

*For the executive and account audience primarily. Two kinds of number appear on this page and they are never mixed in one table:* ***OURS, MEASURED*** *and* ***THEIRS, REPORTED****.*

**Unresolvable today, and not because our side is unmeasured.** Cost per adjudication is measured over 42 live adjudications; what is missing is one input from the customer — **how many applications per day go through the full eight-agent forensic underwriting path.** At ~3,000/day the measured p50 annualises to roughly **$158K**; at 300,000/day, roughly **$15.8M**. Same architecture, opposite conclusion.

## Where the comparison breaks




One missing input, three answers

We did not measure Snowflake, we have no access to their account, and nothing on this site is a benchmark of their system. Where the two kinds of figure appear near each other, every row is labelled. A blended table would be the exact failure mode this site exists to avoid.

## Our side, measured

| Metric | Value | n | Status |
|---|---|---|---|
| Cost per adjudication, min | $0.1177 | 41 priced runs | OURS, MEASURED |
| **Cost per adjudication, p50** | **$0.14396** | 41 priced runs | OURS, MEASURED |
| Cost per adjudication, p95 | $0.2271 | 41 priced runs | OURS, MEASURED |
| Cost per adjudication, max | $0.2536 | 41 priced runs | OURS, MEASURED |
| Cost per adjudication, mean | $0.15616 | 41 priced runs | OURS, MEASURED |
| Total Bedrock spend for the benchmark | **$6.46592** | 42 runs, 336 specialist calls | OURS, MEASURED |
| p99 | *"refused: 41 sample(s), need >= 100. A p99 over 41 sample(s) is not a p99."* | — | refusal recorded verbatim |

Source: `evals/results/pipeline_14x3_us-east-1.json`, run `2026-07-28T07:42:42+00:00` in `us-east-1`, git `9bb569bac801`. 14 applications x 3 repetitions, 9 model calls per adjudication. One run is excluded from the cost distribution because it was degraded — two specialists lost to `InternalServerException` — and therefore not fully priced. It remains in the latency distribution, which is why latency is n=42 and cost is n=41.

Two caveats on our own number, both from the artifact:

1. **This ran under the previous model assignment.** `rings` and `synthetic` were on Opus 5; both moved to Sonnet 5 afterwards. The current assignment's cost is a **projection** — $0.12475 at 2026-07-28 rates, $0.17734 once Sonnet 5's introductory rate lapses on 2026-09-01. The measured $0.14396 is the figure with an n behind it, so it drives the arithmetic below, and the projections are shown separately.
2. **It excludes AgentCore Runtime overhead and signal generation.** Benchmarked calls went to Bedrock directly from the host recorded in `run.host`, and `build_payload` runs outside the timed and priced window because the customer's own architecture precomputes signals. **This is model spend, not total cost of ownership**: no AgentCore Runtime compute, no Aurora, no ETL, no CloudWatch, no engineering time.

## Their side, as they reported it

| Their reported figure | What it is | Status |
|---|---|---|
| **$700K–$1M** | Their **projected annual Snowflake AI spend** | **THEIRS, REPORTED** — we have not seen an invoice, a credit rate, or a breakdown |
| **4,500 applications in over 48 hours** | Their reported throughput on the current forensic path — the pain point that started the engagement | **THEIRS, REPORTED** |
| ~10 GB in Aurora PostgreSQL, ~3.4M reads/day | Their reported data footprint, already in AWS | **THEIRS, REPORTED** |
| Under 1 minute per application | Their stated target | **THEIRS, STATED** |

What the $700K–$1M covers is also unknown, independently of volume. It is described as an **AI** spend, and we do not know whether it includes Cortex Analyst message charges, Cortex Agents charges, or both; warehouse compute for the SQL that Analyst generates and then executes; storage, serverless compute or other Snowflake platform costs; or the RETRO and batch workloads as against only forward-flow underwriting. Each of those

changes what our figure should be compared *against*. Both this and the volume question need asking — volume first, because it moves the answer by four orders of magnitude.

## Arithmetic, at three volumes

**OURS x AN ASSUMED VOLUME.** The per-adjudication figure is measured; the volumes are **illustrative brackets chosen to show the range** and are **not** the customer's figures. Formula throughout: `volume x 365 x $0.14395847`.

| Applications/day through the full 8-agent path | Daily | Monthly (30d) | Annualised (365d) | Against a reported $700K–$1M | Conclusion |
|---|---|---|---|---|---|
| 3,000 | $431.88 | $12,956 | **~$158K** | 4.4x to 6.3x lower | Migration case is made on cost alone |
| 15,000 | $2,159.38 | $64,781 | **~$788K** | inside the reported band | **Genuinely ambiguous** — the case rests on latency |
| 300,000 | $43,187.54 | $1,295,626 | **~$15.8M** | 16x to 23x higher | **Reverses** — the architecture would need to change first |

At 15,000/day cost stops being the argument and the case rests entirely on latency — 31.45 s p50 against their reported 48+ hours for 4,500 applications — plus removal of eight Cortex Analyst calls and eight warehouse executions per application. That volume is not an implausible one.

At 300,000/day the port is an order of magnitude *more* expensive on model spend, and it would need batch inference for everything that is not forward-flow, a cheaper tier for the agents that measured no worse, and almost certainly a triage stage so that not every

application receives all eight specialists. There is also a hard capacity question at that volume, below.

### Break-even volume

Since the reported figure is a range, the more useful form of the question is what volume makes our measured cost equal to theirs.

| Their reported annual AI spend | Break-even at our measured p50 ($0.14396) | At the current assignment's projection ($0.12475) | At the post-2026-09-01 projection ($0.17734) |
|---|---|---|---|
| $700K | ~13,300 apps/day | ~15,400 | ~10,800 |
| $1M | ~19,000 apps/day | ~22,000 | ~15,400 |

**Below roughly 13,000 applications/day, we are cheaper. Above roughly 19,000, we are not.** Everything between is inside the reported range and undecidable without a tighter figure from them.

Note how much the September rate change moves that window. When Sonnet 5's introductory rate lapses, the break-even against $700K falls from ~13,300 to ~10,800 applications/day — a **19% narrower** window on the same architecture, from a calendar date. If the volume answer lands anywhere near the break-even, the promo expiry is not a footnote. Cost levers.

## Exact question to ask

Do **not** ask "how many applications do you process". Consortium volume is much larger than forensic-underwriting volume, and mixing them is how this question gets answered wrongly — in the direction that makes us look expensive.

Ask: *of the applications you ingest, how many per day receive the full eight-agent forensic underwriting treatment, as opposed to a score-only or single-product path?*

Three secondary figures worth asking for in the same breath, because each changes the arithmetic rather than just the total:

| Question | Why it changes the arithmetic |
|---|---|
| Is the eight-agent path applied to every application, or only above a score threshold? | If a triage stage already exists, the relevant volume may be a small fraction of ingestion — and the whole comparison resolves in our favour immediately |
| What fraction is re-run? | Repeat submissions and RETRO backfills are the workload where batch inference at −50% applies, and forward-flow is the workload where it does not. Their own reported "4,500 applications in over 48 hours" sounds exactly like a batch workload |
| Does the $700K–$1M include warehouse compute, or only the AI/Cortex line items? | Decides whether our model-spend figure is being compared with a like quantity |

## On our side of the ledger, not as a dollar figure

Two things the port removes per application. **Neither is priced here**, because pricing them would need the customer's Snowflake credit rate, their warehouse size, and their Cortex Analyst message pricing — none of which we have. What is claimable is the **count**:

| Per application | Incumbent path | Here |
|---|---|---|
| Cortex Analyst messages | 8 | 0 |
| Warehouse executions | 8 | 0 |
| LLM calls | 9 | 9 |

Their leadership often finds the count more persuasive than the latency, because they know what a warehouse-second costs and we do not. Handing them a count they can price themselves is stronger than handing them a dollar figure we had to assume our way into.

A latency argument attaches to the same change. A published production decomposition of a Cortex Agent call attributes **~27 s of a 55 s total** to Cortex Analyst SQL generation alone. **That figure is not our measurement** — it is a published third-party decomposition and is labelled as such wherever it appears. For a fixed set of fraud signal groups the queries are known at design time, so regenerating them per application buys nothing. See `docs/semantic-layer-mcp.md` and the semantic layer.

## Capacity question that only exists at high volume

If the volume answer is at the top of the range, there is a second thing to raise **before any load test**, not after.

At 300,000 applications/day x 9 calls x ~2,250 tokens the workload approaches **6.1B tokens/day**. Claude Sonnet 5's daily token ceiling is **8.64B** and is **not adjustable** — it is an account-team escalation, not a self-serve quota ticket. That is 71% of a non-adjustable ceiling on the model six of the eight specialists run on. It does not block anything at 3,000/day. It is a design constraint at 300,000/day, and it depends entirely on the answer to the same question.

## What must not be said

Recorded explicitly, because each is an easy sentence to say in a room and each is unsupported.

| Do not say | Why |
|---|---|
| *"We are 5x cheaper than Snowflake."* | True only at the low end of an unknown volume range, and it quotes their reported figure as if we had measured it |
| *"$0.14 per application versus their $700K."* | A per-unit figure against an annual total, with no volume. Not a comparison |
| *"We benchmarked Snowflake at 48 hours."* | We did not benchmark Snowflake at all. That is **their reported figure about their own platform**, and the UI deliberately contains no "AgentCore vs Snowflake" comparison for exactly this reason — the string "Snowflake" does not appear in it |
| *"Per-agent model tiering delivers the cost saving."* | The tier shape saves **$0.00** structurally, and a test asserts the arithmetic. Detail |
| *"Prompt caching cuts the input cost."* | It does not engage on this workload at all. Detail |
| *"Batch inference halves the bill."* | Worth **3.58%** on this assignment, because Bedrock offers no batch price for Sonnet 5 or GPT-5.6. Detail |
| Any annualised figure without its assumed volume stated in the same sentence | The volume is the entire uncertainty |

**The single most important open item in this engagement**

**Our measured cost is $0.14396 per adjudication over 41 priced runs; their reported spend is $700K–$1M a year; the two meet somewhere between 13,000 and 19,000 applications per day, and we do not know which side of that line they are on.**

That sentence is the honest state of the cost case, and it is one customer answer away from being a conclusion.

# Still unknown

| Missing input | What it would settle |
| --- | --- |
| **Their forensic-path volume per day** | One number. It converts this page from unresolvable to decided, in either direction |
| What the $700K–$1M line item covers | Whether it is AI-only or includes warehouse compute, and whether it includes RETRO |
| Cost of the *current* assignment | `AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --pipeline --repetitions 3` — roughly $5 of spend, and it replaces the $0.12475 projection with a measured p50 |
| Behaviour under concurrency | Applications were run **one at a time**. Nothing here says what happens when N adjudications overlap, and at any interesting volume they will |


**Cost levers**
The three levers that survive arithmetic, and the rate change that moves the break-even.

**Measured results**
Where the $0.14396 and the 31.45 s p50 come from, with their n.

‹ **What does not reduce cost**
Three things that look like cost levers and a...

**AgentCore CLI** ›
What @aws/agentcore 1.0.0-preview.22 su...

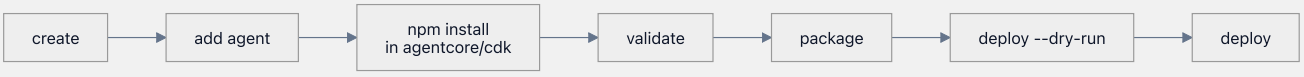
# AgentCore CLI

What @aws/agentcore 1.0.0-preview.22 supports — the two verbs that do not exist, which of the two colliding binaries silently no-ops, the real deploy order, and the rename rule that destroys resources.

*For the customer's engineer and whoever owns the deployment pipeline. Everything here was verified against the installed CLI, not read from a doc page.*

Two commands that appear in older AgentCore instructions — `configure` and `launch` — do not exist in this CLI, and one of the two programs named `agentcore` on a typical machine accepts them anyway and does nothing. Verified against `@aws/agentcore` 1.0.0-preview.22 with `bedrock-agentcore` 1.14.0, Node v22.22.0, npm 10.9.4, Python 3.12.9. Sources: `deploy/deploy.md` and `deploy/ci.sh`, both committed.

## Deploy pipeline




The order that actually deploys this project

Step 3 is the one most likely to be skipped, and the only one whose omission fails inside a TypeScript build rather than in an AWS call. Step 6 is spelled `--dry-run`, not `--plan`.

## Which binary is which

Both programs answer to `agentcore`, and `PATH` order decides which one a CI job invoked. Only one is the real CLI, and only one is dangerous.

| Behaviour on `agentcore configure` / `launch` | npm `@aws/agentcore` (real) | pip `bedrock-agentcore-starter-toolkit` (deprecated) | Verdict |
|---|---|---|---|
| Does the verb parse? | No — `error: unknown command 'configure'` | Yes | npm CLI |
| Exit code | 1 | 0 | npm CLI: a non-zero code stops a `set -e` script |
| AWS resources created | none, and it says so | **none, and it says nothing** | npm CLI |
| Effect on a green CI run | job fails, correctly | **job passes having deployed nothing** | npm CLI |
| Recommended action | keep | `pip uninstall -y bedrock-agentcore-starter-toolkit` | remove the pip package |

**A pipeline built on those verbs reports success while deploying zero infrastructure**

Failure shape: `agentcore configure --entrypoint …` prints something resembling help, `agentcore launch` prints something similar, the shell reports **exit 0**, the job goes green, and nothing was created, updated or contacted. Discovery happens when someone tries to invoke a runtime that does not exist.

`deploy/ci.sh` step 0 closes both halves before anything else runs. It greps every tracked-or-untracked-but-not-ignored file for the literal strings `agentcore configure` and `agentcore launch` and fails the build if either appears — with four documented exemptions, the files whose purpose is to warn about them. It then fails if `bedrock-agentcore-starter-toolkit` is installed at all, because with that package on `PATH` every later `agentcore` call in the script is ambiguous.

That guard exists because **this repository shipped with the failure**: the earlier `deploy/deploy.md` instructed two commands that do not exist. `requirements.txt` now lists the pip package under a heading reading DELIBERATELY ABSENT rather than simply omitting it.

## Real verb set

```
create    add       remove    validate    package    deploy
dev       invoke    logs      traces      status
fetch     exec      import    run
```

1 **Install the toolchain, and remove the collider.**

```
npm install -g @aws/agentcore      # requires Node >= 20
agentcore --version                # expect 1.0.0-preview.22
pip uninstall -y bedrock-agentcore-starter-toolkit
```

Telemetry ships **on by default**, which matters for a regulated customer. Opt out per machine with `agentcore config telemetry.enabled false`, then confirm with `agentcore telemetry status`.

2 **Install the CDK app's dependencies. Not optional.**

```
(cd agentcore/cdk && npm install)
```

Skip it and `agentcore deploy` dies inside the TypeScript build:

```
tsconfig.json(5,25): error TS5107: Option 'moduleResolution=node10' is deprecated
and will stop functioning in TypeScript 7.0.
```

Cause: `deploy` runs `npm run build` (bare `tsc`) inside `agentcore/cdk/`, and the vended project pins `typescript: ~5.9.3` as a devDependency. With no local install, `tsc` resolves to whatever is global and the vended `tsconfig.json` fails under a newer compiler. Install the pinned compiler; do **not** edit the vended tsconfig.

3 **Validate.**

```
agentcore validate --json --directory "$REPO_ROOT"
```

Non-interactive and machine-readable. Exits 1 with `{"success":false,…}` on any Zod failure. Creates nothing, needs no credentials.

4 **Package, then dry-run.**

```
agentcore package                          # writes agentcore/<runtime>.zip
agentcore deploy --dry-run --target default --json
```

`--dry-run`, not `--plan`. AWS's docs' `--plan` flag does not exist here: `error: unknown option '--plan'`. Internally the CLI maps `--dry-run` onto a `plan` option, which is probably the origin of the docs bug. A dry run returns after the deployability check and before `deployStack`, so it synthesizes CloudFormation and touches nothing.

5 **Deploy.**

```
agentcore deploy --target default --yes
```

## Project layout, and what the config demands of the code

`agentcore/` here is hand-authored, declares two runtimes and one memory, and `agentcore validate` passes on it.

- agentcore
  - agentcore.json
  - aws-targets.json
  - .cli
  - cdk

| Declared in `agentcore.json` | What `app/` must supply |
|---|---|
| `codeLocation: app/underwriter/`, `entrypoint: main.py` | `app/underwriter/main.py` and `app/underwriter/pyproject.toml`. Without the second, `agentcore package` fails with `Required project file not found: …/pyproject.toml` — the config alone is not enough. |
| Same for `app/francis/` | Same two files. |
| `runtimeVersion: PYTHON_3_12` | `requires-python = ">=3.12"` in each `pyproject.toml`. |
| `instrumentation.enableOtel: true` | `aws-opentelemetry-distro>=0.18.0` in **each** `pyproject.toml`, not only in `requirements.txt` — packaging builds each runtime's closure from its own pyproject. |
| `protocol: HTTP` | An `@app.entrypoint` whose second parameter is literally named `context`. See Runtime entrypoint contract. |
| `memories[].name: francisMemory` | Deploy injects the id as `MEMORY_FRANCISMEMORY_ID` — the name upper-cased. The session manager must read exactly that name and no-op when unset, so `agentcore dev` still works. |

## Naming constraints the schema enforces

All four are checked before any AWS call, and they contradict each other.

| Thing | Pattern | Note |
|---|---|---|
| Project name | `^[A-Za-z][A-Za-z0-9]{0,22}$` | Alphanumeric only, max 23. No hyphens, no underscores. `ppFraud` is 7. |
| Runtime / memory name | `^[a-zA-Z][a-zA-Z0-9_]{0,47}$` | Underscores allowed, **hyphens are not**. `under-writer` fails. |
| Deployed runtime name | `<project>_<runtime>`, ≤ 48 chars | `ppFraud_underwriter` is 19. |
| Deployment target name | `^[a-zA-Z][a-zA-Z0-9-]*$`, max 64 | Exactly the inverse: hyphens allowed, **underscores are not**. `pp-prod` passes, `pp_prod` fails. |

### Validation is strict at the root and lax one level down

Root spec is Zod `.strict()`, so one unknown top-level key fails outright and names itself:

```
$ agentcore validate --json
{"success":false,"error":"…/agentcore/agentcore.json:\n  - root: unknown keys (remove)
```

Nested objects are **not** all strict — an unknown key inside a `runtimes[]` entry is silently accepted. Do not rely on validate to catch a typo inside a runtime block.

Two documentation sources disagree with the schema and were checked against it. The vended `agentcore/.llm-context/agentcore.ts` declares only 15 of the root's 20 keys, and the generated `AGENTS.md` advertises `onlineInsightsConfigs` and `runtimeEndpoints` as root arrays, which the strict root **rejects** — declare either and validate fails. Authority is the compiled Zod schema at `$(npm root -g)/@aws/agentcore/dist/schema/schemas/agentcore-project.js`, nothing else.

## Where region and account come from

`agentcore create` writes `aws-targets.json` as `[]`, and this repo ships it that way. That file is the **only** place region and account are configured.

```
[
  {
    "name": "default",
    "description": "AnyCompany fraud underwriting - primary",
    "account": "123456789012",
    "region": "us-east-1"
  }
]
```

- `account` must be exactly **12 digits**; validate rejects anything else.
- `region` is a closed enum of 20 AgentCore regions. Memory is region-restricted *beyond* that enum, so a valid target does not guarantee `francisMemory` can be created there.
- `agentcore deploy` has no `--region`, no `--profile`, no `--agent`. Credentials come from the ambient chain; region and account come from this file only.

Export `AWS_REGION` explicitly, and never rely on `AWS_DEFAULT_REGION`. Python SDK region fallback is `os.getenv("AWS_REGION") or boto3.Session().region_name or "us-west-2"` (`bedrock_agentcore/identity/auth.py:339`); it **ignores** `AWS_DEFAULT_REGION` and lands in us-west-2. `MemoryClient` and `GatewayClient` do the same. A cross-region surprise here presents as an authorization failure rather than as a configuration failure, which is what makes it expensive to diagnose.

```
export AWS_REGION=us-east-1
export AWS_PROFILE=your-bedrock-profile
```

## Committed state that looks disposable and is not

`agentcore/.cli/deployed-state.json` is a build artifact that must be committed. `agentcore/.gitignore` ignores `.cli/*` and then negates this one path; do not tidy up that negation.

Reason: credential providers are created **before** CloudFormation runs, out of band, in an identity pre-step. Their ARNs are recorded **only** in that file, and the CDK app reads them back on the next synth. Losing the file orphans the providers and breaks the next deploy.

## Renaming a resource destroys it

**A resource's `name` is its CloudFormation logical id.** Renaming a runtime or a memory in `agentcore.json` **destroys and recreates** it — new ARN, new endpoint, and for a memory, contents gone. Changing any *other* field updates in place.

> ⚠ **This is a data-loss event, not an untidy diff**
>
> `deploy/iam_policy.json`'s `deployingPrincipal` carries `bedrock-agentcore:Delete*` actions specifically because a rename produces Delete+Create rather than Update. If those actions look over-broad in a review, that is the reason they are there.

# Other verified behaviours

| Behaviour | Detail |
|---|---|
| Deploy exit code 2 is a pass | 0 = deployed, **2 = deployed with post-deploy warnings**, 1 = failed with a log path under `agentcore/.cli/logs/deploy/`. A CI script testing `-ne 0` fails a successful deploy; `deploy/ci.sh` treats 2 as success. |
| `agentcore import` is not local | It publishes CDK assets to S3 **before** it can fail. Do not run it to see what it would do. This repo's runbook lists it under what it deliberately does not use. |
| Container builds are ARM64-only | Both runtimes use `"build": "CodeZip"` to sidestep needing an ARM64 Docker builder; the packaged closure is already `aarch64` (`cpython-312-aarch64-linux-gnu.so` is visible inside the zip). Switching to `Container` requires an ARM64 build path, and CodeBuild in a VPC then needs an explicit `vpcId`. |
| `runtimeVersion` defaults to `PYTHON_3_14` | Not to your interpreter. `agentcore.json` pins `PYTHON_3_12` to match the local interpreter and the tested venv. Regenerate the config and you must re-pin, or the deployed interpreter diverges from the one the 1,238 collected tests ran on. |
| `agentcore package` breaks bare pytest | `agentcore/<runtime>/staging/` is the full dependency closure, and several third-party wheels there ship test modules with colliding basenames (two copies of `annotated_types/test_cases.py`). Bare `pytest` aborts during collection with `import file mismatch` before reaching a single project test, so `deploy/ci.sh` passes `--ignore=agentcore`. Zips and staging trees are gitignored. |
| `agentcore dev --runtime` is required in scripts | With more than one runtime, `agentcore dev --logs` alone stops with `Error: Multiple runtimes found. Use --runtime to specify which one.` Bare `agentcore dev` opens an interactive picker, so the omission only bites in automation. Default port is 8080 offset by runtime index, so `francis` (index 1) lands on 8081 unless you pass `-p`. |
| There is no `agentcore destroy` | Teardown is: empty the config, then deploy the emptiness — `agentcore remove all --dry-run`, `agentcore remove all --yes`, then `agentcore deploy --target default --yes`. `--yes` is **required** on that last call: deploy detects "no resources defined" as a teardown and refuses without it. |

## Still unknown

**No deploy of this project has been run against a live AWS account.** `agentcore validate` passes and `agentcore package` produces both zips, but `agentcore/aws-targets.json` ships as an empty list, so `deploy/ci.sh` skips its dry-run step and no CloudFormation stack exists. Everything above the dry-run line is verified by execution; `deploy` itself is verified by reading the CLI's own source and compiled schema.

**Deployment**
IAM for both principals, the one-time CloudWatch Transaction Search setup, and the five-step CI gate.

**Runtime entrypoint contract**
The literally-named context parameter, async-generator SSE, and errors that arrive after HTTP 200.

‹ Snowflake comparison
Our measured $0.14396 per adjudication ag...

Runtime entrypoint contract ›
Why the second parameter must be literally...

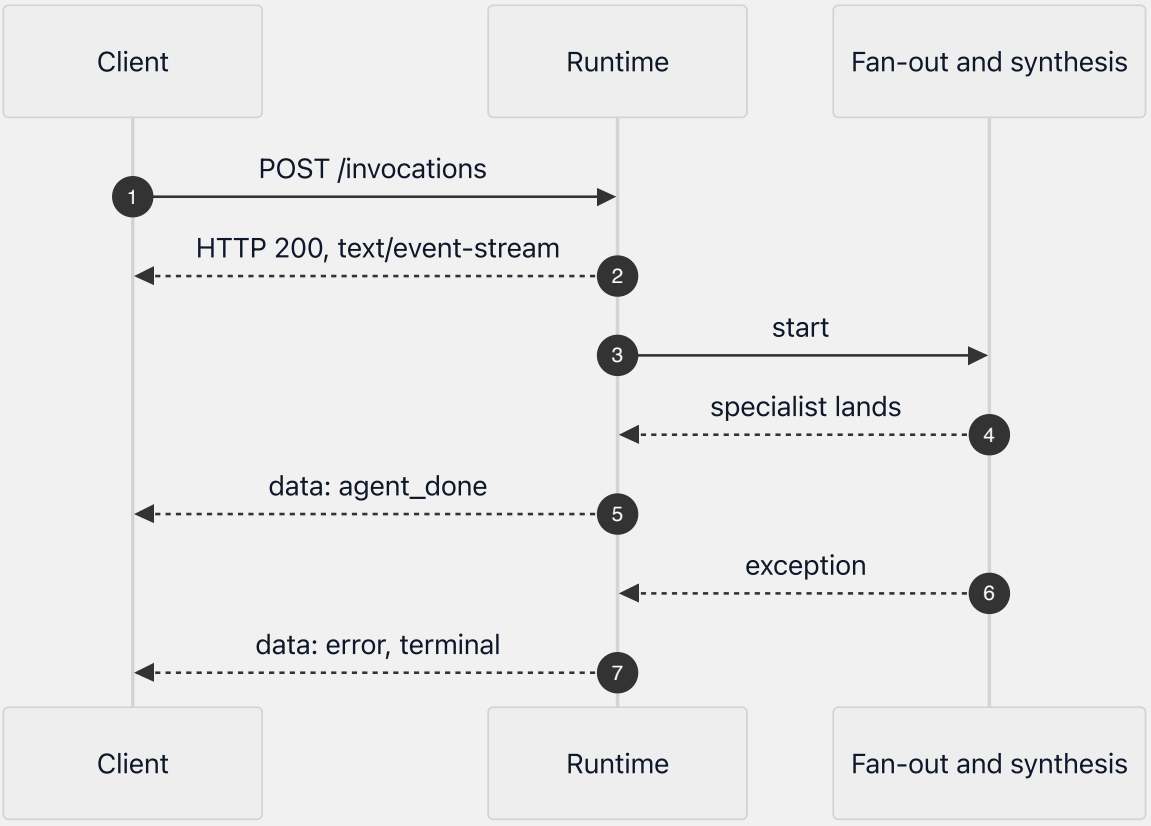
# Runtime entrypoint contract

Why the second parameter must be literally named context, how an async-generator entrypoint becomes SSE, why a failed run still returns HTTP 200, and the quotas that matter at the customer's volume.

*For the customer's engineer and for anyone writing a client against these endpoints.*

Four rules govern an AgentCore Runtime entrypoint, and three of them fail silently when broken: the second parameter must be named `context`, an async generator becomes SSE automatically, an error raised mid-stream arrives **after** HTTP 200, and only one `@app.entrypoint` survives per app. Verified against `bedrock-agentcore` **1.14.0** and re-verified on **1.19.0** — see SDK upgrade. `tests/test_runtime_contract.py` holds **60 tests** asserting everything below.

# Request lifecycle




Where the 200 is committed, and where an error can still appear

Step 2 is the whole hazard. Status is committed the moment the first byte of the stream goes out, so every later failure can only be reported *inside a frame*. A dashboard counting HTTP status codes reports 100% success on a run that failed.

Two runtimes are declared here, both `protocol: HTTP` with one entrypoint each.

| Runtime | Path | Shape |
|---|---|---|
| `underwriter` | `app/underwriter/main.py` | Async-generator entrypoint → SSE. Eight-way fan-out, then synthesis, streaming each specialist as it lands. |
| `francis` | `app/francis/main.py` | Interactive router; the eight specialists as `@tool` wrappers under the customer's maximum-2-tools-per-query cap. |

## Parameter named `context`, matched by name

Two failure shapes, depending on how the handler was written.

| Handler signature | What happens | Why |
|---|---|---|
| `def invoke(payload)` | Runs, but every log line lacks the session id and `RequestContext` is nowhere | One parameter, so the SDK never injects |
| `def invoke(payload, ctx)` | **Every request raises** `TypeError` at invocation time | Second parameter is not named `context` |
| `def invoke(payload, context)` | Correct | `_takes_context` matches by name |

`BedrockAgentCoreApp._takes_context` decides whether to inject `RequestContext` by checking that the handler's **second positional parameter is literally named** `context`. Not by type. Not by annotation. By name.

```
@app.entrypoint
async def invoke(payload, context):     # `context`, exactly
    session_id = session_id_of(context)
    ...
```

`test_second_parameter_is_literally_named_context` asserts the parameter name on both runtimes, and `test_sdk_agrees_the_handler_takes_context` asserts the SDK's own `_takes_context` returns `True` for it — so the guard runs against real SDK behaviour rather than against a restatement of it.

## Async generator returns SSE automatically

Yield plain dicts. Never hand-format SSE.

```python
@app.entrypoint
async def invoke(payload, context):
    yield {"event": "start", "session_id": session_id}
    ...
    yield {"event": "done", ...}
```

`_convert_to_sse` serialises each yielded object as `data: <json>\n\n`. Local `agentcore dev` and the deployed runtime both reply `text/event-stream` **always** — even for a one-line prompt. Every client must parse `data: {...}` frames; there is no "short answers come back as JSON" path.

Frame sequence for the batch runtime, asserted by test:

```
start
  agent_start   x8
  agent_done | failure   x8
synthesis_start
done
```

A terminal `error` frame replaces the tail on an unrecoverable failure.

### Sync versus streaming is a resource decision

The previous entrypoint here was `def invoke(payload) -> dict`. Three consequences, all fixed by the async generator:

1. No session id, per the table above.
2. **A Starlette worker held for the whole fan-out.** `_invoke_handler` routes a *sync* handler through `run_in_threadpool`, so a tens-of-seconds fan-out occupies a worker throughout.
3. Nothing to show until the end — the live UI could not render specialists as they landed.

## Errors arrive after HTTP 200

> **A 200 is not success. Every frame must be inspected for an error key**

> Symptom pair that identifies this: a monitoring dashboard shows 100% success while users report failed adjudications. An exception raised inside the generator can only be reported in a frame — `_stream_with_error_handling` catches it and yields a terminal `{"error", "error_type", "message"}` object, after the 200 has already gone out.

Three keys, deliberately distinct so one client-side check does not swallow another.

| Key | Meaning | Terminal? |
|---|---|---|
| `error` | Unrecoverable failure of the whole run. Same shape the SDK itself uses, so one check covers both. | Yes |
| `failure` | A **single specialist** failed. 7-of-8 graceful degradation; the adjudication proceeds with a documented gap. | No |
| `done` | Run completed, possibly partially. Always emitted when it completed. | Yes |

A client treating any `error`-shaped frame as fatal therefore does not abort on a degraded-but-valid run. The benchmark contains a live example: APP-1011 repetition 3 lost `synthetic` and `rings` to `server_5xx`, finished in **21.215 s**, and still validated all 21 keys (`evals/results/pipeline_14x3_us-east-1.json`).

## Timeouts

| Path | Limit | Adjustable? |
|---|---|---|
| Synchronous request | **15 minutes** | **No** |
| Streaming response | 60 minutes | — |
| Session ceiling | 8 hours | Not a request timeout at all |

That 8-hour figure appears in enough places to be worth naming: it is the session ceiling. If you have seen "8 hours" quoted as an invocation limit, this is what it was.

## Session id has a 33-character client-side minimum

`uuid4().hex` is 32 characters and is rejected with `ParamValidationError`. `str(uuid.uuid4())` is 36 with the hyphens and is accepted.

```
SESSION_ID_MIN_LENGTH = 33

def new_session_id() -> str:
    return str(uuid.uuid4())        # 36, not .hex's 32
```

`app/runtime_support.py` centralises this so nobody re-derives it, and `test_generated_session_id_meets_the_33_char_floor` asserts *both* halves — that `uuid4().hex` is below the floor, so the test's premise is checked rather than assumed, and that the generated id clears it. Note also that `RequestContext.session_id` is `Optional[str]` and is `None` whenever the header is absent, which makes a fallback mandatory rather than defensive.

## One entrypoint per app

A second `@app.entrypoint` **silently overwrites** the first. No error, no warning. Both runtimes declare exactly one, in separate `codeLocation` directories precisely so the batch and interactive paths cannot collide.

## Ping handler must reproduce the task check

`@app.ping` is consulted **before** the SDK's automatic behaviour — precedence is forced > custom > automatic. Automatic behaviour already reports `HEALTHY_BUSY` while any tracked async task is open, so a custom handler that omits that check makes a long fan-out advertise itself as idle and invites more work onto a busy microVM.

```
@app.ping
def ping_status() -> PingStatus:
    if app.get_async_task_info()["active_count"] > 0:
        return PingStatus.HEALTHY_BUSY
    return PingStatus.HEALTHY
```

One wrinkle: `@app.async_task` raises `ValueError: @async_task can only be applied to async functions` on an async generator — and a streaming entrypoint must be one. So the streaming entrypoint registers by hand with `app.add_async_task(...)` / `app.complete_async_task(...)` in a `try/finally`, keeping ping status honest for the whole run. The synthesis leg, an ordinary coroutine, does use the decorator.

## Keep `debug=False`

```
app = BedrockAgentCoreApp(debug=False)
```

Security decision, not a log-level one. With `debug=True` the SDK routes any payload carrying `_agent_core_app_action` to `_handle_task_action`, which lets a caller force the ping status and enumerate running jobs — a health-state backdoor on the `/invocations` path. `test_debug_mode_is_off` asserts it on both runtimes.

## Payload tolerance, because `agentcore invoke` only sends prompts

`agentcore invoke --runtime underwriter "Run a full underwriting review of APP-1004"` delivers a *prompt*, not a structured payload. So the entrypoint accepts all of these and extracts the application id from each:

```
{"application_id": "APP-1004"}
{"prompt": "Run a full underwriting review of APP-1004"}
{"prompt": "check app 1004 please"}
{"prompt": "app_1004"}
{"messages": [{"role": "user", "content": [{"text": "look at APP-1003"}]}]}
```

A payload with no recoverable id yields a structured `error` frame naming the accepted shapes, not a stack trace.

## Observability: no token or cost attributes come free

| Concern | What AgentCore gives you | What this repo had to add |
|---|---|---|
| Spans at all | Only with `instrumentation.enableOtel: true` and `aws-opentelemetry-distro>=0.18.0` in the runtime's own `pyproject.toml`. Missing the dependency means the `opentelemetry-instrument` wrapper cannot start and the runtime emits **zero spans**, service metrics only | the dependency, per runtime |
| Token / cost / latency span attributes | **None.** Strands also records its token histograms *without* attributes, so they cannot be sliced per agent | per-agent spend telemetry in our code, namespace `AgentCoreFraudDemo`, needing namespace-scoped `cloudwatch:PutMetricData` |
| Trace indexing | Requires the one-time account-level CloudWatch Transaction Search setup plus ~10 minutes. An empty `agentcore traces list` right after the first deploy is expected | see Deployment |
| Log and trace destinations | **Runtime only.** Memory and Gateway get none | a manual `PutDeliverySource` → `PutDeliveryDestination` → `CreateDelivery` sequence |
| Memory under `agentcore dev` | **Does not work** | the Francis session manager no-ops when `MEMORY_FRANCISMEMORY_ID` is unset, so local runs are stateless by design |

One fabrication guard belongs beside that table. When a run is offline, token counts and cost do not exist and are emitted as `null` all the way to the browser and to CloudWatch — where the datapoint is **omitted** rather than sent as `0`. A fabricated `0` in a spend metric is worse than a gap, because a gap is visible and a `0` averages in. `test_cloudwatch_emission_omits_unmeasured_datapoints_rather_than_sending_zero` asserts it.

## Quotas that matter at the customer's stated volume

> ⚠ **Raise the Sonnet 5 ceiling before any load test, not after**
>
> Arithmetic on the customer's own aspiration, labelled as theirs: **300,000 applications/day × 9 model calls × ~2,250 tokens ≈ 6.1 billion tokens/day**. Claude Sonnet 5's daily token ceiling is **8.64 billion and is not adjustable** — an account-team escalation, not a self-serve service-quota ticket.
>
> Nobody has told us how many applications/day actually take the full eight-agent forensic path. At ~3,000/day none of this matters; at 300,000/day it is the first thing to fix, and it is **the single unresolved input in the cost case**.

Two smaller quotas already shaped code here:

- **Memory `CreateEvent` throttles per (actor, session) at roughly 5 TPS.** The Francis session manager therefore uses `batch_size=5` (`FRANCIS_MEMORY_BATCH_SIZE`), asserted by `test_francis_memory_batch_size_avoids_the_create_event_tps_cap`.
- **Throttling measured zero on this workload.** `ModelThrottledException` occurred **0 times across 336 specialist calls** (`evals/results/tail_analysis.json`). Real observation, and a narrow one: the benchmark ran one application at a time.

## What this page's figures do not cover

**AgentCore Runtime overhead is NOT MEASURED anywhere on this site.** The 42-run benchmark called Bedrock directly from a developer host rather than through a deployed runtime, and the artifact says so in its own caveats: container cold start, session setup and SSE framing are not in the end-to-end figures. So the 31.45 s p50 (n=42) excludes everything on this page, and the repo names that as the single largest known gap between the measured figure and what a deployed customer would see. It is not estimated.


**AgentCore CLI**
The verbs that exist, the two that do not, and the rename-destroys-resources rule.

**Strands Agents SDK**
Why the fan-out is asyncio.gather over fresh Agent instances and not GraphBuilder.

‹ AgentCore CLI

Strands Agents SDK ›

What @aws/agentcore 1.0.0-preview.22 su...

Fresh-Agent-per-invocation and Concurren...

# Strands Agents SDK

Fresh-Agent-per-invocation and ConcurrencyException, the gather-versus-GraphBuilder decision with a column per verified constraint, two invisible concurrency ceilings, retry tuning, structured output, and where usage really lives.

*For the customer's engineer. Every constraint here was hit in this repository and is now asserted by a test.*

Eight specialists run through `asyncio.gather` over eight freshly constructed `Agent` objects, and both halves of that sentence are forced rather than chosen. Verified on `strands-agents` 1.26.0 and re-verified on 1.50.2 (see SDK upgrade). Sources: `agents/fanout.py`, `agents/models.py`, `agents/synthesize.py`.

## Fan-out primitive: gather versus GraphBuilder

`strands.multiagent.GraphBuilder` is the obvious-looking choice for a multi-agent fan-out and it was rejected on five verified grounds. Any one of them alone would decide it.

| Constraint | `asyncio.gather` (chosen) | `GraphBuilder` | Verdict |
|---|---|---|---|
| Partial failure | `return_exceptions=True` plus a per-agent `try/except` gives 7-of-8 graceful degradation | **Fail-fast on any node error** — one transient 5xx loses the whole adjudication | **gather.** One benchmark run lost two specialists and still validated 21 keys |
| Shared `Agent` instances | Not needed; one fresh Agent per specialist per request | **Forbidden** — correct in itself, but it prevents assembling the graph the way it appears to allow | gather |
| `session_manager` on nodes | Unrestricted | **Forbidden** — rules out the interactive path's memory-backed agents outright | gather |
| Typed results between stages | Objects carrying `usage`, `latency_ms`, `wall_ms`, `cost_usd`, `source` pass through intact | **Stringified between nodes** — destroys exactly the telemetry this engagement exists to produce | gather |
| Measured duration | This process's own `perf_counter()` per leg | `GraphResult.execution_time` comes back **zeroed** | **gather.** A benchmark cannot be built on a zero |

```
results = await asyncio.gather(*(run(domain) for domain in DOMAINS), return_exceptions
```

A thread pool was also tried and is redundant: `BedrockModel.stream` **already** offloads its blocking boto3 call through `asyncio.to_thread`, so wrapping a coroutine runner in `ThreadPoolExecutor` adds a layer of threads that buys nothing.

## An `Agent` instance cannot be invoked concurrently

**Build a fresh `Agent` per specialist per invocation. Nothing at module level.**

`Agent.stream_async` takes a `threading.Lock`. An `Agent` is a stateful conversational object, not a stateless client, so reusing one across concurrent invocations raises `ConcurrencyException` from code that looks perfectly ordinary. Worse, if overlap never happens in testing: `result.metrics.accumulated_usage` is **cumulative** on a reused Agent, so a module-level Agent silently inflates every measurement after the first.

Constructing eight fresh Agents per application is therefore not a workaround — it is what makes each one's `accumulated_usage` exactly that request's usage.

## Two concurrency ceilings, invisible until you hit them

Symptom for both: adding concurrency stops helping, wall clock per application rises with load in a way no single model's latency explains, and nothing errors or warns.

| Ceiling | Default | Effect at eight-way fan-out | Fix in this repo |
|---|---|---|---|
| `BedrockModel` HTTP pool | `max_pool_connections=10`, botocore's default | Eight specialists × N applications all queue on the pool | `agents.models.shared_bedrock_client` raises it to **32** and shares one client across every model |
| `asyncio.to_thread` default executor | `min(32, cpu_count + 4)` — 15 threads on an 11-core box | Caps throughput at roughly two applications in flight **regardless of the pool size you just fixed** | `widen_default_executor`, `EXECUTOR_MAX_WORKERS=64`, once per loop |

Admission is then gated by an `asyncio.Semaphore` sized deliberately **above** the pool, so the pool stays the binding constraint and the queue forms in Python where it is observable rather than inside botocore where it is not.

## `read_timeout` is silently dropped when you pass a config

**Supplying boto_client_config resets read_timeout from 120 to 60**

Strands sets `read_timeout=120` on the client it builds for you. Supplying your own `boto_client_config` **silently replaces** that, dropping the timeout back to botocore's default of

> 60. Nothing warns you, and the symptom is reads failing at 60 s on calls that used to survive to 120.
>
> This bites concretely here: the benchmark's slowest single specialist call was **69.78 s** (`bustout`, `evals/results/pipeline_14x3_us-east-1.json`). At a 60 s read timeout that call does not return at all.

```
Config(
    max_pool_connections=32,     # MAX_POOL_CONNECTIONS
    read_timeout=120,            # READ_TIMEOUT_S — must be restated, not inherited
    connect_timeout=10,          # CONNECT_TIMEOUT_S
    retries={"mode": "adaptive"},
)
```

## Unknown config keys are accepted with only a `UserWarning`

`BedrockConfig` accepts unknown `**model_config` keys with **only a** `UserWarning`. A kwarg renamed between releases, or simply mistyped, does not raise — it stops taking effect. Run CI with `-W error::UserWarning` or a typo becomes permanent silence.

That behaviour is also why `requirements.txt` pins the three libraries whose API surface this repo reads directly to **exact** versions rather than floors, with the reasoning written in the file beside each pin.

## Retry tuning: the SDK default costs 4 s on the first throttle

```
from strands.agent import ModelRetryStrategy      # note the import path

ModelRetryStrategy(max_attempts=4, initial_delay=1, max_delay=8)
```

Read against `strands.event_loop._retry`, that is a ladder of **[1, 2, 4] s** — 7 s worst case. The SDK's default is `[4, 8, 16, 32, 64, 128] s`, so **one throttle costs 4 s against a sub-60-second SLA** and a second costs 12.

Two properties bound what a retry can explain:

- It fires on `ModelThrottledException` **and nothing else.** A 5xx is not retried, which is why the one degraded run failed *fast* — 21.215 s end to end — rather than slowly.
- A retry sleeps **inside** the call, so it appears as wall clock exceeding the model's own reported `latencyMs`. That gap is computable per call, which made "is the tail retries?" falsifiable rather than assumed. Measured: **0** `ModelThrottledException` **in 336 specialist calls**, wall-clock-minus-SDK-latency gap **0.73 s p50 / 1.69 s p95 over n=334** (`evals/results/tail_analysis.json`). The tail sits inside the model's own reported latency. See Latency tail.

## Structured output is a kwarg, and `Optional` is a trap

```
Agent(..., structured_output_model=Adjudication)   # correct
agent.structured_output(...)                        # DEPRECATED
```

An `Optional` field drops out of the generated JSON Schema's `required` array. On a contract whose whole point is **exactly 21 keys in the customer's order**, a field the model is not required to emit is a contract violation waiting to happen. `agents/contracts.py` declares all 21 as required, in order, with no `Optional`. Across the 42 benchmark runs `repair_attempts` totalled **0** and every run validated all 21 keys.

## Two settings that are wrong by default for a stateless call

```
Agent(
    ...,
    callback_handler=None,                          # the default PRINTS EVERY TOKEN to
    conversation_manager=NullConversationManager(),
)
```

`callback_handler` is the surprising one: a production runtime on the default handler streams every token of every specialist to stdout, and therefore into CloudWatch Logs.

## Where usage and latency actually live

Everything the benchmark reports comes from these paths and nowhere else.

| Field | Path | Note |
|---|---|---|
| Token usage | `result.metrics.accumulated_usage` | **Cumulative on a reused Agent** — a fresh Agent per invocation is what makes it per-request |
| Cache tokens | `…accumulated_usage.get("cacheReadInputTokens")` / `…("cacheWriteInputTokens")` | **Optional keys** in the TypedDict. Use `.get()`; direct indexing raises |
| Non-cached input | `…["inputTokens"]` | Converse counts **only non-cached** tokens here. Responses' `input_tokens` is the *total* including cached — `agents/mantle.py` normalises to the Converse convention so one rate table prices both families |
| Model latency | `result.metrics.accumulated_metrics["latencyMs"]` | The model's own reported latency, narrower than wall clock |
| Wall clock | this process's own `time.perf_counter()` | Reported separately from model latency, never merged |

Offline means `None`, never zero: in `MOCK_MODE` there is no usage, so `usage` is `None` and `cost_usd` is `None` — never a plausible-looking guess. The `source` field records which transport produced a row (`bedrock` / `bedrock-mantle` / `mock`) so a report can separate them rather than pooling a Converse call, a Responses call and a mock under one label.

## Two transports, chosen by the model id alone

`openai.gpt-5.*` ids are **not on Bedrock Converse** — `converse` rejects them with `ValidationException: The provided model identifier is invalid` — and are reachable only

through the `bedrock-mantle` Responses endpoint. So a specialist's model assignment alone decides its transport:

```
transport_for(model_id)   # reads agents.models.SPECIALIST_MODELS; mantle.is_mantle_mo
```

No flag, no second switch. `agents.models.SPECIALIST_MODELS` is the only place the decision is expressed. The mantle result is wrapped in an adapter carrying the same surface the Converse path returns (`message`, `usage`, `latency_ms`, `stop_reason`) so result assembly is **shared** rather than duplicated per family.

Three things the mantle path cannot honestly report, and does not:

| Field | Value on a mantle call | Why not something more plausible |
|---|---|---|
| `stop_reason` | `None` | `mantle.complete` does not surface the Responses `status`, so `"end_turn"` would be a guess |
| `will_cache` | `None` | `cachePoint` and minimum cacheable prefixes are Converse concepts; `False` would be a Converse verdict about a request that carried no cache point |
| `cost_usd` | raises on an unpriced id | An unpriced GPT id surfaces as a recorded error rather than as a call reported at $0 |

## Still unknown

**Nothing here was measured under concurrent load.** Both concurrency ceilings above are read from the libraries' own defaults and from the code that widens them; the benchmark ran **one application at a time**, so no measurement on this site says where the widened pool or the widened executor actually binds. The semaphore's sizing rationale is an argument, not a measurement.

**GPT-5.6 on bedrock-mantle**
The two base-URL paths, bearer-token auth,

**SDK upgrade**
1.26.0 to 1.50.2, verified by running the whole

and the four measured combinations.

suite on both, and the one real incompatibility.

‹ Runtime entrypoint contract

Why the second parameter must be literally...

GPT-5.6 on bedrock-mantle ›

The endpoint serving OpenAI models on A...

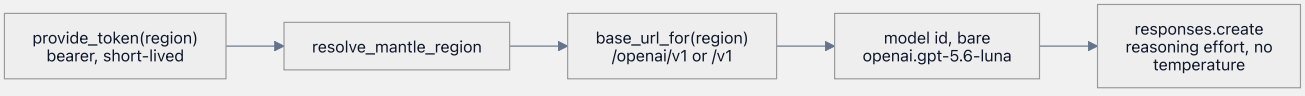
# GPT-5.6 on bedrock-mantle

The endpoint serving OpenAI models on Amazon Bedrock — hostname, the two mutually exclusive base-URL paths with all four measured combinations, bearer-token auth, the third IAM prefix, rates, and token accounting against Converse.

*For the customer's engineer. Every claim was verified live on 2026-07-28 against us-east-1.*

`bedrock-mantle` is a different endpoint from Bedrock Converse, with a different auth mechanism, a different IAM action prefix, no inference-profile prefixes, and two base-URL paths that each serve a disjoint set of models. Sources: `agents/mantle.py`, which carries the measurements as module data rather than as comments, and `tests/test_mantle.py` (82 tests).

## Auth and routing path




From credentials to a model, in the order the module does it

Each stage has its own failure mode and none of them says what it means: a missing token presents as `MantleUnavailable`, a wrong path presents as a 400 blaming the *model*, and a wrong hostname presents as NXDOMAIN that reads like a VPC problem.

## Why this endpoint is in the architecture at all

Master synthesis runs on **GPT-5.6 Luna**, and it is there on a measurement rather than a preference. GPT-5.x models are **not served on Bedrock Converse or InvokeModel** — `strands.models.BedrockModel` cannot reach them, and `converse` rejects the id with `ValidationException: The provided model identifier is invalid`. Only the `bedrock-mantle` endpoint serves them, through the OpenAI Responses API.

Eight *specialists* stay on Claude. That decision and the one deterministic metric that made it are on Cross-family study.

## Base URL: two paths, disjoint model sets

Hostname is `bedrock-mantle.{region}.api.aws` — **not** `.amazonaws.com`, which does not exist. Two paths sit on that host and they are **mutually exclusive**. All four combinations were measured, 2026-07-28, us-east-1.

| Base URL path | Model | Result | Serves |
|---|---|---|---|
| `/v1` | `openai.gpt-oss-120b` | 200 | open-weight `gpt-oss` |
| `/v1` | `openai.gpt-5.6-luna` | 400 `validation_error` — *"does not support the '/v1/responses' API"* | — |
| `/openai/v1` | `openai.gpt-oss-120b` | 400 `validation_error` | — |
| `/openai/v1` | `openai.gpt-5.6-luna` | 200 | frontier `gpt-5.6` |

Neither path serves both families.

> ⚠ **Strands' own documentation leads you into the 400**
>
> Strands' docs for `OpenAIResponsesModel` show `/v1` with `openai.gpt-oss-120b`. That pair is correct — and **wrong for GPT-5.6**. Following it verbatim produces the 400 above, whose

wording points at the model rather than at the path, which is what makes it cost an afternoon.

Both templates are named constants in `agents/mantle.py` so the distinction is discoverable rather than folklore:

```
MANTLE_BASE_URL_TEMPLATE            = "https://bedrock-mantle.{region}.api.aws
MANTLE_OPEN_WEIGHT_BASE_URL_TEMPLATE = "https://bedrock-mantle.{region}.api.aws
```

Nothing in the module calls the second — `gpt-oss` models are also on Converse and go through Strands normally — but it is recorded so nobody re-derives the four-way table.

## Bearer token, not SigV4 request signing

```
from aws_bedrock_token_generator import provide_token

OpenAI(base_url=base_url_for(region), api_key=provide_token(region=region))
```

Tokens are short-lived, so `agents/mantle.py` mints one **per call** rather than caching across a long-running runtime. Holding a model object for hours — which the native Strands provider invites, because the token is baked into `client_args` at construction — requires rebuilding it, and `build_responses_model`'s docstring says so for that reason.

**IAM needs** `bedrock-mantle:CreateInference`, AWS managed policy `AmazonBedrockMantleInferenceAccess`. That is a **third** action prefix, distinct from both `bedrock:` and `bedrock-agentcore:`, so a principal with full `bedrock:*` cannot call this endpoint. Missing it raises `MantleUnavailable` at token-mint time, with the module's own message naming the action.

## Model ids, regions, and rates

There is **no** `global.*` **or** `us.*` **prefix and no cross-region inference** here. The id is bare (`openai.gpt-5.6-luna`) and the call is **in-region only**.

Rates in $ per 1M tokens, read from `MANTLE_RATES_PER_MTOK`:

| Model | Input | Cache read | Output | Available in |
|---|---|---|---|---|
| `openai.gpt-5.6-luna` | 1.10 | 0.11 | 6.60 | us-east-1, us-east-2, us-west-2 |
| `openai.gpt-5.6-terra` | 2.75 | 0.28 | 16.50 | us-east-1, us-east-2, us-west-2 |
| `openai.gpt-5.6-sol` | 5.50 | 0.55 | 33.00 | **us-east-1, us-east-2 only** |
| `openai.gpt-5.5` | 5.50 | 0.55 | 33.00 | us-east-1, us-east-2 |
| `openai.gpt-5.4` | 2.75 | 0.275 | 16.50 | us-east-1, us-east-2, us-west-2, us-gov-west-1 |

**Cache-write tokens bill at 1.25x input, and omitting them under-reported a call by 62%**

GPT-5.6 bills tokens written to cache at **1.25× the uncached input rate** on a 30-minute cache, and implicit caching is **on by default**. So a first call against a large prompt reports most of its input as `cacheWriteInputTokens`, and that term dominates the bill.

Measured: a Luna synthesis reported `cacheWriteInputTokens=6740`, worth **$0.00927 of a $0.01499 call — 62% of the total.** Drop the term and you under-report by that much.

GPT-5.4 and GPT-5.5 cache automatically with **no write charge**, so their multiplier is `0.0`. Both sit in `CACHE_WRITE_MULTIPLIER` rather than being assumed. `rates_for()` **raises** on an unpriced id instead of defaulting, so a new model cannot enter a cost-reported path silently.

## Token accounting against Converse

Same word, two meanings. Normalisation is what lets `agents/pricing.py` stay correct across both families without a per-family branch in the cost path.

| Concern | Bedrock Converse | Responses API | How `agents/mantle.py::_usage` reconciles |
|---|---|---|---|
| `inputTokens` / `input_tokens` | non-cached only | total prompt, cached included | subtracts cached and written from the total |
| Cached tokens | separate `cacheReadInputTokens` | inside `input_tokens` | carried out into its own key |
| Cache-written tokens | separate `cacheWriteInputTokens` | inside `input_tokens` | carried out into its own key |
| Net effect on billing | one rate table applies | would double-count without the subtraction | a cached token is billed **once**, at the cache rate, for both families |

```
{
  "inputTokens": max(total_in - cached - written, 0),
  "outputTokens": out,
  "cacheReadInputTokens": cached,
  "cacheWriteInputTokens": written,
  "totalTokens": total_in + out,
}
```

## Reasoning models ignore `temperature`

```
client.responses.create(
    model=model_id,
    input=prompt,
    reasoning={"effort": resolved_effort},   # not temperature / top_p / top_k
    max_output_tokens=max_output_tokens,
    store=False,
)
```

`temperature`, `top_p` and `top_k` are **ignored**. Effort values are `none | low | medium | high | xhigh | max`. Measured on this repo's synthesis prompt, `low` and `medium` both returned a

valid 21-key object; `low` **is the default** here because the master prompt is an explicit instruction set rather than a puzzle.

Effort is turned up deliberately in exactly one place: the GPT judge (`evals/judge_sol.py`, `effort="high"`), where the whole value of a second judge is that it reasons independently instead of pattern-matching prose.

## Region is a measured choice, not a deployment accident

`resolve_mantle_region` deliberately does **not** read `AWS_REGION`. Where the stack happens to be deployed says nothing about which region answers a mantle call fastest, and it measurably matters: with `AWS_REGION=us-west-2` an earlier revision sent every Luna synthesis to us-west-2, which measured **5.02 s p50 against us-east-1's 4.61 s** (n=3 each, on the real 31,343-character prompt) — an 8.9% latency tax paid for nothing. A mantle call is in-region only with no cross-region profile, so co-locating it with the Converse specialists buys nothing either.

Precedence, most explicit first:

1. an explicit `region=` argument,
2. `BEDROCK_MANTLE_REGION`, the operator override for data residency, a new region, or simply disagreeing with the measurement,
3. the measured-fastest region for that model,
4. the model's first declared available region,
5. `DEFAULT_MANTLE_REGION` = `us-east-1`.

Steps 1 and 2 remain subject to the availability check: a requested region the model is not served in is **replaced**, because the alternative is a confusing 404. An *unmeasured* model falls back to declared availability order — absence is treated as "unknown", never as "slow". The full matrix, why a short-prompt probe ranks regions differently, and the third-party readings it is checked against are on **Region matrix**.

## Native Strands provider

`strands.models.openai_responses.OpenAIResponsesModel` is the ergonomic way to reach this endpoint: a GPT model becomes an ordinary `strands.Agent` — same fan-out, same hooks, the customer's verbatim prompt in a real `system_prompt` rather than concatenated into one Responses `input` string, and the same `result.metrics.accumulated_usage` surface.

It was **not usable on** `strands-agents==1.26.0` for two verified reasons: it does not exist there (`ImportError: No module named 'strands.models.openai_responses'`), and it requires `openai>=2.0.0` against an installed 1.107.3. Both were resolved by the SDK upgrade to `strands-agents==1.50.2` and `openai>=2.49.0`. `agents.mantle.responses_provider_available()` reports which path is live, and `build_responses_model()` returns the native object.

```
OpenAIResponsesModel(
    model_id=model_id,
    client_args={
        "api_key": provide_token(region=resolved_region),   # SHORT-LIVED
        "base_url": base_url_for(resolved_region),          # must end /openai/v1
    },
    params={"max_output_tokens": max_tokens,
            "reasoning": {"effort": resolved_effort},
            "store": False},
)
```

Migrating does not delete this module. When the native provider takes over, `agents/mantle.py` collapses to a thin factory — but the **cost and region tables stay**, because Strands prices nothing and chooses no region. The 1.25× cache-write multiplier, the per-variant rate card, the availability map and the measured latency matrix have no home in the SDK.

## Re-deriving the measurements instead of trusting them

The latency matrix is re-derivable, and the command is double-gated exactly like the benchmark's live mode, because it is *trials × models × regions* billed calls on a 31,343-character prompt.

```
# Print the recorded table and spend nothing.
python3 -m agents.mantle

# Re-measure. Needs BOTH the flag and the env var.
AGENTCORE_MANTLE_ALLOW_MEASURE=1 python3 -m agents.mantle --measure

# Price it first.
AGENTCORE_MANTLE_ALLOW_MEASURE=1 python3 -m agents.mantle --measure --dry-run
```

Four refusals are built into that path, each because the alternative is a misleading number.

| Refusal | Reason |
| --- | --- |
| `--measure` without `AGENTCORE_MANTLE_ALLOW_MEASURE=1` | Two gates, deliberately |
| `--trials` below 3 | A cell below the minimum is dropped from the ranking anyway, so the run would spend money and change nothing |
| An **unpriced** model id | Refuses to measure what it cannot cost |
| A prompt more than 25% away from 31,343 chars | Labelled **NOT comparable** with the recorded table, because the ranking turns on the TTFT-versus-throughput balance, a function of prompt and reply size |

## Still unknown

**The spend estimate for a measurement run is an upper bound, not a measurement**, and its basis is stated where it is printed: input estimated at chars/4, because there is no `CountTokens` API on this endpoint and a real pre-flight count is unavailable, priced at the 1.25× cache-write rate; output priced at the full `max_output_tokens` cap, which no measured call reached.

**Region matrix**
Eight measured cells, n=3 each, and why a 32-token probe ranks regions differently.

**Synthesizer selection**
The measurement that moved synthesis to Luna, and why Sol is the worst GPT variant here.

‹ Strands Agents SDK
Fresh-Agent-per-invocation and Concurren...

SDK upgrade ›
strands-agents 1.26.0 to 1.50.2 and bedrock...

# SDK upgrade

strands-agents 1.26.0 to 1.50.2 and bedrock-agentcore 1.14.0 to 1.19.0 — verified by running the whole suite on both versions, the one real incompatibility, and what a future upgrade should check.

*For the customer's engineer and whoever maintains this after the engagement.*

One coupled upgrade, one capability motivating it, and exactly one incompatibility — in a test, not in product code. Version numbers and the verification method are recorded in `requirements.txt` itself, with the reasoning in comments beside each pin.

## What moved

| Package | Was | Now | Pin style | Why that style |
|---|---|---|---|---|
| `strands-agents` | 1.26.0 | **1.50.2** | exact | this repo reads its API surface directly |
| `strands-agents-tools` | — | **0.8.5** | exact, in lockstep | same release train |
| `bedrock-agentcore` | 1.14.0 | **1.19.0** | exact | the entrypoint contract depends on near-private behaviour |
| `openai` | 1.107.3 | **>=2.49.0** | floor | required by the native Responses provider |
| `boto3` | — | >=1.43.54 | floor | ships new Bedrock model ids continuously; pinning would freeze out model releases |
| `pydantic` | — | >=2.13 | floor | structured-output validation only |

Three exact pins and three floors, and the split is deliberate. From `requirements.txt`: every pin is exact for the three libraries whose API surface this repo reads directly, because loose floors are the failure mode being guarded against — `BedrockModel` accepts unknown `**model_config` keys with only a `UserWarning`, so a kwarg that moved between releases does not raise, it stops taking effect. 1.43.54 is the verified-working `boto3` floor: Converse plus `cachePoint` `SystemContentBlock` plus `global.*` inference profiles.

## Why upgrade at all

One reason, and it is a capability rather than hygiene: `strands.models.openai_responses.OpenAIResponsesModel` **is absent from 1.26.x.** It is the *native* way to reach GPT-5.6 on the `bedrock-mantle` endpoint, and with it a GPT agent becomes an ordinary `strands.Agent` — same fan-out, same hooks, the customer's verbatim prompt in a real `system_prompt` rather than concatenated into one Responses `input` string — instead of a second bespoke code path. It also needed `openai>=2.0.0` against an installed 1.107.3, so this was one coupled move rather than two.

## How it was verified

**The full suite ran against both versions in isolated virtualenvs, and both reported the identical 1206 passed / 0 failed** — after one real incompatibility was found and fixed. That is the claim recorded in `requirements.txt` at the time of the upgrade.

Re-run today on 1.50.2 / 1.19.0, excluding the two docs-guard files, the committed suite reports:

```
$ MOCK_MODE=1 python3 -m pytest tests/ -q -p no:cacheprovider \
    --ignore=tests/test_docs_site.py --ignore=tests/test_docs_readability.py
1214 passed, 1 skipped, 51 warnings in 11.63s
```

That count differs from 1206 because tests were added after the upgrade landed, not because the result changed. Current collection across the whole suite is 1,238; see Test suite for the per-file breakdown.

Running the suite on both versions is the only method that answers the question that matters here, which is not "does it import" but "does anything this repo *relies on* still behave the same way". Two reasons that mattered concretely:

- Unknown `BedrockConfig` keys are accepted with only a `UserWarning`, so a kwarg that moved would have produced a **silent** behaviour change rather than a failure.
- `1.35.0` alone added `plugins` and `concurrent_invocation_mode` to `Agent`, and `service_tier` to `BedrockConfig`. Nine minor releases separate 1.26 from 1.50, so the API surface moved materially even though nothing this repo used was removed.

## One real incompatibility

**A test named a private attribute, and the rename broke the test for no product reason**

A test asserted `BedrockModel._supports_caching`. On 1.50 that private attribute is refactored into `_cache_strategy`, so the test failed on the new version while the *product* behaviour was identical on both.

Fix was to stop naming a private attribute, not to rename it. The test now asserts this repo's own public guard, which holds on both versions:

```
assert m.supports_prompt_caching(m.LIGHT_MODEL) is True
assert m.supports_prompt_caching("global.amazon.nova-pro-v1:0") is False
```

The comment left in `tests/test_pricing.py` states the rule for next time: a test naming either private name breaks on the other version for no product reason, and what must hold across both is that this repo's own guard agrees the model can cache.

That is the whole list. One failure, in a test, caused by reaching into a private attribute. **No product code changed for the upgrade.**

## What was re-verified as still true

This upgrade was additive — `Agent` kwargs were added, none removed, and every symbol this repo imports resolves. Each depended-on behaviour was re-checked rather than assumed.

| Library | Behaviour | Still true |
|---|---|---|
| strands 1.50.2 | `BedrockModel`'s flat `**model_config` | Yes — still no `params` kwarg |
| strands 1.50.2 | `structured_output_model=` on `Agent(...)` | Yes, and `Agent.structured_output()` remains deprecated |
| strands 1.50.2 | `ModelRetryStrategy` import path `from strands.agent import …` | Yes |
| strands 1.50.2 | `threading.Lock` in `stream_async` raising `ConcurrencyException` on a concurrently reused `Agent` | Yes — the constraint the whole fan-out design rests on |
| bedrock-agentcore 1.19.0 | `_takes_context()` matching the second parameter **literally by the name** `context` | Yes |
| bedrock-agentcore 1.19.0 | Async-generator entrypoint auto-converted to `text/event-stream` | Yes |
| bedrock-agentcore 1.19.0 | `tests/test_runtime_contract.py` (**60 tests**) | Passes on both versions |

Those last two behaviours are the entrypoint contract in its entirety, and they are exactly the pair a silent change would be hardest to notice. See **Runtime entrypoint contract**.

## What a future upgrade should check

Not a generic checklist — the specific things that would break *this* repository, in priority order.

1. **Run the whole suite in an isolated venv on both the old and the new version.** Not just the new one. A pass on the new version alone cannot distinguish "still correct" from "the test was always vacuous".
2. **Run with `-W error::UserWarning`.**

`BedrockConfig` swallows unknown keys with a warning. Without this flag a kwarg that moved between releases stops taking effect and no test fails. Highest-value single guard for a Strands upgrade.

3 **Re-assert the two AgentCore entrypoint behaviours by *calling the SDK*, not by restating them.**

`test_sdk_agrees_the_handler_takes_context` invokes the SDK's own `_takes_context` rather than re-implementing the name check. A restated rule cannot detect the rule changing.

4 **Check `ConcurrencyException` still fires on a reused `Agent`.**

If it ever stops, that is not permission to share instances — `accumulated_usage` is still cumulative on a reused Agent, so per-request telemetry would go wrong silently while the exception stopped warning about it.

5 **Grep the test suite for private attributes before starting.**

The one incompatibility found here was exactly that. `_`-prefixed names in assertions are upgrade debt.

6 **Re-check whether `agents/mantle.py` can shrink.**

The native `OpenAIResponsesModel` now exists. Check each release against `responses_provider_available()` and against whether base-URL and effort handling can move into the provider. The **rate card, availability map and measured region matrix stay regardless** — Strands prices nothing and chooses no region.

7 **Re-pin `runtimeVersion` if `agentcore.json` was regenerated.**

CLI default is `PYTHON_3_14`; this project pins `PYTHON_3_12` to match the interpreter the suite ran on. An AgentCore-CLI concern rather than a Python-package one, but it belongs in the same pass. See AgentCore CLI.

## What the upgrade did not do

**No live measurement was re-run.** Every latency and cost figure on this site was measured under `strands-agents 1.26.0` / `bedrock-agentcore 1.14.0`, and the 42-run benchmark records exactly those versions in its `run.packages` block (`evals/results/pipeline_14x3_us-east-1.json`).

There is no reason to expect client-library versions to move Bedrock's own latency, and the one behaviour that could have — the retry ladder — was re-verified as unchanged. But the honest statement is that the benchmark was taken on the older pair and has not been re-taken on the newer one. Nothing here presents it otherwise.


**Strands Agents SDK**
Every constraint the upgrade had to preserve, and why the fan-out is shaped the way it is.

**Test suite**
What the mutation tests prove, and why a test never names a private SDK attribute.

‹ **GPT-5.6 on bedrock-mantle**
The endpoint serving OpenAI models on A...

**Deployment** ›
The real deploy path end to end — the five-...

# Deployment

The real deploy path end to end — the five-step CI gate, IAM for both the runtime execution role and the deploying principal, the one-time CloudWatch Transaction Search setup, and what is still open before production.

*For the customer's engineer and their platform team.*

Seven steps deploy this project, five CI checks gate it, and **two** IAM documents govern it — one per principal, and merging them is the mistake to avoid. Summarised from the committed runbook at `deploy/deploy.md`, the gate at `deploy/ci.sh`, and the policies at `deploy/iam_policy.json`. Verified against `@aws/agentcore` 1.0.0-preview.22. Read AgentCore CLI first if you have not: the two dead commands are why this runbook exists.

## Path

1. **Install the toolchain, remove the colliding binary, install the CDK deps.**

```
npm install -g @aws/agentcore
agentcore --version                                  # 1.0.0-preview.22
pip uninstall -y bedrock-agentcore-starter-toolkit   # not optional — see the CL
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
(cd agentcore/cdk && npm install)                    # MANDATORY
```

2. **Author the deployment target.** `agentcore/aws-targets.json` ships as `[]`; region and account come from **that file only**, and `deploy` has no `--region` / `--profile` / `--agent`.

```
export AWS_REGION=us-east-1            # explicitly. AWS_DEFAULT_REGION is IGNORE
export AWS_PROFILE=your-bedrock-profile
```

3. **Enable Bedrock model access in that region — for the `global.*` prefix.**

Configuration targets `global.*` inference profiles because `us.*` costs exactly **1.10×** on every rate category. A policy allowing only `us.anthropic.*` **silently forbids the cheaper prefix**, and presents as an access-denied error rather than as a cost decision. See **Cost levers**.

4 **Run the gate.**

```
bash deploy/ci.sh
```

5 **Deploy.**

```
agentcore deploy --target default --yes
# --verbose for resource-level CloudFormation events; --json for machine-readabl
```

Order of operations: resolve the target → validate the project spec → `npm run build` in `agentcore/cdk/` → **create credential providers out of band** → synthesize CloudFormation → bootstrap the environment if needed (**only with** `--yes`; otherwise it stops and tells you) → check stack deployability → deploy stack `AgentCore-ppFraud-default` → persist deployed state → enable Transaction Search.

Exit **2 means deployed with post-deploy warnings — a pass.** `ci.sh` treats it as one.

6 **Invoke.**

```
agentcore invoke --runtime underwriter --target default --stream \
  "Run a full underwriting review of APP-1004"

SESSION_ID="$(python3 -c 'import uuid; print(uuid.uuid4())')"   # 36 chars, not
agentcore invoke --runtime francis --session-id "$SESSION_ID" --user-id analyst-
```

`--user-id` becomes the memory `actorId`, so it must be stable per analyst for long-term memory to accumulate against the right namespace.

## CI gate

`deploy/ci.sh` is non-interactive and **creates no AWS resources**.

| # | Step | Needs AWS? |
|---|---|---|
| 0 | **Dead-command guard** — fail if `agentcore configure` / `agentcore launch` appear anywhere in tracked-or-untracked-not-ignored files; fail if the pip starter toolkit is installed | no |
| 1 | `agentcore validate --json` — Zod-validate `agentcore/*.json`, root spec is `.strict()` | no |
| 2 | `pytest -q --ignore=agentcore` | no |
| 3 | `agentcore package` — build each runtime's CodeZip closure | no AWS, but hits PyPI |
| 4 | `agentcore deploy --dry-run --target <target> --json` | **read-only** credentials |

Step 4 skips automatically while `aws-targets.json` is still `[]` — a dry run could then only fail with `Target "default" not found`, which is not a code defect — or on `CI_SKIP_DRY_RUN=1`. It also asserts the region is explicit:

```
: "${AWS_REGION:?AWS_REGION must be set explicitly before a dry-run deploy}"
```

Three details in that script exist because of a real failure, and are worth not "simplifying":

- `--ignore=agentcore` on pytest. A previous `agentcore package` leaves `agentcore/<runtime>/staging/`, the full dependency closure, and several third-party wheels there ship test modules with colliding basenames (two copies of `annotated_types/test_cases.py`). Bare `pytest` aborts during collection with `import file mismatch` **before reaching a single project test.**
- **Exit code 2 treated as success.** Otherwise the gate fails a successful deploy.
- **Four exemptions to the dead-command guard** — `deploy/ci.sh`, `deploy/deploy.md`, `evals/README.md`, `tests/test_bench.py` — the files whose *purpose* is to warn about the commands. `tests/test_bench.py` additionally asserts the warning is present and is not inside a runnable code block.

# IAM: two documents, two principals

`deploy/iam_policy.json` holds two policy documents with `_purpose` annotations on every statement. IAM ignores unknown keys, but some tooling rejects them, so strip them on the way in:

```
ACCOUNT_ID=123456789012

jq --arg a "$ACCOUNT_ID" '
  .deployingPrincipal
  | {Version, Statement: [.Statement[] | del(._purpose)]}
  | walk(if type == "string" then gsub("<ACCOUNT_ID>"; $a) else . end)
' deploy/iam_policy.json > /tmp/deployer.json

aws iam put-role-policy --role-name <your-deploy-role> \
  --policy-name AgentCorePpFraudDeploy \
  --policy-document file:///tmp/deployer.json
```

| | `runtimeExecutionRole` | `deployingPrincipal` | **Recommendation** |
|---|---|---|---|
| Assumed by | `bedrock-agentcore.amazonaws.com` | a human or CI identity | keep separate |
| Creates infrastructure? | **No** | Yes — CDK CloudFormation deploy | keep separate |
| Deletes resources? | No | **Yes**, because a rename is Delete+Create | keep separate |
| Calls models? | Yes: `bedrock:`, plus `bedrock-mantle:CreateInference` | No | keep separate |
| Blast radius if leaked | one runtime's model and memory access | the whole stack | **do not merge them** |

runtimeExecutionRole deployingPrincipal

What the `underwriter` and `francis` containers run as. `agentcore deploy` synthesizes one execution role per runtime via CDK; attach this document to those roles, or supply a pre-

made role per runtime through `agentcore.json`'s `executionRoleArn`. Trust policy is `bedrock-agentcore.amazonaws.com` with `aws:SourceAccount` / `aws:SourceArn` conditions, which CDK writes for you.

| Statement | What it is for |
| --- | --- |
| `InvokeClaudeFoundationModels` | On-demand model invocation. Kept **alongside** the profile statement because an inference profile fans out to the underlying foundation-model ARN in each member region, and that ARN must also be allowed. |
| `InvokeClaudeInferenceProfiles` | Both `global.*` and `us.*`. `global.*` MUST be listed — a policy allowing only `us.anthropic.*` silently forbids the cheapest profile. |
| `InferenceProfileDiscovery` | Resolve a profile to its member regions and let a healthcheck assert the model is `ACTIVE` before serving traffic. Read-only; these actions are not resource-scopable. |
| `AgentCoreMemoryDataPlane` / `AgentCoreMemoryDescribe` | Francis's session manager. Both AgentCore planes sign as `bedrock-agentcore`, so the **resource scope** is what separates control from data. |
| `AgentCoreWorkloadIdentity` | The runtime exchanges its execution identity for a workload access token before fetching any outbound credential. Required as soon as a credential provider exists. |
| `AgentCoreOutboundCredentials` + `ReadCredentialProviderSecret` | Fetch the Snowflake PAT / OAuth token through the provider instead of baking a secret into the image. AgentCore Identity keeps the material in a Secrets Manager secret under the `bedrock-agentcore-identity!` prefix — **without** `GetSecretValue` **on that prefix the token fetch fails.** |
| `InvokeSemanticLayerGateway` | SigV4-signed outbound call to the Gateway fronting the signal layer. Only needed once `agentCoreGateways` is non-empty. See Semantic layer question. |
| `RuntimeLogs` | Its own log group. Runtime — **and only Runtime** — gets its log destination wired automatically; the role still needs write access. |
| `SpanAndTraceExport` + `WriteSpansToLogs` | `aws-opentelemetry-distro`'s exporters, plus the Transaction Search destination. Neither X-Ray write action supports resource-level scoping. |
| `PerAgentSpendMetrics` | Namespace-scoped `cloudwatch:PutMetricData`. Needed because **the AgentCore SDK publishes no token or cost span attributes and Strands records its token histograms without attributes** — per-agent spend telemetry is this repo's own code. |

`bedrock-mantle` is a separate prefix. Synthesis runs on GPT-5.6 Luna, which needs `bedrock-mantle:CreateInference` (managed policy `AmazonBedrockMantleInferenceAccess`), not covered by any `bedrock:*` grant. See GPT-5.6 on bedrock-mantle.

Both documents keep `region: *` so one policy covers a us-east-1 and a us-west-2 target. Narrow it once the customer settles on a single region.

## One-time: CloudWatch Transaction Search

Skip this and you get AgentCore service metrics and **zero spans** — no per-agent latency and no token attribution to look at, which is precisely what this engagement needs to show.

`agentcore deploy` attempts it whenever the project has a Python runtime, as both of these do, and reports that transaction search is enabled and takes roughly 10 minutes to become fully active and index traces from invocations. It makes four idempotent account-level calls, which is exactly what the `EnableTransactionSearchOnce` statement grants:

1. `application-signals:StartDiscovery`
2. a CloudWatch Logs resource policy named `TransactionSearchXRayAccess` granting `logs:PutLogEvents` to principal `xray.amazonaws.com` on log groups `aws/spans` and `/aws/application-signals/data`, conditioned on `aws:SourceAccount` and an `aws:SourceArn` `ArnLike` of `arn:aws:xray:<region>:<account>:*`
3. `xray:UpdateTraceSegmentDestination` → `CloudWatchLogs`
4. `xray:UpdateIndexingRule` — rule `Default`, probabilistic, 100% sampling

If the customer will not grant those actions to the deploy role, have an admin run the equivalent once and then stop the CLI trying:

```
agentcore config disableTransactionSearch true
# or keep it on and sample less:
agentcore config transactionSearchIndexPercentage 10
```

An empty `agentcore traces list` right after the first deploy is expected rather than a bug — the ~10 minute indexing delay is real.

## Operating

```
agentcore status --json                          # all resources and their state
agentcore logs   --runtime underwriter --since 30m --level error
agentcore logs   --runtime francis --query "ConcurrencyException" --json
agentcore traces list --runtime underwriter --limit 20 --since 1h
agentcore traces get <traceId>                   # downloads to a JSON file
agentcore exec   --runtime underwriter --it      # shell into a running container
```

## Local development

```
agentcore dev --runtime underwriter --logs
agentcore dev --runtime francis --logs           # index 1 → port 8081
```

- `--runtime` is **required in non-interactive mode** with more than one runtime.
- The local server exposes `POST /invocations` and `GET /ping`, and — like the deployed runtime — replies `text/event-stream` **always**.
- **Memory is not available under** `agentcore dev`. The Francis session manager no-ops when `MEMORY_FRANCISMEMORY_ID` is unset, so local runs are stateless by design.
- Local-dev identity writes `.agentcore.json` into the **current working directory**. It is gitignored; do not commit it.

## Production hardening still open

Six gaps between this demo and production, in the order they should be closed.

| Gap | What closing it takes | Blast radius if skipped |
|---|---|---|
| Data layer | Replace the fixture backend with the Lambda Gateway target fronting Snowflake or Aurora. Nothing above the signal layer changes. Semantic layer question | the demo reads fixtures, not the customer's data |
| Guardrails | Attach a Bedrock Guardrail so SSN redaction is enforced *server-side*. General Rule 23 is enforced in the signal layer today, which is the right place but should not be the only one. General Rules coverage | one code path is the sole barrier to a raw SSN in a prompt |
| Identity | `authorizerType: CUSTOM_JWT` on the endpoints, with an `authorizerConfiguration.customJwtAuthorizer` block. Validate rejects either without the other, in both directions | calls are not attributable per underwriter |
| In-platform evaluation | `agentcore add evaluator` / `add dataset` / `run eval` / `add online-eval`. The evaluator configs and golden dataset are written and schema-validated but not registered against a deployed runtime. Eval harness | no continuous calibration check |
| Model pinning | Pin model ids to an exact version. Two current ids (`claude-sonnet-5`, `claude-opus-5`) carry no date suffix | adjudications are not reproducible across a model refresh |
| Throughput | A concurrency benchmark. Nothing measured says what happens to p95 or the throttle rate when adjudications overlap | unknown behaviour under real load |

## Still unknown

**No `agentcore deploy` has run against a live account from this repository.** Steps 1 through 4 of the CI gate pass; step 5's dry run is skipped because `aws-targets.json` is still `[]`. The one `server_5xx` pair observed at concurrency 1 is a reason to expect the throughput row above to matter, not evidence about what it will show.

AgentCore CLI
The verb set, the project layout, and the two commands that exit 0 without doing anything.

Eval harness
What each measurement script does, how to re-run it, and its spend gate.

‹ SDK upgrade
strands-agents 1.26.0 to 1.50.2 and bedrock...

Semantic layer question ›
What is fictional in the original Cortex Analy...

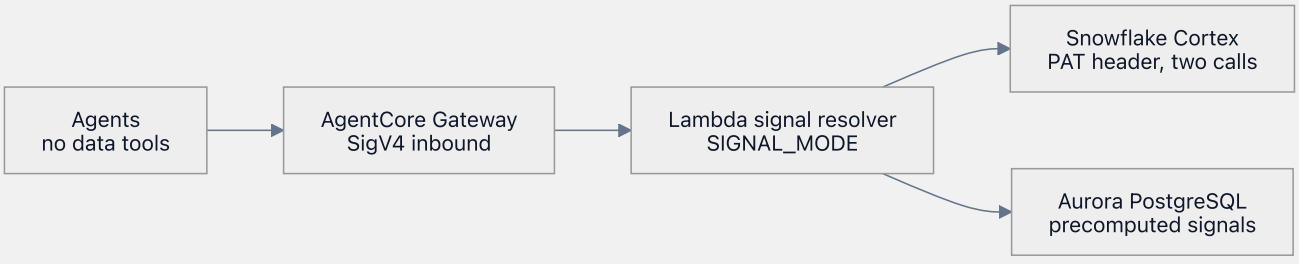
# Semantic layer question

What is fictional in the original Cortex Analyst MCP design, what Snowflake's managed MCP server actually is, the Lambda Gateway target recommendation, and the latency argument for precomputing signals.

*For the customer's engineer and for the account team. This page corrects a design still committed in this repository.*

Keeping the Cortex semantic layer is possible; the integration described in this repo's own `docs/semantic-layer-mcp.md` is not. That file and `tools/cortex_analyst_mcp.py` describe an MCP call that does not exist as written, and neither has been rewritten yet — both are on the cleanup list, and this page is the accurate statement.

## Recommended shape




Lambda Gateway target with a SIGNAL_MODE flag

Signals are computed **before** the agents run, so the Gateway leg is not on the per-application agent path at all. `SIGNAL_MODE` is a flag on a function, which makes live-Cortex and precomputed-Aurora one config change rather than a rearchitecture.

## What the customer will ask

Their stated concern, recorded in `docs/semantic-layer-mcp.md`, is that the hard part of this migration is not porting the agents but replicating the SQL guardrails built into the Cortex semantic layer. Those guardrails encode institutional knowledge: join rules, lender name normalisation, fake-record exclusion, never surfacing a raw SSN, the alert groupings, product-code decoding, active-dealer filtering, invalid-credit-value handling.

That concern is correct, and this port answers it two ways. The 29 numbered General Rules are **implemented as tested code**, 27 of 29 deterministically; and the semantic layer itself can be **kept** rather than rebuilt. What follows is what is actually possible on the second option.

## Four claims in the original design that do not hold

**Ship any of these and the failure lands after the Gateway target is already built**

1. `cortex_analyst_query(semantic_view=…, question=…)` **does not exist as written.** That tool name and argument shape are invented. Snowflake's managed MCP server has **user-defined** tool names — the fixed part is `type: "CORTEX_ANALYST_MESSAGE"` — the tool takes a single `message` argument, and the semantic view is **bound server-side** rather than passed per call. The `arguments={"semantic_view": …, "question": …}` in `tools/cortex_analyst_mcp.py::_live_retrieve` would not be accepted. Symptom: a `tools/call` failing on an unknown tool name or on unexpected arguments.

2. **Cortex Analyst** `/message` **returns generated SQL, not rows.** A second call is needed to execute it. That module's mock envelope returns `signals` and `context` populated alongside `generated_sql`, which is a **two-call** interaction presented as one. Any latency or architecture estimate built on the one-call shape is wrong by a round trip plus a warehouse execution.

3. **SigV4 outbound auth to Snowflake is impossible.** The diagram in that file shows `AgentCore Gateway --(SigV4 / OAuth)--> Snowflake`. Only **AgentCore Gateway/Runtime, API Gateway and Lambda Function URLs** verify SigV4. Snowflake does not. SigV4 authenticates *to* AWS; it is not a credential Snowflake can check.

4. **It is presented as the recommended posture.** Snowflake's own official AgentCore guide uses an **OpenAPI** Gateway target with a PAT as an API-key header, not an MCP target.

## What Snowflake's managed MCP server actually is

| Aspect | Reality |
|---|---|
| Tool names | **User-defined.** The fixed element is the tool `type: "CORTEX_ANALYST_MESSAGE"` |
| Arguments | A single `message` |
| Semantic view | **Bound server-side** in the server's configuration, not selected per call |
| Semantic model support | **Views only.** A stage-based semantic model YAML is not supported by the managed server |
| `/message` response | The **generated SQL**. Executing it is a separate call |
| Auth | Not SigV4. A PAT or OAuth token, presented as a header |

**One unanswered question gates the whole "keep the semantic layer over MCP" option: is their Cortex Analyst backed by a semantic VIEW or a stage-based semantic model YAML?** The managed server supports views only. If theirs is a stage-based YAML, the managed form of this option does not exist and the conversation becomes the OpenAPI path or porting. See Open questions.

## Why a Lambda target rather than an MCP target

Three shapes were considered, and the differences are not stylistic.

| Concern | Lambda Gateway target (recommended) | MCP Gateway target to Cortex | Data tool on the agent | Verdict |
|---|---|---|---|---|
| Two-call Cortex interaction | ordinary code inside the function | a second agent turn | a second agent turn | **Lambda** |
| Live-Cortex vs precomputed swap | one env var, `SIGNAL_MODE` | rebuild the target | rewrite the agent | **Lambda** |
| Does the agent get a data tool? | **No** | yes | yes — and every specialist prompt forbids the agent from querying | **Lambda** |
| Outbound auth | PAT/OAuth from AgentCore Identity, no secret in the image | same credential problem, fewer places to put it | credential reaches the agent | **Lambda** |
| Where governed SQL lives | in Snowflake, single source of truth | in Snowflake | ambiguous | Lambda or MCP |
| Trust boundary | no Snowflake credential on the agent side | no credential on the agent side | **credential on the agent side** | **not the agent** |

Auth on the recommended path works because each leg uses the mechanism it actually supports. The runtime fetches the Snowflake PAT or OAuth token through an AgentCore Identity credential provider (`bedrock-agentcore:GetResourceApiKey` / `GetResourceOauth2Token`, plus `secretsmanager:GetSecretValue` on the `bedrock-agentcore-identity!` prefix), so **no secret is baked into the image** — those statements already exist in `deploy/iam_policy.json`'s `runtimeExecutionRole`. The Gateway leg is SigV4, which is the leg SigV4 works on.

One open dependency on the customer's side: AWS publishes no stable Gateway egress IP ranges, so if their Snowflake account enforces network policies by IP, the outbound leg may need PrivateLink. Better asked now than discovered at cutover.

## The latency argument for precomputing signals

Whether the semantic layer stays *on the per-application hot path* is a different question from whether it stays at all, and their own architecture already answered it: their process guide (§2.2) says signals exist to ease review and cut processing time by preventing the agents from re-issuing the same SQL for every application. The split this port implements is theirs.

Three reasons the hot path should not contain text-to-SQL:

**1. The queries are known at design time.** For a *fixed* set of fraud signal groups — exactly what the eight specialists consume — regenerating SQL per application buys nothing. Natural-language-to-SQL earns its cost on ad-hoc investigation, which is the interactive Francis path rather than the batch underwriting path.

**2. It removes 16 per-application operations.** Precomputing eliminates **8 Cortex Analyst messages and 8 warehouse executions per application** from the hot path. That is often more persuasive to the customer's leadership than any latency figure, because it is a Snowflake-side consumption reduction visible on their own bill.

**3. The measured budget has no room for it.**

| Measurement | Value | n |
|---|---|---|
| End-to-end adjudication, agents only, p50 | **31.45 s** | 42 |
| Same, p95 | **46.55 s** | 42 |
| Runs under the customer's 60 s target | **40 of 42** | 42 |
| Slowest single specialist observed | 69.78 s (`bustout`) | 336 calls |
| Headroom to 60 s at the median | 28.6 s | — |
| Headroom to 60 s at p95 | 13.5 s | — |

Source: `evals/results/pipeline_14x3_us-east-1.json`.

Be precise about what that budget covers. The artifact's own caveat records that `build_payload` runs **outside** the timed window, because the customer's architecture precomputes signals, so an end-to-end figure here is agent latency and not pipeline-including-ETL latency. Correct statement: the measured 31.45 s p50 is agent latency with signals already available, and the remaining headroom is the two rows above.

## Still unknown

**Cortex Analyst latency has never been measured here.** This repo has never run the customer's Snowflake pipeline and cannot, so any figure for what Analyst adds per call would be invented and none appears anywhere on this site. What can be said is structural: whatever that number is, it is paid **8 times per application** on the hot path and **zero times** if signals are precomputed, and the p95 headroom above is 13.5 s.

Also still unknown, and it is the blocker: whether their semantic layer is a view or a stage-based YAML.

## What the two stale files still say

For the engineer reading the repository alongside this site, so nothing is a surprise.

| File | Status |
|---|---|
| `docs/semantic-layer-mcp.md` | **Stale.** Carries the invented tool signature, the one-call rows assumption, and the SigV4-to-Snowflake diagram. Correct on the layer-separation argument; wrong on the integration |
| `tools/cortex_analyst_mcp.py` | **Stale.** Its `MOCK_MODE` envelope is a useful shape for the demo; its `_live_retrieve` calls the fictional tool |
| `signal_layer/` | **Current.** This is the layer that runs, and it is backend-agnostic by construction |

Read the mock module for what it gets right: it demonstrates *the envelope the analysis layer consumes* — semantic view, question, generated SQL, guardrails applied, signal rows — which is genuinely backend-agnostic and is the right seam. Only the live call on the other side of that seam is wrong.


**Signal layer**
The registry, the per-domain projection, and what is reconstructed rather than received.

**General Rules coverage**
The 29 numbered rules — 27 enforced deterministically, 2 prompt-enforced.

‹ **Deployment**
The real deploy path end to end — the five-...

**Signal layer** ›
The 60-signal registry, the per-domain proj...

# Signal layer

The 60-signal registry, the per-domain projection that bounds what an agent can see, the payload schema, the score bands — and what is reconstructed from the prompts rather than received from the customer.

*For the customer's engineer. Everything here was read from the modules at import, not transcribed.*

**Agents interpret; they never query.** Governed, deterministic signal values are produced with no LLM involved, and the analysis layer receives them and reasons. The same agent never both queries and investigates. Sources: `signal_layer/registry.py`, `signal_layer/schema.py`, `signal_layer/bands.py`, `signal_layer/alert_categories.py`.

## Per-domain projection

60 signals in, one DomainView per specialist out

`project_view` filters by `registry.signals_for(domain)` and `alert_categories.alerts_for_domain(domain)`, and the resulting `DomainView` is the **complete universe** the specialist reasons over — its own docstring says so, because there is no data tool on the agent side. "The agent only saw its own domain" is therefore a property of the data path rather than an instruction a model might ignore.

That split is the customer's own finding rather than an AWS preference — their process guide (§2.2) states signals exist to ease review and cut processing time by preventing the

agents from re-issuing the same SQL per application. It is also what keeps text-to-SQL out of the per-application hot path (**latency argument**).

```
signal_layer/
  registry.py          60 signals: owner domain(s), the customer's own interpretation
  schema.py            SignalPayload / DomainView / AlertFinding / render_for_agent
  alert_categories.py  the ten alert-ID categories
  bands.py             fraud-score and dealer-score bands, PRODUCT decode, credit vali
  rules/               one module per numbered General Rule + catalog.py coverage tabl
```

The previous revision of this repo fetched the full record **nine times per application** and gave every specialist every signal, with the split existing only inside a CLI display function. Now a specialist is constructed with no tools and receives one `DomainView`, so nothing it could be told to ignore is present to ignore.

## Signal registry

`signal_layer/registry.py` is the contract every other layer resolves names through. Read from the module: **60** signals in `SIGNAL_OWNER` / `SIGNAL_SPEC`, of which 55 are owned by exactly one domain and **5** are legitimately shared by two (`SHARED_SIGNALS`).

Per domain, distinguishing *sole-owned* from *visible in its view*:

| Domain | Sole-owned | Visible in its view | Difference |
|---|---|---|---|
| `dealer` | 14 | 14 | — |
| `synthetic` | 9 | 10 | `rapid_app_velocity` |
| `straw` | 8 | 8 | — |
| `employment` | 8 | 8 | — |
| `income` | 6 | 7 | `ssn_income_high_variance_inconsistent_reporting` |
| `identity` | 4 | 7 | the three reuse counts |
| `bustout` | 5 | 7 | velocity and income-variance |
| `rings` | 1 | 4 | the three reuse counts |

The five shared names and who shares them:

| Signal | Shared by |
|---|---|
| `phone_reuse_distinct_ssn_count` | identity, rings |
| `address_reuse_distinct_ssn_count` | identity, rings |
| `email_reuse_distinct_ssn_count` | identity, rings |
| `rapid_app_velocity` | synthetic, bustout |
| `ssn_income_high_variance_inconsistent_reporting` | income, bustout |

`rings` owns exactly **one** name of its own because most of what it reasons over is shared with identity. Not a gap in the registry — it is what the rings prompt names.

## Each entry carries the customer's own sentence

`SignalSpec` has five fields and none is a paraphrase:

```
SignalSpec(
  name            = "ssn_days_in_system",
  owners          = ("identity",),
  context_key     = "ssn_days_in_system_context",
  interpretation  = "ssn_days_in_system = 0 (new to system): common for first-time "
                    "buyers. Only concerning with other velocity signals.",
  source_doc      = "prompts/verbatim/identity_agent_prompt.txt",
)
```

`interpretation` is read out of `prompts/verbatim/` **at import**, so no threshold in this module can drift from the customer's text. `source_doc` names which of her nine prompts it came from.

**Near-duplicate names are both materialised, never aliased.** The customer spells the same *concept* differently in different domains, and collapsing those spellings would silently degrade whichever agent uses the other name. So these are separate keys on purpose:

| Concept | One prompt's name | Another prompt's name |
|---|---|---|
| other SSNs at the employer | `employer_other_ssns_count` (rings) | `employer_distinct_other_ssns_count` (employment) |
| SSNs on the phone | `phone_ssn_count` (synthetic) | `phone_reuse_distinct_ssn_count` (identity, rings) |
| 20% income jump | `ssn_income_increase_20_percent_or_more` (income) | `ssn_income_increased_20_percent_last_30d_rapid_growth` (bustout) |

**A specialist handed a signal its prompt never mentions is being invited to reason about something it has no interpretation for.** That is the failure de-duplication would introduce, and it would be invisible: the agent would still produce fluent prose.

## `DomainView` and `AlertFinding`

```python
@dataclass(frozen=True, slots=True)
class DomainView:
    domain: str
    signals: Mapping[str, Any]
    context: Mapping[str, Any]
    alerts_fired: tuple[AlertFinding, ...] = ()

@dataclass(frozen=True, slots=True)
class AlertFinding:
    alert_id: int
    categories: tuple[str, ...]
    finding_text: str
```

`finding_text` exists because every specialist prompt forbids citing alert *numbers* and requires citing findings, which is only **satisfiable** if the payload describes what the alert found. `alert_id` and `categories` are retained for routing and audit, not for the agent to quote.

## `render_for_agent` is deterministic, and why that matters

Fixed headings, registry-ordered keys, sorted stable JSON:

```
=== APPLICATION CORE FIELDS ===
=== PRECOMPUTED FRAUD SIGNALS ===
=== CONTEXT FIELDS (validate signals against these) ===
=== ALERTS FIRED ===
```

Application fields come **first** because that block is the part most likely to be identical across domains for one record, and the signal block last because it is the most domain-specific. That is the ordering a cache point wants.

Rendered sizes for APP-1004, read from the module — note how far apart the domains sit:

| Domain | Rendered characters |
|---|---|
| `dealer` | 8,602 |
| `synthetic` | 7,463 |
| `identity` | 5,856 |
| `employment` | 5,699 |
| `straw` | 5,056 |
| `income` | 4,981 |
| `bustout` | 4,218 |
| `rings` | 3,949 |

Determinism is necessary for caching and not sufficient for it. A deterministic renderer is what *could* make a cacheable prefix identical across applications; today it produces no cache hit, because the specialist prompts are 547–956 tokens against per-model minimums of 512 / 1024 / **4096** and the per-application payload differs every time. Every specialist call in the 42-run benchmark measured a **miss**. See Cost: what does not work.

## `SignalPayload`: the whole-application object

```python
@dataclass(frozen=True, slots=True)
class SignalPayload:
    record_id: str
    hash_lender_id: str
    application: Mapping[str, Any]
    views: Mapping[str, DomainView]      # one per domain, eight of them
    guardrails_applied: tuple[str, ...]  # General Rule IDs
    semantic_layer_rev: str
    computed_at: str
    notes: ...
```

Two fields exist for the customer's benefit specifically:

- `guardrails_applied` carries rule **IDs**, not prose — for APP-1004, `['23', '2', '4', '5', '10', '15', '17', '3']`. That lets a demo show *which governance rules the layer actually enforced for this record* instead of asserting that governance was preserved.
- `semantic_layer_rev` (`pp-semantic-layer-2026-06`) pins which revision produced the values, so a signal set traces to its source version.

> **Raw SSNs never enter a payload**
>
> `application` is expected to carry `hash_ssn` only, and `signal_layer/rules/r23_no_raw_ssn.py` is the gate that enforces it before anything leaves the layer — General Rule 23, which the customer writes in capitals. `assert_no_raw_ssn` / `find_raw_ssn` are tested functions, not a convention.
>
> A Bedrock Guardrail enforcing the same thing server-side is still open for production, which matters because one code path should not be the sole barrier.

## Bands

`signal_layer/bands.py` holds the customer's numeric bands, each read from her text rather than retyped as a magic number.

**Fraud score** (General Rule 0.5, restated as Rule 6):

| Band | Range |
|---|---|
| low | 0–299 |
| medium | 300–699 |
| high | 700–998 |
| **very high** | **999** |

999 is not merely the top of the range — it means *at least one fraud accelerator fired*. Two associated rules are carried as data because they constrain analysis rather than classify a score: fraud scores must **not be averaged**, since they are standard-scaled (Rule 22), and investigation must **not** be pre-filtered to scores above 700 (`FRAUD_SCORE_NO_PREFILTER_RULE`).

**Dealer score** (Rule 21): `low 0 / moderate 500 / elevated 700 / high 900`.

The dealer bands share their endpoints in the source, so exactly **500, 700 and 900 are ambiguous**. `dealer_score_band()` resolves each boundary **upward** — 500 → moderate, 700 → elevated, 900 → high — matching the dealer orchestration document's own phrasing for the top band. `DEALER_BAND_BOUNDARY_NOTE` records the resolution in the module and adds that it should be flagged to the customer if their SQL rounds the other way. Resolve, record, and tell them: that is the correct handling of an ambiguity in a source document.

Also in `bands.py`, all sourced the same way:

| Constant / function | What it encodes |
|---|---|
| `DEALER_SCORE_IS_PORTFOLIO_RISK` | High dealer default / EPD / consortium scores are **credit risk context, not fraud evidence**. The rule APP-1007 exists to test |
| `DEALER_SCORE_BINDING` | `dealer_risk_score` → (`dealer_id`, `lender_id`); `dealer_consortium_score` → (`ppi_dealer_id`,). Two different grains (Rule 21) |
| `is_valid_time_in_file` / `TIME_IN_FILE_ERROR_SENTINELS` | Rule 25: time in file is invalid at `<= 0` or `9999`. **Only those two are rejected** — the note states it deliberately invents no upper bound, because any other sentinel must come from the customer's data dictionary rather than a guess |
| `decode_product` / `PRODUCTS` | Six product acronyms, decoded by **substring containment**, exactly as Rule 24 prescribes. The note is honest that containment cannot distinguish an intended acronym from one straddling two concatenated codes, and that the imprecision is the customer's and is preserved rather than second-guessed |
| `ACTIVE_DEALER_RULE` | Rule 10: only dealers with `consortium_number_of_apps_last_1_year > 0` |
| `INCOMEPASS_SEMANTICS` | Rule 20: `income_misrep_score` is IncomePass, distinct from fraud score; overstatement in isolation is not correlated to EPD |
| `OS15_DEFINITION` | The customer's own `CASE` expression for the 15% overstatement flag, carried as SQL |

## Alert categories

Ten categories, keyed by the customer's own names, from General Rule 3:

| Category | IDs |
|---|---|
| Identity | 60 |
| Default | 33 |
| Employment | 21 |
| Income | 12 |
| Employment Deviation | 10 |
| Straw Borrower | 10 |
| Dealer Risk | 9 |
| Collateral | 6 |
| Vehicle Risk | 3 |
| Bust Out Risk | 2 |

**166 entries, 148 distinct IDs, 18 IDs in more than one category.** The overlap is the customer's rather than a data error — every one of the ten Employment Deviation ids also appears under Employment, for instance. Full treatment on Alert categories.

## Reconstructed rather than received

**The registry is a reconstruction and must be described as one**

`full_underwriting_signal_dictionary.xlsx` **— the authoritative signal contract — was never received.** It appears only as an embedded thumbnail on page 3 of the process-guide PDF, which has zero extractable attachments. `CLEAN_MAIN_STREAM_FILE.sql` was likewise never received.

> So the 60-signal registry is **reconstructed from the nine prompts**. Every name in it is a name one of her prompts uses and every `interpretation` is her sentence — but the *set* is inferred from what the prompts mention, which cannot be the same thing as the set her SQL produces.
>
> Those two files would settle three things at once: the full signal set, the output DDL for `UNDERWRITING_REPORT_OUTPUT`, and **every enum conflict**. Requesting them is the highest-value single ask on **Open questions**.

What is received and verified, so the distinction is unambiguous:

| Element | Provenance |
|---|---|
| The nine prompts | Byte-identical, sha256-verified at import. **Prompt fidelity** |
| Every signal `interpretation` | Read out of those prompts at import |
| The 29 General Rules' text | Extracted from the verbatim orchestration documents, not retyped |
| The ten alert categories and their 166 IDs | From General Rule 3's own list |
| The bands | From Rules 0.5 / 6 / 21 / 24 / 25 |
| **The signal set itself** | **Inferred.** ← the gap |
| **The output DDL** | **Never received** |

**Fixtures**
The 14 applications that exercise this registry, and the nine anti-false-positive scenarios.

**General Rules coverage**
27 of 29 rules enforced deterministically; the two that are prompt-enforced and why.

‹ Semantic layer question
What is fictional in the original Cortex Analy...

Fixtures ›
The 14 applications, the nine anti-false-posi...

# Fixtures

The 14 applications, the nine anti-false-positive scenarios and which customer rule each one isolates, the pinned expectations, and the requirement id every pin cites.

*For the customer's engineer — this is the page where she can check whether we understood her rules.*

Fourteen synthetic applications, 8 LOW / 4 MEDIUM / 2 HIGH, every one pinned to a requirement id and a customer sentence so a disagreement lands on a specific rule rather than on a vibe. Source: `fixtures/applications/*.json` (14 files, regenerated by `python3 -m fixtures.regenerate` from `fixtures/_scenarios_legacy.py`, `fixtures/_scenarios_fp.py` and `fixtures/_scenarios_extra.py`). `tests/test_fixtures.py` holds 58 tests.

# The 14 applications

| Id | Scenario | Pinned | Recommendation | Alerts | Non-LOW domains |
|---|---|---|---|---|---|
| APP-1001 | Clean first-time buyer; thin file explained by age | LOW RISK | APPROVE | 2 | — |
| APP-1002 | Spousal co-borrower, shared phone and address, authorized-user tradelines | LOW RISK | APPROVE | 5 | — |
| APP-1003 | Self-employed income variance, 31% overstatement; ambiguous MEDIUM | MEDIUM RISK | REVIEW AND APPLY STIPULATIONS | 3 | income |
| APP-1004 | Synthetic identity, exact-unit ring overlap, shell employer, negative-file hit | HIGH RISK | DECLINE | 8 | identity, dealer, employment, income, synthetic, bustout, rings |
| APP-1005 | Bust-out velocity (5 apps/7d, 9/30d) with elevated loan-to-income | MEDIUM RISK | REVIEW AND APPLY STIPULATIONS | 5 | income, bustout |
| APP-1006 | Employer with every address, phone, city, state and ZIP field null | LOW RISK | APPROVE | 2 | — |
| APP-1007 | Dealer consortium score 940 and 15% EPD rate, no coordination evidence | LOW RISK | APPROVE | 3 | — |
| APP-1008 | Military-base employer: 3,100 SSNs, 210 new in 90 days, 34% dealer concentration | LOW RISK | APPROVE | 5 | — |

| Id | Scenario | Pinned | Recommendation | Alerts | Non-LOW domains |
|---|---|---|---|---|---|
| APP-1009 | Dense 240-unit apartment: 96 SSNs at the building, 1 at the unit | LOW RISK | APPROVE | 4 | — |
| APP-1010 | Eleven fired alerts, every one from a single shared household phone | LOW RISK | APPROVE | 11 | — |
| APP-1011 | No co-borrower at all; every other signal at baseline | LOW RISK | APPROVE | 0 | — |
| APP-1012 | Co-borrower on 5 applications with 4 unrelated primaries at one dealer | HIGH RISK | DECLINE | 6 | identity, dealer, straw, bustout, rings |
| APP-1013 | Employer unverifiable AND SSN-clustered AND velocity-linked | MEDIUM RISK | REVIEW AND APPLY STIPULATIONS | 5 | dealer, employment, income, rings |
| APP-1014 | Three weak but genuinely independent concerns across three domains | MEDIUM RISK | REVIEW AND APPLY STIPULATIONS | 4 | identity, income, synthetic |

Band distribution is deliberate calibration: six of the eight specialist prompts state that LOW should be the most common outcome, and a fixture set skewed toward fraud would reward a model that escalates everything.

Three of these are the demo candidates — APP-1010 (eleven alerts, one behaviour), APP-1002 (spousal co-borrower) and APP-1011 (no co-borrower, zero alerts) — all stable across three benchmark repetitions and all matching their pins. See Executive summary.

## Shape of one fixture

```
{
  "application_id": "APP-1004",
  "scenario": "…", "why_this_fixture_exists": "…",
  "synthetic_data_notice": "…", "semantic_layer_rev": "…", "computed_at": "…",
  "application": { … 26 core fields, hash_ssn only … },
  "signals":  { … every registry signal … },
  "context":  { … the paired *_context row for each … },
  "alerts_fired": [ … {alert_id, categories, finding_text} … ],
  "expected": { … the pinned adjudication + per-domain band, flag, requirement_id, cus
  "notes": [ … why the hard calls are what they are … ]
}
```

Two properties worth stating for a reviewer:

- **Every signal in the registry has a value and a paired `_context` value.** The context row is not decoration: specialists are required to validate signals against it, and these fixtures are built so the benign explanation lives *only* there. An agent that ignores `_context` gets them wrong.
- `expected` **is more than a band.** Each domain entry is `{band, flag, requirement_id, customer_rule}`, and the `requirement_id` — `EMP-017`, `MSTR-039`, `STRW-008` — is the addressable unit of disagreement.

No customer data is in this repository. Every application, name, SSN hash, dealer and employer is fabricated to exercise a specific rule, which each file states in its own `synthetic_data_notice`. That is also why a pin disagreement is a fixture question at least as much as a model question.

## Nine anti-false-positive scenarios, five designated fixtures

Both counts are correct about different things, and mixing them up misstates the coverage.

| Count | What it counts | Where it comes from |
|---|---|---|
| 5 | Applications built *entirely* as false-positive traps, LOW across every domain | `evals/set_comparison.py::ANTI_FP_APPS` — APP-1006 … APP-1010 |
| 4 | Anti-FP controls embedded in records that are **not** themselves LOW overall, where a single domain must clear independently | APP-1001, APP-1002, APP-1005, APP-1011 |
| 9 | Total distinct anti-FP scenarios | the two rows above |

The four embedded ones are the harder tests, and they are excluded from `ANTI_FP_APPS` because the *application* is not benign — only that one domain is.

## The five designated fixtures

- APP-1006 — every employer contact field null → EMP-007 data gap
- APP-1007 — dealer consortium 940, 15% EPD, zero coordination → DLR-009 / DLR-010
- APP-1008 — military-base employer, 3,100 SSNs → EMP-017 + DLR-015 together
- APP-1009 — 96 SSNs at the building, 1 at the unit → RNG-013 benign overlap
- APP-1010 — eleven alerts, one behaviour → IDEN-014 / IDEN-018 / RNG-023

## The four embedded controls

| Fixture | Overall pin | The domain that must clear independently, and why |
|---|---|---|
| APP-1001 | LOW | The full control: *every* suspicious value has a benign cause stated only in its own `_context` row — a 40-unit complex, a first-time buyer's `ssn_days_in_system = 0`, a 480-SSN employer that is a hospital. `IDEN-013` calls a zero-day SSN common for first-time buyers and concerning only alongside other velocity signals — and there are none. |
| APP-1002 | LOW | The **alert-volume control**: 5 alerts fire, 4 of them from one shared household phone (`IDEN-014`), so volume must not move the band (`IDEN-018`). Also `fraud_score 388`, in the medium band, which must still resolve LOW — a score is a model output, not corroborated evidence. |
| APP-1005 | MEDIUM | The **staffing-agency control inside a non-benign record.** The employer carries 260 SSNs, 40 new in 90 days, `rapid_employer_change_flag` **and** `employer_spike_flag` — all four with a staffing-agency explanation in their own `_context` rows, and `EMP-017` names staffing agencies. `employment` must be LOW while the adjudication is MEDIUM. Also `fraud_score 712` — the customer's *high* band — on a record that must stay MEDIUM (`MSTR-029`). |
| APP-1011 | LOW | The **anti-padding control.** `alerts_fired` is deliberately **empty** — the only record proving the payload builder and the specialists handle a zero-length array. `STRW-005` caps straw at 2–3 sentences with no co-borrower, and unlike APP-1003 there is nothing else on the record to pad from. Its `_context` rows also distinguish "no co-borrower exists" from "a co-borrower exists with a count of zero" — both render as `0` in the signal, and only the context says which. |

# Pinned expectations, and the hard calls

Each fixture's `notes` array explains the calls a reviewer would question. Four patterns recur, each a rule the customer stated.

Band vs flag | Overlap is not corroboration | Sparse data alone vs plus | A score is not evidence

A domain can be **POSSIBLE with its flag at 0**. `MSTR-039` sets a flag to 1 only on clear, material, corroborated evidence.

- **APP-1004**: `dealer_flag = 0` while the dealer band is POSSIBLE. `DLR-009` forbids escalating on the 930 consortium score, and `DLR-018` treats negative-file and synthetic-linkage flags as *individual borrower* flags that mean dealer fraud only when multiple such borrowers cluster at one dealer.
- **APP-1014: all eight flags are 0 while the overall outcome is MEDIUM.** Legal and important — `MSTR-027`'s MEDIUM covers unresolved concerns on moderate corroborating evidence requiring additional verification, which is exactly this record. **A flag-counting implementation returns LOW.**
- **APP-1005**: `income` is POSSIBLE with `income_flag = 0` — 18% overstatement is in the requires-context band and the context (assignment-based hours) supplies it.

## Golden dataset built on top

`evals/datasets/underwriting_golden.jsonl` — **13 scenarios, 134 assertions** — in the `AGENTCORE_EVALUATION_PREDEFINED_V1` shape, and it parses under the AgentCore CLI's own `parseAndValidate`.

| Group | Scenarios | Expected trajectory |
|---|---|---|
| End-to-end adjudications (APP-1001…1005) | 5 | all eight specialist calls |
| Anti-false-positive, one domain each | 5 | the single relevant specialist |
| Format contracts (straw 2–3 sentences) | 1 | `call_straw_agent` |
| Interactive Francis (2-tool cap + format; RETRO deflection) | 2 | 1 tool / none |

Two extra keys are carried for review and ignored by the CLI's parser:

- `expected_provenance` — which fixture `expected` block the scenario encodes, plus the exact verbatim prompt line each assertion comes from, quoted as `prompts/verbatim/<file>::<SECTION> -> '<line>'`. Every one of the 134 assertions traces to a line in the customer's own prompt.
- `notes` — why a scenario exists where that is not self-evident.

## Regenerating

```
python3 -m fixtures.regenerate      # rewrites fixtures/applications/*.json
```

Scenario modules are deliberately importable without side effects — `_build` writes, `_scenarios_*` only declare, `__init__` never writes — so a test can read a scenario without touching the tree. `regenerate` also fails on duplicate application ids before writing anything.

`tests/test_fixtures.py` asserts that the fixture-side signal mirror agrees with `signal_layer/registry.py` **name for name**: the registry is the authority and the fixtures are checked against it, not the other way round.

## Still unknown

**Whether these pins match the customer's own judgement has not been confirmed by her.** Three pin disagreements surfaced in the benchmark, and the artifact's own caveat says whether the pin or the model is wrong is the customer's call. The fixtures encode our reading of her rules with a requirement id attached to each reading precisely so she can overturn one without re-litigating the rest.


**Signal layer**
The registry these fixtures populate, and the per-domain projection they are read through.

**Eval harness**
Which of these fixtures each study used, and why those and not others.

‹ Signal layer
The 60-signal registry, the per-domain proj...

Eval harness ›
bench.py, tier_quality.py, set_comparison.p...

# Eval harness

bench.py, tier_quality.py, set_comparison.py, judge_sol.py and tail_analysis.py — what each measures, how to re-run it, what it refuses to report, and its spend gate.

*For the customer's engineer, and for whoever has to re-run any of this after the engagement.*

Five modules, ~9,600 lines, one rule they all serve: **a number shown to the customer is either a real measurement from a named run in this repo, or it is the customer's own reported figure with attribution. There is no third category.** Total measured spend across all committed studies is **$28.89**.

## Five modules at a glance

| Module | Question it answers | Committed artifact | Measured spend |
|---|---|---|---|
| `evals/bench.py` | How long does one adjudication take, and what does it cost? | `evals/results/pipeline_14x3_us-east-1.json` + `.md` | $6.4659 |
| `evals/tier_quality.py` | Does the cheaper *Claude* tier analyse as well? | `evals/results/tier_quality.json` + `.md` + 2 sidecars | $4.9088 |
| `evals/set_comparison.py` | Is the GPT set as good at the fraud analysis as the Claude set? | `evals/results/set_comparison.json` + `.md` + 2 sidecars | $16.8869 |
| `evals/judge_sol.py` | (the GPT half of the two-judge design — not a standalone study) | contributes verdicts to the above | included above |
| `evals/tail_analysis.py` | Why is there a 75 s max, and what would remove it? | `evals/results/tail_analysis.json` + `.md` | $0.6278 (the live cap sweep) |

## Why the harness exists at all

Fabrication is why. The pre-port demo computed its headline claim — "26 s sequential → 8.5 s parallel" — from a hand-written `NOMINAL_LATENCY_S` dict, and printed `time.sleep(nominal * 0.05)` beside it as "measured wall-clock". Per-agent cost came from a hard-coded `NOMINAL_TOKENS` table. Every one of those figures was invented, and `agents/models.py` now carries an explicit prohibition against holding any latency table at all.

`evals/bench.py` replaced all of it, and `tests/test_bench.py` (133 tests) fails CI if that style of fabrication returns.

## Two rules that apply to every module

**Percentiles are refused below a documented sample count.** A refused percentile is `null` in the JSON with its reason recorded beside it, for example `"p99": "refused: 42 sample(s), need >= 100. A p99 over 42 sample(s) is not a p99."`

| Percentile | Minimum n | Why that number |
|---|---|---|
| p50 | 5 | A three-sample median is not worth showing a customer. The arithmetic floor would be 2 |
| p95 | 20 | 1/(1−0.95) — below this the tail cannot be observed |
| p99 | 100 | 1/(1−0.99) |

Percentiles are **nearest-rank with no interpolation**, so every published value is a value the run actually observed — which matters most at exactly these sample sizes. The method string is recorded in every output, and `n/a` in a table column means the percentile was **refused**, never that it was zero.

**Live modes are double-gated.** Every billed path requires both a flag and an environment variable, because a flag alone is one keystroke from a billed run.

| Module | Gate |
|---|---|
| `bench.py --live` / `--pipeline` | `AGENTCORE_BENCH_ALLOW_LIVE=1` |
| `tier_quality.py --live` | `AGENTCORE_BENCH_ALLOW_LIVE=1` |
| `set_comparison.py --live` | `AGENTCORE_BENCH_ALLOW_LIVE=1` |
| `tail_analysis.py --live` | `AGENTCORE_BENCH_ALLOW_LIVE=1` |
| `agents/mantle.py --measure` | `AGENTCORE_MANTLE_ALLOW_MEASURE=1` |

Each prints the full plan — applications, repetitions, call count, every model id, and a spend estimate — **before calling anything**, and each has a `--dry-run` that stops there.

## `evals/bench.py`: the headline benchmark

Four modes, and the difference between two of them decides which number you may quote.

| Mode | Calls a model? | What it measures | Quotable as |
|---|---|---|---|
| `--offline` | no | **only** this repo's orchestration overhead | orchestration overhead, labelled |
| `--live` | yes | per-call Bedrock latency/cost via *this module's own* agents | **a** fan-out, plus per-specialist time-to-first-token |
| `--pipeline` | yes, unless `--mock` | **the shipped pipeline**, end to end, per application | **the** fan-out — the program AgentCore runs |
| `--reaggregate` | no | recomputes a committed report's statistics from its own `raw_runs` | nothing new; derivation only |

> **`--live` is not `--pipeline`, and the committed headline comes from `--pipeline`**
>
> `--live` builds its own `strands.Agent` per specialist and its own synthesis call, so it measures **a** fan-out rather than **the** fan-out. It bypasses `agents.fanout`'s semaphore, widened executor and shared botocore pool; it bypasses `agents.synthesize`'s structured-output validation and bounded repair; and **it cannot reach the GPT-5.6 synthesizer at all**, because GPT-5.x is not on Converse.
>
> `--pipeline` calls `agents.fanout.fan_out` and `agents.synthesize.synthesize` **directly**. Its one disadvantage: it does not stream, so `--live` remains the only mode that measures time-to-first-token per specialist.

## Re-running it

```
# Structural dry run. Exercises the whole path, calls nothing, spends nothing.
python -m evals.bench --pipeline --mock --repetitions 1

# The committed benchmark. 14 applications x 3 repetitions = 42 adjudications.
# Measured spend for this exact command: $6.4659.
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --pipeline \
    --repetitions 3 --region us-east-1 \
    --json evals/results/pipeline_14x3_us-east-1.json \
    --markdown evals/results/pipeline_14x3_us-east-1.md

# One application first, to check credentials and the mantle endpoint before spending.
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --pipeline \
    --repetitions 1 --application APP-1001

# Offline orchestration overhead only.
python -m evals.bench --offline --concurrency 1,4,16 --iterations 40

# Price a live run without making one.
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --live --dry-run
```

Why `--repetitions 3`: 14 × 3 = 42 clears the 20-sample minimum for a **pooled** p95. It does **not** clear it per application, where n=3 each, and the report suppresses those rather than printing them. Reaching a p99 needs 100 pooled samples — `--repetitions 8`, roughly $18.

## Re-aggregating without re-spending

```
python -m evals.bench --reaggregate evals/results/pipeline_14x3_us-east-1.json \
    --markdown evals/results/pipeline_14x3_us-east-1.md
```

Every measured field lives in `raw_runs`, so every statistic is recomputable. The provenance block is carried through **unchanged** — re-aggregating does not restamp the numbers with today's git sha — and a `reaggregated_at_utc` field plus a caveat record that the derivation, and only the derivation, is newer than the measurement. The committed report was re-aggregated this way at `2026-07-28T07:46:30+00:00`; no model was called to produce it.

## Two fields decide whether a cost figure is real

`totals.calls_with_real_usage` and `per_agent.<agent>.cost_usd.n_priced_calls`. If either is `0`, no cost in that report was measured, and every cost field is `null`.

### Cacheability verdict is reported separately from the hit rate

`bench.py` emits `cacheable` / `never_cacheable` / `unknown` as its own field, so a 0% hit rate never appears without the sentence that explains it. `unknown` means the run did not measure a prompt token count, which offline runs cannot, and is **not** a guess.

Measured prompt sizes via Bedrock `CountTokens` (us-east-1, 2026-07-28, on the assembled prompt — verbatim system prompt plus projected APP-1004 payload):

| Specialist | Model | Measured prompt tokens | Min cacheable prefix | Verdict |
|---|---|---|---|---|
| `dealer` | haiku-4-5 | **1,829** | 4,096 | **never caches** |
| `straw` | haiku-4-5 | **1,655** | 4,096 | **never caches** |
| the other six | sonnet-5 / opus-5 | not measurable | 1,024 / 512 | unknown |

`CountTokens` is not offered for every model: in us-east-1 `anthropic.claude-haiku-4-5-…` returns a count while `anthropic.claude-sonnet-5` and `anthropic.claude-opus-5` both raise `ValidationException: The provided model doesn't support counting tokens`. It also **rejects inference-profile prefixes** — the bare `anthropic.*` id is required. `bench.py` strips the prefix and reports `None` for the unsupported models rather than substituting an estimate.

---

## `evals/tier_quality.py`: the Claude-only tier study

**48 blind-judged analyses**, 12 (agent, model) cells, one judge (Claude Opus 5), region us-east-1, shuffle seed 20260728. Three **independent layers**, deliberately not one score.

1. **Deterministic contract check** (`contract_findings`). Every rule is read out of the agent's own verbatim prompt at import, so a rule is only enforced for a domain

whose prompt actually carries the sentence — and the sentence is **quoted in the finding**.

This layer holds the study's most load-bearing correction: the header ban appears in **four** of eight prompts, not six. `income` and `synthetic` never received it. A checker hard-coding "6 of 8" would report violations the customer's prompt does not contain. From the artifact's own `contract_scope`:

| Rule | Applies to | n |
|---|---|---|
| `exact_title` | identity, dealer, straw, employment, income, synthetic | 6 |
| `no_alert_numbers` | same six | 6 |
| `no_emoji` | same six | 6 |
| `max_4000_chars` | same six | 6 |
| `no_risk_label_in_headers` | **identity, dealer, straw, employment** | **4** |

`bustout` and `rings` state **none** of them. See Output-contract defects.

2 **Deterministic band and grounding extraction.** The band is what the analysis itself declares, compared against `fixtures.loader.expected_for(app)['domains'][domain]`, which carries the requirement id and the customer sentence justifying it.

Grounding is measured against the **paired `<signal>_context` rows the payload actually contained**: distinctive tokens are lifted out of the example rows and looked for in the prose, so "cites the context field" is evidence rather than the word *context* appearing somewhere.

3 **Blind LLM judge.** Analysis text is anonymised and the order shuffled under a fixed seed before judging, so the judge never sees which tier wrote what and two runs judge in the same order.

Bands are extracted **twice** on purpose. A deterministic extractor and the judge each report the band they read, and the aggregation reports **how often they disagree** — a regex over prose is fallible and so is a model. Two independent readings that agree are evidence, and

where they disagree the row is counted `band_unresolved` rather than scored in either direction.

**The judge conflict is quantified rather than assumed away.** Opus 5 was one of the three candidates *and* the judge. On its own 16 rows it scored substance **+0.78** and calibration **+0.31** higher than on the other 32 rows, while the **deterministic** band-agreement margin moved only **−0.026** and contract violations **−0.12**.

Its caveats state the consequence directly: any conclusion resting on Opus outscoring a cheaper tier by less than the measured self-preference margin is not supported. That is what motivated the two-judge design in the next study. See **Judge methodology**.

```
# Judge analyses already paid for.
AGENTCORE_BENCH_ALLOW_LIVE=1 PYTHONPATH=. python evals/tier_quality.py \
    --reuse <analyses.json> --judge --live

# Extend the sweep, then judge everything.
AGENTCORE_BENCH_ALLOW_LIVE=1 PYTHONPATH=. python evals/tier_quality.py \
    --reuse <analyses.json> --sweep bustout,identity \
    --apps APP-1004,APP-1012,APP-1009 --judge --live

# Price it only.
PYTHONPATH=. python evals/tier_quality.py --dry-run
```

Reuse is why the committed run reports zero live calls: `live_calls` reads `specialist_calls_made: 0`, `judge_verdicts_reused_from_cache: 48`, `reused_analyses: 48`, and `spend_actual_usd` still totals $4.9088, because **rows reused from an earlier sweep carry that sweep's measured cost**. Rows with no recorded usage contribute **nothing**, not an estimate. The `--rescore` / `--reuse` paths are how the aggregation can be corrected without re-spending.

---

## `evals/set_comparison.py`: the cross-family study

**4 agents × 6 applications × 5 models = 120 blind-judged analyses, 240 verdicts**, two judges per analysis. Candidates: Claude Opus 5 and Sonnet 5; GPT-5.6 Luna, Terra and Sol. Judges: Claude Opus 5 and GPT-5.5. `specialist_max_output_tokens: 4096` — **identical**

**across both families on purpose**, because the length gap is the central question and a per-tier cap would put the harness's own configuration inside the measurement.

Three tempting shortcuts are refused, and the refusals are the design.

| Shortcut refused | What it does instead |
|---|---|
| Judging by output length | Measures *evidence per unit length*, **paired on the same application**: distinct signals cited, distinct `_context` fields validated, whether the band's justification is complete. Brevity that drops evidence is a finding against GPT; brevity that drops padding is not. The customer's own master prompt says so (`MSTR-018`) |
| One judge | Two judges biased in **opposite** directions. **Agreement is the signal; disagreement is "NOT ESTABLISHED"** — the study is allowed to return no verdict on an axis |
| Letting the judges decide | The scores driving the recommendation are **deterministic and untouchable by a judge**: band agreement, false positives on the anti-FP fixtures, false negatives, `_context` grounding, and `agents.output_contract.check`. Judges add calibration and substance, reported as subjective |

## Why those six applications and not a random six

Chosen for what they can **falsify**:

- **APP-1009, APP-1010** are anti-false-positive fixtures — APP-1010 fires eleven alerts sharing one underlying behaviour, so **any model that counts alerts escalates it.**
- **APP-1014 rings** and **APP-1004 bustout** are the two calibration failures every Claude tier shared. If GPT repeats them the fault is the prompt; if it does not, that is a point for GPT no latency table shows.
- **APP-1004** carries genuinely corroborated fraud, so **a model that says LOW to everything cannot score well.**
- **APP-1012, APP-1013** supply the POSSIBLE band, which both families are most likely to skip past in either direction.

**Coverage is 4 of 8 agents, and that is reported as the limit it is.** `dealer`, `straw`, `employment` and `income` **never ran on GPT** — and APP-1006/1007/1008, three of the five designated

anti-false-positive fixtures, target exactly those. Only two of the five anti-FP fixtures touch the four agents swept. `income` in particular has **no quality evidence either way**, which is one reason the partial migration was declined. See Cross-family study.

```
# No calls; re-scores the committed analyses.
PYTHONPATH=. python3 -m evals.set_comparison --rescore

# Billed. Prints the plan and estimated spend first.
AGENTCORE_BENCH_ALLOW_LIVE=1 PYTHONPATH=. python3 -m evals.set_comparison \
    --sweep rings,synthetic,bustout,identity --judge --live
```

Four output files: the report, a markdown sidecar, `set_comparison_analyses.json` (the 120 analyses) and `set_comparison_verdicts.json` (the 240 verdicts). Keeping analyses and verdicts separate is what makes `--rescore` free.

---

## `evals/judge_sol.py`: the second, oppositely-biased judge

Not a standalone study — the GPT half of the two-judge design. **GPT-5.6 Sol**, `us-east-1`, `effort="high"`.

Sol specifically, and the reason is a conflict-of-interest argument: Sol is **the only GPT-5.6 variant that is not a candidate for any specialist assignment**. Luna is the leading candidate *and* the incumbent synthesizer; Terra is the middle candidate. So Sol is the one GPT model not scoring a sibling it might be deployed alongside, and it measured the highest reasoning effort of the three, which is what a judge should be. Note the separate finding that **Sol is the worst GPT variant on quality**, at 4× Luna's price — a statement about it as a *specialist*, not as a judge. See Synthesizer selection.

| Held identical to the Claude judge | Differs |
|---|---|
| Judge instructions, verbatim | Model: GPT-5.6 Sol |
| Anonymisation | Transport: Responses API via `agents.mantle.complete` |
| Verdict schema, same Pydantic model | JSON enforcement: stated then validated on the way back, because the Responses API has no forced tool call here |
| The pinned expectation | — |

A test asserts it uses the same instruction text, because if the prompt drifted between judges the comparison would be meaningless. And an invalid verdict **raises** rather than being coerced — `tests/test_set_comparison.py` parametrises the mutations.

---

## `evals/tail_analysis.py`: why the tail exists

Reads the committed benchmark and decomposes its tail into the four causes it could have. Four questions answered, and a fifth refused.

1. **Where is the tail?** Per run: which agent was slowest, and whether it is the same one every time. Per application: how much of the spread is the application's own signal load versus repetition-to-repetition noise on identical input — reported as a **share of sum-of-squares, not an F test**, because n=3 per cell.

2. **Is it output length?** Every specialist call has both a wall clock and a real `outputTokens`, so the line is fitted and `r²` reported per domain — **with and without the runs identified as stalls**, because one stall destroys a correlation and hides the very effect being tested.

3. **Is it retries?** A retry sleeps *inside* the call, so it shows up as wall clock **exceeding** the model's own `latencyMs`. That gap is computable per call, which makes the retry

theory falsifiable rather than assumed. Measured answer: **0 throttles in 336 calls**, gap **0.73 s p50 / 1.69 s p95 over n=334**.

4 **What is the floor?** `asyncio.gather` cannot return before its slowest member, so the distribution of `slowest_specialist_wall_ms` **is** the fan-out floor, and `fanout_wall_ms − slowest_specialist_wall_ms` is everything orchestration adds. **A fix that does not move the floor does not move end-to-end.**

**What it refuses: modelling a configuration it did not run**

`cap_binding_census` reports how many **measured** calls a candidate `max_tokens` would have truncated — a count of observed output lengths, **not** a predicted latency. Any saving from a cap is only ever claimed from `--live`, which re-invokes the agent with the cap actually applied. The module's own words: holding tokens constant across a config change is exactly the error this repo already made once.

That live sweep — 30 calls, $0.6278, caps 1024/1536/2048/3072/4096 on bustout and rings over APP-1004/1012/1013, with **4096 re-run in the same session as the control** so the delta is not against another day's run — is why the proposed `max_tokens` fix is recorded as having **failed**: 5 of 10 comparable cells faster and 5 slower (p50 delta −1.52 s), **14 of 30 truncated**, and only **13 of 30 agreeing on band**. Its own quality gate says a cap is a fix only if it lowers wall clock, truncates nothing, and leaves every band unchanged. It failed all three. See Latency tail.

```
# Offline analysis of the committed benchmark. No calls.
python -m evals.tail_analysis

# The live cap sweep, priced first.
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.tail_analysis --live --dry-run
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.tail_analysis --live \
    --caps 1024,1536,2048,3072,4096 --domains bustout,rings \
    --applications APP-1004,APP-1012,APP-1013
```

A `STALL` in this module means a **single specialist** call crossing 60 s. Not an outlier rule — it is the customer's own sub-minute target applied to one agent, because one specialist consuming the entire end-to-end budget is a different physical event from a verbose specialist and the two must not be pooled.

## In-platform evaluators, and where they stand

Three evaluator configs in `evals/evaluators/` are written and schema-checked but **not yet registered against a deployed runtime**:

| File | Level | Type |
|---|---|---|
| `pp_calibration_judge.json` | TRACE | llm-as-a-judge |
| `pp_false_positive_discipline.json` | TRACE | llm-as-a-judge |
| `pp_output_contract_check.json` | SESSION | code-based |

They were validated against the AgentCore CLI's own Zod schema (`EvaluatorSchema` from `@aws/agentcore/dist/schema/schemas/agentcore-project.js`), and the golden dataset against the CLI's own `parseAndValidate` — because a shape that only satisfies our own tests is not evidence. Registration commands are in `evals/README.md`; the gap is listed under production hardening.

## What has NOT been measured

Reproduced from the harness's own list, because the exclusions are as load-bearing as the figures.

- **A p99 of anything.** n=42; a p99 needs 100. `--repetitions 8` (~$18) would reach it.
- **A per-application p95.** n=3 each, all suppressed. Only the pooled p95 is supportable.
- **Throughput or latency under concurrent load.** The benchmark ran **one application at a time**. The one `server_5xx` pair observed at concurrency 1 is a reason to expect the question to matter.
- **Prompt-cache hit rate under repeat load.** Every specialist call measured a miss, which is documented behaviour rather than a defect.
- **Accuracy.** The benchmark measures whether an adjudication *validated* and whether repeats *agreed* — not whether the verdict is right. Three pin disagreements are surfaced, not resolved.

- **AgentCore Runtime overhead** — cold start, session setup, SSE framing. **The single largest known gap** between 31.45 s p50 and what a deployed customer would see.
- **Any region other than** `us-east-1` for the pipeline as a whole.
- **Any Snowflake comparison.** This repo has never run the customer's Snowflake pipeline and cannot. Their reported figures — 4,500 applications over 48 h, $700K–$1M ARR — are **theirs, by attribution**, and must never sit beside one of ours as though both were measured the same way. See Snowflake comparison.

**Benchmark method**
The percentile policy, the refusal thresholds and the anti-fabrication guards in detail.

**Artifact index**
Every file under evals/results/ with its n, its spend, and what it can and cannot support.

‹ Fixtures
The 14 applications, the nine anti-false-posi...

Test suite ›
1,238 offline tests — what the mutation test...

# Test suite

1,238 offline tests — what the mutation tests prove, the anti-fabrication guards, the schema registry that refuses an unknown artifact, and why a test never names a private SDK attribute.

*For the customer's engineer, and specifically for the question "how do I know the fidelity claims on this site are true and not just asserted?"*

Most of those claims are executable, and several were verified by **deliberately breaking them**. **1,238 tests** are collected by `MOCK_MODE=1 python3 -m pytest tests/`, of which 23 are the two docs-guard files that only apply once the site exists.

## Running it

```
MOCK_MODE=1 python3 -m pytest tests/ -q -p no:cacheprovider
```

No AWS, no credentials, no spend. `MOCK_MODE` defaults to `1`, so the flag is belt-and-braces; every runtime in this repo must be runnable with zero AWS calls, because the demo is rehearsed offline and CI has no Bedrock access. Excluding the two docs-guard files, the product suite reports `1214 passed, 1 skipped`; the one skip is a live smoke test gated behind `AGENTCORE_BENCH_ALLOW_LIVE=1`.

Two other invocation notes that will otherwise cost time:

```
python3 -m pytest -q --ignore=agentcore     # what deploy/ci.sh runs
python3 -m pytest -q -W error::UserWarning  # what a Strands upgrade should run
```

`--ignore=agentcore` is required after any `agentcore package`: the staging tree contains third-party wheels shipping test modules with colliding basenames, and a bare `pytest`

aborts during **collection** with `import file mismatch` before reaching a single project test. `-W error::UserWarning` matters because `BedrockConfig` swallows unknown keys with a warning — see Strands.

`-p no:cacheprovider` disables pytest's `.pytest_cache` writer, and it is worth passing rather than being a matter of taste for two reasons. **The tree is asserted on**: `tests/test_docs_site.py` walks `site/content/docs/**` and `tests/test_prompt_fidelity.py` walks `prompts/verbatim/`, so a run that writes new directories into the working tree is a run that changed its own inputs. And **read-only or parallel contexts** — a packaged runtime directory, a CI container with a read-only mount, or two suites sharing a checkout — fail or interfere on the cache directory rather than on anything real. Nothing in the suite depends on cross-run state, so the cache buys nothing and costs a mutable directory.

# What is collected

| File | Tests | What it gates |
|---|---|---|
| `tests/test_prompt_fidelity.py` | 215 | The nine prompts byte-identical, sha256-verified, and the load-bearing contract strings intact through the loader |
| `tests/test_contracts.py` | 199 | The 21-key adjudication: exact keys, exact order, exact enums, the derived constraints |
| `tests/test_bench.py` | 133 | The measurement harness, the percentile refusals, and the anti-fabrication guards |
| `tests/test_set_comparison.py` | 106 | The cross-family study's shape, re-derived from its own artifact |
| `tests/test_tier_quality.py` | 91 | The tier study's shape, the blinding, and the contract scope |
| `tests/test_mantle.py` | 82 | The GPT endpoint: base URL, rates, region ranking, the measured matrix |
| `tests/test_rules.py` | 67 | The 29 General Rules and the generated coverage table |
| `tests/test_runtime_contract.py` | 60 | Both AgentCore entrypoints: `context`, SSE, frames, session ids, the 2-tool cap |
| `tests/test_pricing.py` | 58 | The rate card, the `us.*` multiplier, the tiering arithmetic |
| `tests/test_fixtures.py` | 58 | The 14 applications against the registry, name for name |
| `tests/test_tail_analysis.py` | 50 | The tail decomposition and the cap sweep's own quality gate |
| `tests/test_fanout_mantle.py` | 39 | Two transports through one shared result-assembly path |

| File | Tests | What it gates |
|---|---|---|
| `tests/test_models.py` | 32 | Model assignment, tier rationales, the recorded usage |
| `tests/test_output_contract.py` | 25 | Rules **derived** from prompt text, not hard-coded |
| `tests/test_docs_site.py` | 13 | This site: no confidential text, no fabricated figures, every citation real |
| `tests/test_docs_readability.py` | 9 | This site's readability: titles, callout budget, diagrams, tables |

Prompt fidelity being the largest file is not an accident. **Fidelity is the product**, because the person reviewing this wrote the originals.

## Mutation tests

A test that has never failed is a test whose premise is unverified. Several guards here were checked by deliberately breaking the thing they guard.

### Prompt integrity — 5 mutation attempts, all caught

1. **Edit a packaged copy.** `test_manifest_hash_matches` recomputes the sha256 of every file in `prompts/verbatim/` against `prompts/manifest.json`. One changed byte fails, naming the file.
2. **Paraphrase a prompt and update its hash.** Caught by a *different* test: `test_verbatim_matches_source` diffs each packaged copy against the customer document in `docs/`. Regenerating the manifest satisfies the hash check and still fails the diff — the two guards are deliberately independent, so **regenerating the manifest is not a bypass.**
3. **Reintroduce the invented output format.** The pre-port code rewrote all nine prompts and invented a `RISK:` / `FINDINGS:` format.

`test_loader_raises_naming_the_file_on_drift` performs exactly that mutation in a tmpdir — replacing `"You are an Identity Fraud Detection Agent."` with `"ROLE: Identity Fraud Detection Agent"` — and asserts `PromptIntegrityError` is raised **and that the message names both the file and the expected hash.**

4 **Delete a packaged file.** `test_loader_raises_on_missing_file` asserts absence is an error naming the file, not a silently empty prompt. An empty system prompt is the worst failure mode here: the agent still answers.

5 **Add or drop a file without updating the manifest.** `test_manifest_covers_exactly_the_packaged_files` asserts the manifest's file list equals the directory listing exactly, and that each recorded byte count matches `stat().st_size`.

> **Why five guards for one property**
>
> This chain prevents the failure that would have lost the technical review outright. The paraphrased prompts were bad; the *parser* was worse — it defaulted to `LOW RISK` whenever it could not find its invented `RISK:` line, so **feeding it the real prompts would have cleared every application silently.** No test failed, no error was logged, and every adjudication looked clean. See Prompt fidelity.

## Docs-site leak guard, mutation-tested three ways

`tests/test_docs_site.py` shingles every confidential document into normalised 12-word n-grams and asserts none appears in any page's **prose**. Three tests, because a guard needs its own guard.

| Test | What it proves |
| --- | --- |
| `test_no_customer_prompt_text_is_published` | The guard fires — the actual check |
| `test_the_leak_guard_would_catch_a_paste` | The guard is **not vacuous**: a 60-word paste from `identity_agent_prompt.txt` *must* trip it. Without this, a shingle size producing no overlap would let the confidentiality test pass on any input |
| `test_the_code_span_exemption_does_not_swallow_prose` | The **exemption** cannot be abused. Code spans are exempt, since a 1:1 port cannot be documented without naming field names and enum values, so fenced blocks are re-checked at a **30-word** shingle — long enough that an enum list cannot reach it and copied prose always will |

Its justification is the line this site is written on: what is confidential is the customer's writing — her instructions, her reasoning, her paragraphs. What is not is the machine-readable contract her prompt defines: the field names, the enum values, the JSON key order. In practice that exemption fired on exactly **one** real passage, the enum-conflict comparison, and every word of the match was inside backticks.

## Mutation-tested elsewhere

- `tests/test_mantle.py:441` — the "requested region is unavailable, use the measured-fastest *available* one" branch. Its docstring records that it was verified by mutation, because without the check that distinction is untested.
- `tests/test_mantle.py:808` — a table's provenance sentence. Verified by mutation: **changing the table cell alone did not fail the test**, which is exactly the gap the extra assertion closes.
- `tests/test_set_comparison.py` parametrises **invalid judge verdicts** and asserts each **raises rather than being coerced** — a fourth `false_positive_verdict` value, an out-of-range score, a missing key.

# Anti-fabrication guards

`tests/test_bench.py` is organised into nine numbered sections, and four exist only to make fabrication a build failure.

Percentiles | Cost | Offline labelling | The README itself

Section 1. Twelve tests, and the ones that matter are the negatives:

- `test_single_sample_yields_no_percentiles_at_all` — n=1 produces **no** p50, not "the maximum".
- `test_p50_suppressed_below_its_minimum` — parametrised over n.
- `test_percentiles_appear_exactly_at_their_minimum_sample_count` — the boundary is tested from both sides, so the thresholds cannot silently loosen.
- `test_percentile_values_are_observed_samples_not_interpolated` — nearest-rank means every published value is a value the run **observed**.
- `test_percentile_method_is_recorded_so_the_number_is_reproducible`.
- `test_pipeline_p95_is_refused_at_three_repetitions_per_application` and `test_pipeline_report_caveats_name_the_refused_p95_when_n_is_too_small` — the refusal must be *stated*, not merely `null`.
- `test_markdown_summary_refuses_a_suppressed_percentile_out_loud` — the human-readable summary must not quietly drop a suppressed row.

## One guard against this engagement's own near-miss

`tests/test_models.py` and `tests/test_pricing.py` carry 12 guards added specifically because a draft summary of this work quoted study figures that appear in **no artifact on disk**. The comment in the test file is the rationale: the grounding margin, the contract counts, the per-agent figures and both judges' self-preference numbers are now all re-derived from the 120 rows rather than trusted.

`tests/test_docs_site.py` carries the same idea for the site. The fabricated figures may not appear on a page **unless the same paragraph marks them as fabricated**, which is how the

landing page is allowed to name them, and every `evals/results/*` or `docs/*` path a page cites as its source **must exist**. That second guard is why a citation on this site cannot be decorative.

## Schema registry

Every JSON in `evals/results/` is either collected as a pipeline-bench report and subjected to the full contract, or it carries a schema **explicitly listed as known-not-to-be-one**.

| Schema | Guarded by |
|---|---|
| `pointpredictive.agentcore.pipeline-bench/1` | `tests/test_bench.py` — provenance, sample counts, caveats, percentile minimums, `raw_runs` recomputability |
| `pointpredictive.agentcore.tail-analysis/1` | `tests/test_tail_analysis.py` |
| `pointpredictive.agentcore.tier-quality/1` + `-analyses/1` + `-verdicts/1` | `tests/test_tier_quality.py` |
| `pointpredictive.agentcore.set-comparison/1` + `-analyses/1` + `-verdicts/1` | `tests/test_set_comparison.py` |

> ⚠ **An unlisted schema fails the build — that is the point**
>
> `test_report_selection_skips_only_recognised_non_bench_schemas` asserts every file falls into one of those two buckets. A new artifact with an unlisted schema **fails**, with a message forcing the decision: either it is a benchmark report and must satisfy the guards, or its schema is added to `known_non_bench` with a reason.
>
> A file that is JSON but carries **no** recognisable schema is collected as a bench report on purpose, so a corrupt or unschema'd report fails loudly instead of being skipped. The alternative — a `glob("*.json")` that quietly ignores what it does not understand — is how an artifact ends up published with no guard on it at all.

Two further guards on the committed benchmark specifically:

- `test_committed_report_carries_provenance_sample_counts_and_caveats` — a committed benchmark must name the commit it measured, its timestamp, its region and its model ids; must carry a non-empty `caveats` list; and `len(raw_runs)` must equal `totals.runs`, so **every aggregate stays recomputable.**
- `test_committed_report_percentiles_obey_the_sample_minimums` — the *published* numbers face the same refusal rule as a fresh run. An artifact cannot be grandfathered in.

## Two conventions a maintainer should keep

**Never assert a private SDK attribute.** The single real incompatibility found in the `strands-agents` 1.26 → 1.50 upgrade was a test naming `BedrockModel._supports_caching`, which 1.50 refactored to `_cache_strategy`, while product behaviour was identical on both versions. The fix was to assert this repo's own public guard instead:

```
assert m.supports_prompt_caching(m.LIGHT_MODEL) is True
assert m.supports_prompt_caching("global.amazon.nova-pro-v1:0") is False
```

`_`-prefixed names in assertions are upgrade debt. See SDK upgrade.

**Derive contract rules from the source; do not restate them.** `agents/output_contract.py` derives which output rule applies to which agent **from the prompt text at import** rather than hard-coding a table, and that is not stylistic — it corrected a real assumption. The header ban appears in **four** prompts, not six; `income` and `synthetic` never received it. A checker with a hard-coded "6 of 8" would report violations the customer's prompt does not contain, and a fidelity report that flags a non-violation is worse than no report, because it teaches the reviewer to distrust the whole thing. `tests/test_output_contract.py` (25 tests) asserts the derivation, including that it agrees with `agents.fanout.ANALYSIS_TITLES` so the two cannot drift. See Output-contract defects.

## Still unknown

**No test in this suite calls AWS.** That is the design — the demo is rehearsed offline and CI has no Bedrock access — but it means the suite verifies the contracts, the refusals and the

arithmetic, and verifies nothing about live service behaviour. The one live smoke test in `tests/test_fanout_mantle.py` is skipped by default and costs roughly a cent when enabled.


**Eval harness**
The five measurement modules, their spend gates, and what each refuses to report.

**Prompt fidelity**
The sha256 chain in full, and the paraphrase-era LOW RISK default bug.

‹ Eval harness
bench.py, tier_quality.py, set_comparison.p...

Artifact index ›
Every committed measurement artifact, wh...

# Artifact index

Every committed measurement artifact, what it contains, when it was produced, what it cost to produce, and which pages on this site draw their figures from it.

*For anyone who wants to check a number on this site rather than take it.*

Four measurement artifacts, **$28.89** of measured spend between them, each committed with its own recorded timestamp, region and git SHA taken from its `run` block rather than from a commit log. Every figure on this site traces to one of these files; this page is the other direction, so a reader can ask what a given artifact does and does not support.

# Four measurement artifacts

| Artifact | What it is | Recorded run | Measured spend |
|---|---|---|---|
| `evals/results/pipeline_14x3_us-east-1.json` | 14 fixtures × 3 repetitions = **42 adjudications**, **336 specialist calls**, 41 complete + 1 degraded | 2026-07-28 07:42 UTC, `us-east-1`, git `9bb569b` | **$6.4659** |
| `evals/results/tier_quality.json` | Claude-only tier study, **48 blind-judged analyses**, judge Claude Opus 5, candidates Haiku 4.5 / Sonnet 5 / Opus 5 | 2026-07-28 17:29 UTC, `us-east-1` | **$4.9088** |
| `evals/results/set_comparison.json` | Cross-family study, **120 analyses / 240 verdicts**, two judges biased in opposite directions | 2026-07-28 21:32 UTC | **$16.8869** |
| `evals/results/tail_analysis.json` | Re-analysis of the pipeline run's tail, plus one live cap sweep of 30 calls. Its `source` block names the pipeline artifact it reads | derived from the 07:42 run | **$0.6278** (the cap sweep only) |

> ⚠ **Note the two different clocks: the benchmark measures the PREVIOUS assignment**
>
> The pipeline benchmark ran at **07:42 UTC**; the two quality studies ran at **17:29** and **21:32** the same day. Model assignment changed *between* them — `synthetic` and `rings` moved off Opus 5 after the benchmark. So the benchmark measures the previous assignment, and the **assignment page** marks the current one's end-to-end cost as a projection for exactly that reason.

Each JSON has a companion `.md` written for a human: `pipeline_14x3_us-east-1.md`, `tier_quality.md`, `set_comparison.md`, `tail_analysis.md`. The Markdown is generated from the JSON, so if the two ever disagree **the JSON is authoritative**.

## Sidecars, and why reuse is recorded

`set_comparison_analyses.json` and `set_comparison_verdicts.json` hold the raw analyses and raw verdicts behind the summary; `tier_quality_analyses.json` and `tier_quality_verdicts.json` do the same for the tier study.

They exist because both studies **reuse** earlier work instead of re-billing for it, and that has to be visible or a spend figure becomes uncheckable. Both artifacts record it explicitly in `live_calls`.

| Artifact | `specialist_calls_made` | Verdicts reused | Analyses reused | Spend still reported |
|---|---|---|---|---|
| `set_comparison.json` | 0 | 240 | 120 | $16.8869 |
| `tier_quality.json` | 0 | 48 | 48 | $4.9088 |

`tier_quality.json` additionally names its `reuse_sources` — three earlier sweeps, `tiering.json`, `tq_bustout_identity.json` and `tq_rings_synth_extra.json`. Both `spend_actual_usd` blocks carry the same note: rows reused from an earlier sweep carry that sweep's measured cost, and **rows with no recorded usage contribute nothing rather than an estimate**. That is why the spend figures are exact rather than round.

## Which page rests on which artifact

| Page | Primary artifact |
| --- | --- |
| Measured results, Pipeline results | `pipeline_14x3_us-east-1.json` |
| Latency tail, Critical path | `tail_analysis.json`, plus the pipeline artifact it derives from |
| Tier re-derivation | `tier_quality.json` |
| Cross-family study, Length vs evidence, Judge methodology | `set_comparison.json` |
| Snowflake comparison | `pipeline_14x3_us-east-1.json` for our side; the customer's own reported figures for theirs, attributed as theirs |

## Narrative documents in the repository

| File | What it holds |
| --- | --- |
| `docs/model-selection.md` | The long-form model-selection record, including the rejected options |
| `docs/general_rules_coverage.md` | The General Rules coverage table, **generated** from `signal_layer/rules/catalog.py` rather than maintained by hand |
| `docs/semantic-layer-mcp.md` | The Cortex Analyst / MCP analysis behind Semantic layer question |
| `CLAUDE.md` | Engineering context: the findings that change decisions, and the open TODOs |
| `CHANGELOG.md` | The human-readable history of the two landing passes |

## Code that produces or guards the numbers

| File | Role |
| --- | --- |
| `evals/bench.py` | The **only** code permitted to publish latency or cost. Refuses a p95 under 20 samples and a p99 under 100 |
| `evals/judge_sol.py` | The GPT judge, deliberately Sol — the one GPT-5.6 variant that is not a candidate for any specialist slot |
| `agents/models.py` | The assignment, and `TIER_RATIONALE`, whose every entry must cite a measurement that exists on disk |
| `agents/pricing.py` | The rate card. Raises on an unpriced model id rather than defaulting |
| `agents/output_contract.py` | Derives each specialist's output rules **from the prompt text**, so a rule cannot be misremembered |

## Deliberately absent

**Three things you will not find, and should not infer**

- **No end-to-end measurement of the current assignment.** The 42-run benchmark predates the `synthetic` / `rings` move to Sonnet 5. Re-running it is one command and it has not been run.
- **No AgentCore Runtime overhead figure.** The benchmarked calls left the benchmark host directly for Bedrock, so Runtime, cold start and SSE framing all sit outside the timed window.
- **No throughput or concurrency figure.** Every measurement ran **one application at a time**. Nothing here says what happens when adjudications overlap.

Each is stated on the page that would otherwise be read as covering it, not only here.

## Reproducing any of it

```
# offline: 1,238 tests collected, no AWS calls
MOCK_MODE=1 python3 -m pytest tests/ -q

# the pipeline benchmark (real Bedrock spend)
AGENTCORE_BENCH_ALLOW_LIVE=1 python -m evals.bench --pipeline --repetitions 3
```

The live gate is `AGENTCORE_BENCH_ALLOW_LIVE=1`, and it exists so a spend-incurring run cannot be started by muscle memory. See Eval harness for each harness's own gate and what it measures.

## Still unknown

**No artifact here was produced from a deployed AgentCore Runtime.** All four record `run.host` as a developer machine calling Bedrock and bedrock-mantle directly. Anything a reader wants to know about Runtime behaviour — cold start, SSE framing, session handling under load — is not in these files and is not estimated in them.


**Eval harness**
What produced each artifact, how to re-run it, and what it refuses to report.

**Benchmark method**
The percentile policy and the refusal thresholds these artifacts obey.

‹ Test suite
1,238 offline tests — what the mutation test…

Glossary ›
Every term this site uses that a reader could…

# Glossary

Every term this site uses that a reader could reasonably read two ways — the eight fraud dimensions and their persona names, band, grounding, calibration, critical path, mantle, tier, pinned expectation, and the words that mean different things at different layers.

*For both audiences. Ordered by what a newcomer hits first rather than alphabetically.*

Terms marked **AMBIGUOUS** are ones where two sources in this engagement use the same word for different things, and those are the entries worth reading even if you know the domain.

## Eight fraud dimensions

Each specialist carries a persona name from the customer's own documents (`prompts.loader.AGENT_NAMES`), lifted from each prompt file's own header line.

| Domain | Persona | Detects | Its anti-false-positive rule |
|---|---|---|---|
| `identity` | `ISAAC_IDENTITY` | SSN sharing, PII mismatch, identity volatility | Multiple alerts from one underlying data point count as **one** signal |
| `dealer` | `DIANA_DEALER` | Dealer-**orchestrated** fraud | Default / EPD / consortium scores are portfolio risk, not fraud |
| `straw` | `SAMMY_STRAW` | Proxy or nominee co-borrowers, role-switching | No co-borrower means the primary mechanism is absent |
| `employment` | `EMILY_EMPLOYMENT` | Fabricated or shell employers | Missing employer fields are a data gap, not fraud |
| `income` | `IRIS_INCOME` | Overstatement, inconsistent reporting | Under 15% variance is normal; self-employment explains variance |
| `synthetic` | `SYLVIA_SYNTHETIC` | Fabricated identity plus manufactured credit | Thin files are legitimate for young adults and new immigrants |
| `bustout` | `BRUCE_BUSTOUT` | Rapid velocity, unsustainable exposure | Geography alone must not move the classification |
| `rings` | `RENEE_RINGS` | Shared identifiers, exact-unit reuse | Multi-unit housing and large employers explain overlap |

`MASTER_FRAUD_AGENT` — the batch synthesizer. This id is **this port's**, derived from the prompt's header by joining its words with underscores. It is *not* a literal customer string, and the loader records that distinction so nobody quotes it back to her as hers.

**Francis** — the *interactive* persona, in the customer's exact casing from `master_agent_orchestration_062026`. Deliberately **not** upper-cased: the only all-caps `FRANCIS` the customer ever writes is inside `RETRO_FRANCIS`, a separate retro system Francis must deflect to, so `FRANCIS` here would collide with the one name she must never answer as.

`RETRO_FRANCIS` — the customer's separate retro-data system. Francis deflects retro questions to it rather than answering them.

## Bands and verdicts

AMBIGUOUS: the mid band has three names, and each is correct in its own place.

| Layer | Vocabulary for the mid band | Correct? |
|---|---|---|
| **Specialist** output | `POSSIBLE RISK`, between `LOW RISK` and `HIGH RISK` | Yes |
| **Master adjudication** — `master_agent_prompt.txt`, the file that runs | `MEDIUM RISK`, between `LOW RISK` and `HIGH RISK` | Yes |
| **The process-guide PDF** | `POSSIBLE RISK`, with **four** recommendation values | **Conflict** |

First two rows are *both* right: a specialist bands with `POSSIBLE RISK`, the master adjudicates with `MEDIUM RISK`. Third is the **unresolved enum conflict**. The prompt file wins here because it is the artifact that executes, and the conflict is surfaced to the customer rather than reconciled quietly.

**band** — the risk classification a specialist declares, or the master's `overall_risk_decision`. On this site, "band agreement" always means agreement with a **pinned expectation**, never agreement between two models.

`recommendation_decision` — one of `APPROVE` / `REVIEW AND APPLY STIPULATIONS` / `DECLINE` in the prompt file's three-value enum. The PDF's fourth value, `MANUAL REVIEW`, has no target in that enum and is the sharpest edge of the conflict.

**flag** — a per-domain `int 0|1` in the adjudication, for example `identity_flag`. **Not** a restatement of the band. `MSTR-039` sets a flag to `1` only on clear, material, corroborated evidence, so **a domain can legitimately be `POSSIBLE RISK` with its flag at `0`** — and one fixture (APP-1014) is MEDIUM overall with all eight flags at `0`. A flag-counting implementation returns LOW on it.

**adjudication** — the master synthesizer's output: **exactly 21 keys in the customer's order**, starting at `master_summary`. There is **no** `application_id` **key** — it travels in the transport envelope. See Adjudication contract.

**fraud score** — a AnyCompany model output on the application, banded by the customer as `0–299` low, `300–699` medium, `700–998` high, `999` very high, where 999 means at least one fraud accelerator fired. **A score is not evidence**: four fixtures deliberately place it in a band that argues against the pinned outcome. It must also not be averaged, being standard-scaled, and must not be used to pre-filter investigation.

**IncomePass** — the `income_misrep_score` field, indicating likely material income misrepresentation. **Distinct from the fraud score** (General Rule 20).

---

## Measurement vocabulary

**pinned expectation** [#pinned-expectation] — the `expected` block on a fixture: the band, flag, requirement id and quoted customer rule recording what *should* come back. The requirement id, for example `EMP-017`, is the addressable unit of disagreement.

**AMBIGUOUS: a pin disagreement is not automatically a model error.** When a model disagrees with a pin, either could be wrong. APP-1003 was perfectly *stable* across three runs and stably disagreed with its pin, and the benchmark's own caveat says whether the pin or the model is wrong is the customer's call. This site reports those disagreements and does not adjudicate them.

**grounded_fraction** — the deciding metric of the cross-family study, and the one no judge touches. Every specialist prompt requires signals to be validated against the paired `<signal>_context` columns, and `grounded_fraction` is the share of the signals in an agent's view whose paired `_context` row is demonstrably used in the prose: **distinctive tokens are lifted out of the context rows and looked for in the text**, so "cites the context field" is evidence rather than the word *context* appearing somewhere.

Measured per-analysis means (`evals/results/set_comparison.json`, 120 analyses): **Claude 0.642 vs GPT 0.357**, over 48 and 72 analyses respectively, with GPT lower on **4 of 4** agents on both a mean and a median basis. Deterministic, invisible in the band, invisible in latency, invisible in cost. See Cross-family study.

**calibration** — how well a band follows the customer's own calibration text, scored 1–5 by a blind judge. **Subjective by construction**, and reported as such. Distinct from *band agreement*, which is deterministic. The judge's measured self-preference (+0.31 calibration) is the noise floor below which a calibration margin means nothing.

**substance** — a judge score for *distinct grounded evidence*, explicitly **not length**. The customer's own master prompt supplies the reason (`MSTR-018`): the amount of analysis written does not necessarily indicate fraud severity.

**padding** — output length not carrying additional distinct evidence. Measured by pairing on identical payloads, pooled across the four swept agents: Opus writes **2.95×** Luna's characters while naming only **1.08×** the payload's signals, and the two agree on band in **23 of 24** pairs. So the length gap is padding rather than lost evidence.

**anti-false-positive fixture** — a record built to *look* bad and resolve LOW anyway, isolating one rule the customer states as an explicit exception. **Five are designated** (`ANTI_FP_APPS`: APP-1006 … APP-1010); **four more are embedded** in records that are not themselves LOW overall, where a single domain must clear independently. See Fixtures.

**false positive / false negative** — in the judges' vocabulary: escalating something the customer's rules call benign, or clearing something the rules say should have been escalated. The customer's stated quality goal is reducing false positives, which is why the study weights anti-FP escalation above every other axis.

**critical path** — the specialist that was the **slowest in a given run**. Because `asyncio.gather` cannot return before its slowest member, the distribution of slowest-specialist wall clock *is* the fan-out floor. `rings` owned it in **25 of 42** runs.

**AMBIGUOUS: the agent that owns the median is not the agent that produces the tail.** `rings` sets the p50 at 20.96 s, the highest of the eight. The two runs that crossed 60 s were owned by `bustout` (69.78 s) and `income` (66.01 s), so **shaving `rings` alone would not have fixed either**. See Critical path.

**stall** — in `evals/tail_analysis.py`, a **single specialist** call crossing 60 s. Not an outlier rule: it is the customer's sub-minute target applied to one agent, because one specialist consuming the whole budget is a different physical event from a verbose one. The two stalls in the sample generated at **16.1 and 4.7 output tok/s** against a **65.50 tok/s p50** over the 332 calls under the threshold (`evals/results/tail_analysis.json`).

**sequential counterfactual** (`sum_of_eight_ms`) — the sum of the eight specialists' own measured wall clocks in a run. **The terms are measured; the arrangement is hypothetical.** It is what eight calls would cost one after another *only if* each took the same time alone.

**parallel speed-up** — fan-out wall clock divided by that counterfactual, **one ratio per run** (p50 3.64×, n=42). Reported as a distribution because the fan-out is bounded by its slowest dimension, so the ratio moves with which specialist ran long.

**refused percentile** — `null` in the JSON *with the reason beside it*, not an omission and never a substituted number. p50 needs 5 samples, p95 needs 20, p99 needs 100. `n/a` in a table means refused, never zero.

**degraded run** — an adjudication that lost one or more specialists and still validated its 21 keys. It **stays in the latency distribution**, because it really took that long, and **leaves the cost distribution**, because eight priced calls presented as a nine-call total is an under-count wearing a total's name. That is why end-to-end is n=42 and cost is n=41.

---

## Platform vocabulary

**mantle** / **bedrock-mantle** — the Amazon Bedrock endpoint serving OpenAI models through the OpenAI Responses API, at `bedrock-mantle.{region}.api.aws`. **Not** Converse: GPT-5.x is not on Converse at all. Needs the separate IAM action `bedrock-mantle:CreateInference`, a **bearer token** rather than SigV4 request signing, and the base URL path `/openai/v1`, since `/v1` on the same host serves only the open-weight `gpt-oss` family. See GPT-5.6 on bedrock-mantle.

**tier** — this port's `light` / `mid` / `heavy` grouping, which sets `max_tokens` (3072 / 4096 / 4096) and the default model per group.

> **AMBIGUOUS: 'tier' does not mean 'saves money'**
>
> Haiku→Sonnet is a **$2/$10-per-MTok step down** and Sonnet→Opus is the **identical step up**, so at equal token counts one LIGHT agent funds exactly one HEAVY agent. Tiering saves **$0.00 structurally**, and a test asserts the arithmetic so the claim cannot return. The measured 20.13% saving was **entirely** Sonnet 5's introductory rate, which reverts on **2026-09-01**. See Cost levers.
>
> There is also **no HEAVY agent left** — `synthetic` and `rings` moved off Opus 5 on 2026-07-28 — so the current shape is two cheap agents and six standard, and what remains is simply not paying Opus rates for output that measured no better.

**inference profile prefix** — `global.*` or `us.*` on a Claude model id. Same weights, same API; `us.*` bills at exactly **1.10×** on every rate category. A policy allowing only `us.anthropic.*` silently forbids the cheaper one.

**effort** — the reasoning-effort control. On Claude via Converse: `additional_request_fields={"output_config": {"effort": "low"}}`. On mantle: `reasoning={"effort": …}`, where `temperature` / `top_p` / `top_k` are **ignored**.

**cache-write tokens** — input tokens written to a prompt cache. Billed at **1.25×** the input rate on GPT-5.6 and on Claude. Omitting the term under-reported a measured Luna call by **62%**, because with implicit caching on and a large prompt most of the input lands in that class.

**minimum cacheable prefix** — the prompt size below which a cache point silently never engages. **Not monotonic across generations**: Opus 5 = 512, Sonnet 5 / Sonnet 4.6 / Opus 4.8 = 1024, **Haiku 4.5 = 4096**. No error, no warning, permanent 0% hit rate. Against 547–956-token specialist prompts, prompt caching engages on **0 of 8** agents. See Cost: what does not work.

**signal** vs `_context` — a signal is a precomputed numeric or boolean value (`address_reuse_distinct_ssn_count = 96`). Its `_context` row is the paired explanatory field, for example a 240-unit apartment complex. **The benign explanation lives only in the context row**, which is why `grounded_fraction` is the metric that separates a real reading from a fluent one.

`DomainView` — exactly what one specialist is entitled to: its own signals, their context, its own alerts. There is no data tool on the agent side, so this object is the **complete universe** the specialist reasons over.

`SignalPayload` — the whole-application object: core fields, eight `DomainView`s, `guardrails_applied` (General Rule **ids**, not prose), `semantic_layer_rev`, `computed_at`.

**guardrail** — **ambiguous by platform.** In this repo it means a numbered **General Rule** from the customer's orchestration documents, enforced in `signal_layer/`. On AWS it also names a **Bedrock Guardrail**, a platform feature this repo does **not** configure — no rule here relies on one as its only enforcement layer.

**General Rule** — one of **29** numbered rules repeated across all eight orchestration documents. **27 are enforced by deterministic code; 2 are prompt-enforced** interpretive guidance. See General Rules coverage.

**requirement id** — the addressable label this port assigns to a single obligation in a customer prompt: `IDEN-014`, `EMP-017`, `MSTR-039`, `STRW-008`, `XAG-003`. The `MSTR-*` prefix is the master prompt; `XAG-*` is a rule appearing across agents. Every fixture pin and every golden assertion cites one, so a disagreement lands on a specific sentence.

**alert category** — one of the **ten** groupings in General Rule 3. **166 entries, 148 distinct ids, 18 ids in more than one category** — the overlap is the customer's, not a data error. Alerts 31–39 are explicitly disregardable.

`SIGNAL_MODE` — the proposed flag on the Lambda Gateway target (`cortex` | `aurora`) that makes live-Cortex and precomputed-Aurora one config change. See Semantic layer question.

---

## Two words to use carefully in front of this customer

> ⚠ **'measured' and '1:1' both have a narrow meaning here**
>
> **measured** means: a real call happened, the number came from that call's own reported telemetry, and the artifact naming the run is cited. Anything else is labelled **PROJECTION**, **NOT**

**MEASURED**, or **the customer's own figure, by attribution**. Their reported figures — 4,500 applications taking over 48 hours, $700K–$1M projected annual Snowflake AI spend — are **theirs**, and must never appear beside one of ours as though both were measured the same way, because this repo has never run their Snowflake pipeline and cannot.

**1:1** means the **prompts and the output contracts are the customer's, byte for byte** — not "equivalent in intent". The phrase *verbatim in intent* appeared in the pre-port code's own docstring while that code had rewritten all nine prompts, so it is a phrase to avoid entirely.

## Still unknown

**Three terms here are ours rather than hers, and are labelled as such where they appear:** `MASTER_FRAUD_AGENT`, every `requirement id`, and the `light` / `mid` / `heavy` tier names. If the customer uses different labels internally, these are the ones to reconcile — none of them changes behaviour, and all three appear in artifacts she may read.

**Signal layer**
Where signals, context, views and bands are actually defined.

**Artifact index**
Every committed measurement file, its n, its spend, and what it can support.

< Artifact index

Every committed measurement artifact, what it contains, when it was produced, what it cost to pr...